



NATIONAL AND KAPODISTRIAN
UNIVERSITY OF ATHENS

School of Science, Department of Physics

Interdepartmental Postgraduate Program in Electronic Automation (E/A)

Reducing Multivariate IoT Data Streams at the Network Edge for Efficient Cloud Storage

Surname: Papageorgiou

First Name: Michail

Registration Number: 7110132300201

Supervisor: Efstathios Chatziefthymiades, Professor NKUA

Athens 2026

ΠΕΡΙΛΗΨΗ

Η παρούσα διπλωματική εργασία αφορά την ανάπτυξη μιας μεθοδολογίας για τη μείωση δεδομένων σε πολυμεταβλητές ροές δεδομένων IoT, με στόχο τη δημιουργία ενός αντιπροσωπευτικού δείγματος που επιτρέπει την αποδοτικότερη αποθήκευσή τους στο υπολογιστικό νέφος (cloud). Η προτεινόμενη προσέγγιση εστιάζει στην επεξεργασία των δεδομένων στην παρυφή του δικτύου, περιορίζοντας τον όγκο των δεδομένων πριν από τη μετάδοσή τους στο υπολογιστικό νέφος.

Για την επίτευξη της μείωσης των δεδομένων, εξετάζονται αλγόριθμοι επιλογής και αντικατάστασης μετρήσεων κατά την εισαγωγή τους σε μια δομή δεξαμενής (buffer). Κεντρικό στοιχείο της προτεινόμενης μεθοδολογίας αποτελεί η χρήση του αλγορίθμου Multivariate Cumulative Sum (MCUSUM) ως μηχανισμού ανίχνευσης μεταβολών στη ροή δεδομένων. Ο αλγόριθμος καθορίζει το τέλος μιας «εποχής», σηματοδοτώντας την αποστολή των δεδομένων που έχουν διατηρηθεί στη δεξαμενή προς το υπολογιστικό νέφος και την έναρξη μιας νέας εποχής. Με τον τρόπο αυτό επιδιώκεται η διατήρηση των σημαντικών στατιστικών χαρακτηριστικών της ροής, παράλληλα με τη μείωση του όγκου των μεταδιδόμενων δεδομένων.

Τέλος, πραγματοποιείται πειραματική αξιολόγηση της προτεινόμενης μεθοδολογίας, εξετάζοντας διαφορετικές στρατηγικές αντικατάστασης, διαφορετικά μεγέθη δεξαμενής και διαφορετικές τιμές των παραμέτρων του MCUSUM. Η αξιολόγηση βασίζεται σε δείκτες που αποτυπώνουν τόσο τον βαθμό συμπίεσης όσο και τη διατήρηση της στατιστικής πιστότητας και της ικανότητας ανίχνευσης. Στόχος είναι η διερεύνηση της ισορροπίας μεταξύ της μείωσης του όγκου των δεδομένων και της διατήρησης της πληροφορίας που περιέχεται στην αρχική ροή.

ABSTRACT

This master's thesis focuses on the development of a methodology for data reduction in multivariate IoT data streams, with the aim of generating a representative sample that enables more efficient data storage in the cloud. The proposed approach focuses on processing data at the network edge, thereby reducing the volume of data before transmission to the cloud.

To achieve data reduction, data selection and replacement algorithms are investigated for the insertion of measurements into a buffer structure. A central component of the proposed methodology is the use of the Multivariate Cumulative Sum (MCUSUM) algorithm as a mechanism for detecting changes in the data stream. The algorithm determines the end of an "epoch", signaling the transmission of the data retained in the buffer to the cloud and the beginning of a new epoch. In this way, the methodology aims to preserve the key statistical characteristics of the data stream while reducing the volume of transmitted data.

Finally, an experimental evaluation of the proposed methodology is conducted, examining different replacement strategies, buffer sizes, and MCUSUM parameter values. The evaluation is based on indices that capture both the degree of compression and the preservation of statistical fidelity and detection capability. The objective is to investigate the trade-off between reducing data volume and preserving the information contained in the original data stream.

Keywords: Multivariate IoT Data Streams; Data Reduction; Edge Computing; Cloud Storage; MCUSUM; Data Compression; Anomaly Detection; Buffer Management; Replacement Strategies

Table of Contents

ΠΕΡΙΛΗΨΗ	2
ABSTRACT	3
LIST OF FIGURES	6
LIST OF TABLES.....	7
1. INTRODUCTION	8
2. MULTIVARIATE STATISTICAL ANALYSIS	10
2.1 BASIC ELEMENTS OF MATRIX AND VECTOR ALGEBRA	10
2.1.1 VECTORS.....	10
2.1.2 MATRICES.....	12
2.2 DATA ANALYSIS.....	15
2.2.1 GRAPHICAL METHODS.....	15
2.2.2 NUMERICAL METHODS	19
2.3 PROBABILITY DISTRIBUTION	21
2.3.1 UNIVARIATE NORMAL DISTRIBUTION	22
2.3.2 MULTIVARIATE NORMAL DISTRIBUTION.....	23
2.3.3 CENTRAL LIMIT THEOREM.....	25
2.3.4 CHI-SQUARE DISTRIBUTION.....	26
3. STATISTICAL PROCESS CONTROL.....	28
3.1 SHEWHART CONTROL CHARTS	28
3.2 CUMULATIVE SUM CONTROL CHARTS	29
3.2.1 UNIVARIATE CUSUM	29
3.2.2 MULTIVARIATE CUSUM	35
4. SYSTEM ARCHITECTURE	41
4.1 THE CONCEPT OF AN EPOCH.....	41
4.2 ALGORITHMIC FLOW AND MEMORY MANAGEMENT.....	41
4.2.1 PHASE 1: EPOCH INITIATION	41
4.2.2 PHASE 2: EPOCH DURATION.....	42
4.2.3 PHASE 3: EPOCH END	43
5. ARCHITECTURE IMPLEMENTATION	44
5.1 DATA BUFFER.....	44
5.1.1 IMPLEMENTATION OF THE DATA BUFFER ALGORITHM	45
5.2 IMPLEMENTATION OF THE VECTOR CUSUM ALGORITHM	50
5.3 REPLACEMENT METHODS.....	52
5.3.1 MAX RANGE REPLACEMENT METHOD.....	53
5.3.2 MAHALANOBIS RANGE REPLACEMENT METHOD.....	57
5.3.3 MAX DISTANCE EXPANSION REPLACEMENT METHOD	62
5.4 SYNOPSIS EXPORT AND STORAGE	65
5.4.1 CLASS IMPLEMENTATION.....	65
6. DATA PROCESSING AND DATASET CREATION	69
6.1 VARIABLE DESCRIPTION, EVALUATION, AND ANALYSIS.....	69
6.2 DATASET CREATION.....	72
7. ALGORITHM EXECUTION AND SYNOPSIS GENERATION.....	76
8. RESULTS EVALUATION METHOD	78
8.1 EVALUATION OF REPLACEMENT METHODS	78
8.1.1 VOLUME COMPARISON METHOD	79

8.1.2	GENERALIZED VARIANCE COMPARISON METHOD	80
8.2	COMPRESSION RATE EVALUATION	81
8.3	CHANGE DETECTION EVALUATION	81
8.4	COMPOSITE EVALUATION METRICS	83
8.4.1	SAMPLING QUALITY SCORE - SQ	83
8.4.2	STATISTICAL EFFICIENCY SCORE - SE	84
8.4.3	TOTAL PERFORMANCE SCORE - TP	84
8.5	IMPLEMENTATION OF THE EVALUATION ALGORITHM	84
9.	RESULTS AND EVALUATION	86
9.1	REPLACEMENT METHODS	89
9.1.1	VOLUME COMPARISON INDEX R	89
9.1.2	GENERALIZED VARIANCE COMPARISON INDEX S	97
9.1.3	PROBLEMATIC EPOCHS	106
9.2	COMPRESSION RATIO	107
9.3	CHANGES DETECTION	115
9.4	COMPOSITE EVALUATION METRICS	127
9.4.1	SAMPLING QUALITY INDEX SQ	127
9.4.2	STATISTICAL EFFICIENCY INDEX SE	133
9.4.3	TOTAL PERFORMANCE INDEX TP	141
10.	CONCLUSIONS	150
10.1.1	CONCLUSIONS ON PARAMETERIZATION	150
10.1.2	CONCLUSIONS ON REPLACEMENT METHODS	151
10.2	LIMITATIONS AND PROPOSALS FOR FURTHER RESEARCH	152
11.	REFERENCES	153
APPENDIX A:	ALGORITHM CORE IMPLEMENTATION	155
	MAIN STREAM MANAGEMENT SCRIPT	155
	VECTOR CUSUM	160
	BUFFER	162
	COVARIANCE MATRIX AND MEAN CALCULATION	164
	MAHALANOBIS RANGE REPLACEMENT METHOD	165
	MAX DISTANCE EXPANSION REPLACEMENT METHOD	168
	MAX RANGE REPLACEMENT METHOD	170
	SYNOPSIS EXPORTER	172
APPENDIX B:	DATASET GENERATION	174
APPENDIX C:	SIMULATION	179
APPENDIX D:	EVALUATION	182
	GLUE CODE	182
	GENERALIZED VARIANCE INDEX	186
	VOLUME COMPARISON INDEX	189
	INDEX PATE-F1	191
	COMPRESSION RATIO	195

LIST OF FIGURES

Figure 1 - Geometric representation of a two-dimensional vector	11
Figure 2 - An $m \times n$ matrix.....	12
Figure 3 - Bar chart.....	17
Figure 4 – Histogram.....	17
Figure 5 - Probability or frequency curve	18
Figure 6 - Normal distribution, standard deviation and coverage in statistics.....	22
Figure 7 - Bivariate normal distribution.....	25
Figure 8 - Chi-Square distribution.....	27
Figure 9 - Typical Shewhart control chart	29
Figure 10 - V-mask application to a CUSUM chart.....	30
Figure 11 - Kenneth Kemp chart	34
Figure 12 - Variation of index R as a function of the decision interval h for allowance values k	93
Figure 13 - Boxplot of index R as a function of the buffer size	95
Figure 14 - Variation of index R as a function of buffer size and replacement method	96
Figure 15 - Variation of index S' as a function of the decision interval h for allowance values k	104
Figure 16 - Boxplot of index S' as a function of the buffer size	105
Figure 17 - Variation of index S' as a function of buffer size and replacement method.....	106
Figure 18 - Variation of number of epochs as a function of the decision interval h for allowance values k	112
Figure 19 - Variation of index C as a function of the decision interval h for allowance values k	113
Figure 20 - Variation of number of epochs and buffer size	114
Figure 21 - Boxplot of index C as a function of the buffer size.....	115
Figure 22 - Variation of index PATE-F1 as a function of the decision interval h for allowance values k	121
Figure 23 - Variation of false alarms as a function of the decision interval h for allowance values k	122
Figure 24 - Variation of true alarms as a function of the decision interval h for allowance values k	123
Figure 25 - Boxplot of index PATE-F1 as a function of the buffer size	124
Figure 26 - Variation of false alarms as a function of buffer size.....	125
Figure 27 - Variation of true alarms as a function of buffer size	126
Figure 28 - Variation of index SQ as a function of the decision interval h for allowance values k	131
Figure 29 - Boxplot of index SQ as a function of the buffer size	132
Figure 30 - Variation of index SQ as a function of buffer size and replacement method	133
Figure 31 - Variation of index SE as a function of the decision interval h for allowance values k	138
Figure 32 - Boxplot of index SE as a function of the buffer size.....	139
Figure 33 - Variation of index SE as a function of buffer size and replacement method.....	141
Figure 34 - Variation of index TP as a function of the decision interval h for allowance values k	146
Figure 35 - Boxplot of index TP as a function of the buffer size.....	147
Figure 36 - Variation of index TP as a function of buffer size and replacement method.....	149

LIST OF TABLES

Table 1 - DataBuffer class variables	46
Table 2 - VectorCUSUM class variables	50
Table 3 - MaxRangeReplacement class variables	54
Table 4 - MahalanobisRangeReplacement class variables	58
Table 5 - Variables of MaxDistanceExpansionReplacement class	63
Table 6 - SynopsisExporter class variables	66
Table 7 - Chosen variables	72
Table 8 - Mean and standard deviation of the chosen variables	73
Table 9 - Correlation matrix	75
Table 10 - Theoretical ARL_0 results	76
Table 11 - Parameter combinations for the algorithm implementation	88
Table 12 - Evaluation metrics	89
Table 13 - Analytic results for R index	92
Table 14 - Index R results for each replacement method	92
Table 15 - Index R results for each allowance and decision interval combination	92
Table 16 - Index R results for each buffer size	92
Table 17 - Analytic results for S and S' indexes	102
Table 18 - Index S' results for each replacement method	102
Table 19 - Index S' results for each allowance and decision interval combination	102
Table 20 - Index S' results for each buffer size	102
Table 21 - Analytic results for C index	110
Table 22 - Index C results for each allowance and decision interval combination	111
Table 23 - Index C results for each buffer size	111
Table 24 - Analytic results for PATE-F1 index	119
Table 25 - Index PATE-F1 results for each allowance and decision interval combination	120
Table 26 - Index PATE-F1 results for each buffer size	120
Table 27 - Analytic results for SQ index	129
Table 28 - Index SQ results for each replacement method	130
Table 29 - Index SQ results for each allowance and decision interval combination	130
Table 30 - Index SQ results for each buffer size	130
Table 31 - Analytic results for SE index	136
Table 32 - Index SE results for each replacement method	136
Table 33 - Index SE results for each allowance and decision interval combination	137
Table 34 - Index SE results for each buffer size	137
Table 35 - Analytic results for TP index	144
Table 36 - Index TP results for each replacement method	144
Table 37 - Index TP results for each allowance and decision interval combination	144
Table 38 - Index TP results for each buffer size	145

1. INTRODUCTION

The development of the Internet of Things (IoT) has significantly increased the volume of data generated by sensors and devices. In industrial facilities, ships, vehicles, buildings, and urban networks, multidimensional measurements are continuously collected. Transmitting all these data to the cloud can create substantial demands on both bandwidth and storage capacity. This issue becomes even more important in applications where cloud connectivity is limited or incurs high costs, such as in maritime transportation, remote industrial plants, and mobile networks.

Transmitting raw data to the cloud for processing is not always a practical solution. Besides transmission and storage costs, this approach can increase latency, particularly when frequent data transfer is required. For this reason, the edge computing paradigm has been developed over the last few years. In this scenario, a portion of the processing is performed close to the data source before being transmitted to the cloud. Consequently, the volume of transferred data is reduced, and only the information necessary for subsequent processing or analysis is retained.

In this context, the present thesis examines the reduction of multivariate IoT data streams at the network edge. The primary objective is to generate a synopsis of the original stream, which can be transmitted to the cloud for storage and further analysis. This synopsis must contain a significantly reduced volume of data, without substantially deviating from the basic statistical characteristics of the initial stream.

To this end, the continuous stream is segmented into discrete time intervals, which are referred to as epochs in this thesis. Each epoch comprises a portion of the stream during which the statistical behavior of the process is considered relatively stable. When a significant shift is detected in the stream, the current epoch concludes. At this point, the buffer is emptied, and the synopsis generated thus far is transmitted to the cloud. Subsequently, the next epoch commences with the new data.

For the detection of process shifts, the Multivariate Cumulative Sum (MCUSUM) algorithm is utilized, specifically its vector version (Vector CUSUM) as proposed by Crosier. This particular algorithm was selected because the data stream consists of multiple variables that are not necessarily mutually independent. In contrast to Shewhart charts, where the decision is primarily based on the deviation of a single observation, the CUSUM algorithm accumulates deviations from the reference state. In this way, it is capable of detecting smaller shifts in the multivariate mean when they occur in a persistent manner.

The detection of process shifts is combined with a limited-capacity buffer, within which the synopsis of the current epoch is maintained. When the available buffer space is exhausted, one of the three replacement strategies examined in this thesis is applied, aiming to capture as much of the data dispersion as possible. These three strategies are based, respectively, on maximizing the Euclidean distance from the mean vector, maximizing the Mahalanobis distance from the mean vector, and maximizing the distance between the data points.

The implementation of the system was carried out in Python using an object-oriented design. Special attention was paid to buffer management and the execution of computations, as the method is intended to operate in a resource-constrained edge environment. For the evaluation, synthetic data simulating ship sensor measurements were utilized. This enables the study of the system's behavior on a multivariate data stream with characteristics mirroring a real-world application. Experiments were conducted for different buffer sizes and various values of the CUSUM parameters. Performance was examined using multiple metrics, including the volume ratio, generalized variance, compression rate, and PATE-F1. Additionally, combined metrics were used to evaluate sampling quality, statistical efficiency, and overall system performance.

The structure of the thesis is as follows. Chapter 2 presents fundamental concepts of multivariate statistical analysis, such as matrix algebra, graphical and numerical methods, and the basic distributions used hereafter. Chapter 3 introduces Statistical Process Control, referencing Shewhart charts and the CUSUM algorithm in both their univariate and multivariate forms. Chapter 4 describes the proposed architecture, the concept of the epoch, and the individual operational stages of the system. Chapter 5 presents the implementation of the core structures, the buffer, the Vector CUSUM, and the three replacement methods. Chapter 6 focuses on data preparation and the generation of the synthetic dataset. Chapter 7 describes the experimental procedure and the parameters used in the executions. Chapter 8 presents the evaluation methodology, and Chapter 9 analyzes the experimental results. Finally, Chapter 10 summarizes the main conclusions of the thesis and outlines proposals for future research.

2. MULTIVARIATE STATISTICAL ANALYSIS

When we try to understand social or physical phenomena, we need to examine different aspects and data that are mixed in complicated ways. The complexity of these phenomena requires the gathering and analysis of many different variables. Multivariate Statistical Analysis is the most commonly used statistical approach for actual applications. It provides the statistical tools needed to examine interconnected metrics as a whole, instead of treating each one in isolation.

2.1 BASIC ELEMENTS OF MATRIX AND VECTOR ALGEBRA

Working with multivariate data leads to matrix and vector algebra, as there is no compact way to describe and manipulate multidimensional observations without them. In an IoT environment, each measurement received at a given timestamp is simply expressed as a vector, and a series of such measurements forms a data matrix. The sections that follow build on this representation, so a brief review of the core algebraic concepts is worthwhile before moving on.

2.1.1 VECTORS

Multivariate data analysis relies on vector algebra, as multidimensional observations are naturally represented as mathematical vectors. An array $\vec{x}$ consisting of ν real numbers $x_1, x_2, \dots, x_\nu$ is called a vector and by convention is structured as a column. When a row representation is needed, the vector is transposed, denoted as $\vec{x}'$.

$$\vec{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_\nu \end{bmatrix}, \quad \vec{x}' = [x_1 \quad \dots \quad x_\nu]$$

In addition to the above representation, a vector $\vec{x}$ can be represented geometrically as a directed line segment in an ν -dimensional space. Each numerical element x_i of the vector corresponds to a component along a specific axis of the multidimensional coordinate system.

The basic characteristics of a vector are its length and its direction. The length of the vector $\vec{x}$, denoted as $|\vec{x}|$, is a non-negative real number and it is given by:

$$|\vec{x}| = \sqrt{\vec{x}' \cdot \vec{x}} = \sqrt{x_1^2 + \dots + x_\nu^2} \quad (1)$$

The direction of the vector is determined by its spatial orientation within the ν -dimensional coordinate system. Mathematically, the direction can be isolated and represented by a unit vector $\vec{u}$ which has length equal to 1 and is obtained by dividing the original vector by its length.

The next figure illustrates these geometric properties using a basic two-dimensional vector. As we can see, the length $|\vec{x}|$ corresponds to the physical length of the vector, while the direction is defined by the angle θ . Although the analysis is performed in ν dimensions, the two-dimensional representation provides a simpler way to visualize the concept.

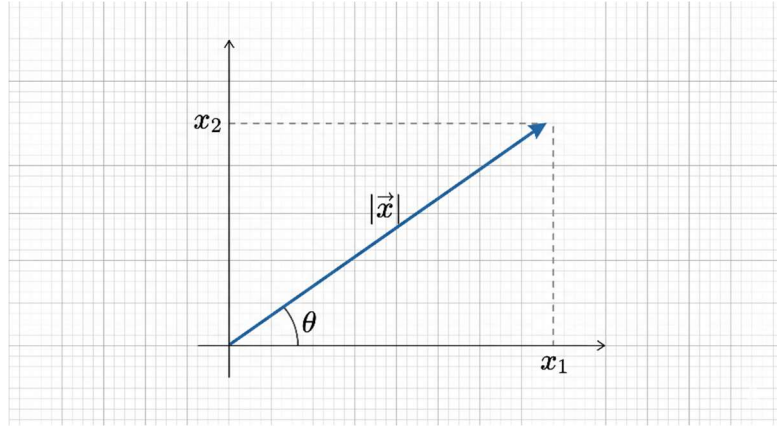


Figure 1 - Geometric representation of a two-dimensional vector¹

2.1.1.1 Vector Operations

The multiplication of a vector $\vec{x}$ by a real number c produces a new vector, wherein each individual component of the original vector is multiplied by c .

$$c \cdot \vec{x} = \begin{bmatrix} c \cdot x_1 \\ \vdots \\ c \cdot x_v \end{bmatrix} \quad (2)$$

The addition or the subtraction between the two v -dimension vectors $\vec{x}$ and $\vec{y}$ produces a new vector $\vec{z}$, wherein each individual component of the new vector $\vec{z}_i$ is the result of the addition or the subtraction of $\vec{x}_i$ and $\vec{y}_i$.

$$\vec{z} = \vec{x} + \vec{y} = \begin{bmatrix} x_1 + y_1 \\ \vdots \\ x_v + y_v \end{bmatrix} \quad (3)$$

The inner product of these two vectors is a real number and it is defined as:

$$\vec{x}' \cdot \vec{y} = x_1 \cdot y_1 + \dots + x_v \cdot y_v \quad (4)$$

The inner product can be used for the calculation of the angle between vector $\vec{x}$, $\vec{y}$. More specifically, the cosine of the θ angle between the two vectors can be calculated by this formula:

$$\cos\theta = \frac{\vec{x}' \cdot \vec{y}}{|\vec{x}| \cdot |\vec{y}|} \quad (5)$$

The projection of the vector $\vec{x}$ on the vector $\vec{y}$ is a vector parallel to $\vec{y}$ and it is defined as:

$$\text{projection of } \vec{x} \text{ on } \vec{y} = \frac{\vec{x}' \cdot \vec{y}}{\vec{y}' \cdot \vec{y}} \cdot \vec{y} \quad (6)$$

The length of the projection can be calculated as:

$$\text{Length of the projection} = \frac{|\vec{x}' \cdot \vec{y}|}{|\vec{y}|} = \frac{\vec{x}' \cdot \vec{y}}{|\vec{y}| \cdot |\vec{x}|} \cdot |\vec{x}| = |\vec{x}| \cdot |\cos\theta| \quad (7)$$

¹ This figure was produced with the assistance of the Gemini (Google) generative AI model.

Linear independence is another basic concept in vector algebra. A set of vectors is considered linearly independent if no single vector within the set can be expressed as a linear combination of the others. Mathematically, a set of vectors $\vec{x}_1, \vec{x}_2, \dots, \vec{x}_k$ is linearly independent if the equation $c_1 \cdot \vec{x}_1 + c_2 \cdot \vec{x}_2 + \dots + c_k \cdot \vec{x}_k = 0$ holds true only when all scalar coefficients c_i are exactly equal to zero.

2.1.2 MATRICES

A matrix is a rectangular array of numbers with elements arranged in rows and columns. In multivariate analysis, matrix theory provides tools not only for data representation but also for their analytical interpretation and the extraction of robust statistical conclusions. Modern IoT deployments typically involve several sensors operating simultaneously, each producing a continuous stream of measurements. When these streams are treated as sequences of multidimensional vectors and organized into data matrices, patterns that would be invisible in any single variable become apparent. This matrix-based perspective is what enables the application of multivariate change-detection techniques — including the Vector CUSUM algorithm examined later in this work — to accurately track shifts in the underlying process and signal the end of an epoch.

In the next figure a matrix with n columns and m rows is shown. As illustrated, this mathematical structure is composed of individual numerical elements a_{ij} where the index i denotes the specific row and j denotes the corresponding column.

$$\begin{matrix} & \begin{matrix} 1 & 2 & \dots & n \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ \vdots \\ m \end{matrix} & \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ a_{31} & a_{32} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \end{matrix}$$

Figure 2 - An $m \times n$ matrix²

A matrix on which the number of columns is equal to the number of rows is called square matrix. A square matrix that has ones on the diagonal and zeros elsewhere is called identity matrix.

2.1.2.1 Matrix Operations

Likewise in arrays, if in a matrix A the columns are changed to rows and rows to columns, this transformation is called transposition and this matrix is denoted as A' . It is obvious that if the matrix A consists of n columns and m rows the matrix A' consists of m columns and n rows. A square matrix that is equal to its transpose is called symmetric. The following example illustrates this mathematical operation.

$$A' = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}' = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

The multiplication of a matrix A by a real number c produces a new matrix, wherein each individual component of the original matrix is multiplied by c .

² Source: Wikipedia, "Matrix (mathematics)". Available at: [https://en.wikipedia.org/wiki/Matrix_\(mathematics\)](https://en.wikipedia.org/wiki/Matrix_(mathematics)) (Accessed: [4/2026])

$$c \cdot A = \begin{bmatrix} c \cdot a_{11} & c \cdot a_{12} \\ c \cdot a_{21} & c \cdot a_{22} \\ c \cdot a_{31} & c \cdot a_{32} \end{bmatrix} \quad (8)$$

The addition and subtraction of two matrices A and B are feasible when they share identical dimensions. The result of these operations is a new matrix C, where each individual component c_{ij} is the result of the operation between a_{ij} and b_{ij} .

However, standard matrix multiplication operates under different mathematical principles. The multiplication of matrix A and matrix B is feasible only when the inner dimensions match. More specific, the number of columns in A must equal the number of rows in B. The elements of the resulting matrix C are not calculated element-wise, but rather through the dot product of the rows of A and the columns of B. In addition, the multiplication between a square matrix and an identity matrix of the same dimensions results in the original matrix itself, leaving its elements entirely unchanged.

The following example illustrates how the standard multiplication of two matrices is mathematically performed. Assume a 2x3 matrix A and a 3x2 matrix B:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \text{ and } B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$$

Because matrix A has 3 columns and matrix B has 3 rows, the multiplication A·B is valid and will result in a new 2x2 matrix C. The calculation for each component c_{ij} is executed as follows:

$$C = A \cdot B = \begin{bmatrix} (1 \cdot 7 + 2 \cdot 9 + 3 \cdot 11) & (1 \cdot 8 + 2 \cdot 10 + 3 \cdot 12) \\ (4 \cdot 7 + 5 \cdot 9 + 6 \cdot 11) & (4 \cdot 8 + 5 \cdot 10 + 6 \cdot 12) \end{bmatrix} = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}$$

Similarly to the inverse numbers in basic algebra, in matrix algebra a matrix B is called the inverse of matrix A if their multiplication results in the identity matrix I. Specifically, this strict relationship dictates that $A \cdot B = B \cdot A = I$. For this property to hold valid, matrices A, B, and I must be square and share the same dimensions. Under these conditions, the inverse matrix B is formally denoted as A^{-1} , yielding the fundamental relationship $A^{-1} = B$.

However, not all matrices have an inverse matrix. The fundamental condition that must be true in order for the inverse of matrix A to exist is that the columns of A must be linearly independent. More specifically, if $\vec{a}_1, \vec{a}_2, \dots, \vec{a}_k$ are column vectors that represent A's columns the equation $c_1 \cdot \vec{a}_1 + c_2 \cdot \vec{a}_2 + \dots + c_k \cdot \vec{a}_k = 0$ holds true only when all scalar coefficients c_i are exactly equal to zero.

To mathematically verify this independence and further characterize the matrix, we use a single scalar value called the determinant. The determinant can only be calculated for square matrices, and a matrix is invertible if and only if its determinant is non-zero. In multivariate statistical analysis, the determinant is also used as a measure of generalized variance. For example, when evaluating a sample covariance matrix, its determinant indicates the overall spread or variability of the individual raw data. A larger determinant means larger differences in the studied variables. However, it is important to note that determinants do not uniquely identify a matrix, because different matrices can have the same exact determinant. For a simple 2 x 2 matrix:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

its determinant is calculated by subtracting the product of its off-diagonal elements from the product of its main diagonal elements. Therefore, the formula is:

$$|A| = a \cdot d - b \cdot c \quad (9)$$

For higher-order matrices, such as a 3 x 3 matrix, their determinants are worked out following a recursive rule that reduces their computation to that of determinants of lower order. Assuming a 3 x 3 matrix $A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$, its determinant is calculated by expanding across its first row, which involves the determinants

of the smaller 2 x 2 sub-matrices:

$$|A| = a \cdot \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \cdot \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \cdot \begin{vmatrix} d & e \\ g & h \end{vmatrix} = a \cdot (e \cdot i - f \cdot h) - b \cdot (d \cdot i - f \cdot g) + c \cdot (d \cdot h - e \cdot g) \quad (10)$$

Once the determinant is calculated and confirmed to be non-zero, the inverse matrix A^{-1} can be analytically derived using the adjugate matrix. The general formula for the inversion of a square matrix is:

$$A^{-1} = \frac{1}{|A|} \cdot adj(A) \quad (11)$$

The term $adj(A)$ represents the adjugate matrix, which is the transpose of the cofactor matrix. To produce the cofactor matrix, each element a_{ij} of the original matrix is replaced by its "cofactor," which is the determinant of the sub-matrix formed by deleting the i -th row and j -th column, multiplied by $(-1)^{i+j}$. If we consider the matrix $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, the cofactors are derived by excluding the respective row and column for each position:

- For position (1,1): Excluding the first row and column leaves the element d . Thus, $C_{11} = (-1)^{1+1} \cdot d = d$.
- For position (1,2): Excluding the first row and second column leaves the element c . Thus, $C_{12} = (-1)^{1+2} \cdot c = -c$.
- For position (2,1): Excluding the second row and first column leaves the element b . Thus, $C_{21} = (-1)^{2+1} \cdot b = -b$.
- For position (2,2): Excluding the second row and column leaves the element a . Thus, $C_{22} = (-1)^{2+2} \cdot a = a$.

The resulting cofactor matrix is $C = \begin{bmatrix} d & -c \\ -b & a \end{bmatrix}$. The adjugate matrix is defined as the transpose of the cofactor matrix. By transposing the rows and columns of $C(C')$, the adjugate is obtained:

$$adj(A) = \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \quad (12)$$

Consequently, by substituting the adjugate and the determinant into the general inversion formula, the inverse of the 2 x 2 matrix is calculated as:

$$A^{-1} = \frac{1}{a \cdot d - b \cdot c} \cdot \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \quad (13)$$

As the dimensions of the data matrix increase, the computational complexity of this recursive algorithm grows significantly. Fortunately, this computation is programmed into most widely available statistical and

mathematical software libraries, allowing the system to handle these intensive linear algebra operations autonomously.

A final crucial property for matrices is the concept of a positive definite matrix. In linear algebra, a specific multiplication sequence of the form $x' \cdot A \cdot x$ (where x is a non-zero vector and A is a symmetric matrix) is referred to as a quadratic form. By mathematical definition, a symmetric matrix A is positive definite if its quadratic form results in a strictly positive scalar for any non-zero vector x , such that $x' \cdot A \cdot x > 0$.

2.2 DATA ANALYSIS

There are two primary types of data: continuous and discrete. A variable is considered continuous when it can take any real value within a specific range. For instance, a laser thermometer can record any temperature reading between -50.0°C and 500.0°C . Conversely, a discrete variable can only take specific, countable values. For example, the number of measurements a sensor records within a given time frame can only be a discrete integer.

Regardless of whether the data are continuous or discrete, their analysis and presentation can be conducted through graphical methods, numerical methods, or a combination of both. Graphical methods, such as histograms, provide a visual understanding of the data's distribution and help identify potential patterns or outliers. On the other hand, numerical methods involve the use of specific statistical measures to describe the data's characteristics more precisely.

In Multivariate Statistical Analysis, these numerical methods are fundamental. They allow us to calculate essential measures, such as means, variances, and the relationships between different variables. By combining both visual and numerical techniques, we can achieve a complete overview of the data, ensuring that the underlying structure of the complex phenomena is accurately captured and understood.

2.2.1 GRAPHICAL METHODS

Although a variety of complex visualizations exist for multivariate data, the primary objective of this thesis is to track structural changes in data distributions. Therefore, this section will exclusively focus on the fundamental graphical representations of frequency, as these provide the necessary foundation for the change-detection techniques analyzed later.

Before we analyze the Graphical Methods, it is essential to define the concept of frequency. For a discrete variable, as the frequency of a value, we define the number of the times that this value appeared. In many cases instead of frequency the normalized (or relative) frequency is utilized. Given N total measurements of a variable, where distinct values $x_1, x_2, \dots, x_v$ appear $n_1, n_2, \dots, n_v$ times, the normalized frequency of the value x_i is defined by the fraction:

$$f_i = \frac{n_i}{N} \quad (14)$$

Obviously, the normalized frequency ratio always lies between 0-1 and the sum of all normalized frequencies equals 1. Utilizing normalized frequency instead of frequency offers a lot of advantages. First of all, it facilitates the comparison between datasets with different number of observations and significantly enhances data visualization. Furthermore, by normalizing the frequency, the values are effectively converted into empirical probabilities, allowing for the application of probability theory and machine learning algorithms. Finally, it acts as a statistical homogenization mechanism, enabling effective data fusion of heterogeneous data by converting primary measurements of different scales and units into a common probabilistic reference basis.

For continuous types of data, we cannot utilize frequency and normalized frequency of a value, as these were described in the previous paragraphs because the probability of observing any single, exact continuous measurement is theoretically zero. Rather than, the continuous range of values is grouped into distinct bins so, the frequency of a bin is defined as the number of measurements that fall within its specific boundaries. If the entire range of values is divided into ν bins and there are $N = n_1 + n_2 + \dots + n_\nu$ total measurements, of which n_i belong to i -th bin then the normalized frequency of the i -th bin is calculated as:

$$f_i = \frac{n_i}{N} \quad (15)$$

As with discrete variables, this normalized frequency ratio remains strictly between 0 and 1, and the sum of all bin frequencies equals 1. To accurately represent the distribution of continuous data, especially when bin widths vary, the concept of frequency density is employed. Frequency density is defined as the frequency per unit of the measured quantity within a bin, and it is calculated using the relation:

$$d_i = \frac{f_i}{\Delta x_i} = \frac{n_i}{N \cdot \Delta x_i} \quad (16)$$

where Δx_i is the range of the i -th bin.

Another important statistical metric is the cumulative frequency. For a specific value or bin in an ordered dataset, as cumulative frequency we define the sum of its frequency and all the preceding frequencies. In the same way, the cumulative normalized frequency provides the running total of probabilities up to a certain point. This cumulative mathematical structure is very important, because it forms the basic foundation for change-detection mechanisms and sequential analysis techniques.

2.2.1.1 Bar chart

Bar charts are used to represent univariate discrete data and they are constructed on a plane of a rectangular coordinate system. On the x -axis the x_i values are represented and, on the y -axis the frequency or the normalized frequency. The next picture depicts a bar chart.

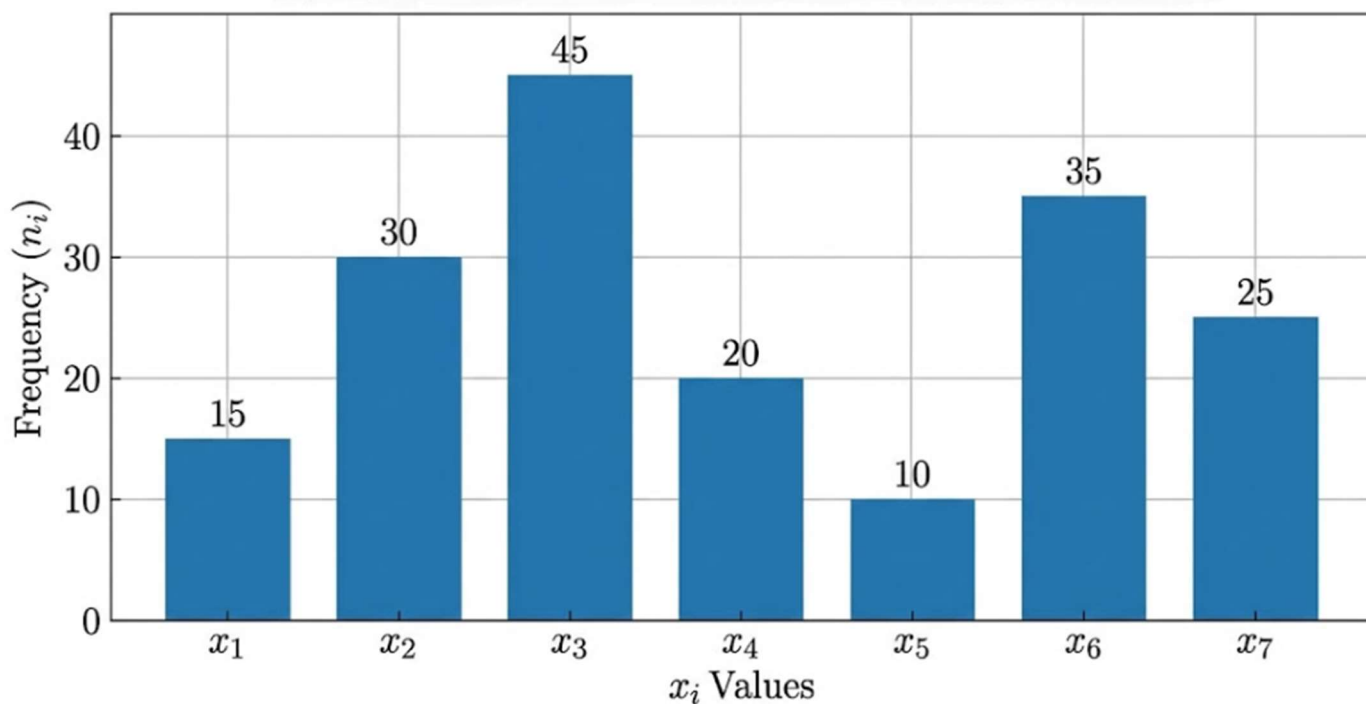


Figure 3 - Bar chart³

2.2.1.2 Histogram

Histograms are used to represent univariate continuous data and they are constructed on a plane of a rectangular coordinate system. On the x-axis the bins are represented and, on the y-axis the frequency, the normalized frequency, or the frequency density. Unlike bar charts, there are no gaps between the bars, reflecting the continuous nature of the data. The next picture depicts a histogram.

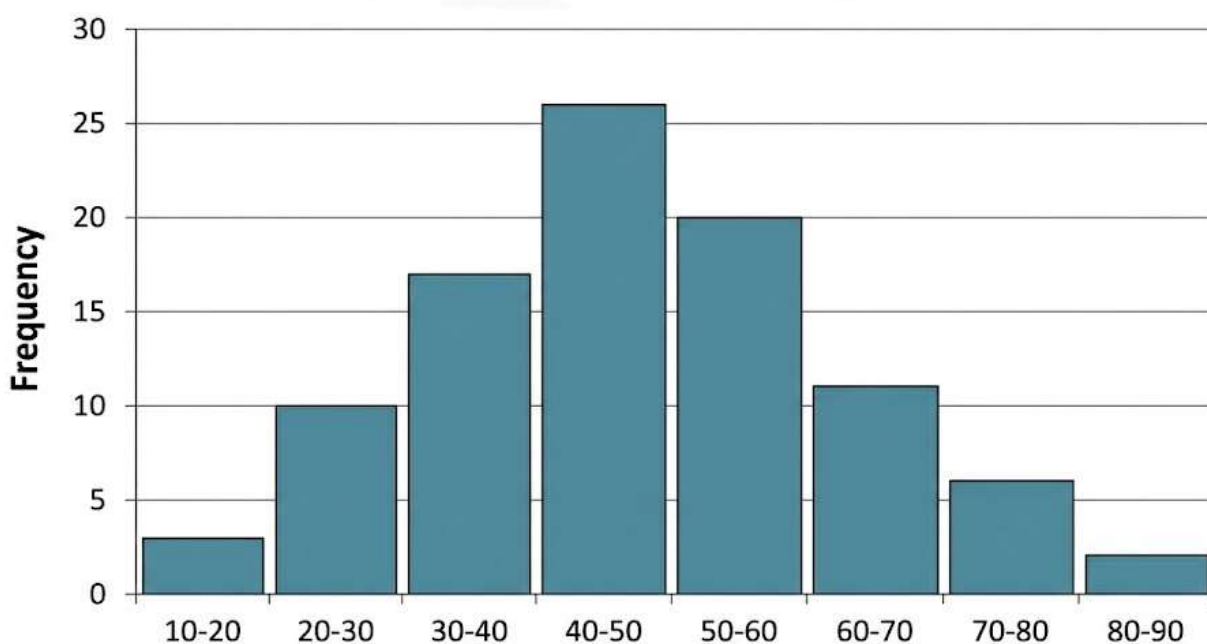


Figure 4 – Histogram⁴

³ This figure was produced with the assistance of the Gemini (Google) generative AI model.

⁴ This figure was produced with the assistance of the Gemini (Google) generative AI model.

Regarding to the number of bins that the values should be divided in two things should be taken into account:

- Changes in the frequency density will be difficult to be noticed if there is a small number of bins.
- Too many bins would lead to few measurements in every bin and consequently to an irregular shape of the histogram.

In order to choose the number of bins there are two main rules:

- The number of bins can be approximately calculated as the square root of the number of total measurements.
- If the number of measurements in a bin is less than five, this bin must be united with an adjacent bin.

Assuming a sufficiently large set of sample values, as the histogram's bin widths are continuously reduced, its discrete enveloping polygon asymptotically approaches a smooth curve. This is formally referred to as a probability or frequency curve, even though the values on the vertical axis represent frequency density.

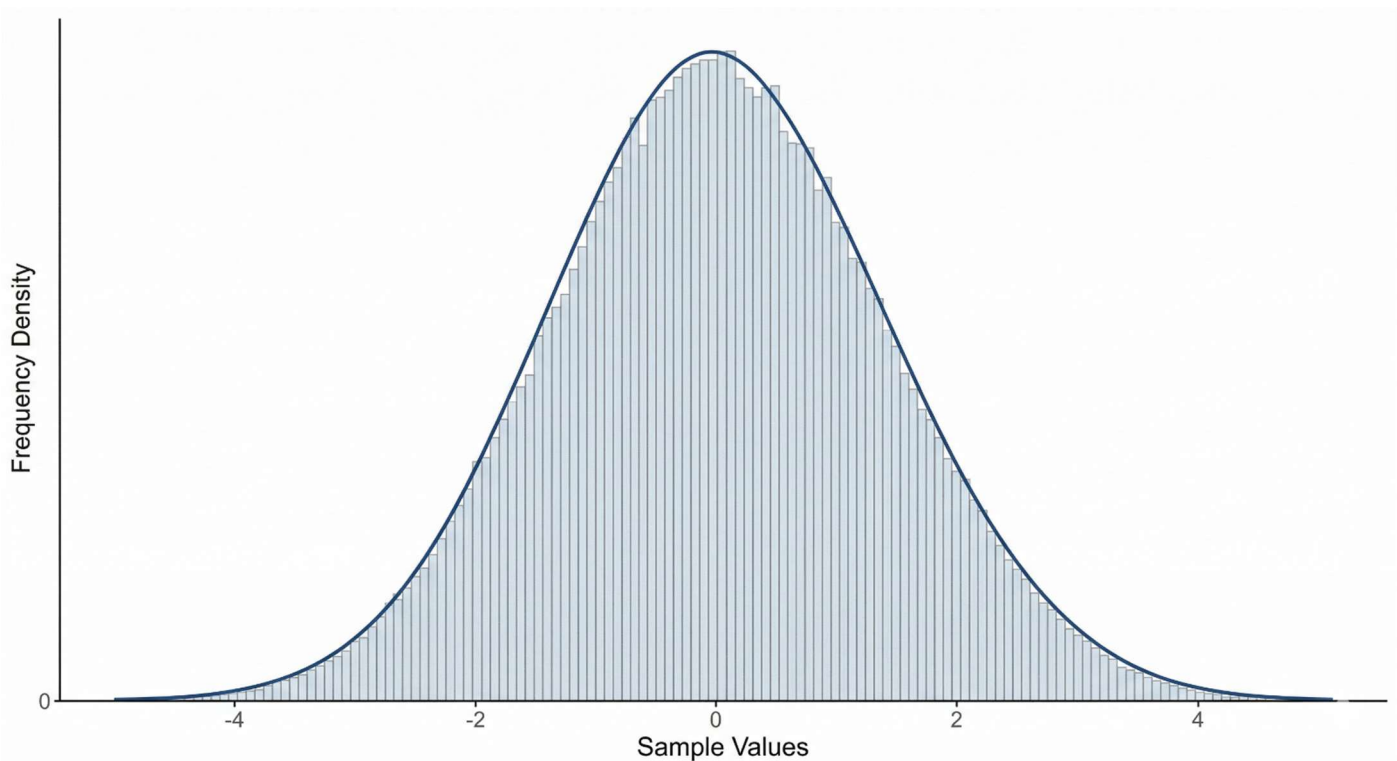


Figure 5 - Probability or frequency curve⁵

The continuous function representing this frequency curve at any given point x is denoted as $f(x)$ and is formally defined as the probability density function. By geometric definition, the total area enclosed between the probability density function curve and the horizontal axis evaluates to unity:

$$\int_{-\infty}^{\infty} f(x)dx = 1 \quad (17)$$

The probability of an exact measurement x occurring is strictly defined over an interval. For a narrow range $[x_0, x_0 + \Delta x]$, the likelihood of an observation is approximated by $f(x_0) \cdot \Delta x$. Consequently, the probability of a data point falling within a defined interval $[x_1, x_2]$ is calculated by integrating the function over that range:

⁵ This figure was produced with the assistance of the Gemini (Google) generative AI model.

$$P(x_1 \leq x \leq x_2) = \int_{x_1}^{x_2} f(x)dx \quad (18)$$

2.2.2 NUMERICAL METHODS

Numerical methods are used to provide a more accurate description of the data's underlying characteristics. Specifically, these methods utilize statistical parameters that quantify the dispersion and the distribution of a variable's values, and they can be applied to both continuous and discrete data types. These statistical metrics are generally categorized into two primary classifications: location parameters and measures of dispersion.

2.2.2.1 Univariate Data

In a univariate dataset the most common location parameter is the mean value. For a dataset consisting of N measurements, where $n_1, n_2, \dots, n_v$ represent the frequencies of occurrence for the distinct values $x_1, x_2, \dots, x_v$ respectively, the mean value can be estimated by this formula:

$$\bar{x} = \sum_{i=1}^v \frac{n_i}{N} \cdot x_i = f_i \cdot x_i \quad (19)$$

Alternatively, if the probability density function is known, the mean can be estimated through the relation:

$$\mu = \int_{-\infty}^{\infty} xf(x)dx \quad (20)$$

Beyond the arithmetic mean, other location parameters utilized to characterize the central tendency of a dataset include the mid-range and the mode. For a sequence of N observations, the mid-range is mathematically defined as the midpoint between the maximum and minimum values recorded within the given sample. Furthermore, the mode (or most probable value) designates the specific measurement with the highest likelihood of occurrence.

The dispersion parameters quantify the extent to which values are distributed around the mean value. The two fundamental parameters of dispersion are utilized in statistical analysis are the variance and the standard deviation. Variance for a dataset consisting of N measurements is defined as the average of the squared differences from the sample mean and is calculated using the following formula:

$$\sigma^2 = \frac{1}{N - 1} * \sum_{i=1}^v n_i \cdot (x_i - \bar{x})^2 \quad (21)$$

In case the probability density function is known it can be evaluated through the expected value of the squared deviation from the theoretical mean:

$$\sigma^2 = \int_{-\infty}^{\infty} (x - \mu)^2 \cdot f(x)dx \quad (22)$$

Consequently, the standard deviation, denoted as σ , serves as a measure of dispersion expressed in the same physical units as the original data. It is fundamentally defined as the principal square root of the variance. For a dataset of N measurements, it is computed via the relation:

$$\sigma = \sqrt{\sigma^2} = \sqrt{\frac{1}{N-1} \cdot \sum_1^v n_i \cdot (x_i - \tilde{x})^2} \quad (23)$$

Alternatively, if the probability density function is known, the standard deviation is estimated by applying the square root to the continuous variance integral:

$$\sigma = \sqrt{\sigma^2} = \sqrt{\int_{-\infty}^{\infty} (x - \mu)^2 \cdot f(x) dx} \quad (24)$$

2.2.2.2 Multivariate Data

In a multivariate dataset the most common location parameter is the mean vector. Specifically, for a dataset of N multidimensional measurements $x_1, x_2, \dots, x_N$, where each observation is a vector of p dimensions, the mean vector $\vec{x}$ is defined as the arithmetic mean of the observed vectors:

$$\vec{x} = \begin{bmatrix} \overline{x_1} \\ \vdots \\ \overline{x_p} \end{bmatrix} \quad (25)$$

where:

$$\overline{x_j} = \frac{1}{N} \cdot \sum_{i=1}^N x_{ij} \quad (26)$$

is the mean of j variable, $j = 1, 2, \dots, p$. Alternatively, if the multivariate probability density function $f(x_1, x_2, \dots, x_p)$ is known, the theoretical mean vector can be estimated through the relation:

$$\vec{\mu_j} = \int_{-\infty}^{\infty} \dots \int_{-\infty}^{\infty} x_j \cdot f(x_1, x_2, \dots, x_p) dx_1 dx_2 \dots dx_p \quad (27)$$

Beyond the mean vector, other location parameters that are very popular for the univariate case, such as the median or the trimmed mean, are seldom utilized in multivariate analysis. This is because these measures are mathematically complex to define in a multidimensional space and, consequently, they are not usually applied in the context of multivariate data. Therefore, the mean vector remains the primary location parameter of complex multidimensional phenomena.

While variance is the primary measure of variability for a single variable, in multivariate datasets, it is also necessary to examine how different variables change together. This relationship is expressed through covariance, which quantifies the joint variability between two variables x_k and x_j . Specifically, the sample covariance σ_{ij} is calculated using the following relation:

$$\sigma_{kj} = \frac{1}{N} \cdot \sum_{i=1}^N (x_{ij} - \overline{x_j}) \cdot (x_{ik} - \overline{x_k}) \quad (28)$$

It is important to note that when $j = k$, the covariance of a variable with itself is equal to its variance $\sigma_{jj} = \sigma_j^2$. In the multivariate case, these measures are structured into the covariance matrix, which is a symmetric

and positive semi-definite matrix. The diagonal of the matrix contains the individual variances, while the off-diagonal elements represent the covariances between the different variables:

$$\Sigma = \begin{bmatrix} \sigma_{11} & \dots & \sigma_{1p} \\ \vdots & \dots & \vdots \\ \sigma_{p1} & \dots & \sigma_{pp} \end{bmatrix} \quad (29)$$

The matrix Σ has specific properties:

- It is symmetric, meaning that $\sigma_{ij} = \sigma_{ji}$.
- The diagonal elements are always positive because they represent variances.
- The matrix is positive semi-definite.

A significant challenge in describing multivariate data is that the covariance matrix consists of $p \times p$ different values, making it difficult to quantify total variability with a single numerical metric. To address this, specific scalar functions of the matrix are commonly utilized to provide a consolidated measure of dispersion. One such measure is the total variation, which is calculated as the trace of the matrix Σ ($\text{tr}(\Sigma)$) and represents the sum of all individual variances. It is estimated by the following relation:

$$\text{tr}(\Sigma) = \sum_{j=1}^p \sigma_{jj} \quad (30)$$

The primary disadvantage of this metric is that it focuses exclusively on the diagonal elements and does not account for the covariances, thereby losing information regarding the internal relationships between variables. Alternatively, the generalized variance is defined as the determinant of the covariance matrix ($|\Sigma|$).

Geometrically, the generalized variance represents the "volume" occupied by the data distribution within the p -dimensional space. Both of these scalar measures are indicative of the total dispersion within the dataset, where higher values suggest that observations are more spread out from the mean vector. Furthermore, in the case of generalized variance, a zero value for the determinant signifies that some variables are linearly dependent. These theoretical metrics provide the necessary foundation for understanding the complex structure of multivariate variability and how it can be summarized into a single reference basis.

2.3 PROBABILITY DISTRIBUTION

In the univariate case, variables describing a physical phenomenon follow a probability distribution, which defines the probability of occurrence for possible events. To identify the underlying distribution of a single variable, empirical tools such as histograms or bar charts are typically employed for initial data visualization and assessment. The distribution function can be either discrete or continuous, depending on the nature of the variable.

However, in multivariate statistical analysis, our objective is not to examine the distribution of each variable in isolation, but rather to treat them as an interconnected mathematical vector. Similarly to the univariate case, for continuous variables the joint probability density function is utilized, whereas for discrete ones, the joint probability mass function is applied. Since this work focuses on continuous sensor measurements from Edge deployments, the joint probability density function is the natural starting point for the analysis that follows.

While a variety of probability distributions can be used to describe stochastic phenomena in both univariate and multivariate statistics, the Normal Distribution, also known as the Gaussian Distribution, is of primary importance for this analysis. This distribution provides the mathematical foundation for most Statistical

Process Control and change-detection mechanisms. Initially, we will examine the fundamental properties of the univariate Normal Distribution. Subsequently, we will extend these concepts to the Multivariate Normal Distribution, which allows us to treat multiple interdependent variables as a single vector.

2.3.1 UNIVARIATE NORMAL DISTRIBUTION

The Normal or Gaussian Distribution is a type of continuous probability distribution for a real-valued random variable and it is the most common distribution with applications in a vast variety of phenomenon. Its shape is bell-shaped, symmetric about the mean, with central tendency, which means that the mean, median, and mode are all equal and located at the center and asymptotic to the x-axis as it moves away from the mean.

The function $f(x)$ of the normal distribution uses the parameter μ which is the mean and the parameter σ^2 which is the variance and it denotes as:

$$f(x) = \frac{1}{\sqrt{2 \cdot \pi \cdot \sigma^2}} \cdot e^{-\frac{(x-\mu)^2}{2 \cdot \sigma^2}} \quad (31)$$

Where $-\infty < x < \infty$.

A fundamental property of the Normal Distribution, is the Empirical Rule, often referred to as the 68-95-99.7 rule. According to this rule, the dispersion of data around the mean value follows a predictable ratio. Specifically, approximately 68.2% of the observations of a normally distributed variable are concentrated within a distance of one standard deviation ($\pm 1\sigma$) from the center. By expanding this interval to two standard deviations ($\pm 2\sigma$), 95.4% of the data is enclosed. Finally, 99.7% of the recorded measurements are located within three standard deviations ($\pm 3\sigma$).

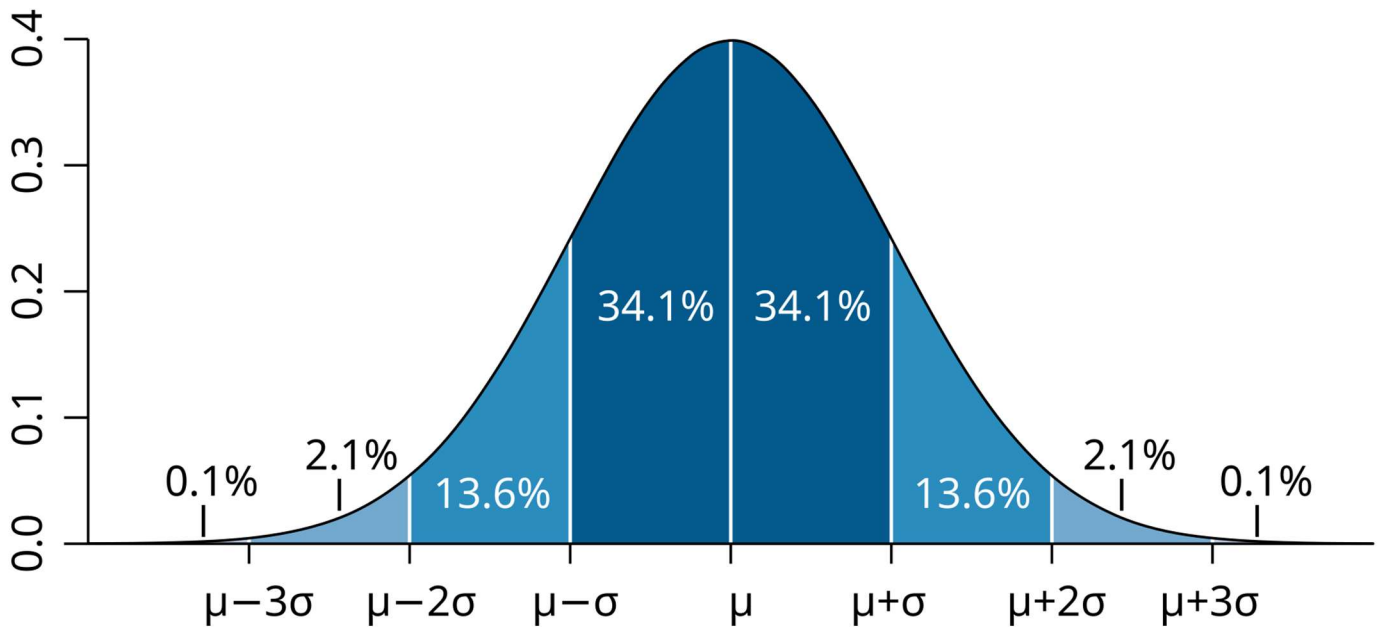


Figure 6 - Normal distribution, standard deviation and coverage in statistics⁶

⁶ Source: Wikipedia, " Normal distribution". Available at: https://en.wikipedia.org/wiki/Normal_distribution (Accessed: [4/2026])

This practically signifies that the probability of a normal measurement occurring in the extreme tails of the distribution (beyond $\pm 3\sigma$) is only 0.3%. This mathematical characteristic provides the basis for defining control limits in anomaly detection mechanisms and change-detection algorithms.

The probability of a continuous measurement falling within a specific interval between the values x_1 and x_2 is mathematically calculated by integrating the probability density function over that range:

$$p(x_1 < x < x_2) = \int_{x_1}^{x_2} f(x)dx = \int_{x_1}^{x_2} \frac{1}{\sqrt{2 \cdot \pi \cdot \sigma^2}} \cdot e^{-\frac{(x-\mu)^2}{2 \cdot \sigma^2}} dx \quad (32)$$

However, due to the computational difficulty of directly calculating this integral for the normal distribution, the dimensionless standardized variable z is commonly utilized. This transformation converts the original measurement into a standard scale and is calculated as:

$$z = \frac{x - \mu}{\sigma} \quad (33)$$

It is proven that the distribution of this dimensionless standardized variable is also a normal distribution, specifically known as the Standard Normal Distribution. This specific distribution always has a mean equal to zero and a standard deviation equal to one. Furthermore, the probabilities of the original variable x and the standardized variable z remain exactly the same over corresponding intervals. In other words, if the boundaries z_1 and z_2 are calculated as:

$$z_1 = \frac{x_1 - \mu}{\sigma}, z_2 = \frac{x_2 - \mu}{\sigma} \quad (34)$$

then the probability equivalence $P(x_1 < x < x_2) = P(z_1 < z < z_2)$ is strictly maintained.

The extensive application of the Normal Distribution stems not only from the fact that many physical measurements inherently follow it, but also from several fundamental statistical properties. Specifically, variables that do not natively follow a Normal Distribution can often be mathematically transformed into one. Additionally, the mean values of samples drawn from a non-normal population tend to approximate a normal distribution. Furthermore, any linear combination of independent normally distributed variables inherently results in a new normal distribution. Finally, under conditions of large sample sizes or high expected values, discrete distributions, such as the Binomial and Poisson distributions, can be adequately approximated by the continuous normal distribution.

2.3.2 MULTIVARIATE NORMAL DISTRIBUTION

The multivariate normal distribution is a generalization of univariate normal distribution for more than one dimensions. In the univariate normal density function with μ mean and σ^2 variance the exponent $\frac{(x-\mu)^2}{\sigma^2}$ calculates, in standard deviation units, the square of difference between the x measurement and the mean μ . The exponent can also be written as:

$$\frac{(x - \mu)^2}{\sigma^2} = (x - \mu) \cdot (\sigma^2)^{-1} \cdot (x - \mu) \quad (35)$$

Fundamentally, this exponent measures a distance. In a p -dimensional space, the square Euclidean distance between a measurement vector $\vec{x}$ and a mean vector $\vec{\mu}$ is calculated as:

$$D^2 = (\vec{x} - \vec{\mu})' \cdot I_p \cdot (\vec{x} - \vec{\mu}) \quad (36)$$

where I_p is an identity matrix with $p \times p$ dimensions. It is obvious that each parameter p_i contributes with the same weight that equals 1. But in the multivariate case, it is typical that observed variables exhibit different variances and are interconnected with one another. The Euclidean distance fails to account for these characteristics, as it is strictly dependent on the measurement units. Consequently, variables with numerically larger values can disproportionately influence the final calculation.

To resolve this issue, the Euclidean distance transforms into the Mahalanobis distance. This multivariate statistical distance counteracts the effect of different scales by replacing the identity matrix I_p with a pre-specified weight matrix. Specifically, it utilizes the inverse covariance matrix Σ^{-1} , which was defined in Sector 2.1.2.1. This matrix applies the appropriate weighting to each individual variable and accounts for their correlations, ensuring that the underlying structure of the complex phenomena is accurately captured.

Therefore, for more than one parameter ($p > 1$), the quadratic form within the exponent of the normal distribution is generalized using the squared Mahalanobis distance as follows:

$$(\vec{x} - \vec{\mu})' \cdot \Sigma^{-1} \cdot (\vec{x} - \vec{\mu}) \quad (37)$$

where the $\vec{x}$ is the vector of each measurement and $\vec{\mu}$ is the mean vector. Both $\vec{\mu}$ and $\vec{x}$ share the same dimensions $p \times 1$ where p is the number of the parameters, and the covariance matrix Σ^{-1} has dimensions $p \times p$.

Similarly to the exponent, the factor $\frac{1}{\sqrt{2 \cdot \pi \cdot \sigma^2}}$ in the univariate normal density function serves as the normalization constant. Its theoretical purpose is to ensure that the total area under the probability density curve equals 1, regardless of the magnitude of the variance σ^2 . For more than one dimension ($p > 1$), this constant is generalized in the multivariate form as:

$$\frac{1}{\sqrt{(2 \cdot \pi)^p \cdot |\Sigma|}} \quad (38)$$

Where $|\Sigma|$ denotes the determinant of the covariance matrix Σ . This factor is essential for normalizing the volume under the multivariate probability density surface—the multidimensional equivalent of the area under a curve—to exactly one, thereby satisfying the fundamental requirement for a valid probability distribution. Finally, the multivariate normal probability density function for a p -dimensional random vector $\vec{x}$ is defined by the following equation:

$$f(\vec{x}) = \frac{1}{\sqrt{(2 \cdot \pi)^p \cdot |\Sigma|}} \cdot e^{-\frac{(\vec{x} - \vec{\mu})' \cdot \Sigma^{-1} \cdot (\vec{x} - \vec{\mu})}{2}} \quad (39)$$

The graphical representation of the multivariate normal distribution is illustrated in the following figure.

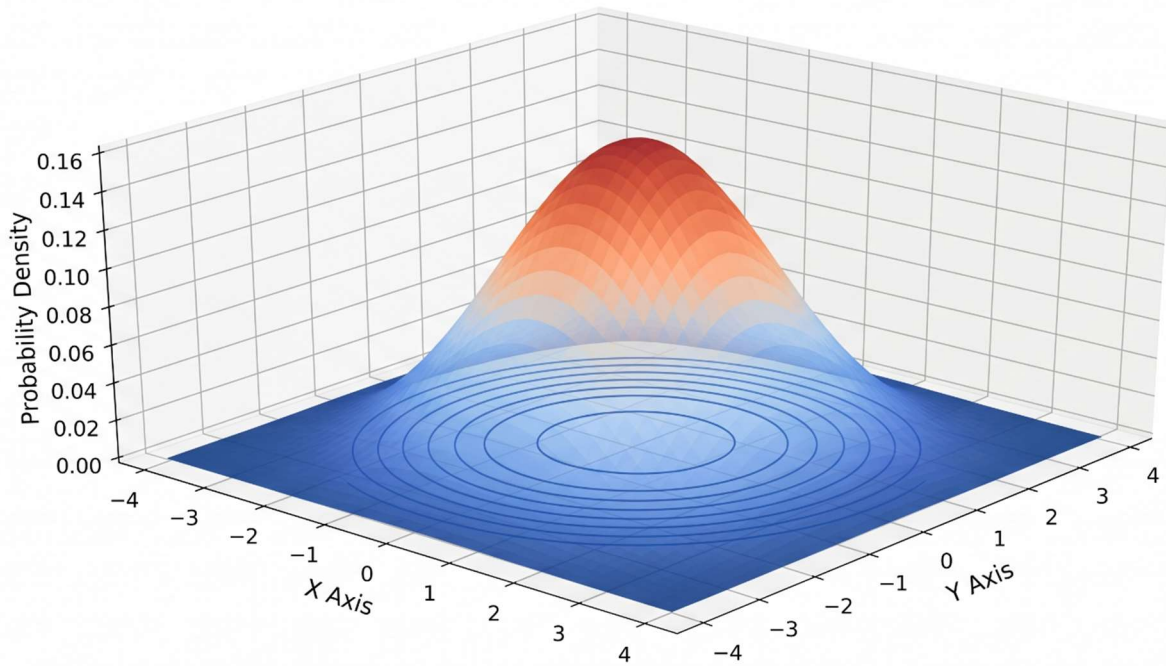


Figure 7 - Bivariate normal distribution⁷

Some very useful properties of the multivariate normal distribution are the following (Johnson & Wichern, 2007):

- If $\vec{x}$ is a random vector having a multivariate normal distribution, all its components' linear combinations are normally distributed.
- If $\vec{x}$ is a random vector having a multivariate normal distribution, all its components' subsets have a normal distribution.
- In a multivariate normal distribution, if the covariance between any corresponding components is zero, it implies that these variables are statistically independent from each other.
- The conditional distribution of any set of components, given the known values of another set of components within the same vector, also maintains a multivariate Normal Distribution.

2.3.3 CENTRAL LIMIT THEOREM

The Central Limit Theorem is a fundamental statistical principle with numerous practical applications. For each dataset of n samples where each of them is consisting of N measurements, the mean value $\bar{x}$ is calculated. If the primary measurements inherently follow a normal distribution with a mean μ and a standard deviation σ , it is proven that the distribution of these sample means is also strictly normal. Its standard deviation, is calculated by the relation:

$$\sigma_{\bar{x}} = \frac{\sigma}{\sqrt{N}} \quad (40)$$

In addition, the dimensionless standardized variable of the sample mean:

$$z = \frac{\bar{x} - \mu}{\sigma_{\bar{x}}} = \frac{\bar{x} - \mu}{\sigma/\sqrt{N}} \quad (41)$$

⁷ This figure was produced with the assistance of the Gemini (Google) generative AI model.

also follows a normal distribution.

Building upon this exact foundation, the Central Limit Theorem extends this concept to non-normal data. The theorem dictates that even if the original measurements are drawn from a population with an unknown or non-normal distribution, the distribution of their sample means will asymptotically approach a normal distribution with standard deviation $\sigma_{\bar{x}} = \frac{\sigma}{\sqrt{N}}$, as the sample size N approaches infinity. The theorem also applies on the dimensionless standardized variable of the sample mean.

In practical statistical applications, this approximation is considered highly satisfactory when the sample size is greater than 30 ($N > 30$), regardless of the base variable's original distribution. Conversely, for smaller samples ($N < 30$), the accuracy of the approximation heavily depends on how closely the primary data initially resembles a normal distribution.

Similarly to the univariate the Central Limit Theorem is applicable in the multivariate case, consequently the sample mean $\vec{M}$ and the sample covariance matrix S exhibit large-sample properties. More specifically, for a set of measurements with a theoretical mean vector $\vec{\mu}$ and a covariance matrix Σ , even when the initial multivariate measurements follow a non-normal distribution, the sample means $\vec{M}$ will, as the samples size reaches infinity, asymptotically approximate a normal distribution with mean equal to $\vec{\mu}$ and covariance matrix equal to Σ/N .

The theorem can also be expressed in terms of a centered and scaled variable. Specifically, if we define:

$$\vec{z} = \sqrt{N} \cdot (\vec{M} - \vec{\mu}) \quad (42)$$

then, for large sample sizes, $\vec{z}$ asymptotically follows a multivariate normal distribution with a zero mean vector and a covariance matrix equal to Σ .

A practical rule for the sample size in the multivariate case is that its size should be large relative to the number of variables p .

2.3.4 CHI-SQUARE DISTRIBUTION

The reason for introducing the Chi-Square distribution is its direct connection to the squared Mahalanobis distance, which was defined in the 2.3.2. Crosier (1988) utilizes this statistical relationship to model the algorithm's theoretical baseline noise and evaluate its Average Run Length (ARL) performance. The property of interest states that if a random vector $\vec{x}$ consisting of p variables follows the multivariate normal distribution $N_p(\vec{\mu}, \Sigma)$, then the quantity,

$$d^2 = (\vec{x} - \vec{\mu})' \cdot \Sigma^{-1} \cdot (\vec{x} - \vec{\mu}) \quad (43)$$

defined as the squared Mahalanobis distance, follows a Chi-Square distribution with p degrees of freedom, denoted as $\chi^2(p)$. This property allows the definition of statistically valid control limits, since the distribution of d^2 is fully characterized by the number of variables p .

The general formula of the Chi-Square probability density function is:

$$f(x^2) \equiv f(x) = \frac{x^{(k/2)-1} \cdot e^{-x/2}}{2^{k/2} \cdot \Gamma(k/2)}, \quad x > 0 \quad (44)$$

where n is the degrees of freedom and $\Gamma(k/2)$ denotes the gamma function, which has closed-form values for integer k . The graphical representation of the Chi-Square distribution for different degrees of freedom is illustrated in the following figure.

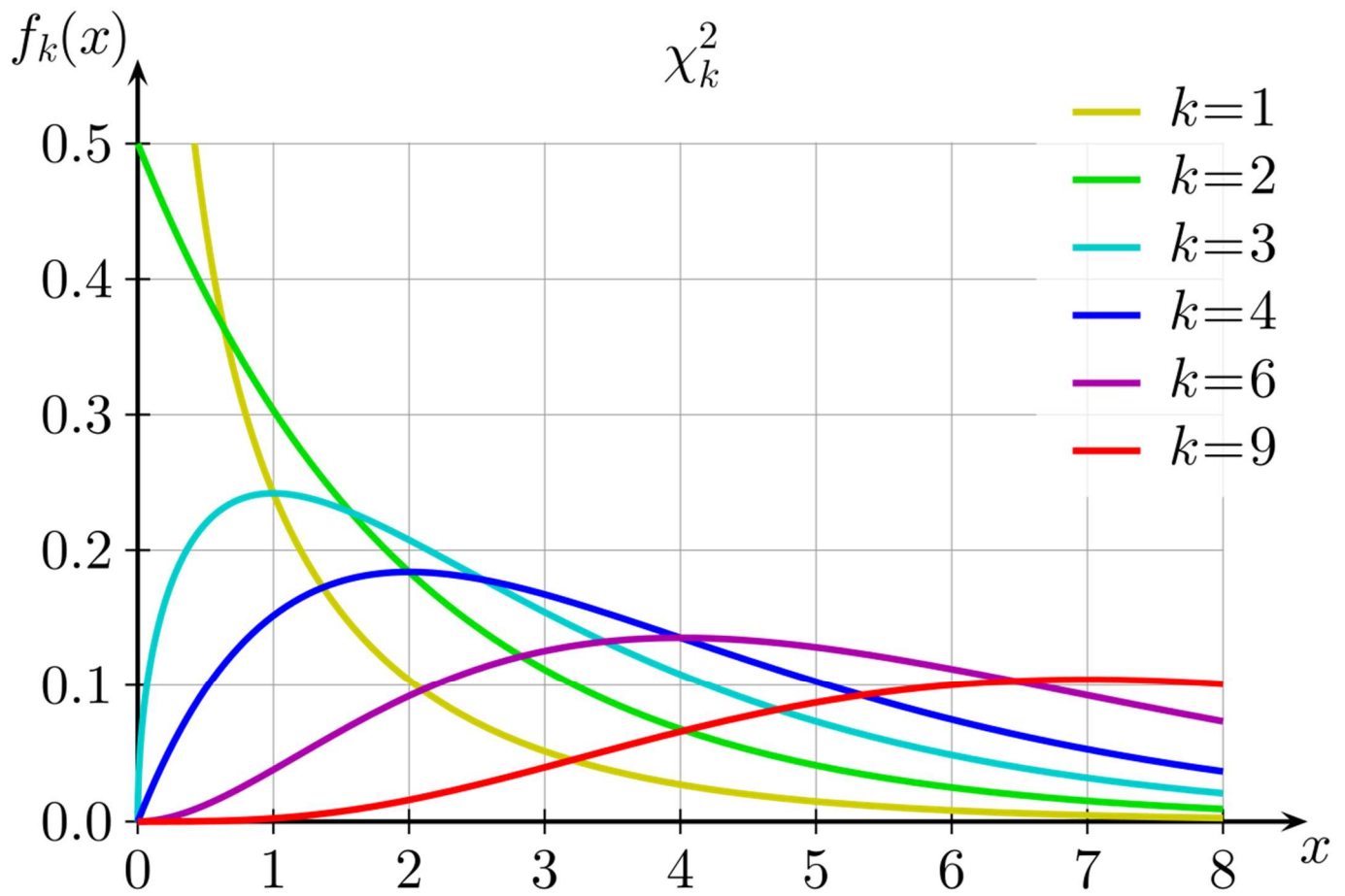


Figure 8 - Chi-Square distribution⁸

If $z_1, z_2, \dots, z_k$ are independent random variables that follow standard normal distribution then their sum of squares:

$$x^2 = z_1^2 + z_2^2 \dots + z_k^2 \quad (45)$$

follows the Chi-Square distribution with k degrees of freedom. This distribution has a theoretical mean μ equal to k and variance σ^2 equal to $2 \cdot k$. As the number of degrees of freedom k approaches infinite, the distribution asymptotically approaches normal distribution. In practice, this approximation is considered highly satisfactory when $k > 30$.

⁸ Source: Wikipedia, " Chi-squared distribution". Available at: https://en.wikipedia.org/wiki/Chi-squared_distribution (Accessed: [4/2026])

3. STATISTICAL PROCESS CONTROL

Quality is a multidimensional and dynamic concept that continuously evolves over time. Regarding a product or a service, a general and objective criterion for evaluating its quality is its fitness for intended use. A more modern approach is based on variability. According to this perspective, quality is inversely proportional to variability (Montgomery, 2013), meaning that the less the characteristics or processes deviate from their expected targets, the higher the quality achieved.

The causes of variability are divided into two main categories. The first comprises common causes, which represent the inherent, random variability that is part of the normal operation of any stable process. For example, in a sugar packaging line, minor variations in the weight of each package may be due to the natural variation in equipment pressure or to minimal sensor tolerances. This random dispersion is continuously present, regardless of how well the production line is designed or how carefully the respective task is executed. Reducing it requires systematic changes to the design of the process itself.

The second category comprises special or assignable causes. These are unusual factors that introduce an unexpected and often large deviation. Continuing the previous example, the appearance of consecutive sugar packages with significantly lower weight could be due, for instance, to a malfunction in the weighing mechanism. In contrast to common causes, this type of cause is not random rather, it constitutes a specific event that can and must be identified, diagnosed, and fixed.

Statistical Process Control (SPC) focuses on controlling and reducing this variability stemming from special or assignable causes. SPC comprises a set of methods that employ statistical techniques to monitor and control a process, ensuring that it operates stably and predictably, thereby producing products that conform to specifications.

3.1 SHEWHART CONTROL CHARTS

Shewhart control charts, invented by Walter Shewhart, are the most widely used type of control charts. They monitor specific parameters to detect anomalies, thereby identifying whether a process is out of control due to a specific cause. An advantage of these charts is that they prevent overreaction to the common cause variation of the process. Additionally, the evaluation of each observation is memoryless, as it relies only on current values without considering previous ones.

The most common Shewhart control charts are the mean chart ($\bar{x}$ control chart) and the range chart (R control chart). A typical chart consists of three horizontal lines. The center line denotes the normal operating state of the process, for instance, the target mean of the measured characteristic. The other two horizontal lines represent the upper control limit and the lower control limit. If the measured values fall outside these control limits, it indicates that the process is out of control and the causes of this anomaly must be investigated.

A typical Shewhart control chart is shown in Figure 9.

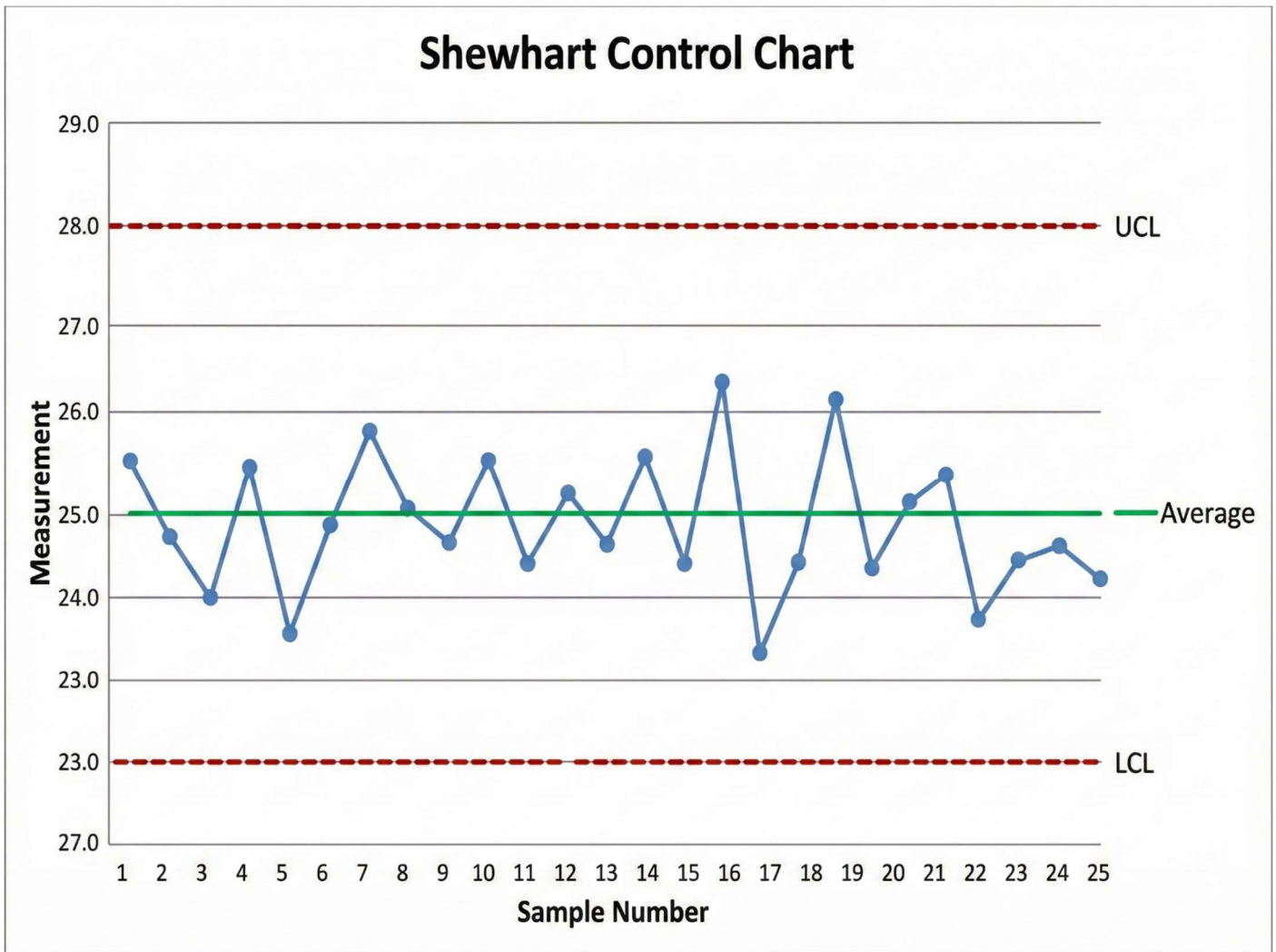


Figure 9 - Typical Shewhart control chart⁹

3.2 CUMULATIVE SUM CONTROL CHARTS

As already mentioned in Section 3.1, control charts are based only on the current value of the sample and do not utilize the information that can be extracted from the previous measurements. As a result, these charts are not sensitive to small changes in the process. In order to deal with this disadvantage, alternative change detection methods have been deployed, with a prominent example being Cumulative Sum control charts (CUSUM).

The main idea behind CUSUM charts is that, instead of evaluating each observation individually, the cumulative sum of the deviations between the measurements and the target value of the process is calculated and monitored. In this way, even small but systematic shifts in the mean accumulate in the chart, making their detection much faster and easier compared to Shewhart control charts.

3.2.1 UNIVARIATE CUSUM

Assuming n samples of measurements for a variable representing a process characteristic, where the variable has a target mean equal to μ and each sample mean is denoted by $\bar{x}_n$ the Cumulative Sum control chart is given by the following equation:

⁹ This figure was produced with the assistance of the Gemini (Google) generative AI model.

$$S = \sum_{n=1}^n (\bar{x}_n - a) \quad (46)$$

The CUSUM chart depicts this cumulative sum as a function of the sample number n . As shown in the preceding equation, if the mean $\bar{x}_n$ of each sample is equivalent with the target mean a then the cumulative sum is zero. In contrast, when the sample means deviate upward (i.e. $\bar{x}_n > a$) the cumulative sum becomes positive and increases progressively. When the sample means deviate downward (i.e. $\bar{x}_n < a$) the sum becomes negative, indicating that the process is systematically operating below the target mean.

Another way to express the CUSUM relationship is by applying standardization, such that the measurements have a zero mean and a unit standard deviation. Assuming that the standard deviation of the samples denotes by σ the standardization CUSUM can be calculated as follows:

$$S'_n = \frac{1}{\sigma} \sum_{n=1}^n (\bar{x}_n - a) \quad (47)$$

These two calculation methods are equivalent, with their main difference lying in the measurement units on the vertical axis. In the first case, the measurement unit is identical to that of the original variable, whereas in the second case, the vertical axis is expressed in dimensionless units, independent of the observed variable's measurement scale. The advantage of standardization is that it allows for the comparison of CUSUM charts representing different quantities.

There are two ways to represent a CUSUM: the tabular (or algorithmic) form and the V-mask. -mask procedure is applied to the CUSUM chart by putting a V-shape mask, with its peak (point O) placed on the most recent CUSUM point and its axis aligned parallel to the horizontal axis. As long as the S_n sum lies between the two lines forming the mask, the process is considered to be under control. When the sum exceeds the boundaries an alarm signal is generated.

The shape of the mask is determined by the lead distance d_{mask} and the angle θ_{mask} , which are equivalent to the k and h parameters of tabular CUSUM discussed below. This technique is not frequently used because it is considered more difficult to implement and its performance characteristics are vaguer compared to the simplest tabular CUSUM. Figure 10 depicts the V-mask application to a CUSUM chart (Montgomery, 2013).

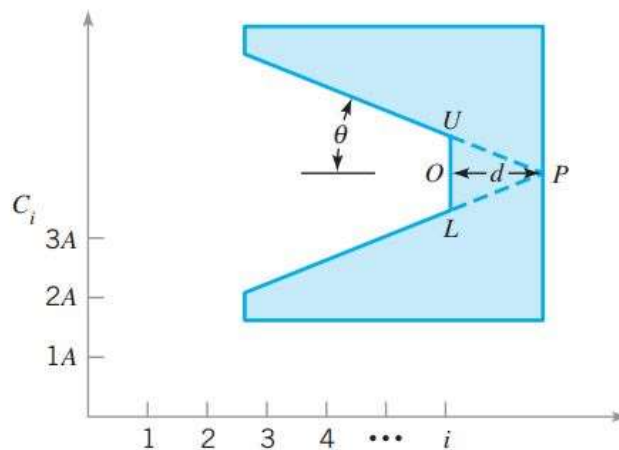


Figure 10 - V-mask application to a CUSUM chart

3.2.1.1 The tabular CUSUM Method

The tabular CUSUM method for monitoring the mean of a process variable can be applied to both individual observations and subgroup mean.

Let x_n be the n -th measurement of a variable in a specific process which under normal conditions has a standard deviation of σ and a mean of a (in other applications we can use the target mean value). The method operates cumulative using the S^+ sum when $x_n > a$ and the sum S^- when $x_n < a$. To sum up, the tabular CUSUM method functions as follows:

$$\begin{aligned} S_n^+ &= \max(0, x_n - (a + K) + S_{n-1}^+) \\ S_n^- &= \max(0, (a - K) - x_n + S_{n-1}^-) \\ S_0^+ &= 0 \\ S_0^- &= 0 \end{aligned} \quad (48)$$

The parameter K is referred to as the reference value or allowance, and it is typically set to half the difference (d) between the true mean (μ) and the target mean (a), namely:

$$K = \frac{\mu - a}{2} = \frac{d}{2} \quad (49)$$

The above equations reveal that when the absolute difference between the measurement x_n and the target mean a is less than the allowance K then the sums S^+ and S^- are reset to zero.

Additionally, a decision interval H is defined. If the sum S^+ exceeds this decision interval or the sum S^- falls below $-H$, the process is considered out-of-control, signaling an epoch change. According to Montgomery (2013), a reasonable value for the decision interval H is five times the standard deviation.

If normalization is applied based on the standard deviation σ , the following is obtained:

$$y_i = \frac{x_i - a}{\sigma} \quad (50)$$

So, the equations of the sums S^+ and S^- are transformed to:

$$\begin{aligned} S_i^+ &= \max(0, y_i - k + S_{i-1}^+) \\ S_i^- &= \max(0, -k - y_i + S_{i-1}^-) \end{aligned} \quad (51)$$

3.2.1.2 Average Run Length

As already mentioned, the sums S^+ and S^- start from an initial state of zero and for every with each newly calculated value, they can become positive, negative, or remain zero. Each such period concludes either with the sums returning to zero or by exceeding the predefined decision decision interval. The sequence from zero until the decision interval is exceeded constitutes a run, while the number of observations elapsed until a signal is generated is referred to as the run length.

In some cases, the algorithm produces signal without an actual change having occurred in the process, resulting in a fake alarm. Such situations are undesirable as they lead to the unnecessary consumption of time and resources, and cause disruptions to the smooth operation of the process.

In cases where change in the process occurs, it is desirable for it to be detected by the algorithm as fast as possible. In contrast, the intervals between the false signals should be as long as possible. However, the objectives of minimizing false alarms and rapidly detecting actual shifts are contradictory, necessitating a trade-off between them.

The run length is a random statistical variable characterized by mean, variance and distribution. The mean of the run length is called Average Run Length (ARL) and is considered as one of the primary criteria for the evaluation of the performance of CUSUM algorithm and a tool for the design and the calibration of its parameters.

The definition of the ARL depends on the starting point of the CUSUM chart. Usually, the CUSUM algorithm starts from the zero value, and the corresponding ARL is defined as the average number of observations between the starting point and the occurrence of the first signal. This choice is justified by the fact that every time CUSUM returns to zero, it loses the memory of its previous path, making the process equivalent to a new start.

When a CUSUM chart generates a signal without an actual shift having occurred in the process, the relevant metric is the in-control Average Run Length (in-control ARL or ARL_0), which represents the average number of observations until a false alarm occurs. Conversely, when the CUSUM correctly detects an actual shift in the process, the metric used is the out-of-control Average Run Length (out-of-control ARL or ARL_1), which corresponds to the average number of observations required to detect the shift after its occurrence. Consequently, when designing a CUSUM chart, the objective is to maximize ARL_0 to limit the frequency of false alarms, while minimizing ARL_1 to ensure the rapid detection of actual shifts.

Choosing CUSUM parameters relies precisely on achieving an acceptable balance between ARL_0 and ARL_1 , depending on the requirements of the process.

3.2.1.3 Design of the CUSUM procedure

The design procedure of a CUSUM chart is separated in to two stages. Initially, choosing the appropriate values for H and K parameters is required, so that the chart exhibits satisfactory performance when the process is in-control, namely achieving an acceptable in-control Average Run Length (ARL_0). Equally important is evaluating the response of the CUSUM chart in out-of-control situations, and specifically the rate at which the chart detects a process shift once it has occurred.

So, the fundamental issue in CUSUM algorithm design is the choice of the K and H parameters, according to the desired value of the Average Run Length (ARL). More specifically, the allowance parameter (K) and the decision interval (H) are directly related to ARL , affecting the frequency of generated signals. Larger K and H values lead to an increased ARL , which implies a lower probability of false alarms, but also a slower detection of actual shifts. For a given distribution of the process measurement variable, the choice of K and H uniquely determines the ARL value. Conversely, for a given K and a predefined ARL greater than one, the corresponding H value can be determined, although the exact relationship between these parameters depends on the underlying process distribution.

In this context, Qiu(2014) proposes the use of Monte Carlo simulations to calculate the parameters of the CUSUM chart. In this method for different values of H and K parameters the in control ARL_0 is calculated. Then, the desired ARL_0 and the allowance K are selected and the corresponding decision interval value H is derived through simulation. For the selection of the allowance parameter K , the use of the previously mentioned formula (Equation 49) is proposed.

Hawkins and Olwell (1998) similarly calculate the decision interval after the values of allowance K and the in-control Average Run Length (ARL_0) are chosen. Parameter K , is calculated to maximize algorithm's sensitivity to a specific mean shift of magnitude d . Regarding the choice of the shift d , in which CUSUM algorithm is tuned, two main approaches can be followed. In the first approach, when the process is out of control, it is likely to shift to a specific and predictable level which can be estimated based on theoretical calculations or prior experimental data. In those cases, the CUSUM chart can be set to detect this specific shift. However, in most applications there is no single expected shift, but rather a range of possible shifts. In this scenario, the selection of d is based on the shift that we desire to detect most rapidly; this shift must be small enough so as not to be immediately noticeable without statistical control, yet large enough to have a substantial impact on the process operation.

The selection of the Average Run Length is not an arbitrary decision but, it is directly connected to practical operational criteria based of the process. Although ARL is defined as the number of observations until a signal is generated the in-control $ARL(ARL_0)$ is practically interpreted as the time interval between consecutive false alarms. For this reason, the ARL_0 is chosen based on the sampling frequency and the acceptable number of false signal for specific application, while taking into consideration and the unavoidable trade-off between minimizing false alarms and rapidly detecting actual shifts.

After selecting the values of the parameters K and ARL_0 , three methods are suggested for calculation of the decision interval H : using tables, charts or software. Regarding the use of tables, there are tables that present either the ARL_0 values for selected combinations of the parameters H and K , or conversely, the required values of the decision interval H for given values of K and ARL_0 . The second category of tables is particularly useful for design purposes, as it allows the direct determination of the H value that corresponds to a predefined value of the ARL_0 parameter. It should be noted that the parameter values are normalized so that they can be applied to different processes. The use of charts is similar to the use of tables. Finally, the authors suggest the use of specialized software, which provides greater flexibility and precision in calculating the parameters of the CUSUM chart.

Montgomery (2013) states that the values for parameters H and K must be selected to yield an acceptable Average Run Length (ARL). Furthermore, he notes that the combination of $K = 0.5 \cdot \sigma$ and $H = 4.0 \cdot \sigma$ or $H = 5.0 \cdot \sigma$, where σ is the standard deviation, typically results in a CUSUM algorithm with good overall ARL performance. This configuration demonstrates satisfactory characteristics in detecting shifts in the process mean of approximately one standard deviation. He also presents an approximation method by Siegmund (1985) for estimating the Average Run Length, according to which, if:

$$\begin{aligned} \Delta &= y_i - K, \text{ for the sum } S^+ \\ \Delta &= -y_i - K, \text{ for the sum } S^- \\ b &= H + 1.166 \end{aligned} \quad (52)$$

Then:

$$ARL^{+/-} = \frac{e^{-2 \cdot \Delta \cdot b} + 2 \cdot b \cdot \Delta - 1}{2 \cdot \Delta^2} \quad (53)$$

$$\frac{1}{ARL} = \frac{1}{ARL^-} + \frac{1}{ARL^+} \quad (54)$$

If $\Delta = 0$ then the formula that calculates the $ARL^{+/-}$ is:

$$ARL^{+/-} = b^2 \quad (55)$$

To determine the CUSUM chart parameters, Devore (2012) uses the Kenneth Kemp nomogram. As shown in Figure 11, this nomogram consists by two vertical scales corresponding to the in-control Average Run Length and the parameter K and also, scales between the two vertical scales that represent the parameter H and the out-of-control ARL . The design process begins by selecting the desired values for the out-of-control and in-control ARL , which are marked on the respective vertical scales and connected by a straight line. The intersection point of this line with the K' parameter scale is used in calculate the value of n . More specifically, using the relation:

$$K' = \frac{d/2}{\sigma/\sqrt{n}} \quad (56)$$

the value of n is determined and rounded up to the nearest integer. Then, the above relation is used to calculate a new K' based on this new n .

Subsequently, on the nomogram, the new K' is connected to the in-control Average Run Length with a straight line to find H' . Once H' is known, the final value of parameter H is calculated using the following formula:

$$H = \frac{\sigma \cdot H'}{\sqrt{n}} \quad (57)$$

The Kemp nomogram method provides a simple graphical approach to determine the CUSUM parameters, enabling the simultaneous control of the chart's behavior under both in-control and out-of-control conditions.

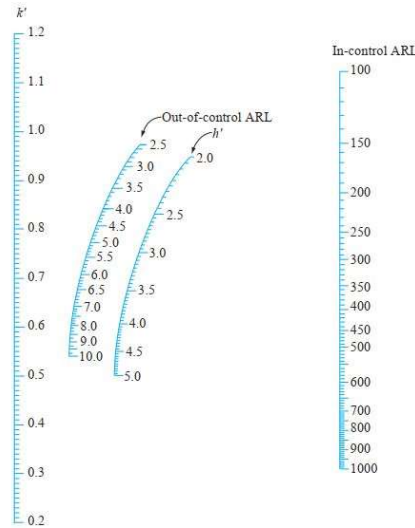


Figure 11 - Kenneth Kemp chart

3.2.1.4 Estimation of the change-point and shift magnitude

Apart from detecting the out-of-control states, the CUSUM algorithm enables the estimation of the time point when the shift began and also its magnitude. That is achieved through the algorithm's property of retaining the information from the previous observations, allowing for a retrospective examination of the data stream.

More specific, to determine the exact starting time of the shift, the counters of positive (T^+) and negative (T^-) deviations are utilized. According to Hawkins and Olwell (1998), the starting point of a shift is defined as the last time point at which the corresponding cumulative statistic was zero. If an alarm signal is triggered at the time point T , the beginning of the shift is placed N time periods earlier, where N is the number of consecutive periods during which the cumulative sum remained non-zero. So, it is concluded that the process was in-control until the time point $T-N$.

After determining the change-point, the new process mean is calculated. The estimation varies depending on the type of deviation.

$$\begin{aligned}\hat{\mu} &= a + K + \frac{S_n^+}{N^+} \\ \hat{\mu} &= a - K - \frac{S_n^-}{N^-}\end{aligned}\quad (58)$$

where a is the target mean and K is the reference parameter. The terms $\frac{S_n^+}{N^+}$ and $\frac{S_n^-}{N^-}$ express the average additional deviation recorded during the shift. In this way, the process shift is quantified by utilizing exclusively the data that led to the control limit violation.

3.2.2 MULTIVARIATE CUSUM

The foundational method for monitoring the mean of a multivariate process was introduced by Hotelling. This technique is a direct expansion of the univariate Shewhart charts to the multivariate space.

The main principle of this method is the reduction of information from a vector of p correlated variables to a single scalar value. This value expresses the distance between the current observation and the mean of the process. Assuming that each observation is a vector $\vec{x}$ that follows multivariate normal distribution with mean vector $\vec{\mu}$ and covariance matrix Σ the control statistic is defined as:

$$T_n = \sqrt{[(\vec{x}_n - \vec{\mu})' \cdot \Sigma^{-1} \cdot (\vec{x}_n - \vec{\mu})]} \quad (59)$$

The chart generates an out-of-control signal when the T_n exceeds a predetermined Statistical Control Limit (SCL).

A major disadvantage of the multivariate Shewhart charts is their lack of robustness, more specific their sensitivity to multivariate outliers. Multivariate outliers are defined as observations that lead to a large T_n value without any single variable being out-of-control. According to Crosier (1988) in many cases this is caused by data entry errors. In addition, when a true shift in the process mean occurs, it tends to manifest along the major axis of the multivariate probability ellipsoid.

The above considerations make robustness an essential characteristic for any multivariate quality control scheme. Furthermore, in many applications, it is highly advisable to employ detection criteria that account for not only the distance of the observation from the target, but also its direction.

3.2.2.1 COT method

To apply the CUSUM to multivariate data, initially Crosier (1988) proposed the conversion of the problem to scalar one, introducing the Cumulative Sum of T (COT) method. So, he used Hotelling's formula and modified the Cumulative Sum equation as follows:

$$S_n = \max(0, T_n - K + S_{n-1}) \quad (60)$$

The algorithm initializes at S_0 where $S_0 \geq 0$ is the allowance K strictly positive ($K > 0$). The algorithm triggers an alarm signal when the value S_n exceeds the decision interval H . While in the Shewhart charts there is no practical difference whether T_n or T_n^2 is used, Crosier chooses T_n , as it is calculated by the relation $T_n = [T_n^2]^{1/2}$, in order to measure distance rather than squared distance. Additionally, regarding the parameter K , he notes that based on his research, the optimal value approximates the square root of the number of variables.

The major advantage of the COT method in comparison to the multivariate Shewhart chart is that it can detect the shift of process' s mean. Additionally,

The main advantage of the COT method compared to multivariate Shewhart charts is that it can detect a process mean shift. Furthermore, enhancement techniques of the CUSUM method can also be applied here. Such techniques include the two-in-a-row rule, where isolated outliers are ignored while two successive outliers in different variables generate a signal, or the Fast Initial Response (FIR) feature, which gives the chart a "head start" by setting the initial value S_0 equal to half of the control limit.

3.2.2.2 Vector CUSUM

To develop Vector CUSUM Crosier (1988) starts from the univariate CUSUM formula which is the following:

$$S_n = \max(0, (x_n - a) - k \cdot \sigma + S_{n-1}) \quad (61)$$

where α is the target mean, σ is the standard deviation, the initial S_0 equals zero and the values x_n and k are positive number. By replacing the scalars with vectors, the above equation becomes:

$$\vec{S}_n = \max(\vec{0}, (\vec{x}_n - \vec{a}) + \vec{S}_{n-1} - \vec{k}) \quad (62)$$

The problems with the above equation the determination of $\vec{k}$ and furthermore, the interpretation of the comparison between a vector and the zero vector.

In the univariate case, the quantity $S_{n-1} + x_n - a$ shrinks towards zero for k standard deviations. So, in order to maintain this property, the equation $\vec{k}' \cdot \Sigma^{-1} \cdot \vec{k} = |\vec{k}|_{\Sigma^{-1}}^2$ must hold, where $|\vec{k}|_{\Sigma^{-1}}$ is the length of $\vec{k}$, calculated using the covariance matrix. Furthermore, to shrink towards $\vec{0}$ the vector $\vec{x}_n - \vec{a} + \vec{S}_{n-1}$ should point in the same direction as $\vec{k}$. Defining C_n as the length of the vector $\vec{x}_n - \vec{a} + \vec{S}_{n-1}$, namely:

$$C_n = \sqrt{(\vec{x}_n - \vec{a} + \vec{S}_{n-1})' \cdot \Sigma^{-1} \cdot (\vec{x}_n - \vec{a} + \vec{S}_{n-1})} \quad (63)$$

it follows that:

$$\vec{k} = \frac{|\vec{k}|}{C_n} \cdot (\vec{x}_n - \vec{a} + \vec{S}_{n-1}) \quad (64)$$

Instead of subtracting the vector $\vec{k}$ from the vector $\vec{x}_n - \vec{a} + \vec{S}_{n-1}$ Crosier proposes setting $\vec{S}_n$ to zero when the length of $\vec{k}$ is greater than C_n and shrinking it by $1 - |\vec{k}|/C_n$ in the opposite case. Namely:

$$\vec{S}_n = \vec{0}, \text{όταν } C_n \leq |\vec{k}|$$

$$\vec{S}_n = (\vec{x}_n - \vec{d} + \vec{S}_{n-1}) \cdot \left(1 - \frac{|\vec{k}|}{C_n}\right), \text{όταν } C_n > |\vec{k}| \quad (65)$$

The initial vector $\vec{S}_0$ is assumed to be equal to the zero vector and $|\vec{k}|$ positive.

By defining the test statistic as:

$$Y_n = \sqrt{\vec{S}'_n \cdot \Sigma^{-1} \cdot \vec{S}_n} \quad (66)$$

the Vector CUSUM algorithm triggers an alarm signal when $Y_n > H$. The Y_n can also be calculated using the equation $Y_n = \max(0, C_n - k)$.

A major advantage of the Vector CUSUM over the COT method and Shewhart control charts is its directional nature. According to Crosier (1988), unlike COT method which ignores direction, the Vector CUSUM allows observations that deviate in opposite directions from the target to cancel each other out. This fact improves algorithm's performance when the process is in-control, leading to larger in-control *ARL*. In addition, the analysis of the *ARL* curves, demonstrates that this method achieves faster detection of small shifts, while the cumulative vector provides a useful indication of the direction in which the mean shifted upon the triggering of a signal.

3.2.2.3 Basic requirements for applying the CUSUM algorithm

The classical statistical foundation and the mathematical design of CUSUM control charts, in both their univariate and multivariate forms (Vector CUSUM), are based on three fundamental assumptions (Hawkins & Olwell, 1998; Qiu, 2013):

- **Multivariate Normality Assumption:** It is assumed that the observation vectors follow a multivariate normal distribution. This assumption is necessary for the mathematical validity of the Mahalanobis distance and the statistical properties of the test statistic Y_n .
- **Statistical Independence of Measurements:** Consecutive observations over time are assumed to be statistically independent and identically distributed, with no presence of serial correlation.
- **Known In-Control Parameters:** Both the target mean vector and the covariance matrix of the process are assumed to be known in advance, usually based on a historical calibration period.

As pointed out by Montgomery (2009) and Mahmoud & Maravelakis (2013), violations of the above assumptions in practical applications can significantly reduce the reliability of the algorithm. In particular, the presence of serial correlation or the estimation of parameters from limited samples can lead to a reduction in the in-control Average Run Length (*ARL*), resulting in a large number of false alarms.

3.2.2.4 Design of the Vector CUSUM procedure

For the design of the Vector CUSUM procedure, a correct estimation of the decision interval H and the allowance K is required. Regarding the allowance K Crossier(1988) suggest that it should be equal to the half of the non-centrality parameter d . This parameter represents the distance between the target mean and the shifted mean of the process, and is equal to:

$$d = \sqrt{(\vec{\mu} - \vec{a})' \cdot \Sigma^{-1} \cdot (\vec{\mu} - \vec{a})} \quad (67)$$

where $\vec{\mu}$ is the shifted mean vector, $\vec{a}$ the target mean vector and Σ the covariance matrix. The determination of the non-centrality parameter d can be made depending on whether prior statistical data for the specific process exist or not. If we know the target mean vector, the covariance matrix, and the mean vector, the parameter is derived from the above formula. For example, if for a process we have measurements both during its in-control operation and during the occurrence of a specific shift (error), the values for the target mean vector and the covariance matrix can easily be obtained from the in-control measurements, while the value of the mean vector can be obtained from the measurements resulting during a shift. In cases where there is not just a single shift we wish to detect, the choice of d is based on the shift we wish to detect most rapidly. This shift should be sufficiently small so as not to be immediately noticeable without statistical process control, yet sufficiently large to have a substantial impact on the process operation.

If there are not previous data the value of the allowance K must not be very low so there the shifts cannot be detected without utilizing the algorithm and enough grater so it will not play significant role in the quality of the algorithm. A typical choice for the non-centrality parameter is equal to standard deviation, from which the value of K is $K = 0.5$.

Regarding the decision interval H , according to Crosier (1988), its choice cannot be based on a simple formula. On the contrary, it is determined empirically with the exclusive objective of achieving the desired in-control Average Run Length. For its precise determination, is suggested a looping simulation procedure combined with linear regression of $\log(ARL)$ on H , in order to determine the value corresponding to the accepted frequency of false alarms. Additionally, it is important to be noted that the decision interval value is directly related to the number of variables. More specific, as the dimensionality of the problem increases, higher decision intervals are required to maintain the same Average Run Length.

3.2.2.5 Data preparation

To run the algorithm, the data fed into it must be statistically independent. In cases with strong serial correlation between the measurements the algorithm will not produce correct results, as the statistical noise is not self-cancelled but is systematically accumulated, leading to an artificial inflation of the test statistic Y_n . This results a dramatic reduction of the in-control Average Run Length, causing an large number of false alarms and the premature finish of the current epoch.

The measurements in the dataset that will be used in this master thesis exhibit strong serial correlation, since the generation of each measurement takes into consideration the previous value of the respective variable. Furthermore, even though the dataset used is not artificial, this specific type of data, namely ship sensor measurements, inherently exhibits strong serial dependence due to the physical, mechanical, and hydrodynamic inertia that characterizes the processes on board a ship. Variables such as speed, revolutions per minute, and generated power cannot change instantaneously between consecutive time points, making serial correlation an inherent structural characteristic of real-world data streams.

Given that data processed via statistical process control (SPC) methods are time series in many cases, several techniques have been developed to address this issue. Qiu (2013) mentions four such methods. The first method involves grouping the data into batches. According to this method, adjacent observations are grouped into batches of m size, the mean of the batch is calculated and the CUSUM algorithm is applied to these means. The advantage of that method is that as the size of the batch increases, time correlation is effectively eliminated. The major disadvantage is that it introduces a time delay in shift detection, since the system must wait for all observations within the batch to be collected before triggering an alarm.

An alternative method that Qiu (2013) suggests is to feed the algorithm with data points separated by a time interval t . In this way, temporal correlation between the data is reduced, specifically when the time interval t is large enough. However, a significant volume of data is discarded and the behavior of the process between consecutive time intervals is ignored.

According to Qiu (2013), in cases where a sufficiently large volume of historical in-control data is available, it is feasible to apply of control charts directly into the raw, autocorrelated data, bypassing the complexity of time series modeling. To counterbalance of the effect of temporal autocorrelation and avoid false alarms, the control limits are mathematically adjusted and typically widened, incorporating the estimated correlation function of the historical sample.

Finally, the method that Qiu (2013) suggests for dealing with the serial dependence of the measurements is utilizing Residual-based Control Charts. This method does not try to retrospectively modify the control limits or to reduce the volume of data through grouping, but it targets to eliminate the autocorrelation from the outset, before feeding the raw data into the statistical process control algorithm.

The behavior of an autocorrelated process, when operating under in-control conditions, can be effectively described by a First-Order Autoregressive Model - AR(1). The mathematical equation defining this process is given by the following relationship:

$$x_n = \mu + \varphi \cdot (x_{n-1} - \mu) + e_n \quad (68)$$

where μ is the theoretical target mean of the process, φ is the autoregressive coefficient ($|\varphi| < 1$) which quantifies the degree of the temporal inertia or memory of the system and e_n represents a sequence of independent and identically distributed errors (white noise) with a mean value equal to zero and a constant variance σ_e^2 . The relationship between the variance of the white noise and the observations x_n is as follows:

$$\sigma_{x_n}^2 = \frac{\sigma_e^2}{1 - \varphi^2} \quad (69)$$

Once the parameters of the AR(1) model—namely the target mean of the process and the autoregressive coefficient—are accurately estimated from a reference sample, the pre-whitening procedure can be applied. During this process, the expected value of the model is subtracted from the actual current observation, thereby isolating the process residuals:

$$\widehat{e}_n = x_n - \widehat{x}_n \quad (70)$$

Consequently, temporal dependence is completely eliminated from the data. If the process operates under in-control conditions, the generated residuals have no autocorrelation, behave as pure statistical white noise, and follow the normal distribution with a mean of zero and a variance σ_e^2 . The CUSUM algorithm can now be applied to these independent residuals with absolute mathematical consistency.

In the event of a sustained mean shift in the raw process data — for instance, from the value μ_0 to the value μ_1 , with a shift magnitude of $\delta = \mu_1 - \mu_0$ — it is mathematically proven that at the exact time point the change manifests, the mean of the residuals shifts instantaneously by δ . However, in all subsequent time steps, due to the subtraction of the previous value via the coefficient φ , the mean of the residuals stabilizes at a new, sustained but attenuated shift, which is equal to:

$$\delta^* = (1 - \varphi) \cdot \delta \quad (71)$$

Given that the coefficient φ is strictly less than one, the value δ^* does not become zero, a fact that allows the control algorithm to detect the sustained process shift by monitoring its residuals.

Although the aforementioned methods described by Qiu (2013) concern the univariate case, they can also be applied to a multivariate dataset. Within the scope of this thesis, the residual-based method will be applied, adapted for a multivariate system. Consequently, the multivariate Vector CUSUM algorithm, instead of being applied directly to the data, is applied to the residual vectors, utilizing the inverse of their covariance matrix to calculate the control statistic and ensure the early detection of phase changes in the system.

4. SYSTEM ARCHITECTURE

The proposed architecture targets to reduce the volume of multivariate IOT data at the edge of the network, in order to minimize bandwidth and cloud storage costs, while largely preserving the basic statistical characteristics of the initial data stream. So, instead of being transmitted continuously, the data streams originating from the sensors and entering the edge node are processed by the algorithm. They are sampled according to the selected sampling technique, temporarily stored in a buffer, and sent to the cloud whenever a shift in the system is detected.

This architecture converts massive data volumes (Big Data) into manageable, representative summaries (Small Data). Data transmission to the cloud is not continuous; rather, it is triggered by statistical shifts detected in real time, ensuring that only the segments of the stream carrying high informational value are permanently retained.

4.1 THE CONCEPT OF AN EPOCH

Given that the data stream is theoretically infinite and the state of the system does not remain constant but is subject to change, the proposed architecture divides the continuous stream into discrete time intervals called epochs.

The operation and lifecycle of each epoch are defined by its start, duration, and end. Each epoch begins with the filling of the buffer with continuous data. During the epoch, the CUSUM change detection algorithm monitors the stream for any shift in the system's state. The end of the epoch is signaled by the detection of a statistically significant shift in the behavior of the data stream. This event triggers the emptying of the buffer, the transmission of its stored data to the cloud, the reinitialization of the CUSUM algorithm, and the start of a new epoch.

4.2 ALGORITHMIC FLOW AND MEMORY MANAGEMENT

The operation of the network edge node is divided into three main operational phases, which ensure the strict statistical foundation of the system and the preservation of a low computational footprint.

4.2.1 PHASE 1: EPOCH INITIATION

At the beginning of each epoch, the new multivariate measurements are stored sequentially in the buffer, until it is full. Once the buffer is full, the system calculates the baseline statistical parameters from the measurements stored in the buffer, which will determine its subsequent behavior.

At this point, it should be mentioned the differentiation of the proposed methodology from the classic assumption of CUSUM theory where the parameters are known in advanced. In a real-world ship telemetry application, there is no single, stable reference state, as the operating conditions and the cruising environment change continuously. For this reason, the requirement for parameters known in advance is addressed dynamically through the introduction of the epoch concept.

More specifically, phase 1 serves as a self-starting calibration stage. The measurements collected during the initial filling of the buffer are utilized as the local reference sample to calculate the mean vector and the covariance matrix. In this way, the parameters do not need to be known in advance; rather, they are adaptively recalculated at the beginning of each epoch, allowing the vector CUSUM to operate effectively as a shift detection mechanism in an edge environment.

The parameters used in the CUSUM change detection algorithm and the Mahalanobis Range Replacement strategy fall into two categories:

- 1) Difference parameters ($\mu_\Delta, \Sigma_\Delta$): These refer to the mean vector and the covariance matrix that will be used in the vector CUSUM. As mentioned in Section 3.2.2.5, in cases where there is temporal interdependence among the values, the direct application of the vector CUSUM to the raw data can lead to an incorrect estimation of the shift due to the temporal correlation of the observations. To address this problem, the residual-based method and a First-Order Autoregressive Model (AR(1)) are adopted. Given that the simulation data have been generated using this specific model, the application of the residual-based method essentially translates to a simple calculation of the first differences of successive measurements, $\Delta x_n = x_n - x_{n-1}$, and consequently, the algorithm is applied directly to these differences.
- 2) Raw parameters (μ_{raw}, Σ_{raw}): These refer to the mean vector and the covariance matrix that will be used in the Mahalanobis Range Replacement strategy and they are calculated directly from the measurements stored in the buffer.

It is worth noting that while, according to the Residual-based Control Charts method, the multivariate CUSUM is applied to the noise, in practice—as mentioned above—it is applied to the difference between the current and the previous value. Normally, solving Equation 68 for e_n yields:

$$e_n = x_n - \mu - \varphi \cdot (x_{n-1} - \mu) = x_n + \mu \cdot (\varphi - 1) - \varphi \cdot x_{n-1} \quad (72)$$

In cases of strong temporal correlation, the coefficient φ approaches one. Therefore, the above equation is simplified to:

$$e_n = x_n - \mu - \varphi \cdot (x_{n-1} - \mu) = x_n - x_{n-1} \quad (73)$$

which is then used in the algorithm. During the creation of the dataset, this coefficient was set to $\varphi = 0,95$. Although its exact value is known in this specific case, the use of the simplified first-difference formula was preferred. Utilizing this exact parameter would constitute an idealized scenario, artificially inflating the algorithm's estimated performance under conditions that typically do not exist in reality.

Consequently, the adoption of first differences ensures the system's independence, given that in a real-world multivariate application, the correlation coefficient can be difficult to calculate. Furthermore, this choice subjects the algorithm to a more rigorous and conservative evaluation, as the imperfect removal of the $0.05 \cdot (x_{n-1} - \mu)$ term leaves a minimal residual noise. The fact that the CUSUM algorithm successfully detects changes even under this conservative assumption confirms the high robustness of the proposed architecture.

To avoid the creation of singular covariance matrices that would lead to a system failure during their inversion, a small regularization factor ε , equal to 0.0001, is added to the diagonal of the covariance matrix. This technique ensures invertibility without altering the statistical control parameters. The computationally expensive matrix inversion is performed once per epoch, during the transition from Phase 1 to Phase 2, optimizing resource utilization.

4.2.2 PHASE 2: EPOCH DURATION

During the second phase, two main operations are performed: monitoring for any system state change (concept drift) using the vector CUSUM, and maintaining the buffer using one of the three replacement methods under consideration.

When a new measurement arrives and after its difference from the previous one is computed, the new statistic C_n , the cumulative sum S_n and the test statistic Y_n are calculated. If $Y_n > H$ (where H is the decision interval) the system triggers an alarm signal and it transitions to the end of the epoch phase.

While the system remains in phase 2 each new raw measurement is evaluated by the replacement algorithm. The aim is the optimal sampling of the data's shell. To this end, three replacement methods are examined and evaluated.

In the Max Range Replacement method, it is evaluated whether replacing the point with the minimum Euclidean distance from the mean increases the maximum distance from it. The Mahalanobis Range Replacement method follows a similar logic, but utilizes the Mahalanobis distance instead of the Euclidean distance. Finally, in the Max Distance Expansion Replacement Method, the distances between all points within the buffer are calculated. If the new point increases the maximum distance, it replaces one of the two points that form the minimum distance.

From the perspective of computational cost and memory management, the algorithms were designed to efficiently process new measurements without the need to create buffer copies. The evaluation of a potential replacement is performed hypothetically using temporary variables and intermediate calculations, without modifying the primary data storage structure. This approach minimizes memory requirements, as the intermediate data structures are strictly limited to the elements essential for making the replacement decision.

The Max Range Replacement and Mahalanobis Range Replacement methods are based on the temporary estimation of the effect of replacement on the statistical characteristics of the buffer, requiring limited additional memory resources. The Mahalanobis method exhibits increased computational requirements due to the use of the covariance matrix, whereas the Max Distance Expansion Replacement Method requires the calculation of distances between all points and thus incurs a higher computational overhead. To reduce the computational cost in the case of the Mahalanobis distance, the inverse covariance matrix is computed once at the beginning of each epoch and remains constant throughout its duration.

Nevertheless, in all three cases, the use of a temporary data pool is avoided, as intermediate structures are restricted to the elements strictly necessary for making the replacement decision, achieving more efficient memory management and rendering the methods suitable for data stream processing applications.

4.2.3 PHASE 3: EPOCH END

When the CUSUM algorithm detects a change, it triggers an alarm that signals the transition to the third phase. In this phase, the epoch is terminated and a transition to a new epoch takes place.

Upon the termination of the epoch, the buffer is emptied. In a real-world environment, its stored data are sent to the cloud for processing. Within the scope of this thesis, summaries are generated to evaluate them.

Concurrently, the CUSUM and replacement algorithms cease their operation as the system transitions back to Phase 1. Upon the completion of Phase 1, both the raw parameters and the difference parameters are recalculated, and the execution of the algorithms is restarted.

5. ARCHITECTURE IMPLEMENTATION

The implementation of the architecture analyzed above was carried out in the Python programming language, utilizing object-oriented programming techniques. The different structures that comprise the system were implemented as separate classes, consisting of respective methods. Each structure is isolated from the rest, and the overall system execution is handled by a main stream management script that orchestrates the various operations. This approach ensures data integrity, while significantly facilitating code testing, modification, debugging, and maintenance.

The main stream management script functions as the main control point, managing the data flow between the classes without permitting their direct interaction. This design approach significantly reduces the coupling among the components, allowing for the easy replacement or upgrade of a class without affecting the rest of the system. Concurrently, the isolation of these structures makes thorough unit testing feasible, as each class can be tested independently. Thanks to the clear separation of roles, the logic remains readable and well-organized, while the addition of new features is achieved by simply creating new classes and appropriately orchestrating them via the main stream management script.

The classes were implemented with the objective of reducing computational load and minimizing memory overhead. This philosophy is not limited to isolated optimizations rather, it cuts across all design stages, from data structure management to the execution of algorithmic operations.

At the memory level, a static pre-allocation strategy for the required space is adopted, thereby avoiding dynamic reallocations that cause fragmentation and slow down execution. Arrays are allocated once at their maximum expected size and are reused. Their reset is achieved by zeroing out the existing data, rather than destroying and recreating the data structures. Concurrently, lightweight numeric types are utilized wherever possible, reducing the data's memory footprint without a significant loss of precision. Furthermore, data access is performed through views, ensuring that reading occurs directly within the existing memory space, without the additional allocation of temporary copies.

At the computational level, a fundamental design choice is the utilization of vectorized operations through software libraries. Consequently, operations on arrays and vectors are performed using libraries such as NumPy, which leverage parallel processing (SIMD) and multithreaded execution, achieving a significant speedup compared to iterative element-by-element processing. This approach effectively avoids the use of explicit loops, which introduce execution overhead, particularly when processing large volumes of data.

Beyond vectorized execution, algorithmic complexity reduction strategies are employed wherever possible. For example, instead of recalculating statistical measures from scratch each time, incremental updates are utilized, whereby the new mean or variance is derived from the previous one using simple corrective operations, irrespective of the volume of stored data. This advantage is particularly crucial, as it transforms calculations of linear complexity ($O(N)$) into constant-cost operations ($O(1)$). Additionally, each class maintains an internal state containing precalculated intermediate values—such as means, distance matrices, or extreme values—so that recalculation is avoided when evaluating a new measurement. Finally, any quantities that remain constant throughout the lifecycle of an object (e.g., inverse covariance matrices, static parameters) are computed upfront during initialization, thus preventing redundant operations.

5.1 DATA BUFFER

Data buffer as a concept plays a crucial role in the data processing architecture at the network edge. Regarding data streams management, buffer does function merely as a temporary storage space, but it is

also the mechanism that bridges the gap between the rapid acquisition of measurements and their asynchronous transmission to the cloud.

According to the proposed architecture, the data buffer has the role of accumulating a representative sample of measurements corresponding to the current epoch. The system's efficiency relies initially on managing the stored in the data. Instead of transmitting every new measurement of the data stream, the systems processes locally the data and it transmits to the cloud only the data that are stored to the buffer, when the vector CUSUM triggers the alarm signal.

5.1.1 IMPLEMENTATION OF THE DATA BUFFER ALGORITHM

The implementation of the buffer algorithm relies on a three-stage structure, which depends on the number of the samples that are stored and the state of the change detector.

Assuming that B is the set of measurements that are stored in a buffer with capacity K , and x_t is the current measurements in the time point t . The procedure is described as follows:

Phase 1: Initialization

During the initialization of the system or after it is cleared, the buffer is empty. In this phase, the aim is the quick creation of a reference base. Until the number of stored elements N becomes equal to K ($N = K$), the state of the buffer is referred to as non-full, and each new measurement x_t , is stored directly, namely:

$$B_t = B_{t-1} \cup \{x_t\} \quad (74)$$

In this phase the replacement criteria are not implemented, because each measurement is considered crucial to full the buffer.

Phase 2: Stable operation state

When the stored data in the buffer reaches its capacity limit ($N = K$), the operation of the structure changes. Now, the buffer stops to accept new records uncontrollably and transitions into a standby state, awaiting further instructions.

At this stage, the buffer class signals the main stream management script that its memory is full. In response to this signal, the stream of new data is no longer routed directly for storage, but is instead directed to the class implementing the replacement strategy. There, the new measurements are evaluated based on their contribution to the statistical quality of the sample. If the algorithm determines that the new measurement should be retained, it issues a command to the buffer specifying exactly which existing element must be replaced, leaving the storage structure to simply execute the update.

Phase 3: Flushing and transmission

If the Vector CUSUM algorithm triggers an alarm signal, it indicates that the stream has transitioned into a new epoch. At this point, the current contents of the buffer are transmitted to the cloud, and the buffer is flushed, returning the algorithm to Phase 1.

For the implementation of the data buffer, developing the `DataBuffer` class was considered the optimal solution. This object-oriented approach not only serves the need to encapsulate the measurements within a single entity, but also ensures the architectural independence of the structure. Specifically, the class was designed to remain autonomous and free of any functionality or data relating exclusively to the replacement algorithm (such as computed distances or auxiliary variables). In this way, a clear separation of the data

from their processing strategy is achieved, rendering the system modular and allowing the interchange of different sampling algorithms without necessitating structural changes to the core of the buffer.

Regarding the internal architecture of the class, the decision to separate the identifiers (`samples_ids`) and the global indices (`global_indices`) into discrete structures, rather than encapsulating them within the main data matrix (`data`), is a critical design choice. This approach ensures the computational efficiency of algorithms such as vector CUSUM, as it allows matrix operations to be performed on pure measurement arrays without the overhead of continuous slicing.

Another characteristic of this design choice is optimal memory management, as the unnecessary allocation of resources is avoided by using appropriate data types. More specifically, discrete storage allows the data matrix to retain a floating-point format for maximum measurement precision, while the `samples_ids` and `global_indices` arrays remain integers. If these were encapsulated within a single matrix, the system would be forced to cast the identifiers and indices into a floating-point format, thereby increasing the overall memory footprint of the application.

Regarding the maintenance of chronological order, the shifting method was rejected, as the $O(K)$ complexity it introduces was deemed prohibitive for edge computing environments. Instead, the in-place replacement method with $O(1)$ complexity was selected, thereby offloading the computationally expensive sorting process to the cloud.

The attributes of the `DataBuffer` objects are presented in Table 1.

<u>Variable name</u>	<u>Type</u>	<u>Description</u>
<code>data</code>	<code>np.ndarray</code>	Array of stored measurements.
<code>variables_num</code>	<code>int</code>	The number of variables in each measurement vector.
<code>samples_ids</code>	<code>np.ndarray</code>	Identifiers of the stored measurements. The IDs are unique within an epoch and function as timestamps.
<code>size</code>	<code>int</code>	Buffer capacity.
<code>total_samples_counter</code>	<code>int</code>	Total counter of incoming measurements, used to generate a unique identifier.
<code>is_full</code>	<code>bool</code>	Takes the value <code>True</code> when <code>total_samples_counter</code> $\geq$ <code>size</code> .
<code>global_indices</code>	<code>np.ndarray</code>	Array storing the global indices of the measurements from the original data stream.

Table 1 - `DataBuffer` class variables

The methods that are implemented are the `init`, `insert`, `reset`, `isFull`, `getGlobalIndices`, `getDataAndIds` and `replacement`.

Method `init`

Class constructor, responsible for instantiating the structure and allocating the necessary memory resources.

Inputs:

- `size`: The desired capacity of the buffer (K), which determines the maximum number of measurements to be retained.
- `variables_num`: The number of variables in each measurement vector.

Outputs: None.

Functionality:

Initializes the internal structures of the class. It creates the empty arrays `data` (for the measurements), `samples_ids` (for the identifiers), and `global_indices` (for the global stream indices) with dimensions defined by the input parameters, thereby preparing the buffer to ingest data. It also sets the `is_full` boolean variable to `False`.

Method reset

Full reset method. It is typically called after the completion of an epoch and the transmission of the data to the cloud. It takes no inputs and returns no outputs.

Functionality:

The method restores the buffer to a zero-baseline state. Specifically, it flushes the `data`, `samples_ids`, and `global_indices` arrays by filling them with zeros and deleting the data of the previous period. Additionally, the `total_samples_counter` is reset to zero, while the `is_full` control variable is set to `False`. In this way, the system becomes ready to initiate Phase 1 of the new monitoring epoch.

Method insert

Direct insertion method. It is called during Phase 1, as long as the buffer is not completely full and there is available space to store new measurements.

Inputs:

- `new_data`: The new measurement to be inserted.
- `global_index`: The global index of the measurement from the overall data stream.

Outputs: None.

Functionality: The method implements the sequential insertion process by utilizing the pre-allocated space of the buffer. Initially, the `new_data` vector is written into the `data` array at the position defined by the current value of the `total_samples_counter`, thus avoiding computationally expensive dynamic memory allocation. Simultaneously, the corresponding unique identifier is recorded in the `samples_ids` array, as well as the global index in the `global_indices` array. Subsequently, the `total_samples_counter` is incremented, preparing the index for the next entry. Finally, it checks whether the number of samples has equaled the buffer's capacity (`size`), in which case the `is_full` variable is set to `True`, signaling the completion of Phase 1.

Method isFull

Method for retrieving the buffer's fullness state. It acts as an accessor (getter) for the boolean variable that determines the operating phase of the algorithm. It requires no input parameters.

Outputs:

- `is_full`: Returns `True` if the buffer is completely full, otherwise `False`.

Functionality: The method returns the stored value of the `is_full` state variable. This value, which is dynamically updated during the data insertion procedures, indicates whether the buffer has reached its capacity (`True`) or is still in the initial filling stage (`False`), allowing external control mechanisms to adapt the sampling strategy.

Method getDataAndIds

Data retrieval method (getter). It is used to feed either the replacement algorithm (for evaluation) or the transmission process to the cloud. It takes no inputs.

Outputs:

- `data`: The array containing the stored measurements.
- `samples_ids`: The array containing their corresponding unique identifiers.

Functionality: It returns the current contents of the buffer, providing access to the "snapshot" of the current epoch without modifying its state.

Method getGlobalIndices

Measurement global indices retrieval method (getter). It requires no input parameters.

Outputs:

- `global_indices`: The array containing the global indices of the stored samples.

Functionality: It returns a view of the `global_indices` array, allowing external classes (such as `SynopsisExporter`) to read the indices for diagnostic checks or export.

Method replacement

Replacement execution method. It is called in Phase 2, provided that the replacement algorithm decides that a new measurement must be inserted into the buffer.

Inputs:

- `new_data`: The new measurement vector to be inserted.
- `old_data_index`: The index of the measurement selected for deletion. Since the control algorithm already knows this position, passing it as an argument allows direct access and writing to memory. This architectural choice transforms the replacement operation into a process of $O(1)$ time complexity.
- `global_index`: The global index of the new measurement in the stream.

Outputs:

- Boolean: Returns True upon a successful replacement.

Functionality: The method implements the replacement process in a direct and efficient manner, while concurrently incorporating a bounds check for the index (`old_data_index`). If the index is valid, the new vector `new_data` is written instantaneously into the data array at the specified position, avoiding any costly search operations. Simultaneously, the new sample receives the current value of the `total_samples_counter` as its ID (timestamp) in the `samples_ids` array, while its global index is stored in the `global_indices` array. Immediately after, the counter is incremented by 1, preparing for the next insertion.

It is noted that the replacement is strictly performed "in-place" (in the same memory location) and not through shifting the remaining elements. This approach was selected because it ensures a constant time complexity of $O(1)$, drastically minimizing resource consumption. Given that the elements within the buffer lose their physical order due to this in-place replacement, the correct chronological order is recovered later and asynchronously in the cloud, through the sorting of the progressive values of the `samples_ids` array. The

pseudocode of the control algorithm is presented next, while the corresponding Python implementation is located in the Appendix.

```
CLASS DataBuffer {
  METHOD init (size, variables_num){
    Size <- size
    data <- ZEROS(size, variables_num)
    samples_ids <- ZEROS(size)
    global_indices <- ZEROS(size)
    total_samples_counter <- 0
    is_full <- False
  }

  METHOD reset (){
    data <- ZEROS(size, variables_num)
    global_indices <- ZEROS(size)
    samples_ids <- ZEROS(size)
    total_samples_counter <- 0
    is_full <- False
  }

  METHOD getDataAndIds(){
    return data, samples_ids
  }

  METHOD getGlobalIndices(){
    return global_indices
  }

  METHOD replacement (new_data, old_data_index, global_index){
    IF (old_data_index < 0 OR old_data_index >= size) THEN {
      RETURN false }
    data[old_data_index] <- new_data
    samples_ids[old_data_index] <- total_samples_counter
    global_indices[old_data_index] <- global_index
    total_samples_counter <- total_samples_counter + 1
    RETURN true
  }

  METHOD insert (new_data, global_index){
    data[total_samples_counter] <- new_data
    samples_ids[total_samples_counter] <- total_samples_counter
    global_indices[total_samples_counter] <- global_index
    total_samples_counter <- total_samples_counter + 1
    IF (total_samples_counter >= size) THEN {
      is_full <- TRUE
    }
  }

  METHOD isFull (){
    return is_full
  }
}
```

```
}
}
```

5.2 IMPLEMENTATION OF THE VECTOR CUSUM ALGORITHM

In the context of this thesis, the Vector CUSUM method will be utilized to signal the change between epochs. Implementing this via the VectorCUSUM class was deemed the optimal solution, as the object-oriented approach more efficiently serves the need to retain the "memory" of the process. In contrast to a simple procedural implementation, using a class allows for the integration of the cumulative sum vector S_n and the control parameters into a single unified structure. This ensures that the current state of the algorithm is maintained intact across successive measurements of the data stream. Additionally, this architecture contributes to the conservation of computational resources, as computationally expensive operations, such as the inversion of the covariance matrix, are executed during initialization rather than repeatedly for each new sample.

The variables of the VectorCUSUM objects are presented in Table 2.

Variable name	Type	Description
mean_vector	np.ndarray	The mean vector.
covariance_matrix	np.ndarray	The covariance matrix Σ .
decision_interval	float	The decision interval H .
reference_value	float	The allowance K .
inverse_covariance_matrix	np.ndarray	The inverse covariance matrix Σ^{-1} .
s_vector	np.ndarray	The vector S_n .
variables_num	int	The number of variables.
length_c	float	The length C_n .
test_statistic	float	The test statistic Y_n .
out_of_control	bool	Its value is True when there is an epoch change.
run_length	int	Counts the length of the sequence from the last reset.

Table 2 - VectorCUSUM class variables

The methods implemented in the class are init, reset, and update.

Method init

This is the constructor of the class, invoked upon object instantiation. Its purpose is to configure the control algorithm and initialize the necessary mathematical structures.

Inputs:

- mean_vector: The mean vector representing the in-control operational target of the process.
- covariance_matrix: The covariance matrix Σ .
- decision_interval: The decision interval value H , which determines the sensitivity of the system.
- reference_value: The allowance value K , associated with the algorithm's tolerance to small shifts.

Outputs: None.

Operation:

The method initializes the internal state parameters of the object. The key step is the computation and storage of the inverse covariance matrix Σ^{-1} . In addition, it automatically determines the number of variables (based on the dimension of the inputs) and initializes the state variables required to begin process monitoring.

Method reset

A reset routine utilized to clear the memory of the algorithm. It is typically invoked following the detection of an epoch change, preparing the system for a new sampling cycle.

Inputs: None.

Outputs: None.

Operation:

Restores the algorithm to its baseline state. Specifically, it resets the cumulative sum vector S_n to zero, clears the run length counter (run_length) to zero, and sets the deviation flag (out_of_control) to False.

Method update

The primary execution method of the algorithm. It is invoked iteratively at each time step and is responsible for updating the statistics and making the control decision.

Inputs:

- new_sample: The new multivariate observation vector received from the data stream.

Outputs:

- out_of_control: A Boolean flag that returns True if a statistical shift is detected, and False otherwise.

Operation:

- Computes the length C_n (length_c).
- Recursively updates the cumulative sum vector S_n (s_vector).
- Calculates the final test statistic Y_n (test_statistic).
- Increments the run_length counter.
- Compares test_statistic (Y_n) against decision_interval (H) to evaluate whether the threshold is exceeded.

Below, the pseudocode of the control algorithm is presented, while the corresponding Python implementation is provided in the Appendix.

```
CLASS VectorCUSUM{
    METHOD init(mean_vector, covariance_matrix, decision_interval, reference_value){
        mean_vector <- mean_vector
        covariance_matrix <- covariance_matrix
        decision_interval <- decision_interval
        reference_value <- reference_value

        inverse_covariance_matrix <- INVERSE(covariance_matrix)

        variables_num <- LENGTH(mean_vector)

        s_vector <- ZEROS(variables_num)
        length_c <- 0.0
        test_statistic <- 0.0
        out_of_control <- FALSE
    }
}
```

```

        run_length <- 0
    }

    METHOD reset(){
        s_vector <- ZEROS(variables_num)

        run_length <- 0

        out_of_control <- FALSE
    }

    METHOD update(new_sample){
        s_temp <- new_sample - mean_vector + s_vector

        length_c <- SQUARE_ROOT(TRANPOSE(s_temp)*inverse_covariance_matrix*s_temp)

        run_length <- run_length + 1

        IF (length_c ≤ reference_value) THEN {
            s_vector <- ZEROS(variables_num)
            test_statistic <- 0.0
            out_of_control <- FALSE
        }
        ELSE{
            s_vector <- (s_temp)*(1-reference_value/length_c)

            test_statistic <- length_c - reference_value

            IF test_statistic > decision_interval THEN{
                out_of_control <- TRUE
            }
            ELSE{
                out_of_control <- FALSE
            }
        }
        RETURN out_of_control
    }
}

```

5.3 REPLACEMENT METHODS

The replacement strategy, which dictates whether an existing data point within the buffer will be replaced by a new measurement, is of crucial importance to the algorithm's operation. The sample selection mechanism determines the extent to which the data ultimately transmitted to the cloud for processing adequately represent the data stream and capture the actual state of the system.

The replacement methods presented and evaluated in this thesis aim to preserve the most extreme values of the sample. According to this approach, information located at the tails of the distribution is considered more critical for describing the variability of the process. In this way, the optimal representation of the data envelope is achieved. In contrast to techniques that attempt to preserve the central tendency of the

distribution, focusing on extreme values allows for a more accurate delimitation of normal operation, ensuring that significant deviations will not be lost during the data reduction process.

In the following subsections, three different strategies implementing this logic are presented. The underlying principle is common to all three: the replacement of a data point is justified only if the introduction of the new measurement expands the dispersion of the stored data. What differentiates the methods is the manner in which distance is defined and calculated—whether as the Euclidean distance from the mean, the Mahalanobis distance (which accounts for the correlations among variables), or the distance between the data points themselves. These three approaches are detailed below.

5.3.1 MAX RANGE REPLACEMENT METHOD

These methods is based on the selective replacement of the data points located closest to the centroid of the dataset, aiming the expansion of the boundaries of the distribution. The replacement is executed only if the new point insertion leads to an increase of the maximum Euclidean distance from the mean vector.

More specifically, assuming that there are N stored vectors $D = \{D_1, D_2, \dots, D_N\}$ in an R^N space, which represent the stored in the buffer measurements. The mean vector of these points is given by:

$$\vec{M}_1 = \frac{1}{N} \sum_{i=1}^N \vec{D}_i \quad (75)$$

For each point D the Euclidean distance from the mean vector is calculated by:

$$d_i = |\vec{D}_i - \vec{M}_1| = \sqrt{\sum_{j=1}^n (D_{ij} - M_{1j})^2} \quad (76)$$

The point with the minimum distance from the mean is calculated by:

$$d_{min} = \min (|\vec{D}_1 - \vec{M}_1|, |\vec{D}_2 - \vec{M}_1|, \dots, |\vec{D}_N - \vec{M}_1|) \quad (77)$$

The point with the maximum distance from the mean is calculated by:

$$d_{max} = \max (|\vec{D}_1 - \vec{M}_1|, |\vec{D}_2 - \vec{M}_1|, \dots, |\vec{D}_N - \vec{M}_1|) \quad (78)$$

When a new measurement is received, represented by the vector $\vec{D}_k$ the algorithm evaluates the possibility of replacing the existing data point that has the minimum distance, d_{min} with the new point. The new vector tentatively replaces the vector closest to the mean vector subsequently, the new mean vector $\vec{M}_2$, the updated distances $|\vec{D}_i - \vec{M}_2|$ and the new vector with the maximum distance from the mean d'_{max} are calculated. If the distance of the new most distant point exceeds the previous maximum distance, namely $d'_{max} > d_{max}$, replacement is finalized. Otherwise, the new measurement is rejected and the buffer retains its previous state.

5.3.1.1 Implementation of the Max Range Replacement algorithm

The implementation of the replacement algorithm was carried out via the `MaxRangeReplacement` class which, was designed aiming for the smallest possible memory footprint and computational complexity.

More specifically, to evaluate the feasibility of replacement upon the arrival of a new measurement, the use of a temporary buffer was avoided. Incorporating such an intermediate structure would cause unnecessary dynamic resource allocation and require the continuous copying of matrix elements. Instead, the `MaxRangeReplacement` class evaluates the cluster expansion criterion without temporarily modifying the buffer data. To this end, it calculates the new mean vector and the corresponding distances that would result if the new measurement replaced the vector with the minimum distance from the mean, thereby performing a hypothetical replacement check.

In the same direction of computational optimization, the calculation of the new temporary sample mean is performed vectorially. Thus, the system does not recalculate the mean by rescanning the entire contents of the buffer. Instead, it utilizes the already calculated initial mean and readjusts it by linearly subtracting the contribution of the departing measurement d_{min} which is represented by the vector $\overrightarrow{D_k}$ and adding the contribution of the new measurement, which is represented by the vector $\overrightarrow{D_k'}$. This operation is mathematically expressed by the following equation:

$$\overrightarrow{M_2} = \overrightarrow{M_1} + (\overrightarrow{D_k'} - \overrightarrow{D_k}) / N \quad (79)$$

where N is the total number of the stored samples in the buffer. Through this vectorized approach, the new mean is calculated immediately and efficiently, subsequently allowing the calculation of the new distances and the identification of the new maximum distance in order to make the final replacement decision is made.

The variables of the `DataBuffer` objects are presented in the Table 3.

Variable name	Type	Description
buffer	np.ndarray	Reference to the array of stored measurements in the buffer.
samples_ids	np.ndarray	Reference to the array containing the unique identifiers of the stored measurements.
mean_vector	np.ndarray	The mean vector (centroid) of the stored measurements for the current epoch.
Distances	np.ndarray	Array storing the calculated Euclidean distances of each point from the mean.
min_idx	int	The internal array index corresponding to the measurement with the minimum distance from the mean (designated for replacement).
min_distance_id	int	The unique identifier (ID) of the measurement with the minimum distance.
max_distance_id	int	The unique identifier (ID) of the measurement with the maximum distance from the mean.
max_distance	float	The numerical value of the maximum distance, serving as a comparative threshold for evaluating the dispersion expansion.

Table 3 - MaxRangeReplacement class variables

Method init

The class constructor, responsible for initializing the variables of the replacement algorithm based on the current content of the buffer.

Inputs:

- `buffer`: The array containing the currently stored measurements.
- `samples_ids`: The array containing the corresponding unique identifiers of the measurements.

Outputs: None.

Operation:

Initializes the internal variables of the class, assigning the references for the data buffer and the identifiers. It calculates the initial sample mean (`mean_vector`) in a vectorized manner along the variable axis. Subsequently, it calls the `distance_calc` method to calculate the Euclidean distances of all points from the mean vector, and the `min_max_calc` method to retrieve the identifiers associated with the minimum and maximum distance, as well as the initial value of the maximum distance.

Method `distance_calc`

A method for calculating the distances of the points from a central reference vector.

Inputs:

- `buffer`: The measurements array.
- `mean_vector`: The mean vector.

Outputs:

- `distances`: An array containing the calculated Euclidean distances for each point in the buffer.

Operation:

The method implements the vectorized calculation of the Euclidean distance (Euclidean norm) of all points from the mean vector.

Method `min_max_calc`

A method for identifying the extreme points of the sample based on their distances from the mean.

Inputs:

- `distances`: The array containing the calculated distances.
- `samples_ids`: The array containing the measurement identifiers.

Outputs:

- `min_distance_id`: The identifier of the measurement with the minimum distance (the point closest to the mean).
- `max_distance_id`: The identifier of the measurement with the maximum distance.
- `max_distance`: The numerical value of the maximum distance.

Operation:

It searches for the indices of the minimum and maximum distance within the `distances` array. Using these indices, it extracts and returns the corresponding unique identifiers from the `samples_ids` array, as well as the numerical value of the maximum distance. Furthermore, it stores the index of the minimum distance (`min_idx`) as an internal variable for direct access during the evaluation of future measurements.

Method check_new_measurement

The primary evaluation method, responsible for assessing the suitability of inserting a new measurement based on the dispersion expansion criterion.

Inputs:

- `new_measurement`: The new incoming measurement.

Outputs:

- Boolean, int/None: Returns True accompanied by the exact index of the position to be deleted (`min_idx`) if the criterion is satisfied, enabling direct spatial access. Otherwise, it returns False and None.

Operation:

The method implements the core of the Max Range Replacement strategy. Initially, it retrieves the vector with the minimum distance, which is designated for replacement. Then, as analyzed previously, it calculates the new temporary mean in a vectorized manner and extracts the new temporary distances for all points. The new temporary maximum distance is identified. If it is determined that the dispersion increases, the criterion is satisfied, and the algorithm updates the mean vector and the distance array. Finally, it instructs the buffer to proceed with the replacement by passing the position index, thus ensuring $O(1)$ time complexity during the write operation. Otherwise, the new measurement is rejected.

Method sync_state

A method for synchronizing the internal state of the algorithm. It is called exclusively after a successful replacement in the buffer.

Inputs: None.

Outputs: None.

Operation:

Following data replacement by the buffer class, this method ensures the consistency of the algorithm's structure. It calls the `min_max_calc` method utilizing the updated distances and identifiers, thus updating the reference variables (`min_distance_id`, `max_distance_id`, `max_distance`) in preparation for evaluating the next arrival from the stream.

The pseudocode of the control algorithm is presented below, while the corresponding Python implementation can be found in the Appendix.

```
CLASS MaxRangeReplacement {
    METHOD init (buffer, samples_ids){
        buffer <- buffer
        samples_ids <- samples_ids
        mean_vector <- MEAN(buffer)
        distances <- distance_calc(buffer, mean_vector)
        min_distance_id, max_distance_id, max_distance <- min_max_calc(distances,
        samples_ids)
    }

    METHOD distance_calc (buffer, mean_vector){
```

```

distances <- NORM(buffer - mean_vector)
RETURN distances
}

METHOD min_max_calc (distances, samples_ids){
  min_idx <- IND_MIN(distances)
  max_idx <- IND_MAX(distances)
  min_distance_id <- samples_ids[min_idx]
  max_distance_id <- samples_ids[max_idx]
  max_distance <- distances[max_idx]
  RETURN min_distance_id, max_distance_id, max_distance}
}

METHOD check_new_measurement (new_measurement){
  n_samples <- LENGTH(buffer)
  min_vector <- buffer[min_idx]
  temp_mean_vector <- mean_vector + ((new_measurement - min_vector) /
  n_samples) temp_distances <- distance_calc(buffer, temp_mean_vector)
  new_point_distance <- NORM(new_measurement - temp_mean_vector)
  temp_distances[min_idx] <- new_point_distance
  new_max_distance <- MAX(temp_distances)

  IF(new_max_distance > self.max_distance){
    mean_vector = temp_mean_vector
    distances = temp_distances
    RETURN true, min_distance_index
  }
  ELSE{
    RETURN false, none
  }

  METHOD sync_state (){
    min_distance_id, max_distance_id, max_distance <- min_max_calc(distances,
samples_ids)
  }
}

```

5.3.2 MAHALANOBIS RANGE REPLACEMENT METHOD

This method is an evolution of the Max Range Strategy, replacing the Euclidian distance with the Mahalanobis. The basic principle remains the selective replacement of the points that are located closest to the mean vector, aiming the expansion of the boundaries of the distribution. However, using the Mahalanobis distance has the advantage of taking into consideration the correlation between the variables of the multivariate stream and also the different range of values of each one, by utilizing the inverse covariance matrix.

In more detail, assuming that there are N stored vectors $D = \{D_1, D_2, \dots, D_N\}$ representing the elements of the buffer. For each point D the Mahalanobis distance between the point and the mean vector $\overline{M}_1$ is calculated as follows:

$$d_i = \sqrt{(\overline{D}_i - \overline{M}_1)' * \Sigma^{-1} * (\overline{D}_i - \overline{M}_1)} \quad (80)$$

where Σ^{-1} is the inverse covariance matrix.

Similar to the Euclidian approach, the d_{min} and d_{max} are defined. When a new measurement is received, the scenario of replacing the point with the minimum distance (d_{min}) with the new point is evaluated. The new temporary mean vector $\vec{M}_2$ the new distances, and the new maximum distance (d'_{max}) are calculated.

If the Mahalanobis distance between the new most distant point is greater than the previous one ($d'_{max} > d_{max}$), then the replacement is executed. Otherwise, the new measurement is rejected and the data buffer remains in its previous state.

5.3.2.1 Implementation of Mahalanobis Range Replacement algorithm

The implementation of the specific algorithm was carried out via the `MahalanobisRangeReplacement` class. Following the design philosophy of the system for edge computing environments, the class was designed aiming to keep the memory footprint low, operating in-place and avoiding the creation of temporary buffers.

The basic differentiation with the `MaxRangeReplacement` class is the replacement of the Euclidean distance with the Mahalanobis. For that aim the inverse covariance matrix is embedded, it is calculated once during the initialization phase in the begging of the epoch and passed to the class, thereby avoiding costly recalculations.

Although the creation of new structures is avoided, the evaluation of each new measurement demands the vectorized recalculation of the distances for all stored points (using `distance_calc`), making the algorithmic complexity of the decision equal to $O(N)$, where N is the capacity of the buffer.

The variables of the objects created by `MahalanobisRangeReplacement` are shown in Table 4.

Variable name	Type	Description
buffer	np.ndarray	Reference to the array of stored measurements in the data buffer.
samples_ids	np.ndarray	Reference to the array containing the unique identifiers of the stored measurements.
inv_cov_matrix	np.ndarray	The inverse covariance matrix used for the distance calculation.
mean_vector	np.ndarray	The mean vector of the stored measurements for the current epoch.
distances	np.ndarray	Array storing the calculated Mahalanobis distances of each point from the mean vector.
min_idx	int	The internal array index corresponding to the measurement with the minimum distance from the mean vector (designated for replacement).
min_distance_id	int	The unique identifier (ID) of the measurement with the minimum distance.
max_distance_id	int	The unique identifier (ID) of the measurement with the maximum distance from the mean vector.
max_distance	float	The numerical value of the maximum distance, serving as a comparative threshold for evaluating the dispersion expansion.

Table 4 - MahalanobisRangeReplacement class variables

Method `init`

The class constructor, responsible for initializing the variables of the replacement algorithm based on the current content of the buffer.

Inputs:

- `buffer`: The array containing the currently stored measurements.
- `samples_ids`: The array containing the corresponding unique identifiers of the measurements.
- `inv_cov_matrix`: The inverse covariance matrix.

Outputs: None.

Operation:

Initializes the internal variables of the class, assigning the references for the data buffer, the identifiers, and the inverse covariance matrix. It calculates the initial sample mean (`mean_vector`) in a vectorized manner along the variable axis. Subsequently, it calls the `distance_calc` method to compute the Mahalanobis distances of all points from the mean vector, and the `min_max_calc` method to retrieve the identifiers associated with the minimum and maximum distance, as well as the value of the maximum distance.

Method `distance_calc`

A method for calculating the Mahalanobis distances of the points from a central reference vector.

Inputs:

- `buffer`: The measurements array.
- `mean_vector`: The mean vector.

Outputs:

- `distances`: An array containing the calculated Mahalanobis distances for each point in the data buffer.

Operation:

The method implements the calculation of the Mahalanobis distance iteratively for each sample in the buffer, applying vectorized matrix operations to extract the distance of each respective point from the reference vector.

Method `min_max_calc`

A method for identifying the extreme points of the sample based on their distances from the mean.

Inputs:

- `distances`: The array containing the calculated distances.
- `samples_ids`: The array containing the measurement identifiers.

Outputs:

- `min_distance_id`: The identifier of the measurement with the minimum distance.
- `max_distance_id`: The identifier of the measurement with the maximum distance.
- `max_distance`: The numerical value of the maximum distance.

Operation:

It searches for the indices of the minimum and maximum distance within the `distances` array. Using these indices, it extracts and returns the corresponding unique identifiers from the `samples_ids` array, as well as the numerical value of the maximum distance. Furthermore, it stores the index of the minimum distance (`min_idx`) as an internal variable for direct access during the evaluation of future measurements.

Method `check_new_measurement`

The primary evaluation method, responsible for assessing the suitability of inserting a new measurement based on the cluster expansion criterion.

Inputs:

- `new_measurement`: The new measurement.

Outputs:

- Boolean, int/None: Returns True accompanied by the exact index of the position to be deleted (`min_idx`) if the criterion is satisfied, enabling direct spatial access. Otherwise, it returns False and None.

Operation:

The method implements the core of the Mahalanobis Range Replacement strategy. Initially, it retrieves the vector with the minimum distance, which is designated for replacement. Then, it calculates the new temporary mean and extracts the new temporary distances for all points. The new temporary maximum distance is identified. If it is determined that the dispersion increases, the criterion is satisfied, and the algorithm updates the mean vector and the distance array. Finally, it instructs the buffer to proceed with the replacement by passing the position index, thus ensuring $O(1)$ time complexity during the write operation. Otherwise, the new measurement is rejected.

Method `sync_state`

A method for synchronizing the internal state of the algorithm. It is called exclusively after a successful replacement in the buffer.

Inputs: None.

Outputs: None.

Operation:

Following data replacement by the buffer class, this method ensures the consistency of the algorithm's structure. It calls the `min_max_calc` method utilizing the updated distances and identifiers, thus updating the reference variables (`min_distance_id`, `max_distance_id`, `max_distance`) in preparation for evaluating the next arrival from the stream.

The pseudocode of the control algorithm is presented below, while the corresponding Python implementation can be found in the Appendix.

```
CLASS MahalanobisRangeReplacement {  
    METHOD init (buffer, samples_ids, inv_cov_matrix){  
        buffer <- buffer  
        samples_ids <- samples_ids  
        inv_cov_matrix <- inv_cov_matrix
```

```

    mean_vector <- MEAN(buffer)
    distances <- distance_calc(buffer, mean_vector)
    min_distance_id, max_distance_id, max_distance <- min_max_calc(distances,
samples_ids)
}

METHOD distance_calc (buffer, mean_vector){
  n_samples <- LENGTH(buffer)
  distances <- ZEROS(n_samples)

  FOR i FROM 0 TO n_samples - 1 DO {
    diff <- buffer[i] - mean_vector
    distances[i] <- SQUARE_ROOT(TRANSPPOSE(diff) * inv_cov_matrix * diff)
  }
  RETURN distances
}

METHOD min_max_calc (distances, samples_ids){
  min_idx <- IND_MIN(distances)
  max_idx <- IND_MAX(distances)

  min_distance_id <- samples_ids[min_idx]
  max_distance_id <- samples_ids[max_idx]
  max_distance <- distances[max_idx]

  RETURN min_distance_id, max_distance_id, max_distance
}

METHOD check_new_measurement (new_measurement){
  n_samples <- LENGTH(buffer)
  min_vector <- buffer[min_idx]

  temp_mean_vector <- mean_vector + ((new_measurement - min_vector) / n_samples)
  temp_distances <- distance_calc(buffer, temp_mean_vector)

  diff_new <- new_measurement - temp_mean_vector
  new_point_distance <- SQUARE_ROOT(TRANSPPOSE(diff_new) * inv_cov_matrix *
diff_new)
  temp_distances[min_idx] <- new_point_distance

  new_max_distance <- MAX(temp_distances)

  IF (new_max_distance > self.max_distance) THEN {
    mean_vector <- temp_mean_vector
    distances <- temp_distances
    RETURN True, min_idx
  } ELSE {
    RETURN False, None
  }
}

```

```

}

METHOD sync_state (){
    min_distance_id, max_distance_id, max_distance <- min_max_calc(distances,
samples_ids)
}
}

```

5.3.3 MAX DISTANCE EXPANSION REPLACEMENT METHOD

This method is an alternative strategy based on the distance between the points, focusing on the expansion of the overall spatial boundary of the measurements stored within the buffer. In contrast to the Max Range Strategies which measures distances strictly from the central mean vector, the Max Distance Expansion Replacement Method evaluates the geometric relations among all individual data points by monitoring their pairwise distances.

In more detail, assuming that there are N stored vectors $D = \{D_1, D_2, \dots, D_N\}$ in an R^N space, which represent the stored in the buffer measurements. The algorithm calculates a complete matrix of the pair distances. From this matrix, it determines the absolute maximum distance d_{max} across the entire dataset, as well as the indices of the two points that form the pair with the minimum distance.

When a new measurement is received, it is represented by the vector `new_measurement` and its Euclidian distance from all existing points in the buffer is calculated. The algorithm detects the new maximum distance d_{max}' and the index of the point that produces it. If the new maximum distance is greater than the previous one, the replacement criterion is satisfied. In this case, if the new point forms the maximum distance with one of the points from the pair that formed the minimum distance, the algorithm retains that point and replaces the other. Otherwise, the new measurement is rejected, and the buffer remains in its previous state.

5.3.3.1 Implementation of max distance expansion algorithm

The implementation of this specific algorithm was carried out via the `MaxDistanceExpansionReplacement` class, which was developed with the aim of maintaining a low memory footprint and high computational efficiency, making it suitable for edge computing environments.

To evaluate the feasibility of a replacement upon the arrival of a new measurement, the class avoids the use of temporary buffers, operating in-place to prevent unnecessary dynamic memory allocation and continuous data copying. The management of the internal state is handled by the `sync_state` method, which recalculates the pairwise distance matrix to identify the baseline maximum distance and the pair of nearest points.

Although new temporary structures are not used, the evaluation requires the recalculation of distances for all stored points. This operation has a computational complexity of $O(N)$, where N is the buffer capacity.

The variables of the `MaxDistanceExpansionReplacement` objects are presented in Table 5.

Variable name	Type	Description
buffer	np.ndarray	Reference to the array of stored measurements in the data buffer.
samples_ids	np.ndarray	Reference to the array containing the unique identifiers (IDs) of the stored measurements.

max_distance	float	The numerical value of the maximum pairwise distance, serving as a comparative threshold for evaluating the dispersion expansion.
min_idx_1	int	The first index of the pair with the minimum distance within the buffer.
min_idx_2	int	The second index of the pair with the minimum distance within the buffer.

Table 5 - Variables of MaxDistanceExpansionReplacement class

Method init

The class constructor, responsible for initializing the variables of the replacement algorithm based on the current content of the buffer.

Inputs:

- buffer: The array containing the currently stored measurements.
- samples_ids: The array containing the corresponding unique identifiers of the measurements.

Outputs: None

Operation:

Initializes the internal variables by binding the references to the data buffer and the identifiers, and subsequently calls the sync_state method to calculate the initial pairwise distances, the maximum distance, and the indices of the closest pair (minimum distance pair).

Method distance_calc

A method for calculating the Euclidean distances of all points in the buffer from a reference vector.

Inputs:

- buffer: The array containing the currently stored measurements.
- reference_vector: The reference vector (e.g., the new measurement).

Outputs:

- distances: An array containing the calculated Euclidean distances for each point in the buffer from the reference vector.

Operation:

The method performs a vectorized calculation of the Euclidean norm for all points in the buffer relative to the given reference vector, returning a one-dimensional array of distances.

Method check_new_measurement

The primary evaluation method, responsible for assessing the suitability of inserting a new measurement based on the maximum distance expansion criterion.

Inputs:

- new_measurement: The new incoming measurement.

Outputs:

- Boolean, int/None: Returns True accompanied by the index of the position to be deleted if the criterion is satisfied. Otherwise, it returns False and None.

Operation:

The method implements the core of the max distance expansion strategy. Initially, it calculates the distances of the new measurement from all points in the buffer. It identifies the maximum of these distances. If this exceeds the current maximum distance, the criterion is satisfied. Subsequently, it finds the index of the buffer point that is furthest from the new measurement. If this index coincides with the first index of the minimum pair, the second index is replaced; otherwise, the first index is replaced. The method does not directly update the internal variables (synchronization is handled via `sync_state` after the replacement in the buffer), but it returns the appropriate index. Otherwise, the new measurement is rejected.

Method `sync_state`

A method for synchronizing the internal state of the algorithm. It is called exclusively after a successful replacement in the buffer.

Inputs: None.

Outputs: None.

Operation:

Following data replacement by the buffer class, this method ensures the consistency of the algorithm's structure. It completely recalculates the pairwise distance matrix, ignoring the zero diagonal values, and extracts the new maximum distance as well as the new minimum pair. In this way, the reference variables (`max_distance`, `min_idx_1`, `min_idx_2`) are updated in preparation for evaluating the next arrival from the stream.

The pseudocode of the control algorithm is presented below, while the corresponding Python implementation can be found in the Appendix.

```
CLASS MaxDistanceExpansionReplacement {
  METHOD init (buffer, samples_ids) {
    buffer <- buffer
    samples_ids <- samples_ids
    sync_state()
  }

  METHOD distance_calc (buffer, reference_vector) {
    distances <- NORM(buffer - reference_vector)
    RETURN distances
  }

  METHOD check_new_measurement (new_measurement) {
    temp_distances <- distance_calc(buffer, new_measurement)
    new_max_distance <- MAX(temp_distances)
    new_max_idx <- ARGMAX(temp_distances)

    IF (new_max_distance > max_distance) {
```

```

        IF (new_max_idx == min_idx_1) {
            replace_idx <- min_idx_2
        } ELSE {
            replace_idx <- min_idx_1
        }
        RETURN True, replace_idx
    } ELSE {
        RETURN False, None
    }
}

METHOD sync_state () {
    n <- LENGTH(buffer)
    pairwise_distances <- ZEROS(n, n)
    FOR i FROM 0 TO n-1 {
        FOR j FROM 0 TO n-1 {
            pairwise_distances[i, j] <- NORM(buffer[i] - buffer[j])
        }
    }
    FILL_DIAGONAL(pairwise_distances, 0.0)
    max_distance <- MAX(pairwise_distances)

    current_min_distance <- ∞
    min_idx_1 <- -1
    min_idx_2 <- -1
    FOR i FROM 0 TO n-1 {
        FOR j FROM i+1 TO n-1 {
            IF (pairwise_distances[i, j] < current_min_distance) {
                current_min_distance <- pairwise_distances[i, j]
                min_idx_1 <- i
                min_idx_2 <- j
            }
        }
    }
}
}

```

5.4 SYNOPSIS EXPORT AND STORAGE

Data processing at the network edge concludes with the finalization and export of the data collected in the buffer. When the Vector CUSUM algorithm signals the transition to a new epoch, the system must transform the buffer content into a structured format suitable for cloud storage. For this purpose, the SynopsisExporter class was developed, serving as the link between local processing and the persistent storage of the results.

5.4.1 CLASS IMPLEMENTATION

Following the philosophy of modular design, the SynopsisExporter class was designed to operate autonomously. Its primary mission is the aggregation of the statistical parameters of the stream and the selected samples, ensuring that the transmitted information is complete.

The implementation of the class focuses on transforming the reservoir data into a structured physical CSV file, which constitutes the final product of the processing. A central feature of this process is the dynamic generation of the filename to incorporate the allowance parameter, the threshold, the reservoir capacity, and the execution identifier (`run_id`), thereby ensuring traceability and the correct categorization of the information.

The variables of the `SynopsisExporter` objects are presented in Table 6. It should be noted that the structure strictly retains only the variables necessary for the export process, without allocating resources to store the algorithm's parameters.

<u>Variable name</u>	<u>Type</u>	<u>Description</u>
<code>file_name</code>	<code>str</code>	The name of the generated file (in CSV format), which is dynamically constructed based on the parameters of the reservoir and the control algorithm.
<code>synopsis</code>	<code>list</code>	A data structure for the temporary aggregation and accumulation of Epoch data (measurements and metadata) prior to their final bulk write to the file.

Table 6 - SynopsisExporter class variables

The class implements three core methods, which cover the lifecycle of the data export process.

Method `init`

The class constructor, responsible for preparing the export structure.

Inputs:

- `h_value`: Vector CUSUM algorithm parameter.
- `k_value`: Vector CUSUM algorithm parameter.
- `buffer_size`: The capacity of the buffer.

Outputs: None.

Operation:

Initializes the empty `synopsis` list, which will host the epoch data. Simultaneously, it utilizes the input parameters to dynamically construct and store the export filename (`file_name`), ensuring the traceability of the experimental parameters.

Method `log_epoch`

This method is called every time an epoch is completed.

Inputs:

- `Buffer_data`: The data array of the reservoir.
- `global_indices`: The array containing the global indices of each measurement in the overall data stream.
- `epoch_id`: The epoch identifier.
- `start_index`, `end_index`: The start and end boundaries (indices) of the epoch within the overall stream.

Outputs: None

Operation:

The method iterates row by row through the collected buffer data. For each measurement, it initially rounds the values to two decimal places. Subsequently, it concatenates the epoch metadata, which includes the global measurement index (`measurement_index`), with the multivariate data vector into a single structure that is appended to the synopsis list. Recording the global index was deemed necessary for the subsequent diagnostic verification of the replacement strategy's correct operation.

Method export_results

The final method that implements the system output, creating the physical file.

Inputs:

- `feature_names`: The variable names of the multivariate stream.

Outputs:

- A CSV (Comma-Separated Values) file containing the entirety of the generated synopses.

Operation:

During its execution, it constructs the headers of the generated file, incorporating the metadata fields ("Epoch_ID", "Start_Index", "End_Index", "Measurement_Index") along with the names of the physical variables. Then, it opens the file defined by the `file_name` variable and writes the accumulated data of the synopsis list in bulk. The operation concludes by printing a confirmation message verifying the number of reduced data points successfully exported.

The logic of the class is depicted in the following pseudocode.

```
CLASS SynopsisExporter {
  METHOD init (h_value, k_value, buffer_size){
    file_name <- "synopsis__h_" + h_value + "__k_" + k_value + "__size_" +
buffer_size + ".csv"
    synopsis <- []
  }

  METHOD log_epoch (Buffer_data, global_indices, epoch_id, start_index, end_index){
    FOR EACH (index, row) IN Buffer_data DO{
      rounded_row <- ROUND(row, 2)
      global_index <- global_indices[index]
      row_data <- [epoch_id, start_index, end_index, global_index] + rounded_row
      APPEND row_data TO synopsis
    }
  }

  METHOD export_results (feature_names){
    headers <- ["Epoch_ID", "Start_Index", "End_Index", "Measurement_Index"] +
feature_names
    OPEN file_name FOR WRITING AS file
    WRITE headers TO file
    WRITE synopsis TO file
    CLOSE file
  }
}
```

```
        PRINT "Successfully exported data."
    }
}
```

6. DATA PROCESSING AND DATASET CREATION

The data that will feed the algorithm are measurements from ship sensors. Actual measurements were initially used, but the dataset was largely corrupted, possibly due to extraction errors. Based on data collected from these measurements, a new synthetic dataset was generated using the following methodology:

1. Initially, variables that contribute to the algorithm are selected. Cumulative variables (e.g., the total distance traveled by the ship), time-dependent variables such as the Timestamp, and geographic position variables are discarded. Their removal is necessary because such variables cannot contribute to the vector CUSUM algorithm, as they depend on non-statistical factors and are not correlated with the rest of the variables.
2. Once the target variables are selected, their acceptable boundaries are defined, retaining only the in-bound measurements. From these remaining values, the mean and standard deviation of each variable are calculated. This provides an estimate of the magnitude of their values.
3. The new values for each variable are calculated taking into account the previous value, the mean, and a noise percentage derived from a combination of the standard deviation and the cross-correlation of each variable with the others.

The previous value is included in the calculation to preserve the smoothness of the time series, preventing abrupt and physically impossible variations (e.g., an instantaneous transition from zero to maximum speed between two consecutive measurements).

Noise was added to introduce value variance, which is based on the standard deviation and an empirical cross-correlation factor reflecting the natural interdependencies of the variables. For instance, main engine power and fuel flow were directly linked to speed variations. If the values of each variable were generated autonomously without considering their cross-correlation, many zero values would appear in the covariance matrix, making the application of the vector CUSUM algorithm impossible.

6.1 VARIABLE DESCRIPTION, EVALUATION, AND ANALYSIS

The initial dataset originates from the vessel CMA CGM NERVAL, owned by Danaos Corporation. The vessel's type and technical specifications will assist in the data cleaning process to extract the mean and standard deviation for each variable.

Vessel Type: Cellular Container

Length / Beam: 299,97 meters / 40 meters

Design Draft (Summer): 14,5 meters

Nominal Capacity: 6.552 TEU

Main Engine: MAN B&W 10K98MC-C6

Maximum Continuous Rating - MCR: 57.100 kW @ 104 RPM

Sea Speed: Approximately 25,60 knots

Fuel Consumption (At Sea): Approximately 208.4 Metric Tons (MT) of HFO (Heavy Fuel Oil) per day, at a design draft of 12.02 meters (excluding generators and chillers).

Generators: 4 engines DAIHATSU (2 x 2.210 kW and 2 x 1.920 kW)

Bow Thruster: KAWASAKI CPP 2.000 kW

The variables included in the dataset are divided into categories and analyzed below.

Navigation & Positioning

TIMESTAMP: The time and date of the measurement, recorded every five minutes. This variable exhibits inherent time dependence and, therefore, will not be included in the implementation of the CUSUM algorithm.

LATITUDE / LONGTITUDE: The vessel's geographical position. This is a spatial variable that lacks variance around a stable mean, and is thus discarded.

VESSELHEAD (Heading): The direction the vessel's bow is pointing (0-360°). This variable does not fluctuate around a stable mean and can inherently trigger false alarms in the algorithm. For example, if the vessel transitions from a heading of 359° to 1° (a minor physical turn of 2 degrees), the MCUSUM will incorrectly detect a massive numerical shift. Therefore, this variable is rejected.

TRACKDEGREES (Course Over Ground - COG): The vessel's actual direction relative to the Earth's surface, which differs from the vesselhead due to wind and sea currents. It is discarded for the exact same reasons as the vesselhead variable.

Speed & Distance

SPEEDKNOTS: Corresponds to the vessel's speed as derived from the GPS. Since the GPS output in this context does not represent backward movement as negative speed, the minimum value this parameter can take is 0 (when the vessel is stationary). Based on the vessel's technical specifications, its maximum sea speed reaches 25.60 knots. Assuming favorable sea currents, the practical upper limit for this variable is set to 26 knots. This variable will be utilized in the algorithm.

GROUNDSPPEED (SOG): Corresponds to the vessel's Speed Over Ground. This measurement offers no additional contribution to the algorithm since SPEEDKNOTS is already utilized, and is therefore rejected.

WATERSPEED (STW): Corresponds to the vessel's Speed Through Water, measured by the Speed Log. The difference between SOG and STW determines the effect of currents or tides. Similar to SOG, it is rejected as it does not contribute additional value beyond SPEEDKNOTS.

TRANSVERSEGROUNDSPPEED: Corresponds to the vessel's lateral speed relative to the ground, obtaining non-zero values only during maneuvering, mooring, or anchoring. Since its value is zero for the vast majority of the voyage, the variance of this variable approaches zero. Introducing a variable with near-zero variance renders the covariance matrix singular and, consequently, non-invertible. For this reason, the variable is rejected.

DISPANCESLR (Distance Speed Log Reading): Corresponds to the total distance traveled by the vessel based on the Speed Log. This is a cumulative and continuously increasing variable. If included, it would cause the MCUSUM algorithm to trigger continuous false alarms; thus, it is discarded.

TOTALACCUMULATEDDISTANCE: Corresponds to the total distance traveled based on the GPS recording. It is rejected for the same reason as DISTANCESLR.

Engine & Propulsion

MERPM (Main Engine RPM): Corresponds to the revolutions per minute of the main engine. Based on the vessel's specifications, the upper limit is 104 RPM. The minimum value occurs when the vessel moves astern

(backwards), yielding negative RPMs. Assuming that astern propulsion reaches a maximum of 30% of the forward capacity, the minimum limit is set to -31.20 RPM. This variable is kept.

MEPOWER (Main Engine Power): Corresponds to the main engine's power output in kW. This variable takes only positive values, setting its lower limit to zero. According to the technical specs, the upper limit is 57,100 kW. This variable is kept.

METORQUE (Main Engine Torque): Corresponds to the main engine's torque, which can take both positive (ahead) and negative (astern) values. Because there is a strict mathematical dependence between power output, RPM, and torque, this variable is excluded. Retaining all three variables would lead to a singular and non-invertible covariance matrix, causing a mathematical collapse during the Mahalanobis distance calculation in the MCUSUM algorithm.

Fuel Consumption

MEFOFLOW (Main Engine Fuel Oil Flow): Corresponds to the instantaneous fuel consumption of the main engine. While theoretically useful, the measurement units are undocumented in the dataset, making it impossible to reliably estimate its upper limit. Consequently, it was excluded from the CUSUM calculations.

MEFOTOTALVOLUME: Corresponds to the cumulative fuel volume consumed by the main engine. Being a continuously increasing cumulative variable, it is rejected to prevent continuous false alarms.

GEFOFLOW (Generator Engine Fuel Oil Flow): Corresponds to the instantaneous fuel consumption of the auxiliary engines. This faces the same missing-units issue as MEFOFLOW. Furthermore, auxiliary engines are inactive during the majority of the voyage, which would lead to near-zero variance and, consequently, a singular covariance matrix. It is discarded.

GEFOTOTALVOLUME: The cumulative fuel consumed by the auxiliary engines. Rejected for being a cumulative variable.

Steering & Maneuvering

COMMANDEDRUDDERANGLE: The desired rudder angle requested by the autopilot or the bridge operator (0-360°). Similar to VESSELHEAD, it suffers from circularity (modulo 360) and lacks variance around a stable mean. A transition from 359° to 1° would incorrectly trigger a large shift in the CUSUM distance. Thus, it is rejected.

RUDDERANGLE: The actual physical angle of the rudder blade. Like COMMANDEDRUDDERANGLE, it lacks variance around a constant mean since its value depends exclusively on autopilot commands and course alterations. It is not a statically stable variable. If included, MCUSUM would be unable to distinguish between a sharp rudder movement due to a mechanical fault and a standard maneuver, leading to false alarms. It is discarded.

Vessel Motion (Inclinometers)

INCLINOMETERXMAX / INCLINOMETERXMIN / INCLINOMETERYMAX / INCLINOMETERYMIN: These correspond to the vessel's maximum and minimum tilt (roll/pitch) along the x and y axes within a five-minute measurement window. These are crucial as they correlate heavily with the weather conditions encountered.

Under normal operations, the upper and lower limits are reasonably assumed to be 45° and -45°, respectively.

INCLINOMETERXZC / INCLINOMETERYZC (Zero-Crossing): Corresponds to the number of times the vessel passed through the vertical (upright) position within the five-minute window. This provides valuable insights into sea state and weather. The minimum value is zero (vessel at rest). Assuming a worst-case scenario of a full oscillation cycle taking ten seconds, the vessel crosses the vertical position twice per cycle. Therefore, in a 300-second window, the vessel would complete a maximum of 30 cycles, resulting in $30 \times 2 = 60$ crossings. This is defined as the upper limit.

Environment & Hull

DEPTH: The sea depth beneath the vessel's keel. A sudden change in depth (e.g., from 1000 to 1200 meters) is a statistical shift that the algorithm would flag as a state change, even though the vessel's operational performance remains virtually identical. Since MCUSUM must focus on meaningful changes affecting operational status, including this parameter would merely introduce functional noise. It is rejected.

DEPTHOFFSET: The fixed distance from the depth sensor to the vessel's waterline. This is a static calibration constant. It offers no dynamic information regarding sailing conditions, and its inclusion would provide zero statistical contribution to change detection. It is rejected.

WINDANGLE: Wind direction. This is a circular metric (0-360°). As previously explained, the numerical transition from 359° to 1° creates an artificial statistical jump. Additionally, wind direction lacks practical analytical value unless combined as a vector with the vessel's heading. It is rejected.

WINDSPEED: The wind speed acting on the vessel. Unlike direction, speed is not circular and directly expresses environmental impact. However, the measurement units are undocumented in the dataset. Moreover, the wind's effect is indirectly captured by the inclinometer variables. Therefore, this variable is rejected.

6.2 DATASET CREATION

The variables that remained after the evaluation and filtering process are the following:

<u>Variable</u>	<u>Description</u>	<u>Lower limit</u>	<u>Upper limit</u>
SPEEDKNOTS	Vessel speed via GPS.	0 nodes	26 nodes
MERPM	Main engine revolutions per minute (RPM).	-31.20 RPM	104 RPM
MEPOWER	Main engine power output.	0 kW	57.100 kW
INCLINOMETERXMAX	Maximum x-axis tilt within the five-minute window.	-45°	45°
INCLINOMETERXMIN	Minimum x-axis tilt within the five-minute window.	-45°	45°
INCLINOMETERYMAX	Maximum y-axis tilt within the five-minute window.	-45°	45°
INCLINOMETERYMIN	Minimum y-axis tilt within the five-minute window.	-45°	45°
INCLINOMETERXZC	Number of zero-crossings on the x-axis.	0 crossings	60 crossings
INCLINOMETERYZC	Number of zero-crossings on the y-axis.	0 crossings	60 crossings

Table 7 - Chosen variables

The variables INCLINOMETERXMAX – INCLINOMETERXMIN and INCLINOMETERYMAX – INCLINOMETERYMIN will be combined to form range variables in order to prohibit including of highly correlated variables in the covariance matrix, thereby ensuring the numerical stability of the algorithm. So, two new variables will be created, RANGEX and RANGEY which represent the tilt range of the ship. These variables combined with the number of zero-crossings per axe, fully capture changes in weather conditions.

The combined use of the range and zero-crossing variables allows the algorithm to distinguish between various sea states. While the range depicts the intensity of the strain, the zero-crossings indicate the frequency of the oscillation. The synergy of these two metrics gives a comprehensive picture of the vessel's hydrodynamic behavior, whether the rough seas cause slow or rapid oscillations.

<u>Variable</u>	<u>Mean</u>	<u>Standard deviation</u>
SPEEDKNOTS	5,74	5,20
MERPM	17,29	22,22
MEPOWER	3.268,67	5.553,74
RANGEX	12,44	15,05
RANGEY	12,14	8,95
INCLINOMETERXZC	10,79	14,21
INCLINOMETERYZC	6,56	9,27

Table 8 - Nean and standard deviation of the chosen variables

The mean and the standard deviation will be used for the creation of the synthetic dataset. For the calculation of each value the previous measurement, the mean, the standard deviation and the cross-correlation between the variables will be taken into consideration. Thus, the value n of the variable x results from the following equation:

$$x_n = \varphi \cdot x_{n-1} + (1 - \varphi) \cdot \bar{x} + e_n \quad (81)$$

This equation constitutes a simplified version of the AR(1) model, discussed in the Section 3.2.2.5, adapted for the synthrtic dataset creation. In a real-world system, variables influence each other over, which would require the calculation of a more complex multivariate model (Vector Autoregression - VAR). Here, a common inertia coefficient ($\varphi = 0.95$) is used, for simplification reasons. In practice, the coefficient φ determines the weight of the previous measurement and the theoretical mean in the calculation of each new value.

However, to achieve a realistic simulation of the vessel's different operational states and to introduce conceptual shifts that define the various Epochs in the data stream, the theoretical mean does not remain constant. In contrast, it dynamically changes at specific intervals within the 25000 measurements dataset in order to simulate different sailing conditions. Furthermore, aiming for a stricter evaluation of the algorithm's sensitivity, successive smaller-scale variations (sub-epochs) have been embedded within the main epochs, to experimentally investigate which of these become detectable. More specifically, the simulation is divided into the following states:

- Rows 0 – 499: Initial normal operational state (based on the initial mean values).
- Rows 500 – 999: Port conditions (zero speed, RPM, and power output, with near-zero tilt).
- Rows 1000 – 1999: Voyage initiation (speed of 14 knots, proportional RPM/power, and moderate tilt).
- Rows 2000 – 3999: Storm conditions with increased power output and severe tilt. Phased shifts occur at rows 3000, 3400, and 3700.

- Rows 4000 – 7999: Calm sea conditions, returning to low tilt and normal power output. Phased shifts occur at rows 6000, 6800, and 7500.
- Rows 8000 – 15999: High-speed cruising state. Phased shifts occur at rows 11000, 12000, 13000, 14000, 14800, 15400, and 15800.
- Rows 16000 – 25000: Severe storm conditions with reduced speed and severe tilt. Phased shifts occur at rows 18000, 19500, 21000, 22500, and 24000.

It must be emphasized that the epoch transitions and internal variations do not occur at fixed or periodic intervals (e.g., every 1,000 rows). Instead, they are deliberately introduced at irregular points to more accurately simulate an unpredictable marine environment.

The e_n factor expresses the noise added to each value based on the covariance matrix which is calculated from the standard deviation and the correlation matrix of the under-examination variables. This, ensures the physical and mechanical consistency of the vessel's behavior at each time step.

The noise follows a Multivariate Normal Distribution with mean vector equal to the zero vector. This means that the noise is not generated separately, but combines the standard deviation of each variable with its correlation with the others. The implementation utilizes the method `numpy.random.multivariate_normal` of the NumPy library in Python. This method takes as input the mean vector which in this case is equal to zero and the covariance matrix. As it is already mentioned in the Section 3.2.2.5 the relationship between the variance of the noise and the variance of the values x_n is the following:

$$\sigma_{x_n}^2 = \frac{\sigma_e^2}{1 - \varphi^2} \quad (82)$$

Consequently, by applying the above relation, the covariance matrix of the noise results from the multiplication of the variables' covariance matrix by the factor $(1 - \varphi^2)$. The covariance matrix of the variables is automatically created from the multiplication of the standard deviations shown in Table 8 by the correlation matrix we have defined (Table 9).

To ensure that the generated data maintain their internal cohesion within the allowable operational boundaries, a stability factor is also introduced. This factor applies restrictively to the covariance matrix, limiting the range of noise fluctuations. In this way, unrealistic, abrupt changes in sensor values are prevented, while the correlations between the variables remain intact. Ultimately, the covariance matrix of the noise is calculated as follows:

$$\Sigma_e = (1 - \varphi^2) \cdot (\text{stability_factor} \cdot \Sigma_x) \quad (83)$$

The correlation matrix:

	SP/TS	MERPM	MEPOWER	RANGEX	RANGEY	IN/TERXZC	IN/TERYZC
SP/TS	1,00	0,85	0,80	-0,25	-0,30	-0,15	-0,25
MERPM	0,85	1,00	0,95	0,15	0,20	0,10	0,15
MEPOWER	0,80	0,95	1,00	0,20	0,25	0,15	0,20
RANGEX	-0,25	0,15	0,20	1,00	0,60	0,70	0,40
RANGEY	-0,30	0,20	0,25	0,60	1,00	0,40	0,70
IN/TERXZC	-0,15	0,10	0,15	0,70	0,40	1,00	0,50
IN/TERYZC	-0,25	0,15	0,20	0,40	0,70	0,50	1,00

Table 9 - Correlation matrix

Regarding the assumption of the multivariate normal distribution, mentioned in Section 3.2.2.3, within the context of this experimental evaluation, it is satisfied by design in the synthetic data. Since the observation noise is generated using the multivariate normal distribution, the first differences, being a linear combination of multivariate normal variables, also follow a multivariate normal distribution. Although the use of differences retains a minimal residual autocorrelation due to the approximation of the inertia coefficient, this choice deliberately subjects the algorithm to a more adverse evaluation condition. The fact that the statistical assumption holds and the algorithm responds with high performance even under this stricter scenario directly benefits the evaluation, confirming its robustness.

At the same time, even in the case of real ship telemetry measurements, where mild deviations from normality or residual correlations may occur, the application of the algorithm remains statistically consistent. As documented in the literature (Crosier, 1988), the Vector CUSUM algorithm exhibits high robustness against mild violations of normality and residual noise. The recursive accumulation process of CUSUM effectively filters out local random fluctuations, ensuring that the control limits are triggered only when a systematic, structural shift occurs in the process mean vector.

As a final step in the generation process, and to ensure the engineering validity of the simulation, the constraints mentioned in the previous chapter are applied. This ensures that the measurements under no circumstances exceed the defined minimum and maximum operational limits. Additionally, to faithfully represent the operation of real telemetry sensors, all generated values are rounded to two decimal places before their final storage.

Finally, the application of the algorithm was not limited to a single dataset instead, ten statistically independent and structurally equivalent datasets were created. This procedure is implemented through the iterative execution of the generation algorithm. In each run, the model parameters (epoch boundaries, mean vectors, covariance matrix) are kept constant, while only the initial random seed of the noise generator is varied, using the `numpy.random.seed()` function. Setting fixed seed values ensures that the generated noise data are fully controllable and reproducible. The ten generated files are automatically saved in a separate folder to be used for the evaluation of the algorithm.

7. ALGORITHM EXECUTION AND SYNOPSIS GENERATION

The first step in executing the algorithm and generating the synopses is selecting appropriate values for the allowance (k), the decision interval (h), and the reservoir size. To establish an order of magnitude for the selected values, an initial exploration was conducted via a Monte Carlo simulation, during which the theoretical, in-control Average Run Length (ARL_0) was calculated for various parameter combinations.

Specifically, a Python script was developed to generate an idealized Multivariate Normal Distribution dataset, utilizing the `numpy.random.multivariate_normal` function. The mean vector of the distribution was defined as the average of the seven variables, as calculated in the previous chapter, while the covariance matrix was reconstructed based on the standard deviations and the correlation matrix.

During the simulation, allowance values ranging from 0.5 to 5.0 with a step of 0.5, and decision interval values from 5.0 to 20.0 with a step of 1.0 were examined. For each possible parameter combination, the number of samples required until the CUSUM algorithm triggered an alarm for the first time due to statistical noise was recorded. To ensure statistical reliability, each combination was executed 50 times, and ultimately the average of these runs was calculated to derive the ARL_0 .

The goal is for the ARL_0 value to be balanced: a very low value indicates oversensitivity, leading to continuous false alarms and unjustified epoch changes. Conversely, a very high value renders the algorithm inefficient, as it will only be able to detect extreme distributional shifts, ignoring smaller ones. For this reason, an early termination mechanism was integrated into the simulation algorithm. If a combination yielded an ARL_0 greater than or equal to 500, the process did not continue for larger decision interval values at the same allowance. The results table is presented below.

<u>Allowance (k)</u>	<u>Decision interval (h)</u>	<u>Theoretical ARL_0</u>
0,5	5,0	8,68
0,5	6,0	13,78
0,5	7,0	23,88
0,5	8,0	36,76
0,5	9,0	54,44
0,5	10,0	90,56
0,5	11,0	147,14
0,5	12,0	207,76
0,5	13,0	350,24
0,5	14,0	706,86
Decision interval exceeded (>500). Stopping h-search for k=0.5		
1,0	5,0	42,66
1,0	6,0	128,94
1,0	7,0	333,06
1,0	8,0	1314,56
Decision interval exceeded (>500). Stopping h-search for k=1.0		
1,5	5,0	751,14
Decision interval exceeded (>500). Stopping h-search for k=1.5		
2,0	5,0	61.167,00
Decision interval exceeded (>500). Stopping h-search for k=2.0		

Table 10 – Theoretical ARL_0 results

It is evident that there is no merit in continuing the simulation for values of $k > 2$. Based on the above, the parameter pairs (k, h) for which the algorithm will be executed are: (0.5, 10.0), (0.5, 11.0), (0.5, 12.0), (0.5, 13.0), (0.5, 14.0), (0.5, 15.0), (1.0, 6.0), (1.0, 7.0), (1.0, 8.0), and (1.0, 9.0). Among the selected pairs, the lowest resulting ARL_0 is 90.56. Considering that the ship's measurements are theoretically recorded every 5 minutes, this ARL_0 translates to one theoretical false alarm approximately every 450 minutes of operation. Consequently, under the most sensitive algorithm setting, about three false alarms per day are expected.

Regarding the size of the reservoir (buffer), given that the covariance matrix will be calculated from the values stored within it, its capacity must necessarily be strictly greater than the number of variables. Otherwise, the matrix becomes singular, making its inversion—and thus the operation of the algorithm—impossible.

With this in mind, the minimum size is set to 50 samples. At the same time, to maintain execution speed and optimally manage the available computational resources, the maximum size is set to 200 samples. The system's evaluation will be conducted by examining intermediate capacities with a step of 50 samples.

8. RESULTS EVALUATION METHOD

The application of the algorithm will be performed on the ten synthetic datasets across various sampling methods, buffer sizes, decision intervals, and reference values. Each combination produces a synopsis recording the measurements stored in the buffer during the algorithm's execution, the global index of each measurement, the epoch identifier, and its limits. The generated synopses will be compared with the original dataset to evaluate the different parameter values and sampling methods.

The evaluation is based on three axes. The first axis concerns the sampling process. The replacement methods employed were developed based on the rationale of capturing the system's variability; therefore, the information extracted from the extreme values of the dataset is considered the most critical. Consequently, the goal of these methods is to retain the values located at the limits of the distribution to accurately depict the data boundaries and measure the dispersion. It should be noted that in a real-world IoT environment—especially on a vessel where sensors are subjected to numerous and diverse types of stress—it is common for individual measurements to be erroneous and out of bounds. Thus, under normal conditions, prior to applying the sampling methods, there must be a pre-processing and error-filtering stage where measurements are evaluated, and out-of-bounds values are either rejected or subjected to special handling. Examining these cases is beyond the scope of this thesis, given that its purpose is not to present a comprehensive sensor error detection and correction system. Instead, it aims to comparatively evaluate various replacement methods and the h and k parameters to draw conclusions regarding the proposed architecture's ability to compress data while preserving its envelope and the system's overall variability.

The second axis of the evaluation addresses the compression rate, namely the percentage of data reduction compared to the original dataset. This metric is particularly crucial for IoT applications, where available bandwidth and storage capacity often act as limiting factors. However, the compression rate is not evaluated in isolation, but in conjunction with the quality of the retained data. A high compression rate is desirable only if the selected samples successfully capture the variability and critical characteristics of the stream. Otherwise, excessive data reduction can lead to the loss of vital information, thereby degrading the value of the analysis in the Cloud. For this reason, a composite evaluation score will be defined, integrating the first two axes simultaneously. This score will function as a single quantitative measure of the overall effectiveness of each strategy, allowing for a direct and objective comparison between the different approaches.

The third evaluation axis focuses on the effectiveness of stream change detection. It examines the degree to which the vector CUSUM algorithm successfully and promptly detects epoch changes in the multivariate signal. This evaluation is performed by comparing the points at which the algorithm signals a change with the actual change points, which are known from the generation of the synthetic data. In this way, metrics such as detection delay and the number of false alarms are quantified for each parameter combination, enabling the identification of optimal settings.

8.1 EVALUATION OF REPLACEMENT METHODS

The evaluation of the three replacement methods will be based on their ability to describe the system's variability. Specifically, the extent to which extreme values are included in the final synopses will be examined, as well as the degree to which the overall dispersion of the measurements is represented. For this purpose, two distinct techniques will be applied: the volume comparison method for evaluating extreme values, and the variance comparison method for estimating the overall variance.

The comparison will be performed on a per-epoch basis; that is, for each epoch signaled by the Vector CUSUM algorithm, a comparison is conducted between the measurements included in the synopses and the initial measurements of the dataset. Thus, an individual score per epoch is produced for each metric, while

the overall score of the synopsis is calculated from the average of these individual scores. The final score is formed by combining the performance of the two metrics.

8.1.1 VOLUME COMPARISON METHOD

Assuming that there are N points $D = \{D_1, D_2, \dots, D_N\}$ in an n -dimensional space R^n . For each dimension $i \in \{1, 2, \dots, n\}$, the maximum value D_{i_max} and the minimum value D_{i_min} are defined. The volume of the hyper-rectangle (bounding box) that encloses all the points is calculated by the product of the individual ranges:

$$v = \prod_{i=1}^n (D_{i_max} - D_{i_min}) \quad (84)$$

This product determines the total volume of the bounded region that includes the entire dataset. It is noted that this value constitutes an upper bound, as the volume of the hyper-rectangle is a superset and usually larger than the actual volume of the shape formed by the points in space.

The above definition will be used to compare the measurements stored in the synopsis against the original dataset. We consider $e \in Z^+$ the number of epochs resulting from the execution of the vector CUSUM algorithm. For an epoch i , the volume of the hyper-rectangle enclosing the measurements of the original dataset is denoted as $V_{dataset,i}$, while the corresponding volume for the measurements of the synopsis is denoted as $V_{synopsis,i}$. The ratio of the two volumes for epoch i is defined as:

$$R_i = \frac{V_{synopsis,i}}{V_{dataset,i}} \quad (85)$$

This fraction quantifies the degree to which the extreme values of the original dataset are included in the synopsis of each epoch. Special care is required when the volume of the original dataset becomes zero, i.e., when $V_{dataset,i} = 0$. This occurs when, in a given epoch, at least one dimension has a zero range, meaning the corresponding variable remains constant throughout the entire epoch. In this case, the hyper-rectangle degenerates to a zero volume, and the ratio R_i is not directly defined by simple division.

To address this issue, the ratio R_i is defined on a case-by-case basis as follows:

$$R_i = \begin{cases} 1, & \text{if } V_{dataset,i} = 0 \text{ and } V_{synopsis,i} = 0, \\ \text{NaN}, & \text{if } V_{dataset,i} = 0 \text{ and } V_{synopsis,i} \neq 0, \\ \frac{V_{synopsis,i}}{V_{dataset,i}}, & \text{if } V_{dataset,i} \neq 0. \end{cases} \quad (86)$$

In the first case, both the original set and the synopsis exhibit zero volume; therefore, the synopsis preserves the volume property, and we consider the ratio to be equal to one. In the second case, the synopsis introduces dispersion in a dimension where the original set was constant, an impossible event that can only be due to an error. In the experiments of this thesis, whenever a zero volume appeared in the original set, it simultaneously appeared in the synopsis as well hence, undefined ratios did not occur, and the arithmetic mean was calculated normally. The third and general case corresponds to a non-zero volume of the original set, in which case the ratio is calculated by simple division.

The overall evaluation of the synopsis for the entire set of epochs is calculated as the arithmetic mean of the individual ratios:

$$R = \frac{1}{e} \cdot \sum_{i=1}^e R_i \quad (87)$$

8.1.2 GENERALIZED VARIANCE COMPARISON METHOD

The covariance matrix, as defined above, captures the dependencies among the variables of a dataset. However, it cannot be directly used to compare dispersion because it is not expressed as a single scalar value. To address this limitation, the concept of generalized variance is used, which is defined as the determinant of the covariance matrix ($|\Sigma|$). Geometrically, the generalized variance is interpreted as the multivariate volume corresponding to the matrix Σ and indicates the overall variability of the data (Karlis, 2003).

Let $e \in \mathbb{Z}^+$ be the number of epochs resulting from the execution of the Vector CUSUM algorithm. For an epoch i , the covariance matrix of the original dataset is denoted as $\Sigma_{dataset,i}$, while the corresponding matrix for the synopsis measurements is denoted as $\Sigma_{synopsis,i}$. Respectively, the generalized variances are $|\Sigma_{dataset,i}|$ and $|\Sigma_{synopsis,i}|$. Since the synopsis contains a smaller number of measurements than the original set, the goal is for its generalized variance to be as large as possible relative to the generalized variance of the original set. The ratio of the two generalized variances for epoch i is defined as:

$$S_i = \frac{|\Sigma_{synopsis,i}|}{|\Sigma_{dataset,i}|} \quad (88)$$

This ratio quantifies the degree to which the overall dispersion of the original set is expressed in the synopsis of the respective epoch. Special care is required when the generalized variance of the original set becomes zero, i.e., when $|\Sigma_{dataset,i}| = 0$. This occurs when the covariance matrix is singular, an event typically caused by the presence of a constant variable, linear dependence among the variables, or an insufficient number of samples within the specific epoch. In this case, the ratio S_i is not directly defined.

To address this issue, the ratio S_i is defined on a case-by-case basis as follows:

$$S_i = \begin{cases} 1, & \text{if } |\Sigma_{dataset,i}| = 0 \text{ and } |\Sigma_{synopsis,i}| = 0, \\ \text{NaN}, & \text{if } |\Sigma_{dataset,i}| = 0 \text{ and } |\Sigma_{synopsis,i}| \neq 0, \\ \frac{|\Sigma_{synopsis,i}|}{|\Sigma_{dataset,i}|}, & \text{if } |\Sigma_{dataset,i}| \neq 0 \end{cases} \quad (89)$$

In the first case, both the original set and the synopsis exhibit zero generalized variance. Therefore, the synopsis preserves the degenerate nature of the multivariate dispersion, and the ratio is set to one. In the second case, the synopsis introduces multivariate dispersion where the original set was degenerate, an impossible event that can only be due to an error. In the experiments of this thesis, whenever zero generalized variance was observed in the original set, it was simultaneously observed in the synopsis; thus, undefined ratios did not occur, and the average was calculated normally. The third and general case corresponds to a non-zero generalized variance of the original set, in which case the ratio is calculated by simple division.

The overall evaluation of the synopsis for all epochs is calculated as the arithmetic mean of the individual ratios:

$$S = \frac{1}{e} \cdot \sum_{i=1}^e S_i \quad (90)$$

Since the index S can take values greater than one, min-max normalization was applied to its set of values so that it falls within the same $[0,1]$ scale as the other indices. This normalization preserves the ranking order of the methods but transforms the index into a relative comparative measure. The normalized generalized variance index is denoted by S' .

8.2 COMPRESSION RATE EVALUATION

The data compression rate is one of the most important evaluation metrics for the proposed method, as it reflects one of the algorithm's two main objectives: reducing the volume of data sent to the cloud while simultaneously preserving as much of the available information as possible. The compression rate is defined as follows:

$$C = 1 - \frac{M}{N} \quad (91)$$

where N is the total number of measurements in the original data stream and M is the total number of elements in the respective synopsis. The compression rate takes values in the interval $[0,1]$. Values close to one indicate a significant reduction in data volume and, consequently, a high compression rate. Conversely, values close to zero indicate that the volume of the original stream has been minimally reduced.

8.3 CHANGE DETECTION EVALUATION

The evaluation of the Vector CUSUM algorithm is based on a modified version of the PATE-F1 method by Ghorbani et al. (2024). This specific technique constitutes an adaptation of the original evaluation framework for models that directly generate binary predictions, given that in the case of the Vector CUSUM, predictions are discrete time points rather than intervals.

The method relies on the use of temporal tolerance zones combined with proximity weighting. More specifically, an alarm is considered correct if it is triggered within a predetermined time window from the onset of an actual change in the data stream. However, its weight is dynamically adjusted based on its distance from the event.

Applying this technique significantly mitigates the performance overestimation of change detection algorithms. This is achieved because an alarm recorded between two changes is deemed correct only if it falls within the acceptable tolerance zones, thereby rejecting random alarms as false. An additional advantage of this method is its integration of response speed. Consequently, alongside evaluating the correctness of the detection, it simultaneously assesses whether the algorithm promptly identifies the transition to a new epoch.

Let x_t ($1 \leq t \leq T$) denote the observations, $A = \{a_1, \dots, a_N\}$ be the set of anomaly events where the individual event a_k occurs at time t_k ($1 \leq t_k \leq T$), and $P = \{p_1, \dots, p_M\}$ be the set of predictions where the individual prediction p_l occurs at time t_l ($1 \leq t_l \leq T$). Regarding the predictions, we define:

True Detection (True Positive – TP): The subset of predictions made within a temporal tolerance zone from an anomaly event. This specific time interval is equal to one hour, which corresponds to 12 measurements, given that measurements occur every five minutes. The size of this window was selected so as to remain strictly smaller than the minimum buffer capacity, preventing the extensive statistical contamination of the

synopsis with data from the subsequent epoch. These specific predictions are referred to as True Positives (TP).

Outside (False Positive – FP): The subset of predictions recorded outside the tolerance zone. These specific predictions are referred to as False Positives (FP).

Regarding the anomaly events, the following are defined:

Missed anomalies (False Negative – FN): Events for which no alarm was signaled within the temporal tolerance zone. These specific events are referred to as False Negatives (FN).

The calculation of weights per case is performed as follows:

True Detection: The true detection of an anomaly contributes to both the true positive weight (w_{TP}) and the false positive weight (w_{FP}). The w_{TP} is calculated to express how rapidly a change was detected. Assume that the temporal tolerance interval equals d , the event a_k occurs at time t_k , and the prediction p_l occurs at time t_l ($t_k \leq t_l \leq t_k + d$). The true positive weight is calculated as follows:

$$w_{TP} = 1 - \frac{t_l - t_k}{d} \quad (92)$$

It is obvious from the above formula that if the anomaly is detected instantaneously, the weight equals one. At the same time, as the distance from the event increases, the probability of having a false positive prediction increases. The weight of the false positive prediction is calculated as the complement of the true positive weight, namely:

$$w_{FP} = 1 - w_{TP} \quad (93)$$

Outside: An out-of-bounds prediction contributes to the false positive weight (w_{FP}). More specifically, in the case of a false positive alarm, the weight of the false positive prediction equals:

$$w_{FP} = 1 \quad (94)$$

Missed anomalies: An event that has not been detected by the algorithm contributes to the false negative weight (w_{FN}). More specifically, in the event that an anomaly is not detected, the weight of the false negative prediction equals:

$$w_{FN} = 1 \quad (95)$$

Once the weights have been calculated, precision and recall can be computed. Precision measures the reliability of the algorithm's alarms and is calculated as the ratio of correctly identified anomalies to the total number of points the algorithm classified as anomalies. Within the PATE framework, instead of the number of predictions, the corresponding weights are used. Therefore:

$$Precision = \frac{\sum_{t=1}^T w_{TP}}{\sum_{t=1}^T w_{FP} + \sum_{t=1}^T w_{TP}} \quad (96)$$

Recall evaluates the method's ability to detect actual anomalies and is calculated as the ratio of correctly identified anomalies to the total number of actual anomalies present in the data. Within the PATE framework, instead of the number of predictions, the corresponding weights are used. Therefore:

$$Recall = \frac{\sum_{t=1}^T w_{TP}}{\sum_{t=1}^T w_{FN} + \sum_{t=1}^T w_{TP}} \quad (97)$$

Once precision and recall are calculated, the F1-Score can be computed, which is the harmonic mean of these two values:

$$F1 - Score_d = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (98)$$

Finally, the method calculates a weighted score assuming that there may be more than one temporal tolerance interval (buffer zone). Based on the method described, the temporal tolerance interval (buffer zone) is invariant, therefore:

$$PATE - F1 = F1 - Score_d \quad (99)$$

8.4 COMPOSITE EVALUATION METRICS

The evaluation of a multivariate data reduction system cannot rely exclusively on individual metrics, as the specific objectives of the architecture are inherently competing. Specifically, the extreme maximization of the compression rate inevitably leads to the degradation of the statistical fidelity of the data retained in the buffer. At the same time, it must be emphasized that the core research challenge of this thesis is not simple anomaly detection, but rather efficient data reduction aimed at effective Cloud storage.

In this context, the vector CUSUM serves an auxiliary role as a mechanism that determines when an epoch concludes and when the generated synopsis must be flushed from the buffer to the Cloud. If the system fails to compress the data efficiently, or if the synopsis alters the statistical characteristics of the original data stream, the utility of the described method is practically nullified, regardless of the response speed of the detection algorithm.

For this reason, the first two metrics defined below (Sampling Quality and Statistical Efficiency) are considered the most important, as they capture the primary essence of the research: the minimization of data volume while simultaneously preserving its internal structure. The third metric merely provides a supplementary, overall view of the system. In order to draw reliable conclusions without the arbitrary use of weighting factors, three general, composite metrics are introduced, which are calculated via the simple arithmetic mean.

8.4.1 SAMPLING QUALITY SCORE - SQ

The first index consolidates the individual metrics that evaluate the ability of the selected sampling method to preserve the statistical characteristics of the original data stream within the buffer. It is derived as the simple arithmetic mean of the Volume Ratio (R) and the Normalized Generalized Variance Ratio (S'), expressing how well the synopsis retains the natural data envelope of the extreme values and the overall multivariate dispersion. The SQ score is defined by the following equation:

$$SQ = \frac{R + S'}{2} \quad (100)$$

8.4.2 STATISTICAL EFFICIENCY SCORE - SE

The second composite metric incorporates the Volume Ratio (R), the Normalized Generalized Variance Ratio (S'), and the Compression Rate (C). As previously mentioned, this metric captures the core of the research effort, as it quantifies the node's ability to drastically reduce the data volume sent to the cloud while simultaneously maintaining high statistical accuracy. The SE score is calculated as the arithmetic mean of the three quantities:

$$SE = \frac{C + R + S'}{3} \quad (101)$$

8.4.3 TOTAL PERFORMANCE SCORE - TP

The third and final metric provides a comprehensive evaluation of the architecture, equally integrating compression, sampling, and the performance of the CUSUM algorithm in the early detection of changes. The detection component is quantified through the PATE-F1 score, which evaluates when an epoch is correctly delineated. The total score TP is defined as the average of the four metrics:

$$TP = \frac{C + R + S' + PATE - F1}{4} \quad (102)$$

This specific score enables the final comparative evaluation of different parameterization scenarios, taking into account the entirety of the system's mechanisms.

8.5 IMPLEMENTATION OF THE EVALUATION ALGORITHM

To evaluate the results, Python code was developed to implement all the metrics described in the previous subsections. The code is organized into four distinct evaluation classes and a central orchestration class that coordinates the overall process.

The central class `evaluator.py` serves as the entry point of the evaluation system. Initially, the input file paths are defined; these include the synthetic datasets located in the `synthetic_data` folder and the synopses generated by executing the algorithm for each sampling method, which are stored in the `mcusum_outputs` folder. The code sequentially processes the three sampling methods for each of the ten data streams. For every parameter combination—namely, for each decision interval h , allowance k , and buffer size—the synopsis files are extracted and evaluated.

The `VolumeComparisonEvaluator` class implements the volume comparison method. Specifically, for each epoch, the volume of the hyper-rectangle enclosing the points is calculated for both the original dataset and the synopsis. Subsequently, the ratio R_i of the two volumes is calculated. Special care is taken in the event of a zero volume in the original set, where the ratio is set equal to one provided that the synopsis also exhibits a zero volume. The overall score R is derived as the average of the individual ratios across all epochs.

The `GeneralizedVarianceEvaluator` class implements the generalized variance comparison method. For each epoch, the covariance matrix is calculated for both the original dataset and the synopsis, and then their determinant—which constitutes the generalized variance—is computed. The ratio S_i is determined by the quotient of the two generalized variances; in the case of a zero determinant, the same rule described in the volume comparison is applied: the ratio is set to one if the generalized variance is zero for both the original set and the synopsis. The overall index S is derived as the average of the individual ratios.

To evaluate the compression rate, the `CompressionRatioEvaluator` class calculates the ratio C according to Equation 91, taking into account the total number of measurements in the original stream N and the number of elements in the synopsis M .

To evaluate change detection, the `PateF1Evaluator` class implements the modified PATE-F1 method. Upon initialization, it extracts the timestamps of the alarms from the synopsis, excluding the mandatory final buffer flush at the end of the stream, as this does not constitute a change detection. For each actual change point t_k , the temporally first alarm t_l that falls within the tolerance zone d (which is defined as equal to 12 measurements) is identified. If such an alarm exists, the true positive weight w_{TP} is calculated based on the detection delay, while the false positive weight w_{FP} is derived as its complement. This specific alarm is immediately removed from the list of available alarms to ensure that each alarm is mapped to a single change and to prevent double counting. Conversely, if no alarm is detected within the tolerance zone, the event is considered missed and full weight is added to the w_{FN} index. After processing all actual changes, any remaining alarms are considered false positives and contribute with full weight to w_{FP} . Concurrently, the class records both the weighted indices and the absolute numbers of true and false positive alarms. The total weighted values are used to compute precision and recall; their harmonic mean yields the F1-Score, which constitutes the final PATE-F1 score.

Finally, the central orchestration class combines the individual metrics by calculating the three composite evaluation scores. The Sampling Quality Score (SQ) is derived as the average of the R and S indices. The Basic Statistical Efficiency Score (SE) is derived as the average of the C , R , and S indices. The Total Performance Score (TP) is derived as the average of the C , R , S , and PATE-F1 indices. All scores are calculated using the simple arithmetic mean, ensuring the equal participation of the individual metrics.

Upon completing the processing of all combinations, the results are aggregated into a central `DataFrame`. Subsequently, the averages of the metrics are calculated for each combination of sampling method, decision interval h , allowance k , and buffer size, by taking the mean performance across all ten synthetic data streams. The final aggregated result is saved to an Excel file. Concurrently, instances where zero volumes or zero generalized variances were observed are logged in a separate file, facilitating the identification of potential edge cases.

9. RESULTS AND EVALUATION

For the evaluation of the method, ten different synthetic and statistically independent datasets were created, on which the algorithm was executed using various parameter combinations. In total, 120 parameter combinations were tested, detailed as follows:

<u>Replacement method</u>	<u>Allowance k</u>	<u>Decision interval h</u>	<u>Buffer size</u>
Max Range Replacement	0,5	10,0	50
Max Range Replacement	0,5	11,0	50
Max Range Replacement	0,5	12,0	50
Max Range Replacement	0,5	13,0	50
Max Range Replacement	0,5	14,0	50
Max Range Replacement	0,5	15,0	50
Max Range Replacement	1,0	6,0	50
Max Range Replacement	1,0	7,0	50
Max Range Replacement	1,0	8,0	50
Max Range Replacement	1,0	9,0	50
Max Range Replacement	0,5	10,0	100
Max Range Replacement	0,5	11,0	100
Max Range Replacement	0,5	12,0	100
Max Range Replacement	0,5	13,0	100
Max Range Replacement	0,5	14,0	100
Max Range Replacement	0,5	15,0	100
Max Range Replacement	1,0	6,0	100
Max Range Replacement	1,0	7,0	100
Max Range Replacement	1,0	8,0	100
Max Range Replacement	1,0	9,0	100
Max Range Replacement	0,5	10,0	150
Max Range Replacement	0,5	11,0	150
Max Range Replacement	0,5	12,0	150
Max Range Replacement	0,5	13,0	150
Max Range Replacement	0,5	14,0	150
Max Range Replacement	0,5	15,0	150
Max Range Replacement	1,0	6,0	150
Max Range Replacement	1,0	7,0	150
Max Range Replacement	1,0	8,0	150
Max Range Replacement	1,0	9,0	150
Max Range Replacement	0,5	10,0	200
Max Range Replacement	0,5	11,0	200
Max Range Replacement	0,5	12,0	200
Max Range Replacement	0,5	13,0	200
Max Range Replacement	0,5	14,0	200
Max Range Replacement	0,5	15,0	200
Max Range Replacement	1,0	6,0	200
Max Range Replacement	1,0	7,0	200
Max Range Replacement	1,0	8,0	200
Max Range Replacement	1,0	9,0	200
Mahalanobis Range Replacement	0,5	10,0	50

Mahalanobis Range Replacement	0,5	11,0	50
Mahalanobis Range Replacement	0,5	12,0	50
Mahalanobis Range Replacement	0,5	13,0	50
Mahalanobis Range Replacement	0,5	14,0	50
Mahalanobis Range Replacement	0,5	15,0	50
Mahalanobis Range Replacement	1,0	6,0	50
Mahalanobis Range Replacement	1,0	7,0	50
Mahalanobis Range Replacement	1,0	8,0	50
Mahalanobis Range Replacement	1,0	9,0	50
Mahalanobis Range Replacement	0,5	10,0	100
Mahalanobis Range Replacement	0,5	11,0	100
Mahalanobis Range Replacement	0,5	12,0	100
Mahalanobis Range Replacement	0,5	13,0	100
Mahalanobis Range Replacement	0,5	14,0	100
Mahalanobis Range Replacement	0,5	15,0	100
Mahalanobis Range Replacement	1,0	6,0	100
Mahalanobis Range Replacement	1,0	7,0	100
Mahalanobis Range Replacement	1,0	8,0	100
Mahalanobis Range Replacement	1,0	9,0	100
Mahalanobis Range Replacement	0,5	10,0	150
Mahalanobis Range Replacement	0,5	11,0	150
Mahalanobis Range Replacement	0,5	12,0	150
Mahalanobis Range Replacement	0,5	13,0	150
Mahalanobis Range Replacement	0,5	14,0	150
Mahalanobis Range Replacement	0,5	15,0	150
Mahalanobis Range Replacement	1,0	6,0	150
Mahalanobis Range Replacement	1,0	7,0	150
Mahalanobis Range Replacement	1,0	8,0	150
Mahalanobis Range Replacement	1,0	9,0	150
Mahalanobis Range Replacement	0,5	10,0	200
Mahalanobis Range Replacement	0,5	11,0	200
Mahalanobis Range Replacement	0,5	12,0	200
Mahalanobis Range Replacement	0,5	13,0	200
Mahalanobis Range Replacement	0,5	14,0	200
Mahalanobis Range Replacement	0,5	15,0	200
Mahalanobis Range Replacement	1,0	6,0	200
Mahalanobis Range Replacement	1,0	7,0	200
Mahalanobis Range Replacement	1,0	8,0	200
Mahalanobis Range Replacement	1,0	9,0	200
Max Distance Expansion Replacement	0,5	10,0	50
Max Distance Expansion Replacement	0,5	11,0	50
Max Distance Expansion Replacement	0,5	12,0	50
Max Distance Expansion Replacement	0,5	13,0	50
Max Distance Expansion Replacement	0,5	14,0	50
Max Distance Expansion Replacement	0,5	15,0	50
Max Distance Expansion Replacement	1,0	6,0	50
Max Distance Expansion Replacement	1,0	7,0	50

Max Distance Expansion Replacement	1,0	8,0	50
Max Distance Expansion Replacement	1,0	9,0	50
Max Distance Expansion Replacement	0,5	10,0	100
Max Distance Expansion Replacement	0,5	11,0	100
Max Distance Expansion Replacement	0,5	12,0	100
Max Distance Expansion Replacement	0,5	13,0	100
Max Distance Expansion Replacement	0,5	14,0	100
Max Distance Expansion Replacement	0,5	15,0	100
Max Distance Expansion Replacement	1,0	6,0	100
Max Distance Expansion Replacement	1,0	7,0	100
Max Distance Expansion Replacement	1,0	8,0	100
Max Distance Expansion Replacement	1,0	9,0	100
Max Distance Expansion Replacement	0,5	10,0	150
Max Distance Expansion Replacement	0,5	11,0	150
Max Distance Expansion Replacement	0,5	12,0	150
Max Distance Expansion Replacement	0,5	13,0	150
Max Distance Expansion Replacement	0,5	14,0	150
Max Distance Expansion Replacement	0,5	15,0	150
Max Distance Expansion Replacement	1,0	6,0	150
Max Distance Expansion Replacement	1,0	7,0	150
Max Distance Expansion Replacement	1,0	8,0	150
Max Distance Expansion Replacement	1,0	9,0	150
Max Distance Expansion Replacement	0,5	10,0	200
Max Distance Expansion Replacement	0,5	11,0	200
Max Distance Expansion Replacement	0,5	12,0	200
Max Distance Expansion Replacement	0,5	13,0	200
Max Distance Expansion Replacement	0,5	14,0	200
Max Distance Expansion Replacement	0,5	15,0	200
Max Distance Expansion Replacement	1,0	6,0	200
Max Distance Expansion Replacement	1,0	7,0	200
Max Distance Expansion Replacement	1,0	8,0	200
Max Distance Expansion Replacement	1,0	9,0	200

Table 11 - Parameter combinations for the algorithm implementation

As detailed in the previous chapters, the replacement methods were selected to preserve the overall data envelope so that the extreme values of the stream are accurately captured. The allowance and decision interval values were chosen based on the simulation results of the CUSUM algorithm on multivariate normal distribution data, from which the ARL_0 was calculated to estimate the magnitude of the parameters. Finally, the buffer size was chosen considering that it must be larger than the number of variables, with the objective of maintaining execution speed and optimizing the management of computational resources, given that available resources in real-world applications can be limited. The evaluation metrics of the algorithm are briefly presented in the following table.

Index	Symbol	Description
Volume Comparison Index	R	Evaluates the degree of volume/envelope preservation of the data between the original stream and the synopsis.
Normalized Generalized Variance Comparison Index	S'	Evaluates whether the synopsis preserves the overall multivariate dispersion of the original stream.
Compression Ratio	C	Expresses the reduction in the number of transmitted data points compared to the original stream.
PATE-F1	-	Evaluates the system's ability to detect actual changes in the stream and correctly delineate the epochs.
Sampling Quality Index	SQ	Combines the R and S' indices and reflects the statistical quality of the synopsis.
Statistical Efficiency Index	SE	Combines the compression and the preservation of the statistical characteristics of the original stream.
Total Performance Index	TP	Provides an overall evaluation of the architecture, taking into account compression, statistical fidelity, and change detection.

Table 12 - Evaluation metrics

9.1 REPLACEMENT METHODS

As previously discussed, the replacement methods consist of three approaches: the Max Range Replacement Strategy, the Mahalanobis Range Replacement Strategy, and the Max Distance Expansion Replacement Method. Two metrics are employed for their evaluation: the volume comparison index R and the generalized variance index S .

9.1.1 VOLUME COMPARISON INDEX R

The Volume Comparison Index R takes values between zero and one. A value close to one means that the synopsis retains almost the entire range of the initial dataset, indicating high statistical fidelity in capturing the statistical envelope of the data stream. Conversely, a value close to zero implies that the synopsis has significantly shrunk the multivariate volume, rejecting extreme values. The resulting index R for each combination is presented in the following table.

Replacement method	Decision interval h	Allowance k	Buffer size	Mean of R
Mahalanobis Range Replacement	6	1	50	0,690
Mahalanobis Range Replacement	6	1	100	0,709
Mahalanobis Range Replacement	6	1	150	0,742
Mahalanobis Range Replacement	6	1	200	0,755
Mahalanobis Range Replacement	7	1	50	0,593
Mahalanobis Range Replacement	7	1	100	0,619
Mahalanobis Range Replacement	7	1	150	0,643
Mahalanobis Range Replacement	7	1	200	0,689
Mahalanobis Range Replacement	8	1	50	0,538
Mahalanobis Range Replacement	8	1	100	0,582
Mahalanobis Range Replacement	8	1	150	0,616
Mahalanobis Range Replacement	8	1	200	0,659
Mahalanobis Range Replacement	9	1	50	0,51
Mahalanobis Range Replacement	9	1	100	0,535
Mahalanobis Range Replacement	9	1	150	0,619

Mahalanobis Range Replacement	9	1	200	0,670
Mahalanobis Range Replacement	10	0,5	50	0,663
Mahalanobis Range Replacement	10	0,5	100	0,661
Mahalanobis Range Replacement	10	0,5	150	0,707
Mahalanobis Range Replacement	10	0,5	200	0,723
Mahalanobis Range Replacement	11	0,5	50	0,619
Mahalanobis Range Replacement	11	0,5	100	0,618
Mahalanobis Range Replacement	11	0,5	150	0,669
Mahalanobis Range Replacement	11	0,5	200	0,675
Mahalanobis Range Replacement	12	0,5	50	0,579
Mahalanobis Range Replacement	12	0,5	100	0,615
Mahalanobis Range Replacement	12	0,5	150	0,654
Mahalanobis Range Replacement	12	0,5	200	0,666
Mahalanobis Range Replacement	13	0,5	50	0,565
Mahalanobis Range Replacement	13	0,5	100	0,605
Mahalanobis Range Replacement	13	0,5	150	0,616
Mahalanobis Range Replacement	13	0,5	200	0,654
Mahalanobis Range Replacement	14	0,5	50	0,545
Mahalanobis Range Replacement	14	0,5	100	0,596
Mahalanobis Range Replacement	14	0,5	150	0,609
Mahalanobis Range Replacement	14	0,5	200	0,640
Mahalanobis Range Replacement	15	0,5	50	0,533
Mahalanobis Range Replacement	15	0,5	100	0,595
Mahalanobis Range Replacement	15	0,5	150	0,598
Mahalanobis Range Replacement	15	0,5	200	0,633
Max Distance Expansion Replacement	6	1	50	0,544
Max Distance Expansion Replacement	6	1	100	0,581
Max Distance Expansion Replacement	6	1	150	0,618
Max Distance Expansion Replacement	6	1	200	0,658
Max Distance Expansion Replacement	7	1	50	0,458
Max Distance Expansion Replacement	7	1	100	0,492
Max Distance Expansion Replacement	7	1	150	0,510
Max Distance Expansion Replacement	7	1	200	0,550
Max Distance Expansion Replacement	8	1	50	0,423
Max Distance Expansion Replacement	8	1	100	0,453
Max Distance Expansion Replacement	8	1	150	0,463
Max Distance Expansion Replacement	8	1	200	0,518
Max Distance Expansion Replacement	9	1	50	0,402
Max Distance Expansion Replacement	9	1	100	0,415
Max Distance Expansion Replacement	9	1	150	0,460
Max Distance Expansion Replacement	9	1	200	0,527
Max Distance Expansion Replacement	10	0,5	50	0,521
Max Distance Expansion Replacement	10	0,5	100	0,536
Max Distance Expansion Replacement	10	0,5	150	0,579
Max Distance Expansion Replacement	10	0,5	200	0,613
Max Distance Expansion Replacement	11	0,5	50	0,475
Max Distance Expansion Replacement	11	0,5	100	0,503

Max Distance Expansion Replacement	11	0,5	150	0,537
Max Distance Expansion Replacement	11	0,5	200	0,552
Max Distance Expansion Replacement	12	0,5	50	0,449
Max Distance Expansion Replacement	12	0,5	100	0,495
Max Distance Expansion Replacement	12	0,5	150	0,523
Max Distance Expansion Replacement	12	0,5	200	0,529
Max Distance Expansion Replacement	13	0,5	50	0,431
Max Distance Expansion Replacement	13	0,5	100	0,497
Max Distance Expansion Replacement	13	0,5	150	0,484
Max Distance Expansion Replacement	13	0,5	200	0,510
Max Distance Expansion Replacement	14	0,5	50	0,413
Max Distance Expansion Replacement	14	0,5	100	0,472
Max Distance Expansion Replacement	14	0,5	150	0,469
Max Distance Expansion Replacement	14	0,5	200	0,493
Max Distance Expansion Replacement	15	0,5	50	0,415
Max Distance Expansion Replacement	15	0,5	100	0,474
Max Distance Expansion Replacement	15	0,5	150	0,457
Max Distance Expansion Replacement	15	0,5	200	0,488
Max Range Replacement	6	1	50	0,626
Max Range Replacement	6	1	100	0,675
Max Range Replacement	6	1	150	0,708
Max Range Replacement	6	1	200	0,741
Max Range Replacement	7	1	50	0,548
Max Range Replacement	7	1	100	0,577
Max Range Replacement	7	1	150	0,596
Max Range Replacement	7	1	200	0,643
Max Range Replacement	8	1	50	0,498
Max Range Replacement	8	1	100	0,530
Max Range Replacement	8	1	150	0,552
Max Range Replacement	8	1	200	0,619
Max Range Replacement	9	1	50	0,476
Max Range Replacement	9	1	100	0,491
Max Range Replacement	9	1	150	0,549
Max Range Replacement	9	1	200	0,618
Max Range Replacement	10	0,5	50	0,606
Max Range Replacement	10	0,5	100	0,628
Max Range Replacement	10	0,5	150	0,663
Max Range Replacement	10	0,5	200	0,708
Max Range Replacement	11	0,5	50	0,555
Max Range Replacement	11	0,5	100	0,587
Max Range Replacement	11	0,5	150	0,619
Max Range Replacement	11	0,5	200	0,658
Max Range Replacement	12	0,5	50	0,531
Max Range Replacement	12	0,5	100	0,581
Max Range Replacement	12	0,5	150	0,601
Max Range Replacement	12	0,5	200	0,627
Max Range Replacement	13	0,5	50	0,504

Max Range Replacement	13	0,5	100	0,559
Max Range Replacement	13	0,5	150	0,578
Max Range Replacement	13	0,5	200	0,620
Max Range Replacement	14	0,5	50	0,485
Max Range Replacement	14	0,5	100	0,543
Max Range Replacement	14	0,5	150	0,567
Max Range Replacement	14	0,5	200	0,596
Max Range Replacement	15	0,5	50	0,486
Max Range Replacement	15	0,5	100	0,549
Max Range Replacement	15	0,5	150	0,566
Max Range Replacement	15	0,5	200	0,596

Table 13 - Analytic results for R index

Since evaluating the results from the above table is difficult due to the large volume of information, each parameter will initially be examined in isolation. It should be highlighted that the decision interval and allowance values are evaluated together, as different allowance values are combined with various decision intervals. The following tables present the mean, standard deviation, maximum, and minimum values for each parameter.

<u>Replacement method</u>	<u>Mean of R</u>	<u>Standard deviation of R</u>	<u>Minimum value of R</u>	<u>Maximum Value of R</u>
Mahalanobis Range Replacement	0,630	0,058	0,510	0,755
Max Range Replacement	0,587	0,064	0,476	0,741
Max Distance Expansion Replacement	0,500	0,059	0,402	0,658

Table 14 - Index R results for each replacement method

<u>Combination</u>	<u>Mean of R</u>	<u>Standard deviation of R</u>	<u>Minimum value of R</u>	<u>Maximum Value of R</u>
k=0,5 / h=10	0,634	0,066	0,521	0,723
k=0,5 / h=11	0,589	0,065	0,475	0,675
k=0,5 / h=12	0,571	0,066	0,449	0,666
k=0,5 / h=13	0,552	0,067	0,431	0,654
k=0,5 / h=14	0,536	0,069	0,413	0,640
k=0,5 / h=15	0,533	0,068	0,415	0,633
k=1,0 / h=6	0,671	0,067	0,544	0,755
k=1,0 / h=7	0,577	0,068	0,458	0,689
k=1,0 / h=8	0,538	0,072	0,423	0,659
k=1,0 / h=9	0,523	0,082	0,402	0,670

Table 15 - Index R results for each allowance and decision interval combination

<u>Buffer size</u>	<u>Mean of R</u>	<u>Standard deviation of R</u>	<u>Minimum value of R</u>	<u>Maximum Value of R</u>
50	0,523	0,075	0,402	0,690
100	0,559	0,069	0,415	0,709
150	0,586	0,076	0,457	0,742
200	0,621	0,073	0,488	0,755

Table 16 - Index R results for each buffer size

Regarding the combinations of the decision interval and allowance, it is observed that the optimal values occur for an allowance equal to one ($k = 1$), and furthermore, for a constant allowance, as the decision interval increases, the index R decreases. Although this may seem contradictory, it occurs because the value $k = 1$ was evaluated alongside lower decision intervals ($h = 6,7,8,9$), whereas the value $k = 0.5$ was combined with higher values ($h = 10,11,12,13,14,15$). Given that the decision interval h determines the sensitivity of the CUSUM algorithm and the expected epoch length, smaller values lead to more frequent alarms and, consequently, shorter epochs. During a shorter epoch, the measurements of the initial dataset exhibit a smaller range of values per dimension, as the time spent in a particular state is limited. Under these conditions, the buffer capacity covers a larger portion of the total volume, leading to an increase in the index R . The following graph depicts the relationship between the $h - k$ combinations and R .

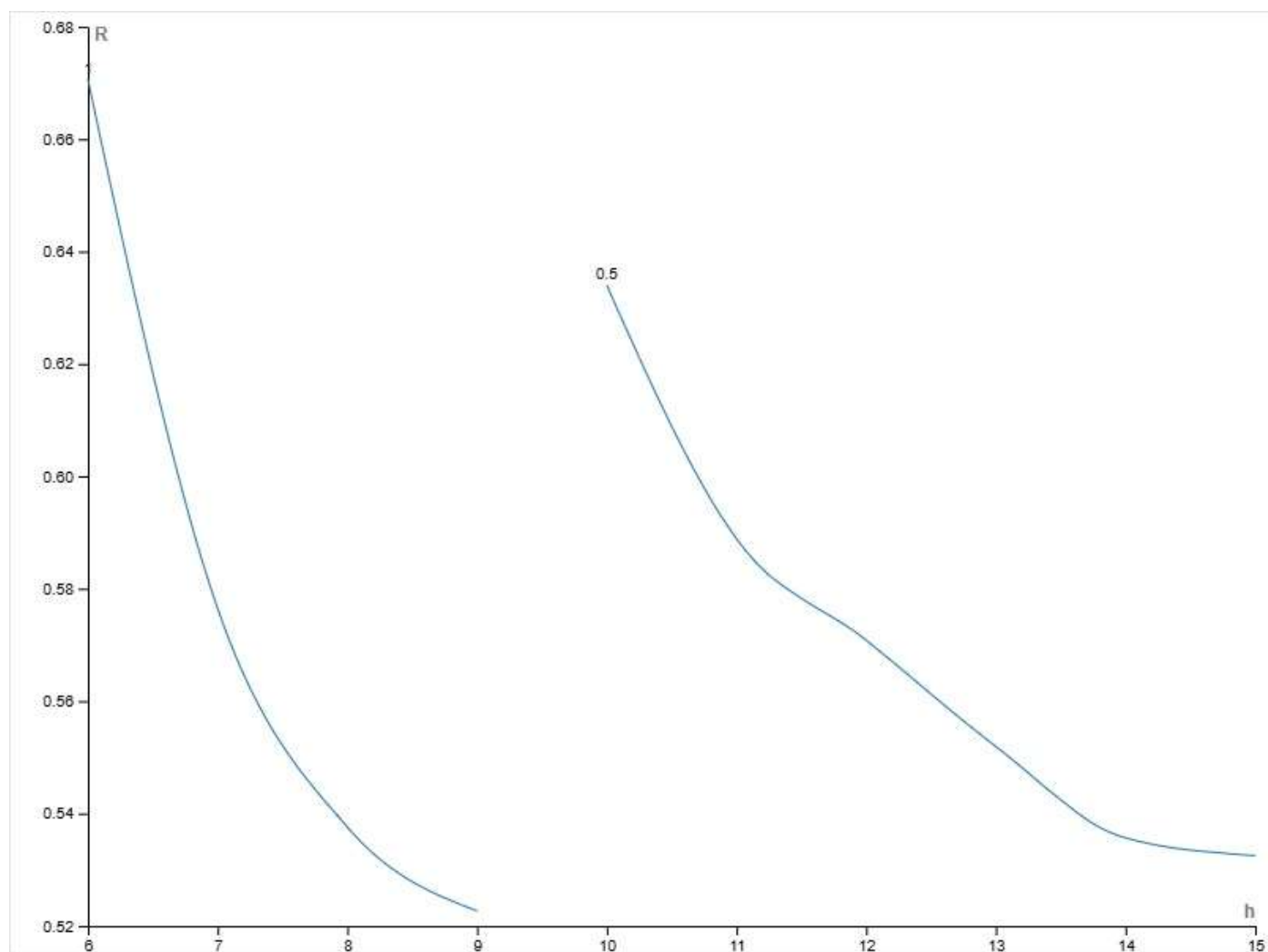


Figure 12 - Variation of index R as a function of the decision interval h for allowance values k^{10}

The strongly decreasing, non-linear relationship between the decision interval h and the index R is clearly depicted in the above graph, where two distinct falling curves can be observed. One is steeper for the range $h = 6$ to 9 where the allowance is $k = 1$, and the other is milder for the range $h = 10$ to 15 with an allowance of $k = 0.5$.

¹⁰ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

Regarding the relationship between the index R and the buffer size, as expected, larger buffer sizes result in higher R values. This increase is fully consistent with the system's operation, as a larger reservoir capacity allows for the storage of more samples from each stream, thereby increasing the probability of capturing the extreme values of the distribution.

The following boxplot presents the distribution of R values per buffer size. The graph depicts the median (the horizontal line inside the box) and the interquartile range (IQR), which contains the central 50% of the observations (blue box). The extension lines (whiskers) do not indicate the absolute extremes of the sample rather, they extend to the most extreme observation that falls within the inner statistical fences, which are mathematically defined as $1.5 \times IQR$ below the first quartile (Q_1) and above the third quartile (Q_3), respectively. Values exceeding these limits are plotted as individual points, clearly highlighting the extreme observations of the distribution.

As shown, the median value of R steadily increases as the buffer size grows, from 0.526 for a buffer size of 50, up to 0.630 for a size of 200. Simultaneously, both the total range (the distance between the maximum and minimum values) and the dispersion around the median (interquartile range) exhibit low variability. Regarding the presence of extreme values, high stability is recorded, with the sole exception of buffer size 150, where a single upper extreme value is located at 0.738. Finally, a slight asymmetry in the distributions is worth noting, as the median is not perfectly centered within the respective interquartile range.

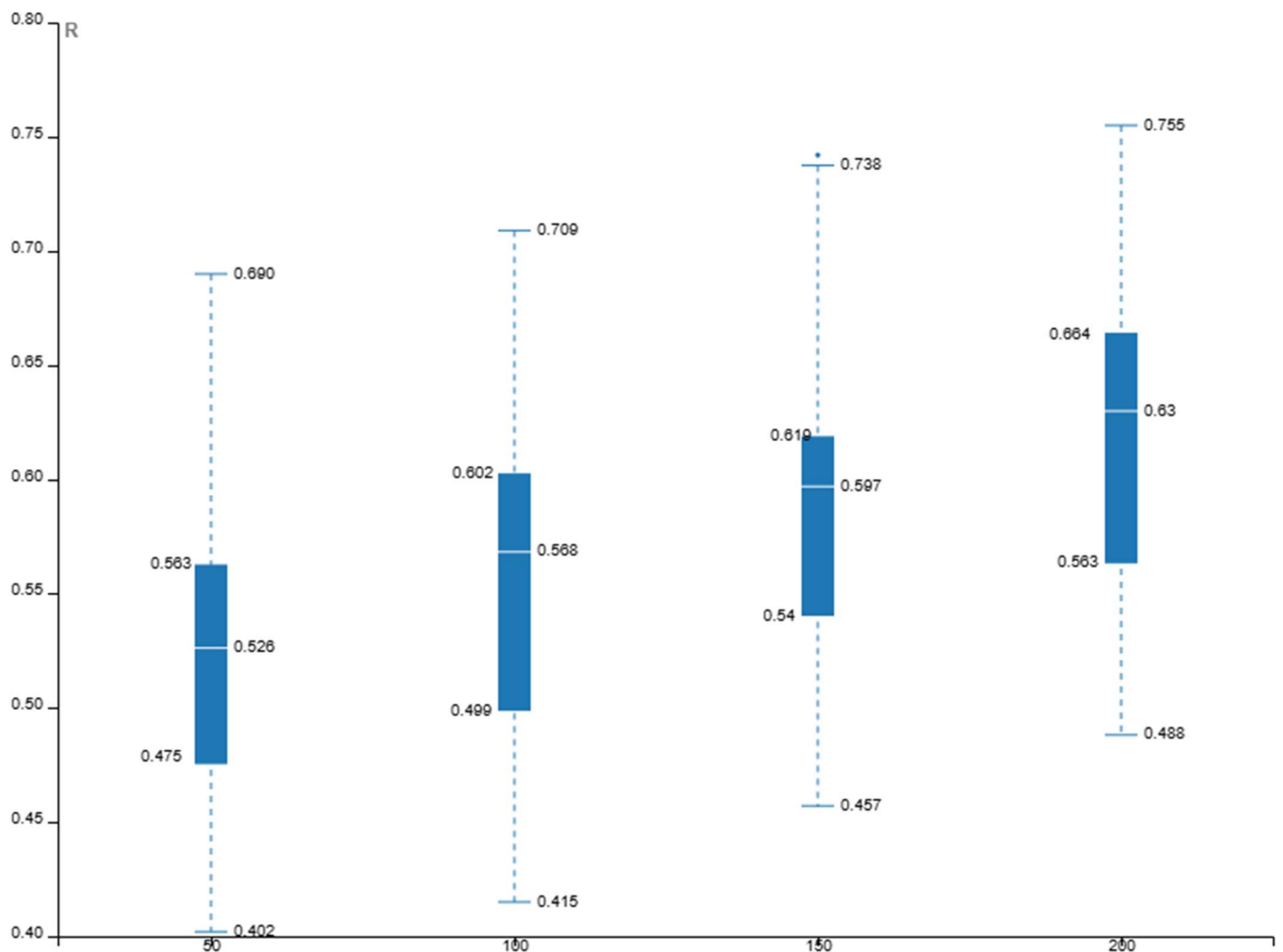


Figure 13 - Boxplot of index R as a function of the buffer size¹¹

Regarding the replacement methods, as shown in the corresponding table, the Mahalanobis Range Replacement strategy exhibits the best performance in terms of the mean, maximum, and minimum values. It is followed by the Max Range Replacement Method, which uses the Euclidean distance, and finally the Max Distance Expansion Replacement Method. The maximum range replacement methods aim to increase the multivariate range of the values relative to the center; this is why they achieve better results for this index compared to the Max Distance Expansion Replacement Method, which aims to maximize the distance among the points themselves. Furthermore, using the Mahalanobis distance yields better results than the Euclidean distance because it takes into account the correlations between the variables and their different measurement scales. In contrast, the Euclidean distance treats all dimensions equally and is sensitive to absolute values.

The low standard deviation values indicate that the index R exhibits relative stability and does not present extreme fluctuations, regardless of the selected method. The Mahalanobis Range Replacement Method achieves the best (lowest) value, indicating greater stability, followed by the Max Distance Expansion Replacement Method and the Max Range Replacement Method (using Euclidean distance). Notably, among all the evaluated parameters, the replacement method is the one that presents the lowest standard deviation. It is characteristic that even the maximum standard deviation value, observed in the maximum Max Range

¹¹ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

Replacement Method ($\sigma = 0.064$), is smaller than the values of the other parameters. This indicates that categorizing the data based on the replacement method produces the most homogeneous data subsets.

The above findings are also depicted in the following graph, which illustrates the relationship between the index R and the buffer capacity for each replacement method. As shown, for all strategies without exception, an increase in buffer capacity leads to an improvement in the index R . Furthermore, the Mahalanobis Range Replacement Method clearly outperforms the others across every evaluated buffer capacity.

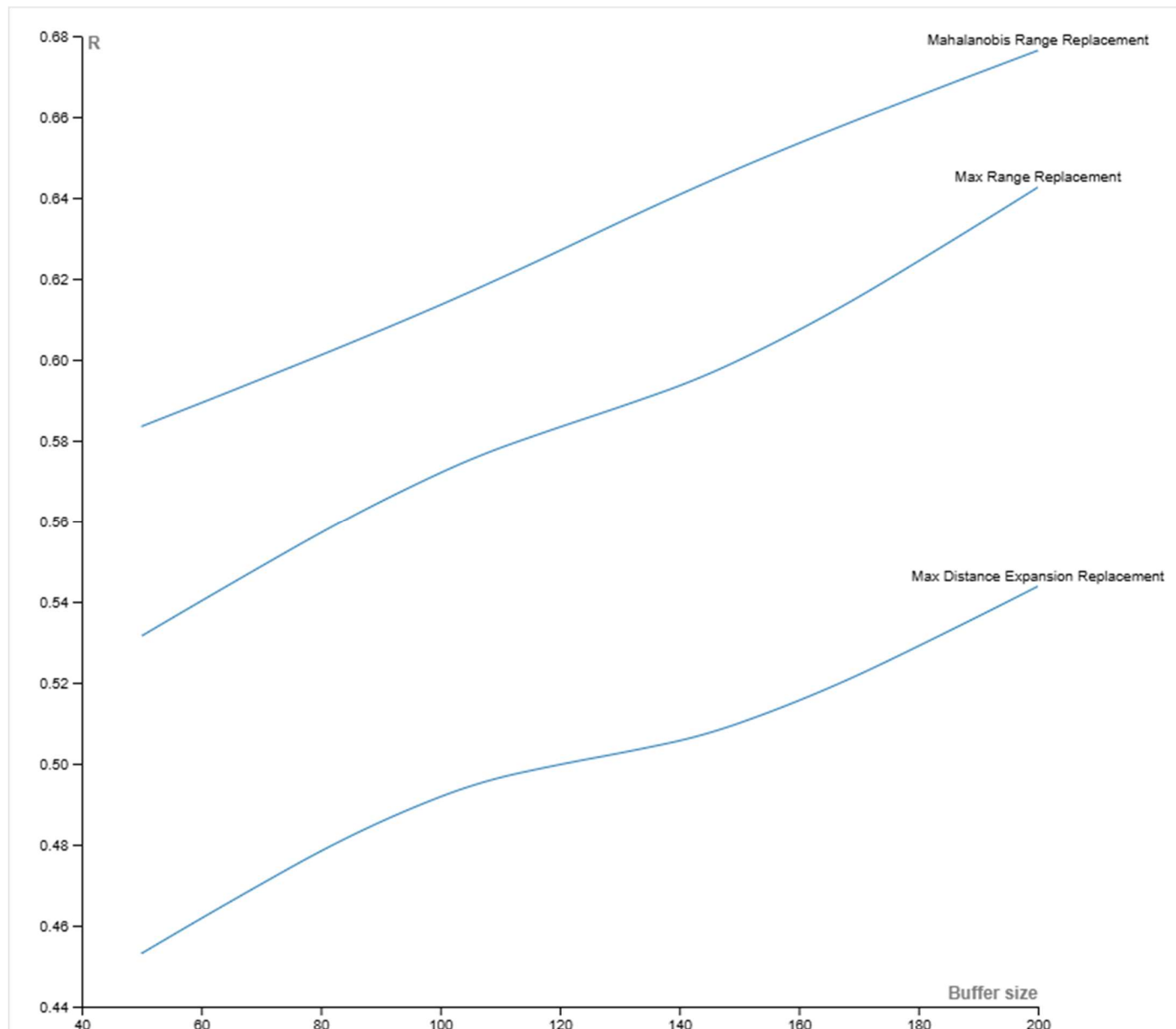


Figure 14 - Variation of index R as a function of buffer size and replacement method¹²

¹² The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

9.1.2 GENERALIZED VARIANCE COMPARISON INDEX S

The Generalized Variance Comparison Index S is unbounded. For this reason, and to facilitate comparisons among different parameters as well as with the other indices, the application of normalization is imperative. Specifically, the values of S will be transformed using the Min-Max method, utilizing the total range of values obtained from the experimental measurements as a reference, so that they are mapped to the closed interval $[0,1]$. In this way, the new normalized value S' will depict the relative position of each measurement, eliminating the issue of incomparable scales.

<u>Replacement method</u>	<u>Decision interval h</u>	<u>Allowance k</u>	<u>Buffer size</u>	<u>Mean of S</u>	<u>Mean of S'</u>
Mahalanobis Range Replacement	6	1	50	1,042	0,129
Mahalanobis Range Replacement	6	1	100	1,180	0,149
Mahalanobis Range Replacement	6	1	150	1,428	0,185
Mahalanobis Range Replacement	6	1	200	1,603	0,211
Mahalanobis Range Replacement	7	1	50	0,998	0,123
Mahalanobis Range Replacement	7	1	100	1,535	0,201
Mahalanobis Range Replacement	7	1	150	2,452	0,335
Mahalanobis Range Replacement	7	1	200	3,661	0,512
Mahalanobis Range Replacement	8	1	50	1,044	0,129
Mahalanobis Range Replacement	8	1	100	1,883	0,252
Mahalanobis Range Replacement	8	1	150	4,114	0,578
Mahalanobis Range Replacement	8	1	200	6,353	0,905
Mahalanobis Range Replacement	9	1	50	1,129	0,142
Mahalanobis Range Replacement	9	1	100	3,121	0,433
Mahalanobis Range Replacement	9	1	150	6,715	0,958
Mahalanobis Range Replacement	9	1	200	7,002	1,000
Mahalanobis Range Replacement	10	0,5	50	1,075	0,134
Mahalanobis Range Replacement	10	0,5	100	1,259	0,161
Mahalanobis Range Replacement	10	0,5	150	1,604	0,211

Mahalanobis Range Replacement	10	0,5	200	1,706	0,226
Mahalanobis Range Replacement	11	0,5	50	1,016	0,125
Mahalanobis Range Replacement	11	0,5	100	1,395	0,181
Mahalanobis Range Replacement	11	0,5	150	1,916	0,257
Mahalanobis Range Replacement	11	0,5	200	2,326	0,317
Mahalanobis Range Replacement	12	0,5	50	1,096	0,137
Mahalanobis Range Replacement	12	0,5	100	1,585	0,208
Mahalanobis Range Replacement	12	0,5	150	2,224	0,302
Mahalanobis Range Replacement	12	0,5	200	2,797	0,386
Mahalanobis Range Replacement	13	0,5	50	1,137	0,143
Mahalanobis Range Replacement	13	0,5	100	1,682	0,223
Mahalanobis Range Replacement	13	0,5	150	2,509	0,343
Mahalanobis Range Replacement	13	0,5	200	4,011	0,563
Mahalanobis Range Replacement	14	0,5	50	1,206	0,153
Mahalanobis Range Replacement	14	0,5	100	1,932	0,259
Mahalanobis Range Replacement	14	0,5	150	2,636	0,362
Mahalanobis Range Replacement	14	0,5	200	4,497	0,634
Mahalanobis Range Replacement	15	0,5	50	1,214	0,154
Mahalanobis Range Replacement	15	0,5	100	2,034	0,274
Mahalanobis Range Replacement	15	0,5	150	2,779	0,383
Mahalanobis Range Replacement	15	0,5	200	4,906	0,694
Max Distance Expansion Replacement	6	1	50	0,250	0,013
Max Distance Expansion Replacement	6	1	100	0,281	0,018
Max Distance Expansion Replacement	6	1	150	0,369	0,031

Max Distance Expansion Replacement	6	1	200	0,459	0,044
Max Distance Expansion Replacement	7	1	50	0,178	0,003
Max Distance Expansion Replacement	7	1	100	0,209	0,007
Max Distance Expansion Replacement	7	1	150	0,292	0,019
Max Distance Expansion Replacement	7	1	200	0,413	0,037
Max Distance Expansion Replacement	8	1	50	0,168	0,001
Max Distance Expansion Replacement	8	1	100	0,186	0,004
Max Distance Expansion Replacement	8	1	150	0,292	0,019
Max Distance Expansion Replacement	8	1	200	0,455	0,043
Max Distance Expansion Replacement	9	1	50	0,160	0,000
Max Distance Expansion Replacement	9	1	100	0,169	0,001
Max Distance Expansion Replacement	9	1	150	0,288	0,019
Max Distance Expansion Replacement	9	1	200	0,417	0,038
Max Distance Expansion Replacement	10	0,5	50	0,217	0,008
Max Distance Expansion Replacement	10	0,5	100	0,255	0,014
Max Distance Expansion Replacement	10	0,5	150	0,330	0,025
Max Distance Expansion Replacement	10	0,5	200	0,409	0,037
Max Distance Expansion Replacement	11	0,5	50	0,187	0,004
Max Distance Expansion Replacement	11	0,5	100	0,227	0,010
Max Distance Expansion Replacement	11	0,5	150	0,310	0,022
Max Distance Expansion Replacement	11	0,5	200	0,393	0,034
Max Distance Expansion Replacement	12	0,5	50	0,171	0,002
Max Distance Expansion Replacement	12	0,5	100	0,237	0,011
Max Distance Expansion Replacement	12	0,5	150	0,293	0,020

Max Distance Expansion Replacement	12	0,5	200	0,382	0,033
Max Distance Expansion Replacement	13	0,5	50	0,162	0,000
Max Distance Expansion Replacement	13	0,5	100	0,223	0,009
Max Distance Expansion Replacement	13	0,5	150	0,288	0,019
Max Distance Expansion Replacement	13	0,5	200	0,386	0,033
Max Distance Expansion Replacement	14	0,5	50	0,159	0,000
Max Distance Expansion Replacement	14	0,5	100	0,213	0,008
Max Distance Expansion Replacement	14	0,5	150	0,279	0,018
Max Distance Expansion Replacement	14	0,5	200	0,379	0,032
Max Distance Expansion Replacement	15	0,5	50	0,160	0,000
Max Distance Expansion Replacement	15	0,5	100	0,207	0,007
Max Distance Expansion Replacement	15	0,5	150	0,270	0,016
Max Distance Expansion Replacement	15	0,5	200	0,380	0,032
Max Range Replacement	6	1	50	0,404	0,036
Max Range Replacement	6	1	100	0,509	0,051
Max Range Replacement	6	1	150	0,640	0,070
Max Range Replacement	6	1	200	0,777	0,090
Max Range Replacement	7	1	50	0,336	0,026
Max Range Replacement	7	1	100	0,466	0,045
Max Range Replacement	7	1	150	0,675	0,075
Max Range Replacement	7	1	200	0,928	0,112
Max Range Replacement	8	1	50	0,313	0,023
Max Range Replacement	8	1	100	0,455	0,043
Max Range Replacement	8	1	150	0,798	0,093

Max Range Replacement	8	1	200	1,152	0,145
Max Range Replacement	9	1	50	0,302	0,021
Max Range Replacement	9	1	100	0,469	0,045
Max Range Replacement	9	1	150	0,901	0,108
Max Range Replacement	9	1	200	1,066	0,133
Max Range Replacement	10	0,5	50	0,384	0,033
Max Range Replacement	10	0,5	100	0,489	0,048
Max Range Replacement	10	0,5	150	0,624	0,068
Max Range Replacement	10	0,5	200	0,743	0,085
Max Range Replacement	11	0,5	50	0,345	0,027
Max Range Replacement	11	0,5	100	0,462	0,044
Max Range Replacement	11	0,5	150	0,622	0,068
Max Range Replacement	11	0,5	200	0,802	0,094
Max Range Replacement	12	0,5	50	0,322	0,024
Max Range Replacement	12	0,5	100	0,479	0,047
Max Range Replacement	12	0,5	150	0,640	0,070
Max Range Replacement	12	0,5	200	0,810	0,095
Max Range Replacement	13	0,5	50	0,315	0,023
Max Range Replacement	13	0,5	100	0,494	0,049
Max Range Replacement	13	0,5	150	0,691	0,078
Max Range Replacement	13	0,5	200	0,928	0,112
Max Range Replacement	14	0,5	50	0,311	0,022
Max Range Replacement	14	0,5	100	0,471	0,046
Max Range Replacement	14	0,5	150	0,696	0,078

Max Range Replacement	14	0,5	200	0,958	0,117
Max Range Replacement	15	0,5	50	0,305	0,021
Max Range Replacement	15	0,5	100	0,493	0,049
Max Range Replacement	15	0,5	150	0,721	0,082
Max Range Replacement	15	0,5	200	1,028	0,127

Table 17 - Analytic results for S and S' indexes

Since evaluating the results from the above table is difficult due to the large volume of information, each parameter will initially be examined in isolation. It should be highlighted that the decision interval and allowance values are evaluated together, as different allowance values are combined with various decision intervals. The following tables present the mean, standard deviation, maximum, and minimum values for each parameter.

<u>Replacement method</u>	<u>Mean of S'</u>	<u>Standard deviation of S'</u>	<u>Minimum value of S'</u>	<u>Maximum Value of S'</u>
Mahalanobis Range Replacement	0,327	0,235	0,123	1,000
Max Range Replacement	0,017	0,013	0,000	0,044
Max Distance Expansion Replacement	0,066	0,035	0,021	0,145

Table 18 - Index S' results for each replacement method

<u>Combination</u>	<u>Mean of S'</u>	<u>Standard deviation of S'</u>	<u>Minimum value of S'</u>	<u>Maximum Value of S'</u>
k=0,5 / h=10	0,088	0,077	0,008	0,226
k=0,5 / h=11	0,099	0,103	0,004	0,317
k=0,5 / h=12	0,111	0,125	0,002	0,386
k=0,5 / h=13	0,133	0,169	0,000	0,563
k=0,5 / h=14	0,144	0,190	0,000	0,634
k=0,5 / h=15	0,153	0,207	0,000	0,694
k=1,0 / h=6	0,086	0,068	0,013	0,211
k=1,0 / h=7	0,125	0,156	0,003	0,512
k=1,0 / h=8	0,186	0,278	0,001	0,905
k=1,0 / h=9	0,241	0,364	0,000	1,000

Table 19 - Index S' results for each allowance and decision interval combination

<u>Buffer size</u>	<u>Mean of S'</u>	<u>Standard deviation of S'</u>	<u>Minimum value of S'</u>	<u>Maximum Value of S'</u>
50	0,055	0,060	0,000	0,154
100	0,097	0,110	0,001	0,433
150	0,164	0,208	0,016	0,958
200	0,231	0,274	0,032	1,000

Table 20 - Index S' results for each buffer size

Regarding the combinations of the decision interval and allowance, the optimal values occur for an allowance equal to one ($k = 1$). Furthermore, for a constant allowance, as the decision interval increases, the index S' increases as well. Given that the decision interval h and the allowance k determine the sensitivity of the CUSUM algorithm, larger values lead to longer epochs, which means that the measurements of the initial dataset exhibit a broader range of values per dimension. Consequently, and given that the replacement algorithms focus on retaining the most extreme values, it is a logical consequence that larger allowance and decision interval values lead to greater generalized variance.

An additional element observed is that, alongside the mean, the standard deviation of the index S' and its range of values also increase. This simultaneous increase in dispersion and the mean is caused by the asymmetric variation of the distribution. This asymmetry arises from the lower boundary, which prevents the distribution from taking negative values (maintaining the minimum value consistently close to zero), and from the reduced sensitivity of the CUSUM algorithm due to larger values for h and k , which leads to the formation of longer epochs. The increased number of measurements per epoch statistically maximizes the probability of encountering extreme values, which the replacement algorithms tend to retain in the synopsis. This mechanism acts actively on the upper tail of the distribution, drastically increasing the maximum value and expanding it disproportionately toward one side. The following graph depicts the relationship between the parameters h, k , and the index S' .

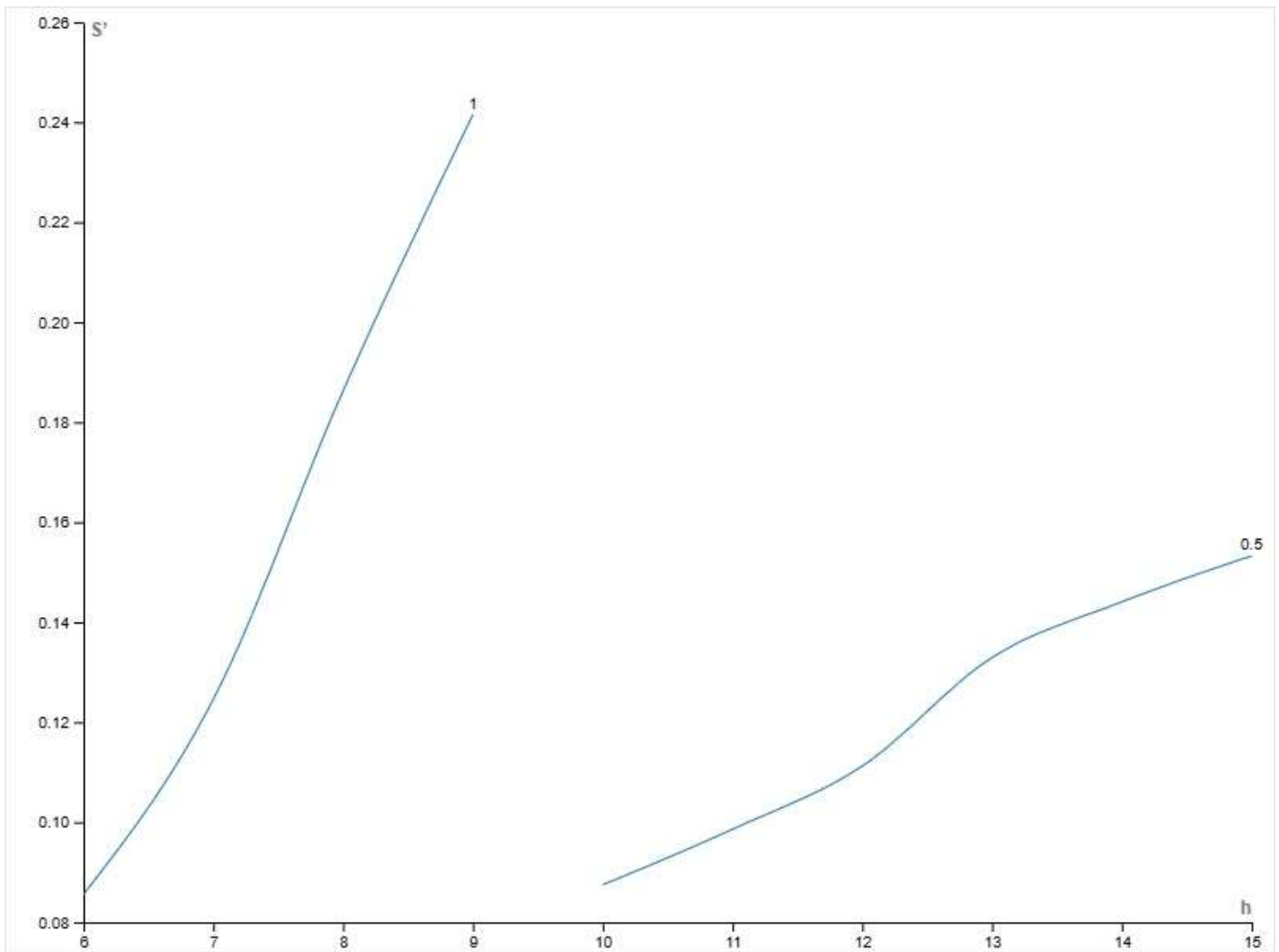


Figure 15 - Variation of index S' as a function of the decision interval h for allowance values k^{13}

Regarding the relationship between the index S' and the buffer size, it is confirmed that larger buffer sizes lead to higher values of the generalized variance index. This observation is consistent with the system's design, as a larger reservoir capacity allows for the storage of more samples, thereby increasing the probability of recording the extreme values of the distribution.

The following boxplot presents the distribution of the generalized variance index values per buffer size. As shown, the median value of S' steadily increases as the buffer size grows, from 0.023 for a buffer size of 50, up to 0.112 for a size of 200. Simultaneously, both the total range and the dispersion around the median exhibit significant and highly asymmetric variability, as the interquartile range expands proportionally to the buffer size (from 0.123 to 0.255). This intense asymmetry, as explained above, is caused by the combination of median values located close to zero, the lower boundary that prevents the distribution from taking negative values, and the structural capability of larger buffers to incorporate more extreme values, thus expanding the upper tail. Finally, regarding the presence of extreme values, these are identified in three out of the four buffer sizes, specifically for sizes 100, 150, and 200. For buffer sizes of 100 and 150 measurements, one extreme value is found respectively, whereas for a size of 200, three extreme values are identified.

¹³ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

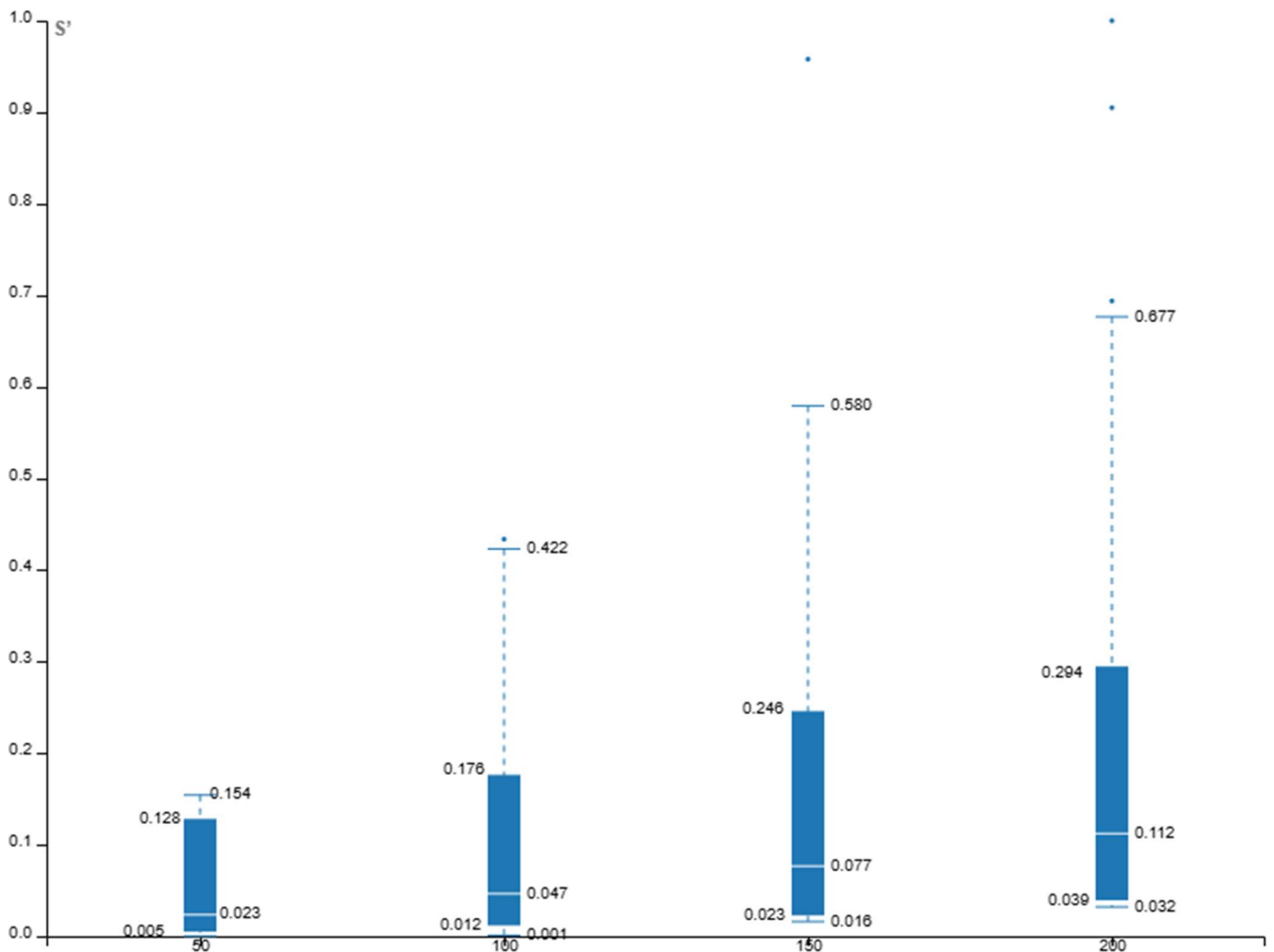


Figure 16 - Boxplot of index S' as a function of the buffer size¹⁴

Regarding the replacement methods, as shown in the corresponding table, the Mahalanobis Range Replacement Method exhibits the best performance in terms of the mean, maximum, and minimum values, followed by the Max Distance Expansion Replacement Method, and finally the Max Range Replacement Method. As observed and reflected in the corresponding graph, the Mahalanobis Maximum Range Replacement Strategy is by far the most efficient of the three in maximizing the generalized variance index. This occurs because this particular method incorporates the Mahalanobis distance, which accounts for the correlations between variables and their different measurement scales, in contrast to the other two methods that use the Euclidean distance, which treats all dimensions equally and is sensitive to absolute values.

Regarding the standard deviation of the index, the Max Distance Expansion Replacement Method and the Max Range Replacement Method record extremely low values (0.035 and 0.013, respectively), in contrast to the Mahalanobis Range Replacement Method, which exhibits a significantly higher dispersion (0.235). This difference is not a consequence of reduced stability in the Mahalanobis method, but rather a byproduct of its efficiency. The two methods based on the Euclidean distance fail to detect the truly extreme states of the system, remaining trapped at maximum values very close to zero (0.044 and 0.145), a fact that compresses their standard deviation. On the other hand, the ability of the Mahalanobis method to account for data

¹⁴ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

correlations allows it to push the maximum values to the limit, asymmetrically expanding the range of the distribution and thereby increasing the dispersion.

The above findings are also depicted in the following graph, which illustrates the relationship between the index S' and the buffer capacity for each replacement method. As shown, for all strategies without exception, an increase in buffer capacity leads to an improvement in the index S' , and the Mahalanobis Max Range Replacement Method significantly outperforms the others regardless of buffer size.

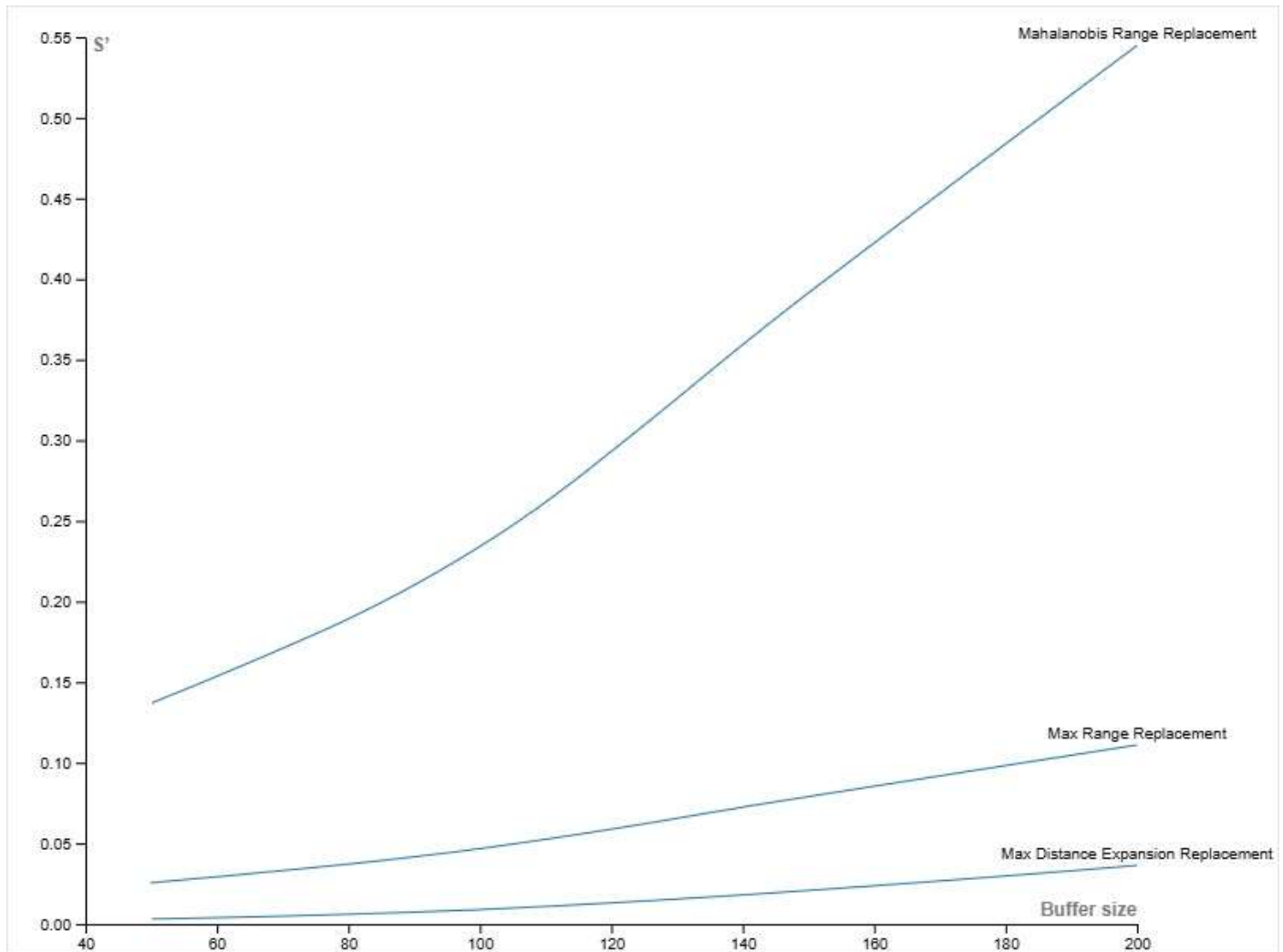


Figure 17 - Variation of index S' as a function of buffer size and replacement method¹⁵

9.1.3 PROBLEMATIC EPOCHS

Problematic epochs are defined as the epochs where either the volume of the initial dataset or the determinant of the covariance matrix becomes zero. Regarding the Volume Comparison Index, a problematic epoch occurs when $V_{dataset,i} = 0$. This happens when, during an epoch, at least one dimension has a zero range; that is, the corresponding variable remains constant throughout the entire epoch. In this case, the hyper-rectangle degenerates to a zero volume, and the index R_i cannot be defined directly through simple division. Based on the definition of the Volume Comparison Index, two cases emerge.

¹⁵ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

If the volume of the synopsis is non-zero, i.e., $V_{synopsis,i} \neq 0$, an error has occurred, as the synopsis does not generate new measurements but rather constitutes a subset of the initial measurements. If the volume of the synopsis is equal to zero, i.e., $V_{synopsis,i} = 0$, it is considered to accurately reflect the dataset, and thus the Volume Comparison Index is assumed to be equal to one ($R = 1$).

Regarding the Generalized Variance Index, a problematic epoch occurs when $|\Sigma_{dataset,i}| = 0$. This happens when the covariance matrix is singular, a fact that mathematically implies either that at least one variable has zero variance, or that a perfect linear dependence exists between two or more dimensions. As with the Volume Comparison Index, two cases are distinguished.

If the determinant of the covariance matrix of the synopsis is non-zero, i.e., $|\Sigma_{synopsis,i}| \neq 0$, then an error has occurred. Since the synopsis does not produce new measurements but constitutes a subset of the original measurements, it is impossible for it to exhibit variance when the superset's variance is zero. Provided that the determinant of the synopsis is also zero ($|\Sigma_{synopsis,i}| = 0$), the synopsis is considered to accurately reflect the degenerate structure of the dataset. For this reason, the Generalized Variance Index for this specific epoch is defined as equal to one.

The evaluation algorithm distinguishes between these two cases and also outputs the details of the specific dataset, parameters, and epoch that produced the result to an Excel file. Our initial observation from the results is that in none of the 72 problematic epochs did a synopsis emerge with a non-zero volume or a non-zero covariance matrix determinant. Additionally, in all cases where a zero volume appeared in the epoch's dataset, the corresponding determinant of the covariance matrix was evaluated as zero. This is expected, given that a zero volume implies a zero range in at least one parameter, which in turn means that the variance is also zero.

Problematic epochs occur in the same datasets and for the same parameters h, k , and buffer sizes across all replacement methods. Regarding buffer sizes, problematic epochs are exclusively recorded at a size of 50. This phenomenon is a direct consequence of the implementation architecture, as the CUSUM detection mechanism is not activated until the buffer is fully populated at the beginning of each new epoch. This means that the buffer size acts as a lower bound for the number of measurements in any epoch. Given that in the examined initial dataset the consecutive zero measurements never exceed 100, an epoch with a minimum size of 50 measurements can consist entirely of zero values, thus recording zero variance. Conversely, for buffer sizes of 100 and above, the length of each generated epoch is mandatorily sufficient to include at least one value change.

9.2 COMPRESSION RATIO

The Compression Ratio of the data stream reflects the reduction in the volume of data that will ultimately be transmitted to the cloud. The following table presents the compression results for each parameter combination.

<u>Replacement method</u>	<u>Decision interval</u> h	<u>Allowance</u> k	<u>Buffer size</u>	<u>Number of epochs</u>	<u>Mean of C</u>
Mahalanobis Range Replacement	6	1	50	212,6	0,575
Mahalanobis Range Replacement	6	1	100	102,5	0,591
Mahalanobis Range Replacement	6	1	150	71,3	0,571
Mahalanobis Range Replacement	6	1	200	56,6	0,548
Mahalanobis Range Replacement	7	1	50	119,3	0,761
Mahalanobis Range Replacement	7	1	100	52,0	0,791
Mahalanobis Range Replacement	7	1	150	35,7	0,786

Mahalanobis Range Replacement	7	1	200	29,8	0,762
Mahalanobis Range Replacement	8	1	50	65,9	0,869
Mahalanobis Range Replacement	8	1	100	33,0	0,868
Mahalanobis Range Replacement	8	1	150	24,3	0,855
Mahalanobis Range Replacement	8	1	200	22,0	0,823
Mahalanobis Range Replacement	9	1	50	46,4	0,909
Mahalanobis Range Replacement	9	1	100	24,4	0,902
Mahalanobis Range Replacement	9	1	150	20,0	0,880
Mahalanobis Range Replacement	9	1	200	17,7	0,857
Mahalanobis Range Replacement	10	0,5	50	192,2	0,615
Mahalanobis Range Replacement	10	0,5	100	73,5	0,706
Mahalanobis Range Replacement	10	0,5	150	51,8	0,69
Mahalanobis Range Replacement	10	0,5	200	43,1	0,655
Mahalanobis Range Replacement	11	0,5	50	128,2	0,744
Mahalanobis Range Replacement	11	0,5	100	51,2	0,796
Mahalanobis Range Replacement	11	0,5	150	37,0	0,777
Mahalanobis Range Replacement	11	0,5	200	30,6	0,756
Mahalanobis Range Replacement	12	0,5	50	90,0	0,820
Mahalanobis Range Replacement	12	0,5	100	40,1	0,839
Mahalanobis Range Replacement	12	0,5	150	29,3	0,823
Mahalanobis Range Replacement	12	0,5	200	26,2	0,791
Mahalanobis Range Replacement	13	0,5	50	66,6	0,866
Mahalanobis Range Replacement	13	0,5	100	33,3	0,868
Mahalanobis Range Replacement	13	0,5	150	25,3	0,847
Mahalanobis Range Replacement	13	0,5	200	23,3	0,814
Mahalanobis Range Replacement	14	0,5	50	52,1	0,897
Mahalanobis Range Replacement	14	0,5	100	29,7	0,881
Mahalanobis Range Replacement	14	0,5	150	24,3	0,857
Mahalanobis Range Replacement	14	0,5	200	22,2	0,822
Mahalanobis Range Replacement	15	0,5	50	47,5	0,906
Mahalanobis Range Replacement	15	0,5	100	28,1	0,889
Mahalanobis Range Replacement	15	0,5	150	23,7	0,860
Mahalanobis Range Replacement	15	0,5	200	21,4	0,826
Max Distance Expansion Replacement	6	1	50	212,6	0,575
Max Distance Expansion Replacement	6	1	100	102,5	0,591
Max Distance Expansion Replacement	6	1	150	71,3	0,571
Max Distance Expansion Replacement	6	1	200	56,6	0,548
Max Distance Expansion Replacement	7	1	50	119,3	0,761
Max Distance Expansion Replacement	7	1	100	52,0	0,791
Max Distance Expansion Replacement	7	1	150	35,7	0,786
Max Distance Expansion Replacement	7	1	200	29,8	0,762
Max Distance Expansion Replacement	8	1	50	65,9	0,869
Max Distance Expansion Replacement	8	1	100	33,0	0,868
Max Distance Expansion Replacement	8	1	150	24,3	0,855
Max Distance Expansion Replacement	8	1	200	22,0	0,823
Max Distance Expansion Replacement	9	1	50	46,4	0,909
Max Distance Expansion Replacement	9	1	100	24,4	0,902

Max Distance Expansion Replacement	9	1	150	20,0	0,880
Max Distance Expansion Replacement	9	1	200	17,7	0,857
Max Distance Expansion Replacement	10	0,5	50	192,2	0,615
Max Distance Expansion Replacement	10	0,5	100	73,5	0,706
Max Distance Expansion Replacement	10	0,5	150	51,8	0,690
Max Distance Expansion Replacement	10	0,5	200	43,1	0,655
Max Distance Expansion Replacement	11	0,5	50	128,2	0,744
Max Distance Expansion Replacement	11	0,5	100	51,2	0,796
Max Distance Expansion Replacement	11	0,5	150	37,0	0,777
Max Distance Expansion Replacement	11	0,5	200	30,6	0,756
Max Distance Expansion Replacement	12	0,5	50	90,0	0,820
Max Distance Expansion Replacement	12	0,5	100	40,1	0,839
Max Distance Expansion Replacement	12	0,5	150	29,3	0,823
Max Distance Expansion Replacement	12	0,5	200	26,2	0,791
Max Distance Expansion Replacement	13	0,5	50	66,6	0,866
Max Distance Expansion Replacement	13	0,5	100	33,3	0,868
Max Distance Expansion Replacement	13	0,5	150	25,3	0,847
Max Distance Expansion Replacement	13	0,5	200	23,3	0,814
Max Distance Expansion Replacement	14	0,5	50	52,1	0,897
Max Distance Expansion Replacement	14	0,5	100	29,7	0,881
Max Distance Expansion Replacement	14	0,5	150	24,3	0,857
Max Distance Expansion Replacement	14	0,5	200	22,2	0,822
Max Distance Expansion Replacement	15	0,5	50	47,5	0,906
Max Distance Expansion Replacement	15	0,5	100	28,1	0,889
Max Distance Expansion Replacement	15	0,5	150	23,7	0,86
Max Distance Expansion Replacement	15	0,5	200	21,4	0,826
Max Range Replacement	6	1	50	212,6	0,575
Max Range Replacement	6	1	100	102,5	0,591
Max Range Replacement	6	1	150	71,3	0,571
Max Range Replacement	6	1	200	56,6	0,548
Max Range Replacement	7	1	50	119,3	0,761
Max Range Replacement	7	1	100	52,0	0,791
Max Range Replacement	7	1	150	35,7	0,786
Max Range Replacement	7	1	200	29,8	0,762
Max Range Replacement	8	1	50	65,9	0,869
Max Range Replacement	8	1	100	33,0	0,868
Max Range Replacement	8	1	150	24,3	0,855
Max Range Replacement	8	1	200	22,0	0,823
Max Range Replacement	9	1	50	46,4	0,909
Max Range Replacement	9	1	100	24,4	0,902
Max Range Replacement	9	1	150	20,0	0,880
Max Range Replacement	9	1	200	17,7	0,857
Max Range Replacement	10	0,5	50	192,2	0,615
Max Range Replacement	10	0,5	100	73,5	0,706
Max Range Replacement	10	0,5	150	51,8	0,690
Max Range Replacement	10	0,5	200	43,1	0,655
Max Range Replacement	11	0,5	50	128,2	0,744

Max Range Replacement	11	0,5	100	51,2	0,796
Max Range Replacement	11	0,5	150	37,0	0,777
Max Range Replacement	11	0,5	200	30,6	0,756
Max Range Replacement	12	0,5	50	90,0	0,820
Max Range Replacement	12	0,5	100	40,1	0,839
Max Range Replacement	12	0,5	150	29,3	0,823
Max Range Replacement	12	0,5	200	26,2	0,791
Max Range Replacement	13	0,5	50	66,6	0,866
Max Range Replacement	13	0,5	100	33,3	0,868
Max Range Replacement	13	0,5	150	25,3	0,847
Max Range Replacement	13	0,5	200	23,3	0,814
Max Range Replacement	14	0,5	50	52,1	0,897
Max Range Replacement	14	0,5	100	29,7	0,881
Max Range Replacement	14	0,5	150	24,3	0,857
Max Range Replacement	14	0,5	200	22,2	0,822
Max Range Replacement	15	0,5	50	47,5	0,906
Max Range Replacement	15	0,5	100	28,1	0,889
Max Range Replacement	15	0,5	150	23,7	0,860
Max Range Replacement	15	0,5	200	21,4	0,826

Table 21 - Analytic results for C index

From the above table, it is evident that the compression rate is independent of the replacement method; for any combination of decision interval, allowance, and buffer size, the compression value remains constant regardless of the selected method. This conclusion is expected, as the total volume of data constituting the final synopsis is calculated as the product of the number of epochs and the buffer capacity (with the possible exception of the final epoch of each stream, which may terminate before the reservoir is fully filled). Consequently, the primary determinant of the compression rate is the number of generated epochs, which depends exclusively on the sensitivity of the CUSUM algorithm in detecting shifts namely, on parameters h and k .

Since evaluating the results from the above table is difficult due to the large volume of information, each parameter will initially be examined in isolation. It should be highlighted that the decision interval and allowance values are evaluated together, as different allowance values are combined with various decision intervals. The following tables present the mean, standard deviation, average number of epochs, maximum, and minimum values for each parameter.

<u>Combination</u>	<u>Mean of C</u>	<u>Standard deviation of C</u>	<u>Minimum value of C</u>	<u>Maximum Value of C</u>	<u>Mean number of epochs</u>
k=0,5 / h=10	0,667	0,040	0,615	0,706	90,150
k=0,5 / h=11	0,768	0,023	0,744	0,796	61,750
k=0,5 / h=12	0,818	0,020	0,791	0,839	46,400
k=0,5 / h=13	0,849	0,025	0,814	0,868	37,125
k=0,5 / h=14	0,864	0,033	0,822	0,897	32,075
k=0,5 / h=15	0,870	0,035	0,826	0,906	30,175
k=1,0 / h=6	0,571	0,018	0,548	0,591	110,750
k=1,0 / h=7	0,775	0,016	0,761	0,791	59,200
k=1,0 / h=8	0,854	0,021	0,823	0,869	36,300
k=1,0 / h=9	0,887	0,024	0,857	0,909	27,125

Table 22 - Index C results for each allowance and decision interval combination

<u>Buffer size</u>	<u>Mean of C</u>	<u>Standard deviation of C</u>	<u>Minimum value of C</u>	<u>Maximum Value of C</u>	<u>Mean number of epochs</u>
50	0,796	0,121	0,575	0,909	102,080
100	0,813	0,098	0,591	0,902	46,780
150	0,795	0,097	0,571	0,880	34,270
200	0,765	0,095	0,548	0,857	29,290

Table 23 - Index C results for each buffer size

As shown in Table 20, the compression rate improves as the value of the threshold h increases. This is expected, given that parameter h determines the sensitivity of the CUSUM algorithm. Consequently, larger values lead to reduced sensitivity and a smaller number of epochs, as evidenced by the last column of the table, which presents the average number of epochs per combination. A further observation emerging from the results is that the standard deviation of the compression rate remains low across all parameter combinations. The relationship between h and the number of epochs for each value of k is illustrated in the following diagram.

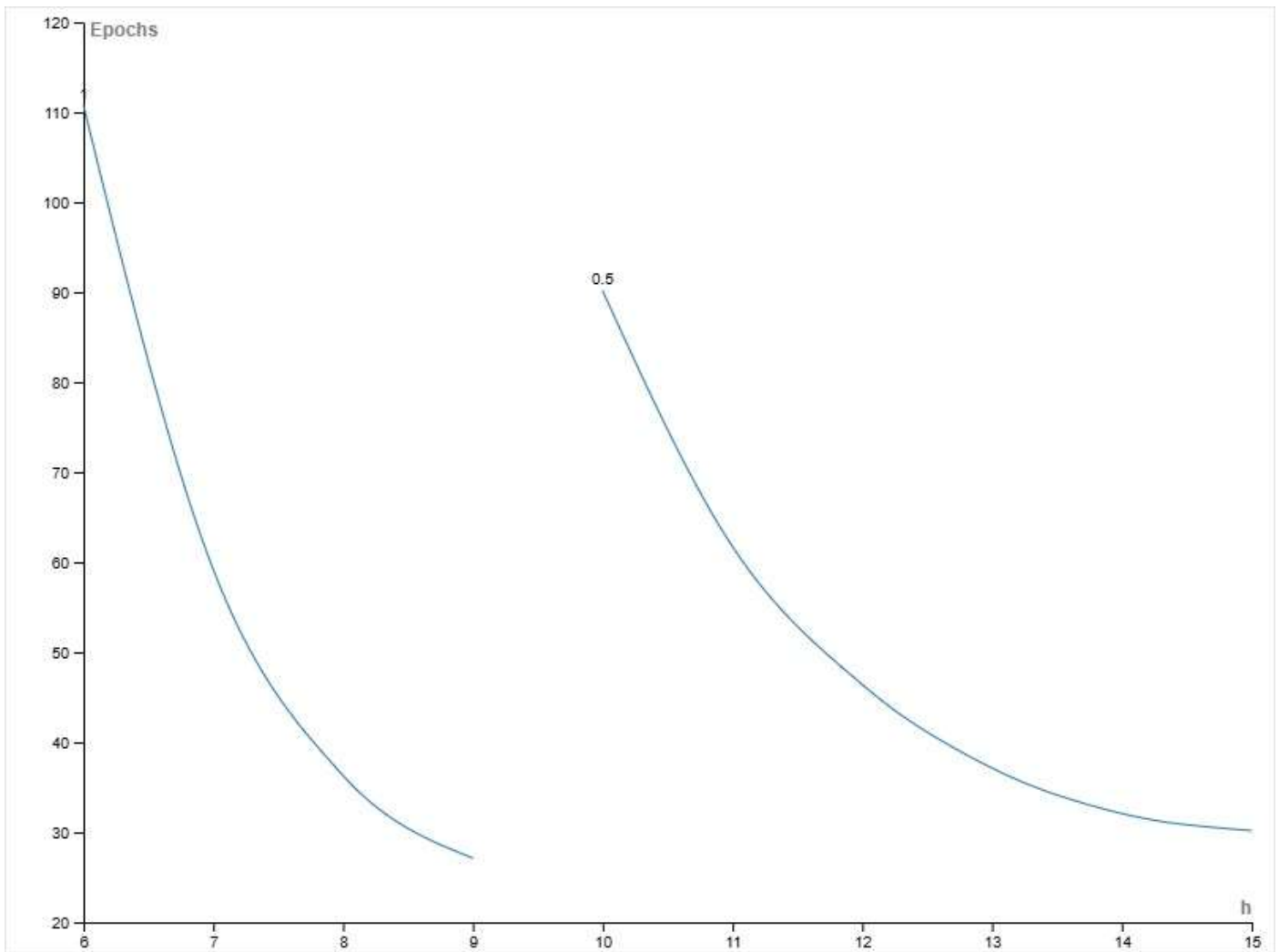


Figure 18 - Variation of number of epochs as a function of the decision interval h for allowance values k^{16}

The graph clearly illustrates the strongly decreasing, non-linear relationship between the decision interval h and the number of generated epochs. Specifically, for both allowance values ($k = 1.0$ and $k = 0.5$), the curves exhibit a distinctly convex shape, indicating that the rate of decrease in the number of epochs is rapid at the initial, lower thresholds. However, as h increases, the decline noticeably decelerates and the curves tend to flatten out, converging toward a common lower bound (approximately 27 to 30 epochs on average). The following graph depicts the relationship between the parameters h and k and the statistic C .

¹⁶ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

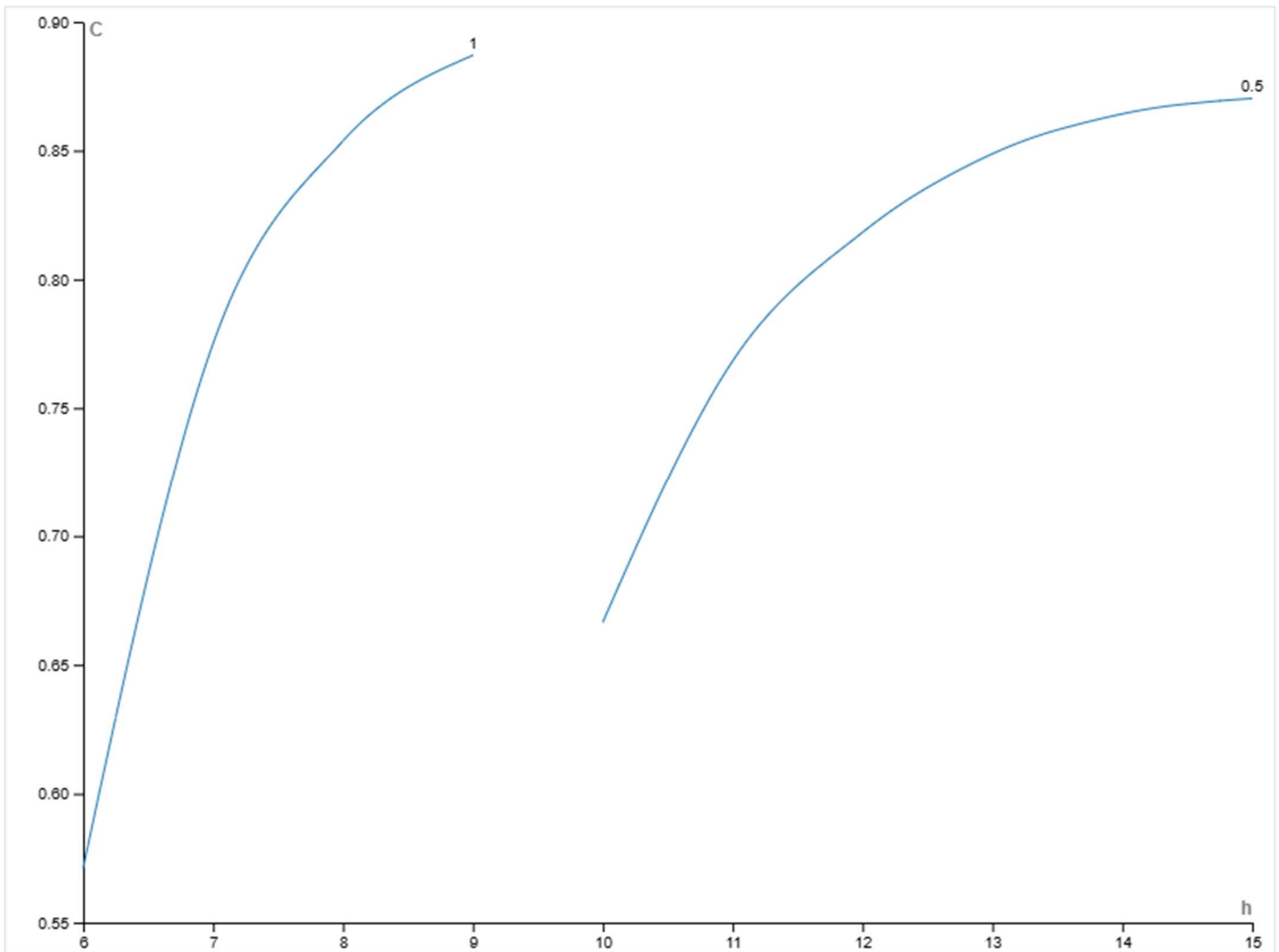


Figure 19 - Variation of index C as a function of the decision interval h for allowance values k^{17}

The above graph, which illustrates the relationship between the decision interval h and the compression rate C , reveals a clear, non-linear increasing trend. Specifically, it is observed that the rate of increase of C is decreasing, giving the curves a strictly increasing, concave shape that resembles a logarithmic function. This behavior indicates the presence of diminishing returns, as for larger values of the decision interval, the improvement in compression slows down, asymptotically approaching the theoretical upper bound of the index.

Regarding the relationship between the compression rate and the buffer size, it is observed that the mean value of index C is maximized at a size of 100 (0.813) and subsequently exhibits a steady decrease for sizes 150 (0.795) and 200 (0.765). This phenomenon is explained by the competitive interaction between the two variables that determine the final volume of the synopsis: the reservoir capacity and the number of generated epochs. The transition from a size of 50 to 100 causes a drastic reduction in the average number of epochs (from 102.08 to 46.78), which overcompensates for the doubling of the capacity, leading to optimal compression. However, for capacities above 100, the rate of decrease in epochs slows down noticeably. Consequently, the additional reservoir capacity burdens the final data volume more than it is alleviated by the reduced production of new epochs, resulting in an inevitable deterioration of index C . The above relationship between the buffer size and the number of epochs is also depicted in the following graph.

¹⁷ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

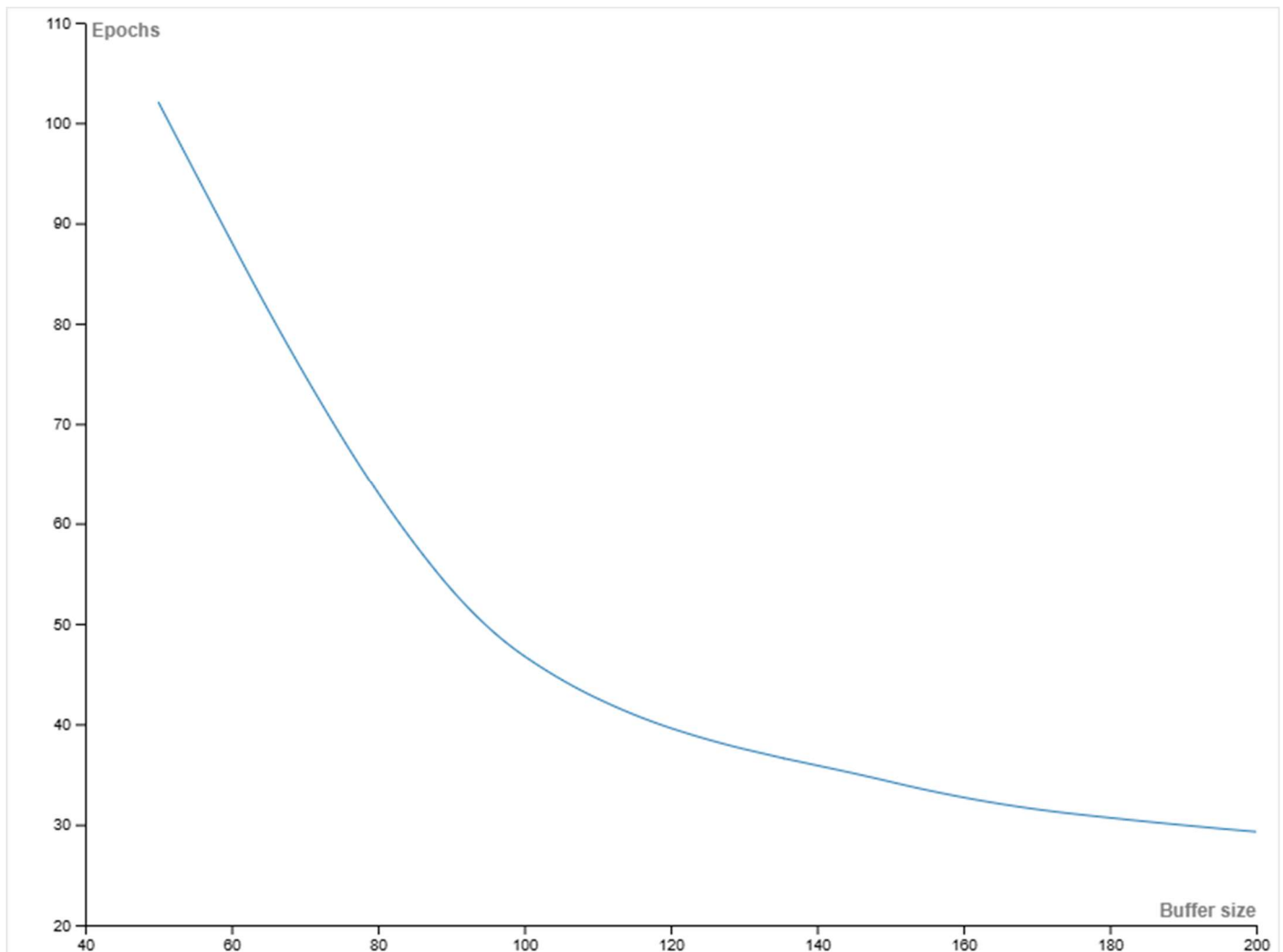


Figure 20 - Variation of number of epochs and buffer size¹⁸

Additionally, it is worth noting that the standard deviation exhibits a steadily decreasing trend as the buffer size increases (from 0.121 at 50 to 0.095 at 200), indicating greater stability and homogeneity in compression levels for larger buffer volumes. The relationship between buffer size and the compression rate is illustrated in the following boxplot.

¹⁸ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

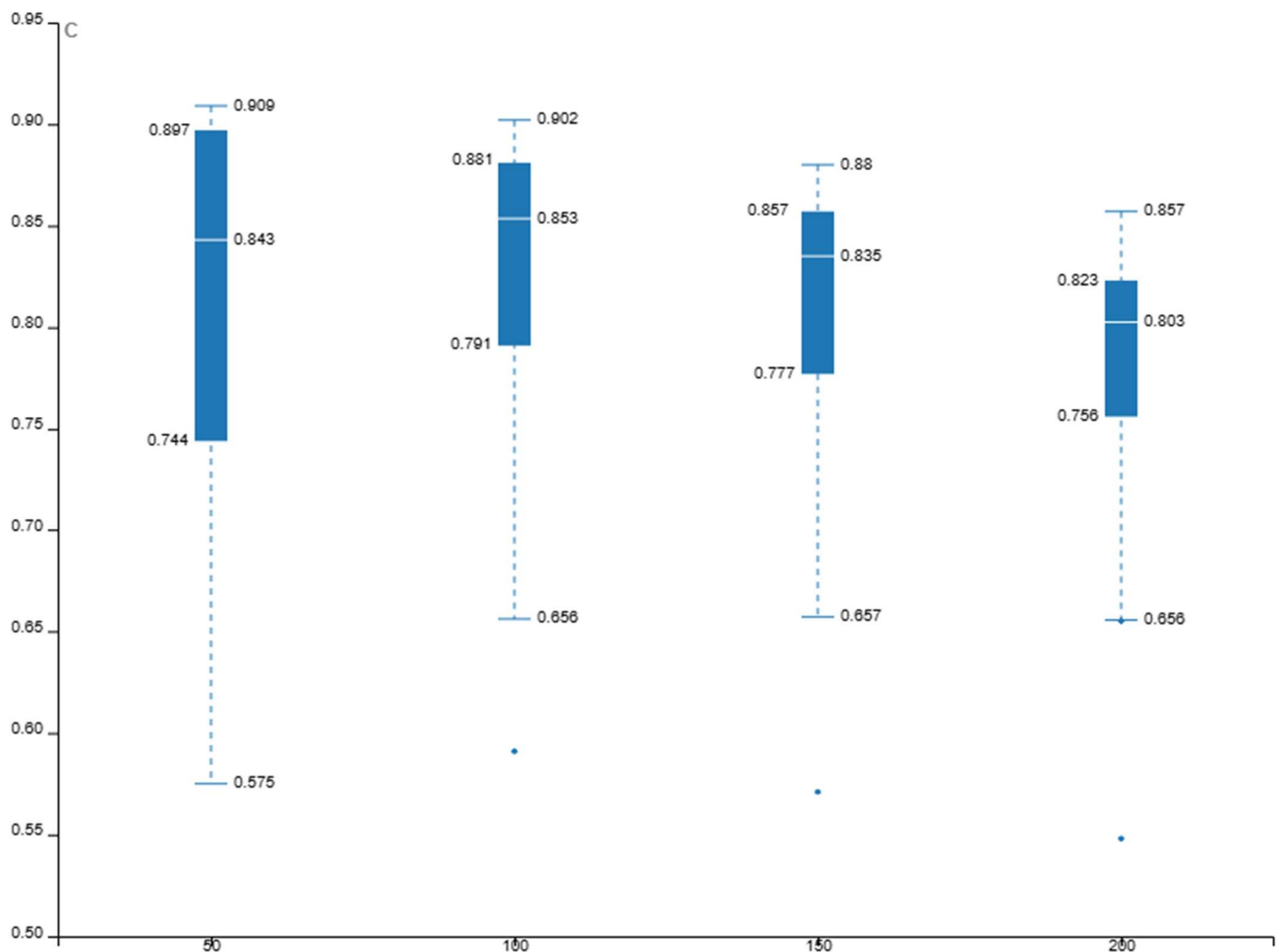


Figure 21 - Boxplot of index C as a function of the buffer size¹⁹

As shown in the above graph, the median value of the generalized variance index indeed peaks at a buffer size of 100 (0.853), visually confirming the optimal performance of this specific capacity. Concurrently, a clear shrinkage of the interquartile range is observed as the buffer size increases from 50 to 200. This visual progression verifies the statistical decrease in dispersion and standard deviation, indicating a more homogeneous and predictable behavior of the algorithm at larger sizes. Characteristic in this regard is the extended lower whisker in the distribution for a buffer size of 50, which equals 0.575. Notably, while for capacities of 100, 150, and 200 the lower compression rates are strictly plotted as individual lower extreme values, at a size of 50 this drop in performance falls within the statistically "expected" bounds of the distribution, confirming the structural instability of the small reservoir. Finally, the placement of the median in the upper half of the box indicates negative asymmetry in the distributions, suggesting that the majority of observations consistently tend toward the upper bound of strong compression.

9.3 CHANGES DETECTION

The evaluation of the change detection mechanism is of secondary importance compared to the evaluation of the replacement methods and the compression ratio. As already mentioned, the performance evaluation of the vector CUSUM will be conducted using a modified version of the PATE-F1 method by Ghorbani et al.

¹⁹ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

(2024). The following table shows the results of the method, as well as the mean number of false and true alarms for each parameter combination.

<u>Replacement method</u>	<u>Decision interval h</u>	<u>Allowance k</u>	<u>Buffer size</u>	<u>True alarms</u>	<u>False alarms</u>	<u>Mean of PATE - F1</u>
Mahalanobis Range Replacement	6	1	50	12,0	199,6	0,081
Mahalanobis Range Replacement	6	1	100	11,3	90,2	0,147
Mahalanobis Range Replacement	6	1	150	10,0	60,3	0,177
Mahalanobis Range Replacement	6	1	200	10,5	45,1	0,209
Mahalanobis Range Replacement	7	1	50	14,9	103,4	0,171
Mahalanobis Range Replacement	7	1	100	13,8	37,2	0,297
Mahalanobis Range Replacement	7	1	150	14,3	20,4	0,394
Mahalanobis Range Replacement	7	1	200	14,1	14,7	0,433
Mahalanobis Range Replacement	8	1	50	15,1	49,8	0,274
Mahalanobis Range Replacement	8	1	100	14,6	17,4	0,412
Mahalanobis Range Replacement	8	1	150	15,5	7,8	0,530
Mahalanobis Range Replacement	8	1	200	15,1	5,9	0,542
Mahalanobis Range Replacement	9	1	50	15,2	30,2	0,349
Mahalanobis Range Replacement	9	1	100	14,7	8,7	0,472
Mahalanobis Range Replacement	9	1	150	14,7	4,3	0,553
Mahalanobis Range Replacement	9	1	200	14,0	2,7	0,547
Mahalanobis Range Replacement	10	0,5	50	12,0	179,2	0,086
Mahalanobis Range Replacement	10	0,5	100	13,6	58,9	0,222
Mahalanobis Range Replacement	10	0,5	150	13,7	37,1	0,276
Mahalanobis Range Replacement	10	0,5	200	13,7	28,4	0,310
Mahalanobis Range Replacement	11	0,5	50	14,3	112,9	0,143
Mahalanobis Range Replacement	11	0,5	100	14,4	35,8	0,292
Mahalanobis Range Replacement	11	0,5	150	14,2	21,8	0,356
Mahalanobis Range Replacement	11	0,5	200	14,9	14,7	0,411
Mahalanobis Range Replacement	12	0,5	50	15,7	73,3	0,209
Mahalanobis Range Replacement	12	0,5	100	15,0	24,1	0,345
Mahalanobis Range Replacement	12	0,5	150	14,6	13,7	0,409
Mahalanobis Range Replacement	12	0,5	200	15,1	10,1	0,439
Mahalanobis Range Replacement	13	0,5	50	14,7	50,9	0,246
Mahalanobis Range Replacement	13	0,5	100	14,1	18,2	0,353
Mahalanobis Range Replacement	13	0,5	150	15,0	9,3	0,441
Mahalanobis Range Replacement	13	0,5	200	15,8	6,5	0,483
Mahalanobis Range Replacement	14	0,5	50	15,5	35,6	0,303
Mahalanobis Range Replacement	14	0,5	100	13,7	15,0	0,355
Mahalanobis Range Replacement	14	0,5	150	15,3	8,0	0,438
Mahalanobis Range Replacement	14	0,5	200	15,5	5,7	0,469
Mahalanobis Range Replacement	15	0,5	50	14,9	31,6	0,308
Mahalanobis Range Replacement	15	0,5	100	13,3	13,8	0,351
Mahalanobis Range Replacement	15	0,5	150	15,0	7,7	0,425
Mahalanobis Range Replacement	15	0,5	200	15,2	5,2	0,453

Max Distance Expansion Replacement	6	1	50	12,0	199,6	0,081
Max Distance Expansion Replacement	6	1	100	11,3	90,2	0,147
Max Distance Expansion Replacement	6	1	150	10,0	60,3	0,177
Max Distance Expansion Replacement	6	1	200	10,5	45,1	0,209
Max Distance Expansion Replacement	7	1	50	14,9	103,4	0,171
Max Distance Expansion Replacement	7	1	100	13,8	37,2	0,297
Max Distance Expansion Replacement	7	1	150	14,3	20,4	0,394
Max Distance Expansion Replacement	7	1	200	14,1	14,7	0,433
Max Distance Expansion Replacement	8	1	50	15,1	49,8	0,274
Max Distance Expansion Replacement	8	1	100	14,6	17,4	0,412
Max Distance Expansion Replacement	8	1	150	15,5	7,8	0,530
Max Distance Expansion Replacement	8	1	200	15,1	5,9	0,542
Max Distance Expansion Replacement	9	1	50	15,2	30,2	0,349
Max Distance Expansion Replacement	9	1	100	14,7	8,7	0,472
Max Distance Expansion Replacement	9	1	150	14,7	4,3	0,553
Max Distance Expansion Replacement	9	1	200	14,0	2,7	0,547
Max Distance Expansion Replacement	10	0,5	50	12,0	179,2	0,086
Max Distance Expansion Replacement	10	0,5	100	13,6	58,9	0,222
Max Distance Expansion Replacement	10	0,5	150	13,7	37,1	0,276
Max Distance Expansion Replacement	10	0,5	200	13,7	28,4	0,310
Max Distance Expansion Replacement	11	0,5	50	14,3	112,9	0,143
Max Distance Expansion Replacement	11	0,5	100	14,4	35,8	0,292
Max Distance Expansion Replacement	11	0,5	150	14,2	21,8	0,356
Max Distance Expansion Replacement	11	0,5	200	14,9	14,7	0,411

Max Distance Expansion Replacement	12	0,5	50	15,7	73,3	0,209
Max Distance Expansion Replacement	12	0,5	100	15,0	24,1	0,345
Max Distance Expansion Replacement	12	0,5	150	14,6	13,7	0,409
Max Distance Expansion Replacement	12	0,5	200	15,1	10,1	0,439
Max Distance Expansion Replacement	13	0,5	50	14,7	50,9	0,246
Max Distance Expansion Replacement	13	0,5	100	14,1	18,2	0,353
Max Distance Expansion Replacement	13	0,5	150	15,0	9,3	0,441
Max Distance Expansion Replacement	13	0,5	200	15,8	6,5	0,483
Max Distance Expansion Replacement	14	0,5	50	15,5	35,6	0,303
Max Distance Expansion Replacement	14	0,5	100	13,7	15,0	0,355
Max Distance Expansion Replacement	14	0,5	150	15,3	8,0	0,438
Max Distance Expansion Replacement	14	0,5	200	15,5	5,7	0,469
Max Distance Expansion Replacement	15	0,5	50	14,9	31,6	0,308
Max Distance Expansion Replacement	15	0,5	100	13,3	13,8	0,351
Max Distance Expansion Replacement	15	0,5	150	15,0	7,7	0,425
Max Distance Expansion Replacement	15	0,5	200	15,2	5,2	0,453
Max Range Replacement	6	1	50	12,0	199,6	0,081
Max Range Replacement	6	1	100	11,3	90,2	0,147
Max Range Replacement	6	1	150	10,0	60,3	0,177
Max Range Replacement	6	1	200	10,5	45,1	0,209
Max Range Replacement	7	1	50	14,9	103,4	0,171
Max Range Replacement	7	1	100	13,8	37,2	0,297
Max Range Replacement	7	1	150	14,3	20,4	0,394
Max Range Replacement	7	1	200	14,1	14,7	0,433
Max Range Replacement	8	1	50	15,1	49,8	0,274
Max Range Replacement	8	1	100	14,6	17,4	0,412
Max Range Replacement	8	1	150	15,5	7,8	0,530
Max Range Replacement	8	1	200	15,1	5,9	0,542
Max Range Replacement	9	1	50	15,2	30,2	0,349
Max Range Replacement	9	1	100	14,7	8,7	0,472
Max Range Replacement	9	1	150	14,7	4,3	0,553
Max Range Replacement	9	1	200	14,0	2,7	0,547

Max Range Replacement	10	0,5	50	12,0	179,2	0,086
Max Range Replacement	10	0,5	100	13,6	58,9	0,222
Max Range Replacement	10	0,5	150	13,7	37,1	0,276
Max Range Replacement	10	0,5	200	13,7	28,4	0,310
Max Range Replacement	11	0,5	50	14,3	112,9	0,143
Max Range Replacement	11	0,5	100	14,4	35,8	0,292
Max Range Replacement	11	0,5	150	14,2	21,8	0,356
Max Range Replacement	11	0,5	200	14,9	14,7	0,411
Max Range Replacement	12	0,5	50	15,7	73,3	0,209
Max Range Replacement	12	0,5	100	15,0	24,1	0,345
Max Range Replacement	12	0,5	150	14,6	13,7	0,409
Max Range Replacement	12	0,5	200	15,1	10,1	0,439
Max Range Replacement	13	0,5	50	14,7	50,9	0,246
Max Range Replacement	13	0,5	100	14,1	18,2	0,353
Max Range Replacement	13	0,5	150	15,0	9,3	0,441
Max Range Replacement	13	0,5	200	15,8	6,5	0,483
Max Range Replacement	14	0,5	50	15,5	35,6	0,303
Max Range Replacement	14	0,5	100	13,7	15,0	0,355
Max Range Replacement	14	0,5	150	15,3	8,0	0,438
Max Range Replacement	14	0,5	200	15,5	5,7	0,469
Max Range Replacement	15	0,5	50	14,9	31,6	0,308
Max Range Replacement	15	0,5	100	13,3	13,8	0,351
Max Range Replacement	15	0,5	150	15,0	7,7	0,425
Max Range Replacement	15	0,5	200	15,2	5,2	0,453

Table 24 - Analytic results for PATE-F1 index

As with the compression ratio, the replacement method plays no role in the performance evaluation of the CUSUM algorithm. For any fixed combination of decision interval, allowance, and buffer size, the PATE-F1 index remains identical regardless of the selected replacement method. This conclusion is expected, as the two processes are functionally decoupled. The replacement method is activated exclusively for buffer memory management during summary generation, without intervening in the control mechanism. In contrast, the operation and sensitivity of CUSUM are directly determined by the allowance and decision interval parameters, and indirectly by the buffer capacity; this is because the covariance matrix—which serves as the algorithm's reference baseline for each epoch—is calculated from the initial data stored in the buffer.

Since evaluating the results from the above table is difficult due to the large volume of information, each parameter will initially be examined in isolation. It should be highlighted that the decision interval and allowance values are evaluated together, as different allowance values are combined with various decision intervals. The following tables present the mean, standard deviation, maximum and minimum values and the true and false alarms for each parameter.

<u>Combination</u>	<u>Mean of PATE - F1</u>	<u>Standard deviation of PATE- F1</u>	<u>Minimum value of PATE - F1</u>	<u>Maximum Value PATE - F1</u>	<u>True alarms</u>	<u>False alarms</u>
k=0,5 / h=10	0,223	0,098	0,086	0,310	13,3	75,9
k=0,5 / h=11	0,300	0,116	0,143	0,411	14,5	46,3
k=0,5 / h=12	0,350	0,102	0,209	0,439	15,1	30,3
k=0,5 / h=13	0,381	0,105	0,246	0,483	14,9	21,2
k=0,5 / h=14	0,391	0,076	0,303	0,469	15,0	16,1

k=0,5 / h=15	0,384	0,067	0,308	0,453	14,6	14,6
k=1,0 / h=6	0,154	0,055	0,081	0,209	11,0	98,8
k=1,0 / h=7	0,324	0,117	0,171	0,433	14,3	43,9
k=1,0 / h=8	0,439	0,125	0,274	0,542	15,1	20,2
k=1,0 / h=9	0,480	0,095	0,349	0,553	14,7	11,5

Table 25 - Index PATE-F1 results for each allowance and decision interval combination

<u>Buffer size</u>	<u>Mean of PATE - F1</u>	<u>Standard deviation of PATE-F1</u>	<u>Minimum value of PATE - F1</u>	<u>Maximum Value of PATE - F1</u>	<u>True alarms</u>	<u>False alarms</u>
50	0,217	0,095	0,081	0,349	14,4	86,7
100	0,325	0,092	0,147	0,472	13,9	31,9
150	0,400	0,111	0,177	0,553	14,2	19,0
200	0,430	0,103	0,209	0,547	14,4	13,9

Table 26 - Index PATE-F1 results for each buffer size

As it is shown in the Table 25 generally the PATE-F1 index increases as the decision interval h increases too. More specifically, for allowance $k = 0.5$, the vector CUSUM algorithm performance increases until the value $h = 14$, where the index drops from 0.391 to 0.384. For $k = 1.0$ the performance of the algorithm increases consistent. Regarding false alarms, their number decreases steadily as the decision interval increases, and simultaneously the mean number of true alarms remains remarkably stable varying between the value 13.3 and 15.1 for $k = 0.5$ and 11.0 to 15.1 for $k = 1.0$. At this point it should be mentioned that the total number of the true shifts that are embedded into the dataset is 24, most of which are small-scale. The stabilization of the true alarms near 14, suggests the algorithm consistently detects the most prominent, structural changes. In contrast, it fails to detect the milder shifts, which are inevitably recorded as false negatives, since their magnitude is insufficient to exceed the statistical thresholds that trigger CUSUM. This asymmetric dynamic explains the improvement of the PATE-F1 index while the higher values of the decision interval make the algorithm stricter, thereby large volume of false alarms without compromising its established ability to detect significant shifts in the stream. The following graph depicts the relationship between the parameters h, k , and the index PATE-F1.

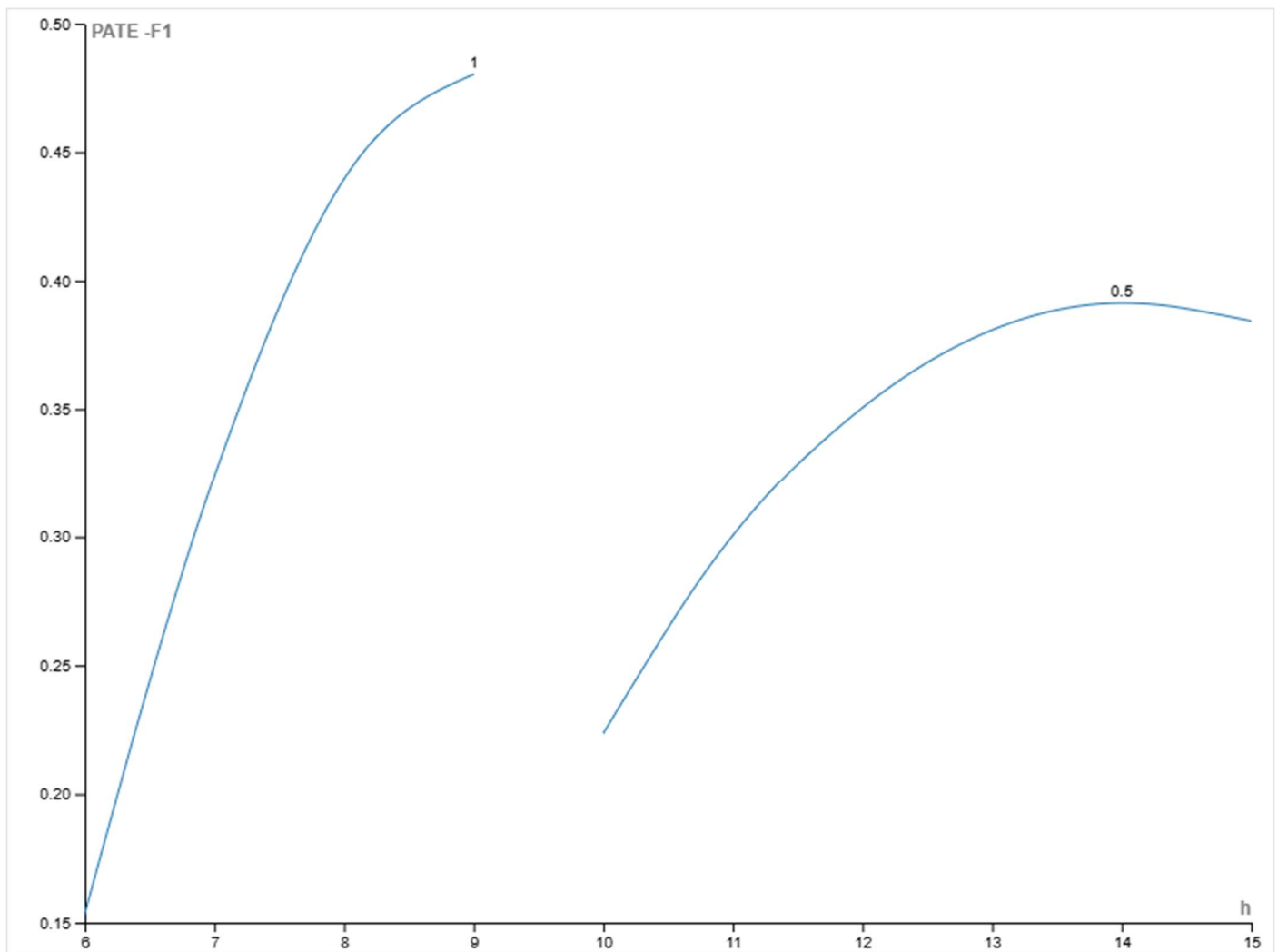


Figure 22 - Variation of index PATE-F1 as a function of the decision interval h for allowance values k^{20}

From the above graph, a clear non-linear trend is evident. Specifically, for $k = 1.0$, it is observed that the growth rate of the PATE-F1 metric is slightly decreasing, giving the curve a strictly increasing, concave shape that resembles a logarithmic one. On the other hand, for $k = 0.5$, the growth rate decreases more sharply and, ultimately, becomes negative at $h = 15$, with the curve reaching its local maximum at $h = 14$. As shown in Table 25, while the transition from $h = 14$ to $h = 15$ continues to reduce false alarms (from 16.1 to 14.6), it simultaneously causes a measurable decrease in true alarms (from 15.0 to 14.6). Given that the PATE-F1 metric represents the harmonic mean of Precision and Recall, the loss in actual detections (drop in Recall) incurs a higher cost to the algorithm than the marginal benefit of further noise elimination (increase in Precision), leading to the inevitable drop in overall performance. The two graphs below illustrate the correlation of true and false alarms with the parameters h and k .

²⁰ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

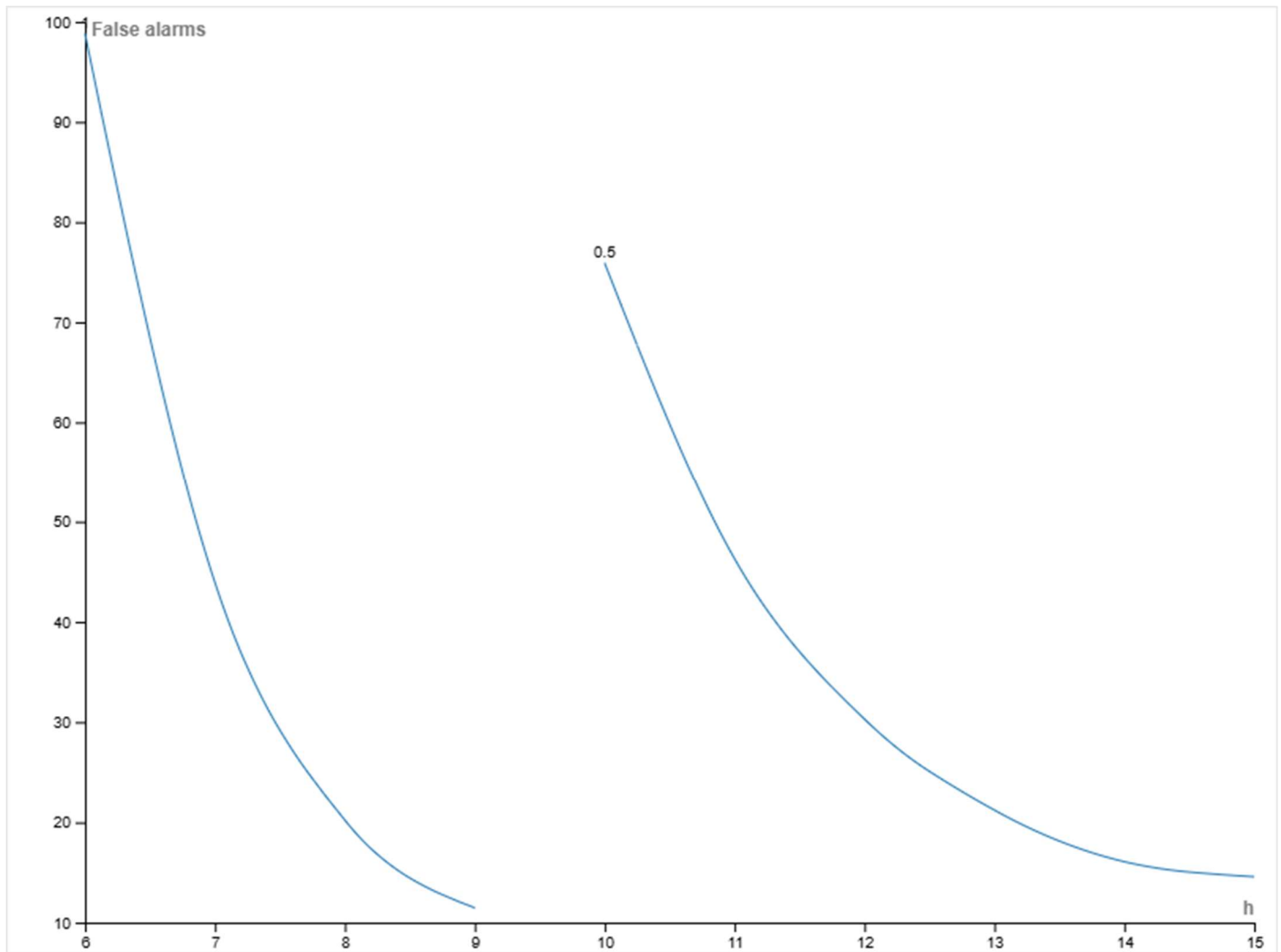


Figure 23 - Variation of false alarms as a function of the decision interval h for allowance values k^{21}

²¹ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

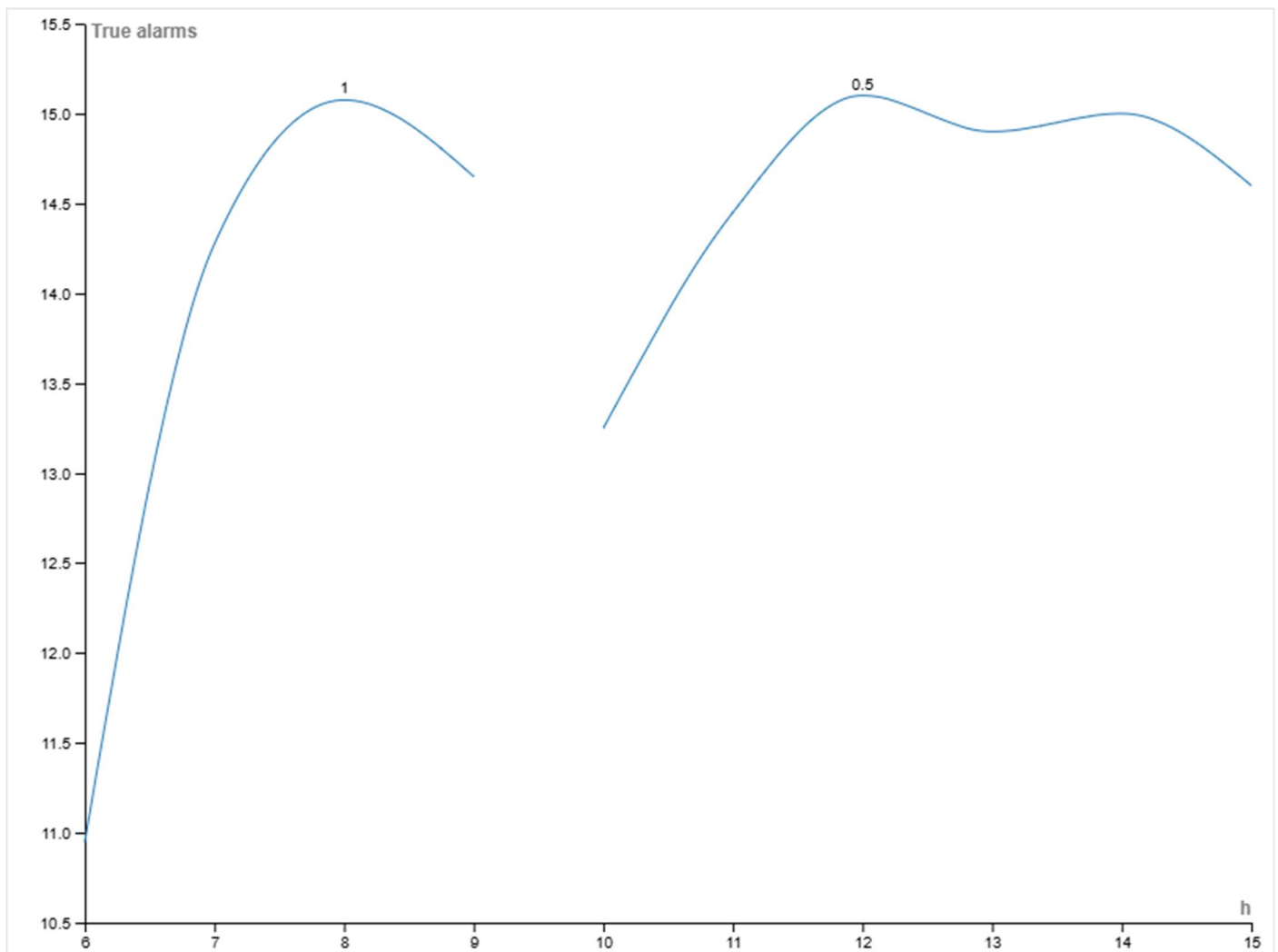


Figure 24 - Variation of true alarms as a function of the decision interval h for allowance values k^2

The relationship between the decision interval h and the generated alarms is illustrated in the two graphs above. In the first graph, a sharply decreasing trend in false alarms is observed as the decision interval increases, for both $k = 1.0$ and $k = 0.5$. Conversely, in the second graph, the number of true alarms stabilizes at the level of 14.5 to 15.0. This visual comparison confirms that an increase in h eliminates noise without improving the detection of actual shifts. Although slight dips are observed in the true alarms chart, such as during the transition to $h = 9$ for $k = 1.0$ or between $h = 12$ and $h = 13$ for $k = 0.5$, these decreases represent small local fluctuations that are overshadowed by the ongoing drop in false alarms. On the contrary, the dip in the curve during the transition from $h = 14$ to $h = 15$ proves that an excessive increase in the decision interval leads to a further loss of actual shifts. However, unlike previous local fluctuations, at this point, the rate of decrease for false alarms has slowed down drastically (from 16.1 to 14.6). Consequently, the marginal gain in Precision is no longer sufficient to offset the reduced Recall, fully justifying the final drop in the overall PATE-F1 metric.

Regarding the relationship between the shift detection capability and the buffer size, it is observed that the mean value of the PATE-F1 metric steadily improves across the entire range of capacities, starting from 0.217 at size 50 and reaching a maximum value of 0.430 at size 200. This phenomenon is explained by the

²² The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

asymmetric dynamics of the two variables that determine the final evaluation, namely the false and true alarms. Increasing the capacity from 50 to 200 causes a vertical drop in the average number of false alarms (from 86.7 to just 13.9), as the larger sample ensures a more stable and representative covariance matrix that shields the algorithm against noise. At the same time, the number of true alarms remains essentially unaffected, fluctuating steadily between 13.9 and 14.4. Consequently, the drastic reduction in false detections spectacularly improves Precision without sacrificing Recall, resulting in the continuous and significant improvement of the overall PATE-F1 metric. The relationship between the buffer size and the PATE-F1 metric is also shown in the boxplot below.

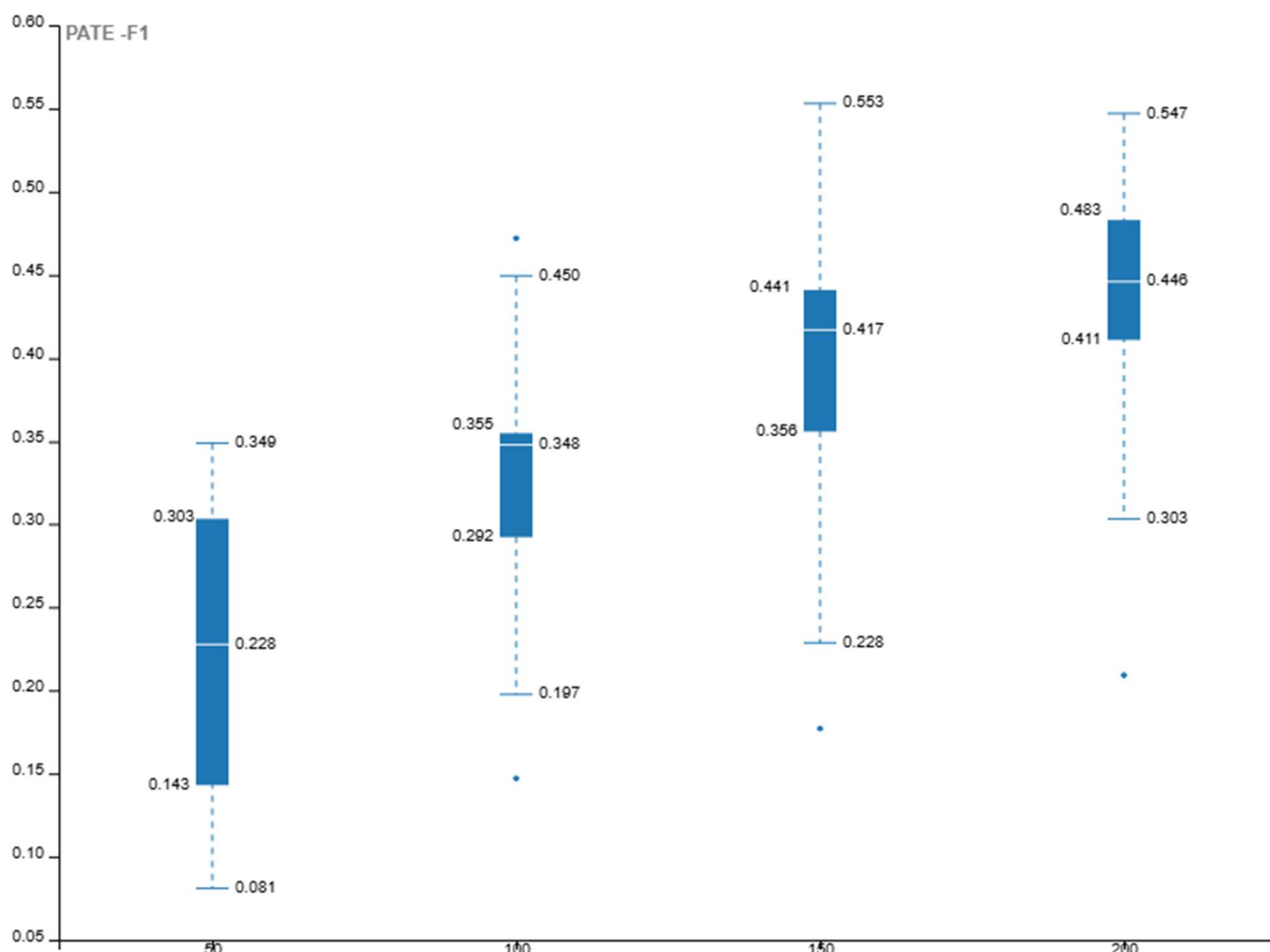


Figure 25 - Boxplot of index PATE-F1 as a function of the buffer size²³

As observed from the above graph, the median value of the PATE-F1 metric indeed peaks at the size of 200 (0.446), visually confirming the steady improvement in performance as the capacity increases. Concurrently, a noticeable shrinkage of the interquartile range is observed during the transition from size 50 to larger capacities, indicating a more homogeneous and predictable behavior of the algorithm when it utilizes a larger reference sample. Characteristic in this regard is the extended lower whisker in the distribution for size 50, which has a value of 0.081. It is noteworthy that, while for capacities of 100, 150, and 200 the lowest detection performances are strictly depicted as isolated lower outliers, at size 50 this drop in performance falls within the statistically "expected" limits of the distribution. This confirms the structural weakness of the

²³ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

small reservoir to effectively filter out noise. Finally, the placement of the median in the upper half of the box, particularly evident in the distributions for sizes 100 and 150, reveals a negative skewness, suggesting that the majority of observations consistently lean towards the upper levels of high accuracy. The two graphs below illustrate the correlation of true and false alarms with the buffer size.

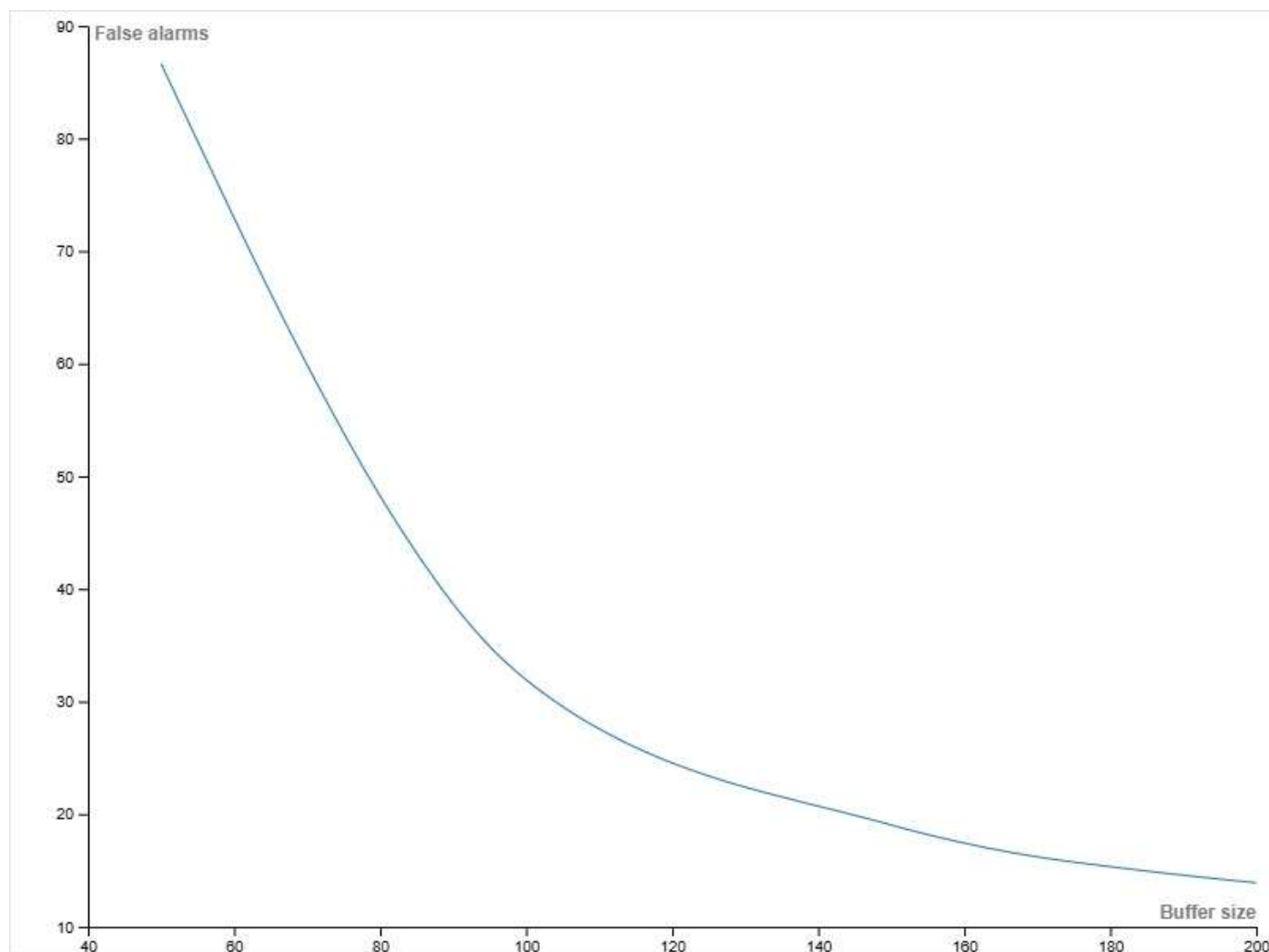


Figure 26 - Variation of false alarms as a function of buffer size²⁴

²⁴ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

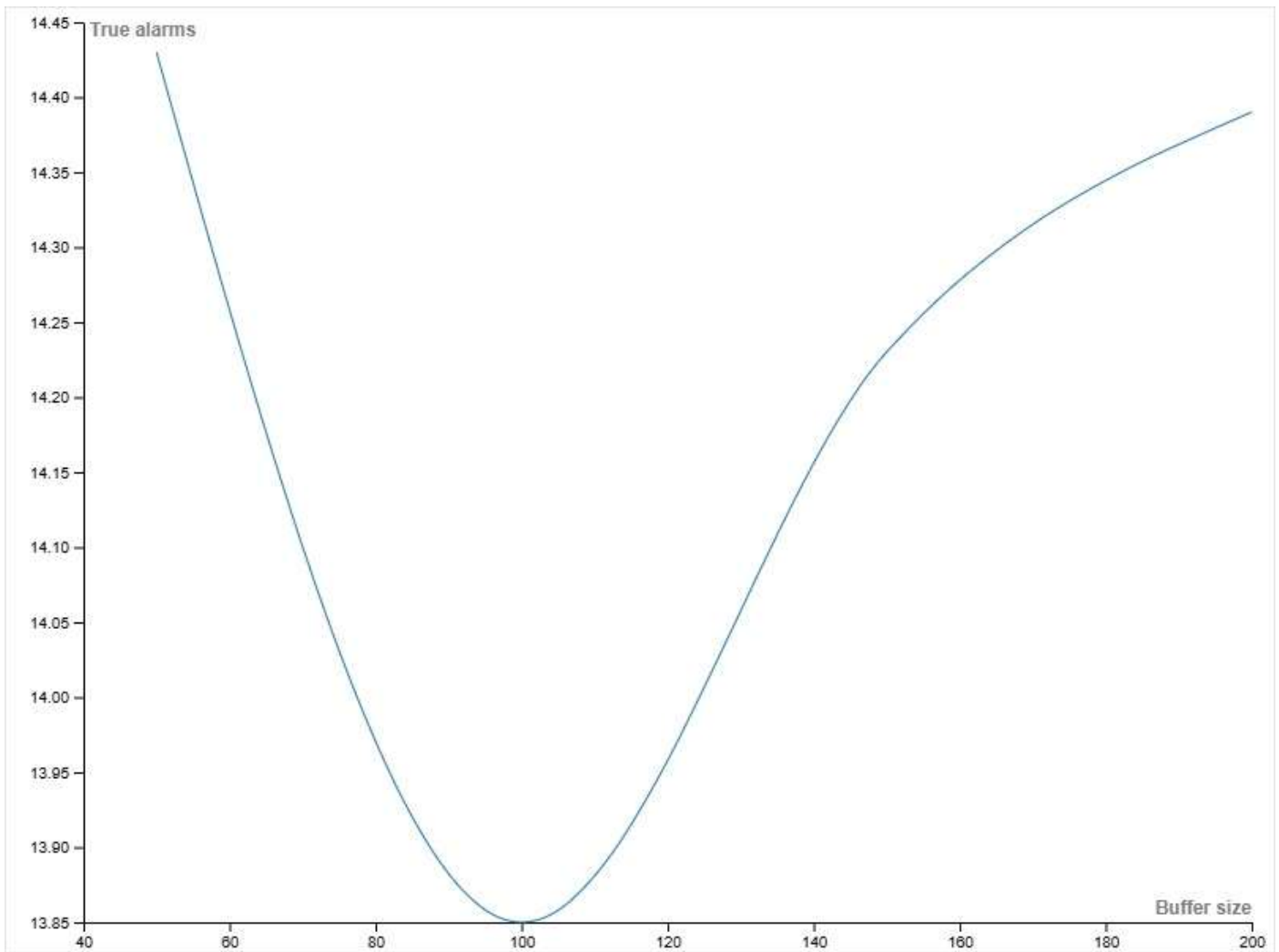


Figure 27 - Variation of true alarms as a function of buffer size²⁵

The first graph illustrates the evolution of false alarms as a function of buffer size. Correspondingly, the second graph depicts the trend of true alarms, whose values remain relatively stable in absolute terms. Examining the curve, a distinct local dip is recorded at size 100, where the mean value drops slightly to 13.9. This visual depression, which is magnified due to the particularly narrow scale of the vertical axis, is actually a marginal statistical fluctuation compared to the peak performance of 14.4 alarms observed at sizes 50 and 200. This temporary loss is deemed negligible for the overall evaluation of the algorithm, given that at the exact same capacity size (100) false alarms have already dropped from 86.7 to 31.9. Consequently, the massive gain in Precision due to noise reduction easily outweighs this slight decrease in Recall, allowing the overall PATE-F1 metric to maintain its strongly upward trajectory.

It is emphasized that the performance of the Vector CUSUM algorithm in the experiments, as reflected in the PATE-F1 metric, is influenced by the nature of the data. Although the initial noise was generated according to a multivariate normal distribution, the first differences used retain a minimal residual remnant due to the approach ($\phi \approx 1$). In a realistic telemetry environment, the differences will most likely not follow a perfectly normal distribution, nevertheless, the vector CUSUM algorithm exhibits high robustness against mild deviations.

²⁵ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

9.4 COMPOSITE EVALUATION METRICS

The composite indexes are three and they combine the performance across different metrics. The Sampling Quality Score (SQ), combines the volume comparison index (R) and the normalized Generalized Variance Comparison Index (S'), the statistical efficiency score (SE) combines the volume comparison index (R), the normalized Generalized Variance Comparison Index (S') and Compression Ratio Index (C). Finally, the Total Performance Score (TP) combines all the above with the PATE-F1 index. This index depicts the total performance of each parameter combination, by integrating the efficiency of the replacement methods, the compression ratio and the efficiency of vector CUSUM.

9.4.1 SAMPLING QUALITY INDEX SQ

This index aims at the evaluation of the sampling quality for each parameter combination (replacement method, allowance, decision interval, and buffer size). This evaluation is achieved through the combination of two individual metrics: the Volume Comparison Index (R) and the normalized Generalized Variance Comparison Index (S'). Given that the aforementioned metrics are defined within the closed interval $[0,1]$, the final index SQ , which is calculated as their arithmetic mean, inevitably ranges within the same interval. The table below presents the performance of each parameter combination in terms of sampling quality.

Replacement method	Decision interval h	Allowance k	Buffer size	Mean of SQ
Mahalanobis Range Replacement	6	1	50	0,410
Mahalanobis Range Replacement	6	1	100	0,429
Mahalanobis Range Replacement	6	1	150	0,464
Mahalanobis Range Replacement	6	1	200	0,483
Mahalanobis Range Replacement	7	1	50	0,358
Mahalanobis Range Replacement	7	1	100	0,410
Mahalanobis Range Replacement	7	1	150	0,489
Mahalanobis Range Replacement	7	1	200	0,600
Mahalanobis Range Replacement	8	1	50	0,334
Mahalanobis Range Replacement	8	1	100	0,417
Mahalanobis Range Replacement	8	1	150	0,597
Mahalanobis Range Replacement	8	1	200	0,782
Mahalanobis Range Replacement	9	1	50	0,326
Mahalanobis Range Replacement	9	1	100	0,484
Mahalanobis Range Replacement	9	1	150	0,789
Mahalanobis Range Replacement	9	1	200	0,835
Mahalanobis Range Replacement	10	0,5	50	0,398
Mahalanobis Range Replacement	10	0,5	100	0,411
Mahalanobis Range Replacement	10	0,5	150	0,459
Mahalanobis Range Replacement	10	0,5	200	0,475
Mahalanobis Range Replacement	11	0,5	50	0,372
Mahalanobis Range Replacement	11	0,5	100	0,399
Mahalanobis Range Replacement	11	0,5	150	0,463
Mahalanobis Range Replacement	11	0,5	200	0,496
Mahalanobis Range Replacement	12	0,5	50	0,358
Mahalanobis Range Replacement	12	0,5	100	0,412
Mahalanobis Range Replacement	12	0,5	150	0,478
Mahalanobis Range Replacement	12	0,5	200	0,526

Mahalanobis Range Replacement	13	0,5	50	0,354
Mahalanobis Range Replacement	13	0,5	100	0,414
Mahalanobis Range Replacement	13	0,5	150	0,480
Mahalanobis Range Replacement	13	0,5	200	0,608
Mahalanobis Range Replacement	14	0,5	50	0,349
Mahalanobis Range Replacement	14	0,5	100	0,428
Mahalanobis Range Replacement	14	0,5	150	0,485
Mahalanobis Range Replacement	14	0,5	200	0,637
Mahalanobis Range Replacement	15	0,5	50	0,344
Mahalanobis Range Replacement	15	0,5	100	0,435
Mahalanobis Range Replacement	15	0,5	150	0,490
Mahalanobis Range Replacement	15	0,5	200	0,663
Max Distance Expansion Replacement	6	1	50	0,279
Max Distance Expansion Replacement	6	1	100	0,299
Max Distance Expansion Replacement	6	1	150	0,324
Max Distance Expansion Replacement	6	1	200	0,351
Max Distance Expansion Replacement	7	1	50	0,230
Max Distance Expansion Replacement	7	1	100	0,250
Max Distance Expansion Replacement	7	1	150	0,265
Max Distance Expansion Replacement	7	1	200	0,294
Max Distance Expansion Replacement	8	1	50	0,212
Max Distance Expansion Replacement	8	1	100	0,228
Max Distance Expansion Replacement	8	1	150	0,241
Max Distance Expansion Replacement	8	1	200	0,281
Max Distance Expansion Replacement	9	1	50	0,201
Max Distance Expansion Replacement	9	1	100	0,208
Max Distance Expansion Replacement	9	1	150	0,239
Max Distance Expansion Replacement	9	1	200	0,282
Max Distance Expansion Replacement	10	0,5	50	0,265
Max Distance Expansion Replacement	10	0,5	100	0,275
Max Distance Expansion Replacement	10	0,5	150	0,302
Max Distance Expansion Replacement	10	0,5	200	0,325
Max Distance Expansion Replacement	11	0,5	50	0,240
Max Distance Expansion Replacement	11	0,5	100	0,256
Max Distance Expansion Replacement	11	0,5	150	0,280
Max Distance Expansion Replacement	11	0,5	200	0,293
Max Distance Expansion Replacement	12	0,5	50	0,225
Max Distance Expansion Replacement	12	0,5	100	0,253
Max Distance Expansion Replacement	12	0,5	150	0,271
Max Distance Expansion Replacement	12	0,5	200	0,281
Max Distance Expansion Replacement	13	0,5	50	0,216
Max Distance Expansion Replacement	13	0,5	100	0,253
Max Distance Expansion Replacement	13	0,5	150	0,251
Max Distance Expansion Replacement	13	0,5	200	0,272
Max Distance Expansion Replacement	14	0,5	50	0,207
Max Distance Expansion Replacement	14	0,5	100	0,240
Max Distance Expansion Replacement	14	0,5	150	0,243

Max Distance Expansion Replacement	14	0,5	200	0,263
Max Distance Expansion Replacement	15	0,5	50	0,208
Max Distance Expansion Replacement	15	0,5	100	0,241
Max Distance Expansion Replacement	15	0,5	150	0,237
Max Distance Expansion Replacement	15	0,5	200	0,260
Max Range Replacement	6	1	50	0,331
Max Range Replacement	6	1	100	0,363
Max Range Replacement	6	1	150	0,389
Max Range Replacement	6	1	200	0,416
Max Range Replacement	7	1	50	0,287
Max Range Replacement	7	1	100	0,311
Max Range Replacement	7	1	150	0,336
Max Range Replacement	7	1	200	0,378
Max Range Replacement	8	1	50	0,260
Max Range Replacement	8	1	100	0,287
Max Range Replacement	8	1	150	0,323
Max Range Replacement	8	1	200	0,382
Max Range Replacement	9	1	50	0,248
Max Range Replacement	9	1	100	0,268
Max Range Replacement	9	1	150	0,329
Max Range Replacement	9	1	200	0,375
Max Range Replacement	10	0,5	50	0,319
Max Range Replacement	10	0,5	100	0,338
Max Range Replacement	10	0,5	150	0,365
Max Range Replacement	10	0,5	200	0,397
Max Range Replacement	11	0,5	50	0,291
Max Range Replacement	11	0,5	100	0,316
Max Range Replacement	11	0,5	150	0,343
Max Range Replacement	11	0,5	200	0,376
Max Range Replacement	12	0,5	50	0,277
Max Range Replacement	12	0,5	100	0,314
Max Range Replacement	12	0,5	150	0,336
Max Range Replacement	12	0,5	200	0,361
Max Range Replacement	13	0,5	50	0,263
Max Range Replacement	13	0,5	100	0,304
Max Range Replacement	13	0,5	150	0,328
Max Range Replacement	13	0,5	200	0,366
Max Range Replacement	14	0,5	50	0,254
Max Range Replacement	14	0,5	100	0,294
Max Range Replacement	14	0,5	150	0,323
Max Range Replacement	14	0,5	200	0,356
Max Range Replacement	15	0,5	50	0,254
Max Range Replacement	15	0,5	100	0,299
Max Range Replacement	15	0,5	150	0,324
Max Range Replacement	15	0,5	200	0,361

Table 27 - Analytic results for SQ index

Since evaluating the results from the above table is difficult due to the large volume of information, each parameter will initially be examined in isolation. It should be highlighted that the decision interval and allowance values are evaluated together, as different allowance values are combined with various decision intervals. The following tables present the mean, standard deviation, maximum, and minimum values for each parameter.

<u>Replacement method</u>	<u>Mean of SQ</u>	<u>Standard deviation of SQ</u>	<u>Minimum value of SQ</u>	<u>Maximum Value of SQ</u>
Mahalanobis Range Replacement	0,478	0,125	0,326	0,835
MaxDistance Expansion Replacement	0,259	0,035	0,201	0,351
Max Range Replacement	0,326	0,044	0,248	0,416

Table 28 - Index SQ results for each replacement method

<u>Combination</u>	<u>Mean of SQ</u>	<u>Standard deviation of SQ</u>	<u>Minimum value of SQ</u>	<u>Maximum Value of SQ</u>
k=0,5 / h=10	0,361	0,068	0,265	0,475
k=0,5 / h=11	0,344	0,080	0,240	0,496
k=0,5 / h=12	0,341	0,092	0,225	0,526
k=0,5 / h=13	0,342	0,113	0,216	0,608
k=0,5 / h=14	0,340	0,124	0,207	0,637
k=0,5 / h=15	0,343	0,132	0,208	0,663
k=1,0 / h=6	0,378	0,064	0,279	0,483
k=1,0 / h=7	0,351	0,107	0,230	0,600
k=1,0 / h=8	0,362	0,169	0,212	0,782
k=1,0 / h=9	0,382	0,216	0,201	0,835

Table 29 - Index SQ results for each allowance and decision interval combination

<u>Buffer size</u>	<u>Mean of SQ</u>	<u>Standard deviation of SQ</u>	<u>Minimum value of SQ</u>	<u>Maximum Value of SQ</u>
50	0,289	0,061	0,201	0,410
100	0,328	0,077	0,208	0,484
150	0,375	0,124	0,237	0,789
200	0,426	0,155	0,260	0,835

Table 30 - Index SQ results for each buffer size

Regarding the combinations of the decision interval h and allowance k , it is observed that the SQ index exhibits stability, with mean values fluctuating within a narrow range (between 0.340 and 0.382) and being slightly higher for an allowance of $k = 1.0$. This behavior is statistically expected, given that SQ is synthesized from the arithmetic mean of the individual indexes R and S' , which demonstrate diametrically opposite responses to changes in the decision interval. As evidenced by the data, as the decision interval h increases, the Volume Comparison Index (R) decreases steadily (e.g., from 0.671 to 0.523 for $k = 1.0$), whereas, conversely, the normalized Generalized Variance Comparison Index (S') follows an upward trajectory (from 0.086 to 0.241 for the same combination). Precisely these opposing forces neutralize each other during the calculation of their arithmetic mean, trapping the mean value of the final SQ index around 0.35.

Although the mean remains relatively stable, the same cannot be said for the standard deviation. Specifically, while the standard deviation of the R index remains practically unchanged across variations in the decision interval, the corresponding dispersion of the S' index exhibits broadening (reaching from 0.068 to 0.364 for

$k = 1.0$). This differentiation is transferred to the SQ index, whose standard deviation more than triples (from 0.064 to 0.216). This statistical evolution reveals a loss of homogeneity in the algorithm's behavior at larger decision intervals. Characteristic in this regard is the sharp increase in the distance between the minimum and maximum recorded values of SQ (from [0.279 - 0.483] for $h = 6$ to [0.201 - 0.835] for $h = 9$). The graph below depicts the relationship between h , k , and SQ .

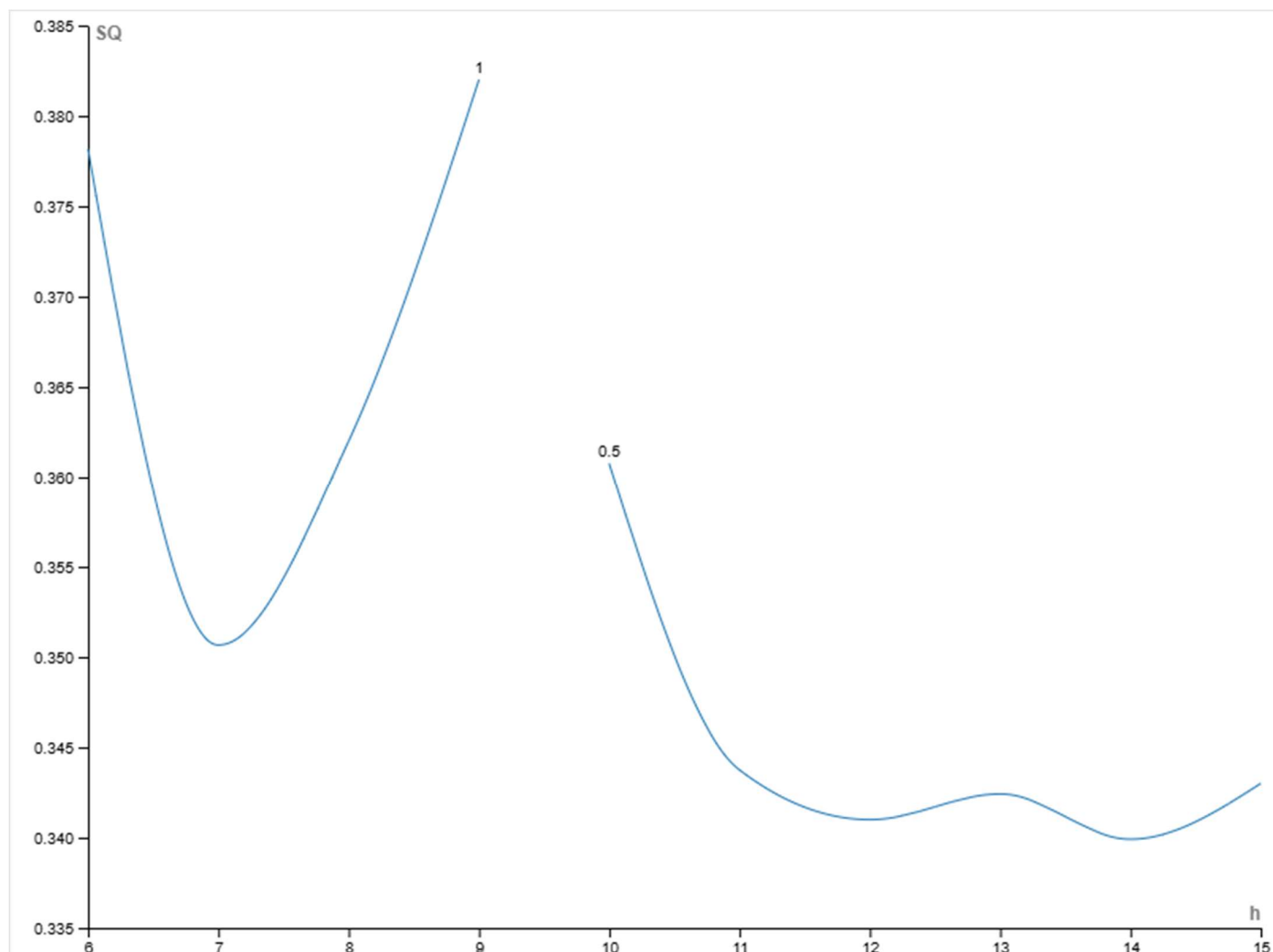


Figure 28 - Variation of index SQ as a function of the decision interval h for allowance values k^{26}

This graph visually confirms the stagnation of the SQ index. The narrow scale of the vertical axis (0.335–0.385) deceptively magnifies the fluctuations, turning marginal variations—such as the curve for $k = 1$ and the apparent drop from $h = 10$ to $h = 11$ for $k = 0.5$ —into seemingly pronounced disturbances. This specific visualization verifies that, due to the opposing dynamics of the individual indexes R and S' , the mean value of SQ is confined within a narrow range.

Regarding the relationship between the SQ index and the buffer size, we observe that performance improves with increasing size, rising from a mean of 0.289 at size 50 to a value of 0.426 at size 200. This behavior is expected given that the two individual indexes S' and R , which co-determine the value of the Sampling Quality Score (SQ), increase as the buffer capacity increases.

²⁶ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

Concurrently with the increase in the mean, the standard deviation and the range of values also increase. Specifically, the standard deviation more than doubles, from 0.061 at size 50 to 0.155 at size 200. At the same time, the span between the minimum and maximum values broadens, from [0.201 – 0.410] to [0.260 – 0.835]. This evolution proves that a larger capacity introduces structural instability, reducing homogeneity and making sampling quality more unpredictable. The above are also reflected in the boxplot below, which illustrates the widening of the interquartile range during the transition from size 50 to 200. Concurrently, a marked elongation of the upper whisker is observed at sizes 150 and 200, culminating in the appearance of isolated outliers at the upper end of the distribution. This visual confirmation of positive skewness reveals that the rise in the mean does not necessarily reflect a uniform and reliable improvement; rather, it is contributed to by sporadic, exceptionally high performances that increase dispersion without guaranteeing algorithm stability.

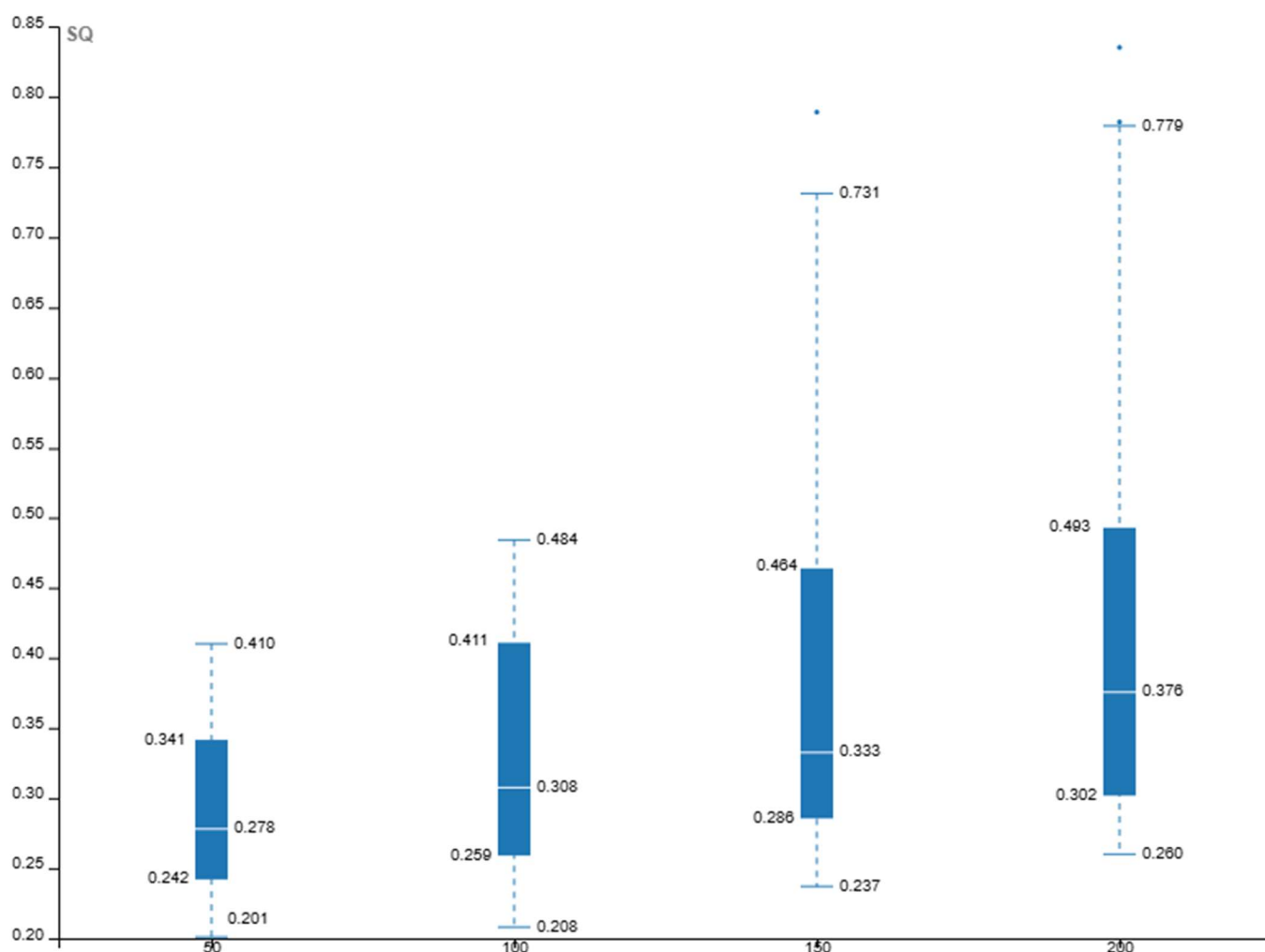


Figure 29 - Boxplot of index SQ as a function of the buffer size²⁷

Finally, regarding the replacement methods, the ranking of the methods in terms of performance follows the ranking of the individual indexes S' and R that form the SQ index. Thus, by far the best performance is achieved by the Mahalanobis Range Replacement Method, followed by the Max Range Replacement Method

²⁷ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

using Euclidean distance, and finally the Max Distance Expansion Replacement Method. This is also illustrated in the graphical representation below.

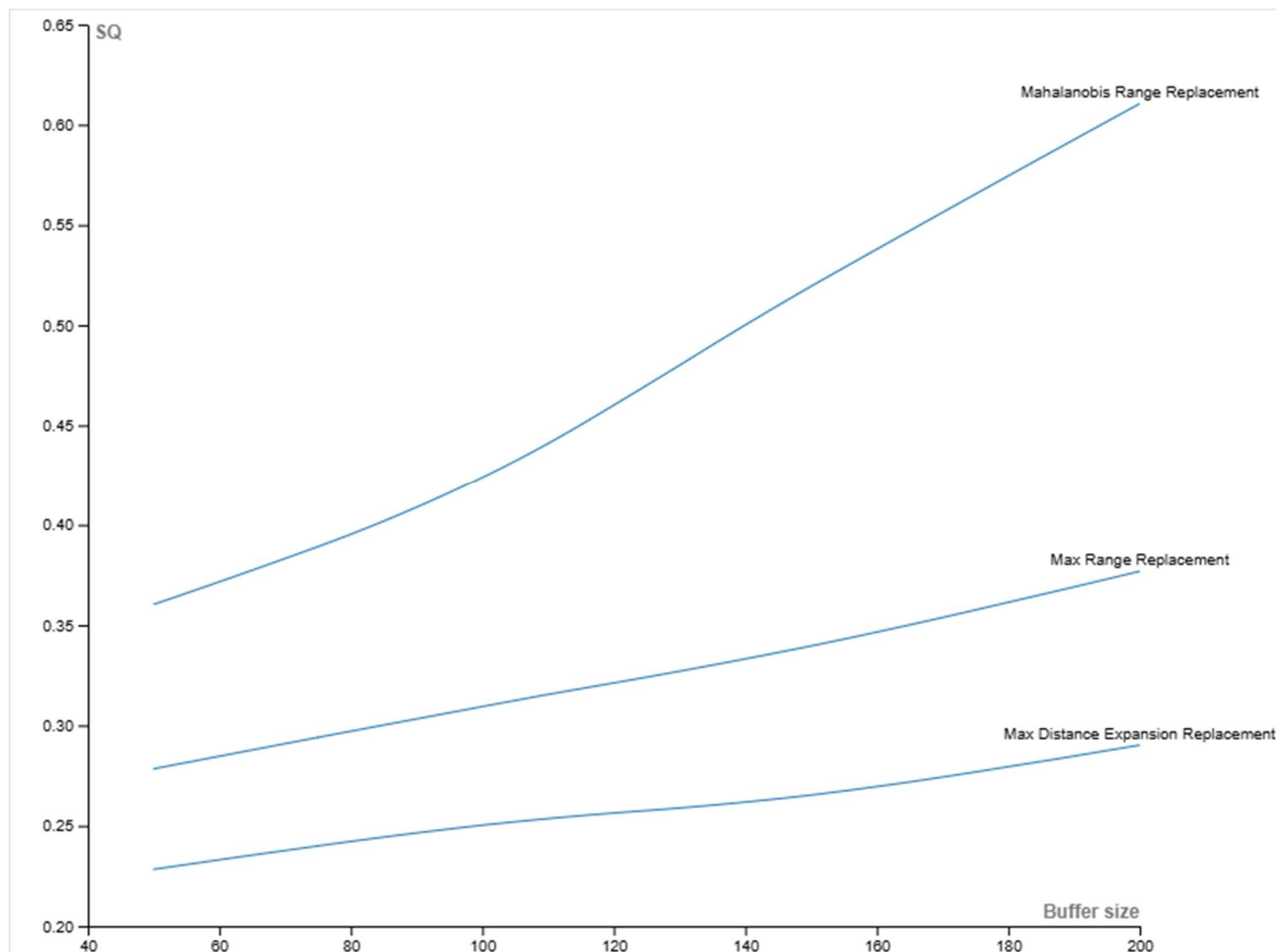


Figure 30 - Variation of index SQ as a function of buffer size and replacement method²⁸

9.4.2 STATISTICAL EFFICIENCY INDEX SE

The Statistical Efficiency Index (SE) integrates the Volume Comparison Index (R), the normalized Generalized Variance Comparison Index (S'), and the Compression Ratio (C) into a single value. Its purpose is to evaluate the algorithm's ability to reduce data volume while preserving statistical accuracy as previously defined. The table below shows the performance of the index as well as the average number of epochs for each parameter combination.

<u>Replacement method</u>	<u>Decision interval</u> <u>h</u>	<u>Allowance</u> <u>k</u>	<u>Buffer size</u>	<u>Number of epochs</u>	<u>Mean of SE</u>
Mahalanobis Range Replacement	6	1	50	212,6	0,465
Mahalanobis Range Replacement	6	1	100	102,5	0,483
Mahalanobis Range Replacement	6	1	150	71,3	0,499
Mahalanobis Range Replacement	6	1	200	56,6	0,505

²⁸ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Uboldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

Mahalanobis Range Replacement	7	1	50	119,3	0,492
Mahalanobis Range Replacement	7	1	100	52,0	0,537
Mahalanobis Range Replacement	7	1	150	35,7	0,588
Mahalanobis Range Replacement	7	1	200	29,8	0,654
Mahalanobis Range Replacement	8	1	50	65,9	0,512
Mahalanobis Range Replacement	8	1	100	33,0	0,567
Mahalanobis Range Replacement	8	1	150	24,3	0,683
Mahalanobis Range Replacement	8	1	200	22,0	0,796
Mahalanobis Range Replacement	9	1	50	46,4	0,520
Mahalanobis Range Replacement	9	1	100	24,4	0,623
Mahalanobis Range Replacement	9	1	150	20,0	0,819
Mahalanobis Range Replacement	9	1	200	17,7	0,842
Mahalanobis Range Replacement	10	0,5	50	192,2	0,471
Mahalanobis Range Replacement	10	0,5	100	73,5	0,509
Mahalanobis Range Replacement	10	0,5	150	51,8	0,536
Mahalanobis Range Replacement	10	0,5	200	43,1	0,535
Mahalanobis Range Replacement	11	0,5	50	128,2	0,496
Mahalanobis Range Replacement	11	0,5	100	51,2	0,532
Mahalanobis Range Replacement	11	0,5	150	37,0	0,568
Mahalanobis Range Replacement	11	0,5	200	30,6	0,583
Mahalanobis Range Replacement	12	0,5	50	90,0	0,512
Mahalanobis Range Replacement	12	0,5	100	40,1	0,554
Mahalanobis Range Replacement	12	0,5	150	29,3	0,593
Mahalanobis Range Replacement	12	0,5	200	26,2	0,614
Mahalanobis Range Replacement	13	0,5	50	66,6	0,525
Mahalanobis Range Replacement	13	0,5	100	33,3	0,565
Mahalanobis Range Replacement	13	0,5	150	25,3	0,602
Mahalanobis Range Replacement	13	0,5	200	23,3	0,677
Mahalanobis Range Replacement	14	0,5	50	52,1	0,532
Mahalanobis Range Replacement	14	0,5	100	29,7	0,579
Mahalanobis Range Replacement	14	0,5	150	24,3	0,609
Mahalanobis Range Replacement	14	0,5	200	22,2	0,699
Mahalanobis Range Replacement	15	0,5	50	47,5	0,531
Mahalanobis Range Replacement	15	0,5	100	28,1	0,586
Mahalanobis Range Replacement	15	0,5	150	23,7	0,614
Mahalanobis Range Replacement	15	0,5	200	21,4	0,718
Max Distance Expansion Replacement	6	1	50	212,6	0,377
Max Distance Expansion Replacement	6	1	100	102,5	0,397
Max Distance Expansion Replacement	6	1	150	71,3	0,407
Max Distance Expansion Replacement	6	1	200	56,6	0,417
Max Distance Expansion Replacement	7	1	50	119,3	0,407
Max Distance Expansion Replacement	7	1	100	52,0	0,430
Max Distance Expansion Replacement	7	1	150	35,7	0,438
Max Distance Expansion Replacement	7	1	200	29,8	0,450
Max Distance Expansion Replacement	8	1	50	65,9	0,431
Max Distance Expansion Replacement	8	1	100	33,0	0,442
Max Distance Expansion Replacement	8	1	150	24,3	0,446

Max Distance Expansion Replacement	8	1	200	22,0	0,461
Max Distance Expansion Replacement	9	1	50	46,4	0,437
Max Distance Expansion Replacement	9	1	100	24,4	0,439
Max Distance Expansion Replacement	9	1	150	20,0	0,453
Max Distance Expansion Replacement	9	1	200	17,7	0,474
Max Distance Expansion Replacement	10	0,5	50	192,2	0,381
Max Distance Expansion Replacement	10	0,5	100	73,5	0,419
Max Distance Expansion Replacement	10	0,5	150	51,8	0,431
Max Distance Expansion Replacement	10	0,5	200	43,1	0,435
Max Distance Expansion Replacement	11	0,5	50	128,2	0,408
Max Distance Expansion Replacement	11	0,5	100	51,2	0,436
Max Distance Expansion Replacement	11	0,5	150	37,0	0,445
Max Distance Expansion Replacement	11	0,5	200	30,6	0,447
Max Distance Expansion Replacement	12	0,5	50	90,0	0,424
Max Distance Expansion Replacement	12	0,5	100	40,1	0,448
Max Distance Expansion Replacement	12	0,5	150	29,3	0,455
Max Distance Expansion Replacement	12	0,5	200	26,2	0,451
Max Distance Expansion Replacement	13	0,5	50	66,6	0,432
Max Distance Expansion Replacement	13	0,5	100	33,3	0,458
Max Distance Expansion Replacement	13	0,5	150	25,3	0,450
Max Distance Expansion Replacement	13	0,5	200	23,3	0,452
Max Distance Expansion Replacement	14	0,5	50	52,1	0,437
Max Distance Expansion Replacement	14	0,5	100	29,7	0,454
Max Distance Expansion Replacement	14	0,5	150	24,3	0,448
Max Distance Expansion Replacement	14	0,5	200	22,2	0,449
Max Distance Expansion Replacement	15	0,5	50	47,5	0,440
Max Distance Expansion Replacement	15	0,5	100	28,1	0,457
Max Distance Expansion Replacement	15	0,5	150	23,7	0,444
Max Distance Expansion Replacement	15	0,5	200	21,4	0,449
Max Range Replacement	6	1	50	212,6	0,412
Max Range Replacement	6	1	100	102,5	0,439
Max Range Replacement	6	1	150	71,3	0,450
Max Range Replacement	6	1	200	56,6	0,460
Max Range Replacement	7	1	50	119,3	0,445
Max Range Replacement	7	1	100	52,0	0,471
Max Range Replacement	7	1	150	35,7	0,486
Max Range Replacement	7	1	200	29,8	0,506
Max Range Replacement	8	1	50	65,9	0,463
Max Range Replacement	8	1	100	33,0	0,480
Max Range Replacement	8	1	150	24,3	0,500
Max Range Replacement	8	1	200	22,0	0,529
Max Range Replacement	9	1	50	46,4	0,469
Max Range Replacement	9	1	100	24,4	0,479
Max Range Replacement	9	1	150	20,0	0,512
Max Range Replacement	9	1	200	17,7	0,536
Max Range Replacement	10	0,5	50	192,2	0,418
Max Range Replacement	10	0,5	100	73,5	0,461

Max Range Replacement	10	0,5	150	51,8	0,474
Max Range Replacement	10	0,5	200	43,1	0,483
Max Range Replacement	11	0,5	50	128,2	0,442
Max Range Replacement	11	0,5	100	51,2	0,476
Max Range Replacement	11	0,5	150	37,0	0,488
Max Range Replacement	11	0,5	200	30,6	0,503
Max Range Replacement	12	0,5	50	90,0	0,458
Max Range Replacement	12	0,5	100	40,1	0,489
Max Range Replacement	12	0,5	150	29,3	0,498
Max Range Replacement	12	0,5	200	26,2	0,504
Max Range Replacement	13	0,5	50	66,6	0,464
Max Range Replacement	13	0,5	100	33,3	0,492
Max Range Replacement	13	0,5	150	25,3	0,501
Max Range Replacement	13	0,5	200	23,3	0,515
Max Range Replacement	14	0,5	50	52,1	0,468
Max Range Replacement	14	0,5	100	29,7	0,490
Max Range Replacement	14	0,5	150	24,3	0,501
Max Range Replacement	14	0,5	200	22,2	0,512
Max Range Replacement	15	0,5	50	47,5	0,471
Max Range Replacement	15	0,5	100	28,1	0,496
Max Range Replacement	15	0,5	150	23,7	0,503
Max Range Replacement	15	0,5	200	21,4	0,516

Table 31 - Analytic results for SE index

Since evaluating the results from the above table is difficult due to the large volume of information, each parameter will initially be examined in isolation. It should be highlighted that the decision interval and allowance values are evaluated together, as different allowance values are combined with various decision intervals. The following tables present the mean, standard deviation, maximum, and minimum values for each parameter.

<u>Replacement method</u>	<u>Mean of SE</u>	<u>Standard deviation of SE</u>	<u>Minimum value of SE</u>	<u>Maximum Value of SE</u>	<u>Mean number of epochs</u>
Mahalanobis Range Replacement	0,583	0,092	0,465	0,842	53,1
Max Distance Expansion Replacement	0,436	0,021	0,377	0,474	53,1
Max Range Replacement	0,481	0,028	0,412	0,536	53,1

Table 32 - Index SE results for each replacement method

<u>Combination</u>	<u>Mean of SE</u>	<u>Standard deviation of SE</u>	<u>Minimum value of SE</u>	<u>Maximum Value of SE</u>	<u>Mean number of epochs</u>
k=0,5 / h=10	0,463	0,048	0,381	0,536	90,2
k=0,5 / h=11	0,485	0,054	0,408	0,583	61,8
k=0,5 / h=12	0,500	0,060	0,424	0,614	46,4
k=0,5 / h=13	0,511	0,072	0,432	0,677	37,1
k=0,5 / h=14	0,515	0,079	0,437	0,699	32,1
k=0,5 / h=15	0,519	0,083	0,440	0,718	30,2

k=1,0 / h=6	0,442	0,041	0,377	0,505	110,8
k=1,0 / h=7	0,492	0,071	0,407	0,654	59,2
k=1,0 / h=8	0,526	0,110	0,431	0,796	36,3
k=1,0 / h=9	0,550	0,141	0,437	0,842	27,1

Table 33 - Index SE results for each allowance and decision interval combination

<u>Buffer size</u>	<u>Mean of SE</u>	<u>Standard deviation of SE</u>	<u>Minimum value of SE</u>	<u>Maximum Value of SE</u>	<u>Mean number of epochs</u>
50	0,458	0,043	0,377	0,532	102,1
100	0,490	0,056	0,397	0,623	46,8
150	0,515	0,088	0,407	0,819	34,3
200	0,539	0,111	0,417	0,842	29,3

Table 34 - Index SE results for each buffer size

As shown in Table 33, for a constant allowance, as the decision interval increases, the performance of the algorithm on the Statistical Efficiency Score (SE) improves. Thus, for $k = 1$, the index ranges within the interval $[0.442 - 0.550]$, while for $k = 0.5$, it ranges between $[0.463 - 0.519]$. This behavior is expected, given that the SE index is the arithmetic mean of three other indexes: the Volume Comparison Index (R), the normalized Generalized Variance Comparison Index (S'), and the Compression Ratio (C). Although the R index decreases as h increases, the increase in the other two indexes is considered decisive for the overall behavior of the SE index. As analyzed in a previous section, the improvement in compression capability at higher decision intervals is attributed to the reduction in the number of generated epochs (from 110.8 to 27.1 average epochs for $k = 1.0$), a factor that ultimately dominates the formation of the mean SE value.

Regarding the standard deviation, although it generally maintains low values, it follows an upward trend as the decision interval increases. This broadening of the dispersion arises primarily from the instability introduced by the S' index when managing prolonged epochs. From the above, it follows that the improvement in mean statistical efficiency, which is mainly driven by the higher compression ratio, is accompanied by a decrease in system stability and the emergence of larger fluctuations in sampling quality. The graph below illustrates the relationship between the SE index and the decision interval for the two allowance values.

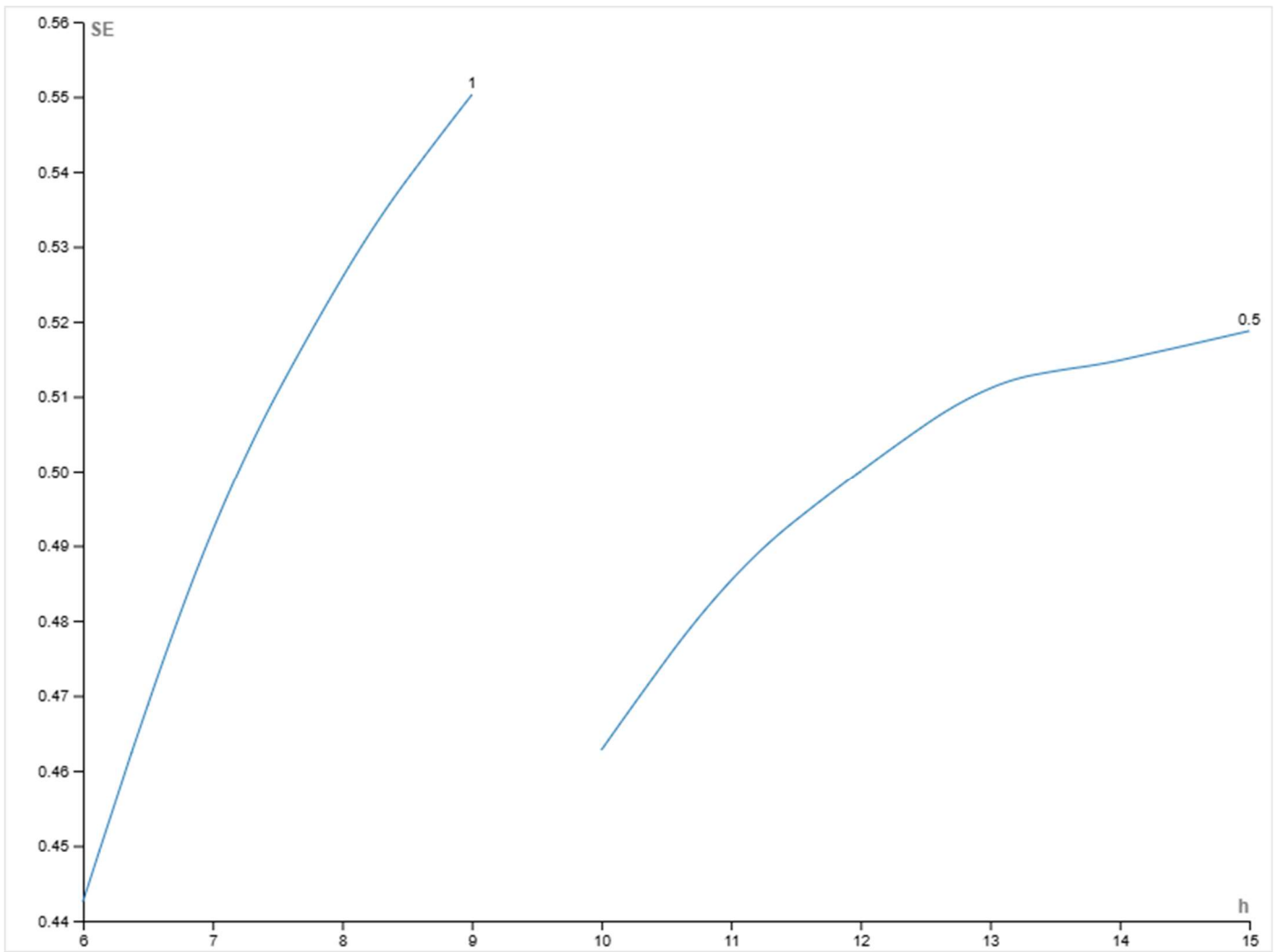


Figure 31 - Variation of index SE as a function of the decision interval h for allowance values k^{29}

From the above graph, a clear non-linear trend is evident. Specifically, for $k = 1.0$, it is observed that the growth rate of the SE index is slightly decreasing, giving the curve a strictly increasing, concave shape that resembles a logarithmic one. On the other hand, for $k = 0.5$, the growth rate decreases more sharply. As shown in Table 33, while in the initial steps the increase in h continues to drastically reduce the number of epochs (boosting the Compression Ratio C), at higher decision intervals the margin for their further reduction is exhausted, causing only marginal changes thereafter. Given that the SE index expresses the arithmetic mean of the C , R , and S' indexes, the marginal benefit of further eliminating epochs is no longer sufficient to overcome the steady degradation of the Volume Comparison Index (R), leading to the inevitable stagnation of the overall performance.

Regarding the relationship between the buffer size and the Statistical Efficiency Score, an increase in capacity leads to better statistical performance. Thus, the SE index rises from 0.458 for a buffer size of 50 to a value of 0.539 for a size of 200. As analyzed in the corresponding chapter, the mean value of the C index is maximized for a buffer size equal to 100 and subsequently shows a steady decline for sizes 150 and 200. On the other hand, the S' and R indexes increase as the buffer capacity increases. This increase in the two indexes is the decisive factor for the overall behavior of the SE index.

²⁹ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

The standard deviation generally maintains low values, but it increases as the buffer capacity increases. This behavior is mainly due to the instability introduced by the S' index when managing longer-lasting epochs, a fact that is also confirmed by the rapid decrease in the average number of generated epochs. Consequently, the overall improvement of the SE index is accompanied by the parallel emergence of larger fluctuations. The boxplot below illustrates the distribution of the Statistical Efficiency Index values per buffer size.

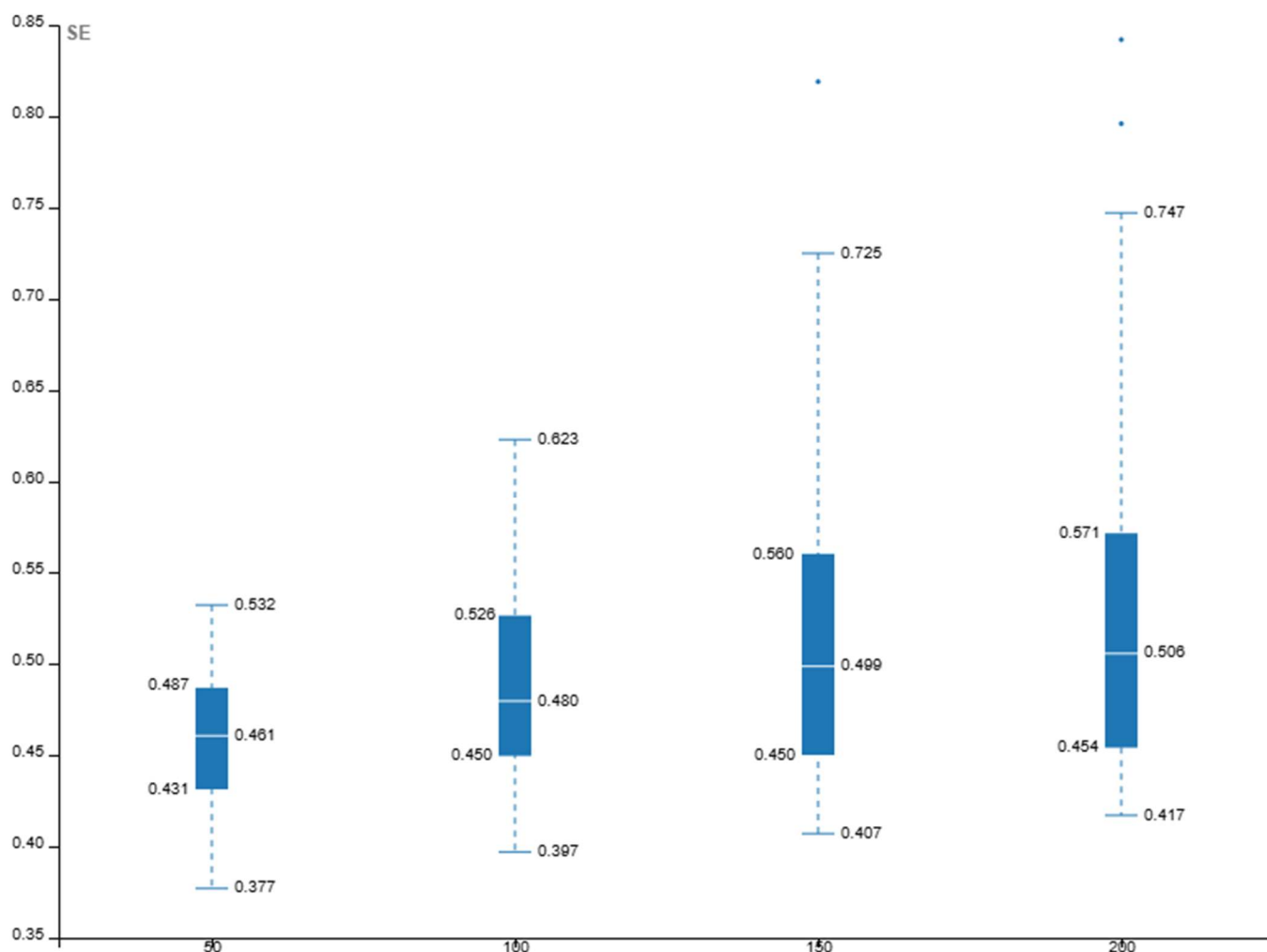


Figure 32 - Boxplot of index SE as a function of the buffer size³⁰

As observed, the median value of the SE index steadily increases with size, starting from 0.461 at size 50 and reaching 0.506 at size 200. Concurrently, both the overall range and the dispersion around the median exhibit an upward trend, as the interquartile range progressively widens (from 0.056 at size 50 to 0.117 at 200). The distribution presents a positive skewness, which becomes more pronounced at larger sizes. This is reflected by the drastic elongation of the upper whisker compared to the lower one, indicating that larger buffers allow the achievement of higher maximums, while the lower bound of the performances remains comparatively more compact. Finally, regarding the presence of outliers, it is observed that they are completely absent from the smaller sizes (50 and 100) and appear exclusively at the upper end of the larger capacities.

³⁰ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

Regarding the replacement methods, as shown in the Table 32, the Mahalanobis Range Replacement Method records by far the best overall performance on the Statistical Efficiency Score (SE)—in terms of the mean, maximum, and minimum values. It is followed by the Max Range Replacement Method, which uses Euclidean distance, and finally, the Max Distance Expansion Replacement Method. It is worth noting that the average number of epochs remains perfectly constant (53.1) regardless of the method, which implies that the resulting compression ratio is practically identical for all strategies. Therefore, the clear superiority of the Mahalanobis method on the SE index (0.583 compared to 0.481 and 0.436, respectively) stems exclusively from its ability to maximize the individual indexes R and S' . As explained above, this is due to the fact that this specific method incorporates the Mahalanobis distance, taking into account the correlations among the variables, as opposed to the other two methods that rely on Euclidean distance.

Regarding the standard deviation of the SE index, the Max Distance Expansion Replacement Method and the Max Range Replacement Method record exceptionally low values (0.021 and 0.028, respectively), in contrast to the Mahalanobis Range Replacement Method, which exhibits a noticeably higher dispersion (0.092). As observed in the case of the S' index, this difference is not an indication of reduced stability, but a natural consequence of the high efficiency of the method. The two Euclidean methods, failing to properly capture the multivariate distribution of extreme values, produce conservative summaries that confine the SE index to a narrow and low range (with maximum values of 0.474 and 0.536). On the other hand, the ability of the Mahalanobis method to respond to the actual fluctuations of the system allows it to push the maximum value of SE to 0.842, a fact that inevitably widens the overall range of the distribution and increases the standard deviation.

The above observations are also depicted in the graph below, which illustrates the relationship between the SE index and the buffer size for each replacement method. As can be seen, for all strategies without exception, an increase in buffer capacity leads to an improvement in the SE index, with the Mahalanobis Range Replacement Method significantly outperforming the rest across every examined size.

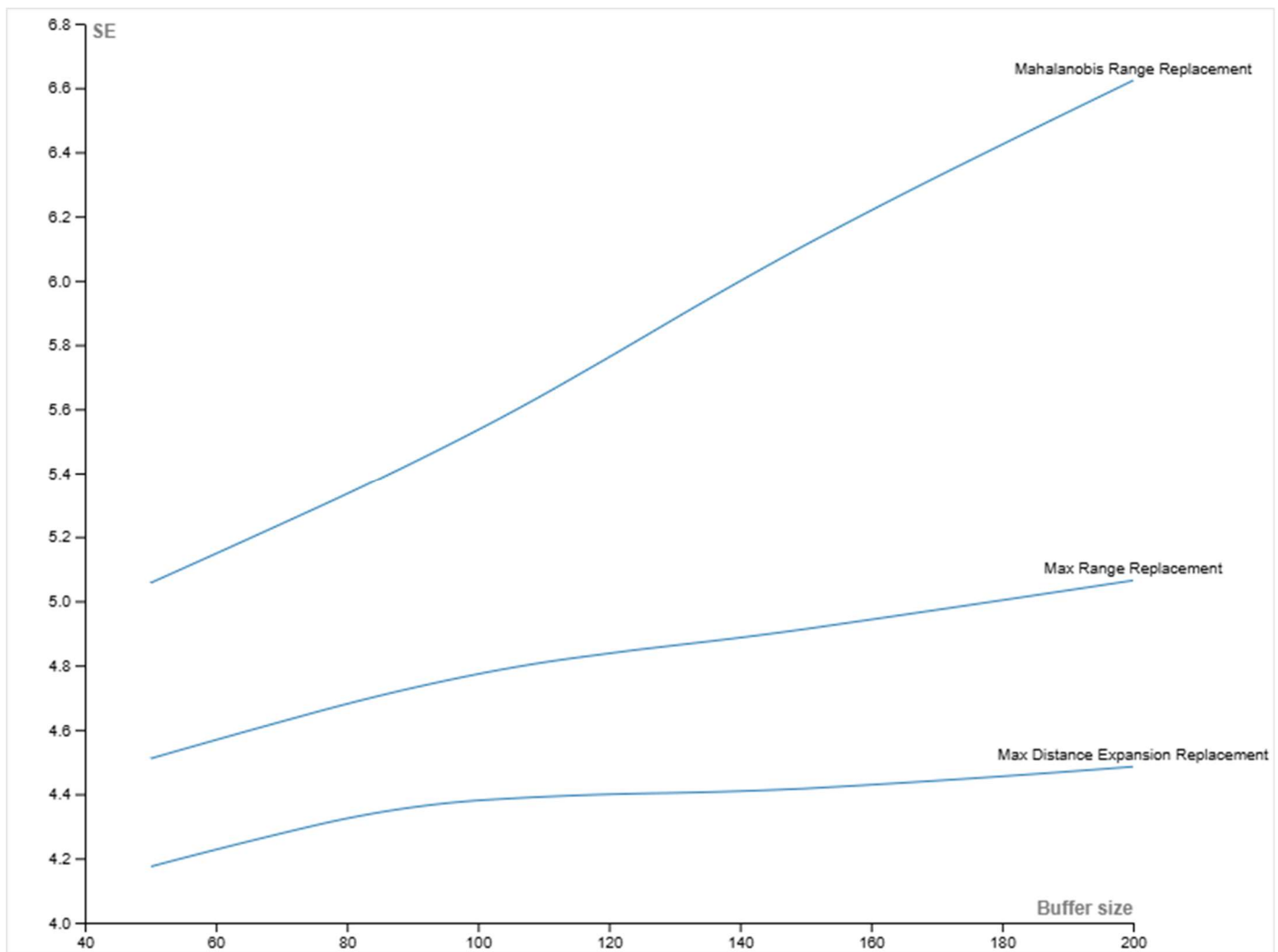


Figure 33 - Variation of index SE as a function of buffer size and replacement method³¹

9.4.3 TOTAL PERFORMANCE INDEX TP

The Total Performance Index (TP) reflects the performance of the algorithm across all individual metrics, namely its statistical precision as previously defined, data compression, and change detection. Thus, the index TP is calculated as the mean of the indexes R , S' , C and PATE - F1. The table below shows the performance of the index as well as the average number of epochs for each parameter combination.

Replacement method	Decision interval h	Allowance k	Buffer size	Mean number of epochs	Mean of TP
Mahalanobis Range Replacement	6	1	50	212,6	0,369
Mahalanobis Range Replacement	6	1	100	102,5	0,399
Mahalanobis Range Replacement	6	1	150	71,3	0,419
Mahalanobis Range Replacement	6	1	200	56,6	0,431
Mahalanobis Range Replacement	7	1	50	119,3	0,412
Mahalanobis Range Replacement	7	1	100	52,0	0,477
Mahalanobis Range Replacement	7	1	150	35,7	0,540
Mahalanobis Range Replacement	7	1	200	29,8	0,599

³¹ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

Mahalanobis Range Replacement	8	1	50	65,9	0,452
Mahalanobis Range Replacement	8	1	100	33,0	0,529
Mahalanobis Range Replacement	8	1	150	24,3	0,645
Mahalanobis Range Replacement	8	1	200	22,0	0,732
Mahalanobis Range Replacement	9	1	50	46,4	0,477
Mahalanobis Range Replacement	9	1	100	24,4	0,586
Mahalanobis Range Replacement	9	1	150	20,0	0,752
Mahalanobis Range Replacement	9	1	200	17,7	0,769
Mahalanobis Range Replacement	10	0,5	50	192,2	0,374
Mahalanobis Range Replacement	10	0,5	100	73,5	0,437
Mahalanobis Range Replacement	10	0,5	150	51,8	0,471
Mahalanobis Range Replacement	10	0,5	200	43,1	0,478
Mahalanobis Range Replacement	11	0,5	50	128,2	0,408
Mahalanobis Range Replacement	11	0,5	100	51,2	0,472
Mahalanobis Range Replacement	11	0,5	150	37,0	0,515
Mahalanobis Range Replacement	11	0,5	200	30,6	0,540
Mahalanobis Range Replacement	12	0,5	50	90,0	0,436
Mahalanobis Range Replacement	12	0,5	100	40,1	0,502
Mahalanobis Range Replacement	12	0,5	150	29,3	0,547
Mahalanobis Range Replacement	12	0,5	200	26,2	0,570
Mahalanobis Range Replacement	13	0,5	50	66,6	0,455
Mahalanobis Range Replacement	13	0,5	100	33,3	0,512
Mahalanobis Range Replacement	13	0,5	150	25,3	0,562
Mahalanobis Range Replacement	13	0,5	200	23,3	0,628
Mahalanobis Range Replacement	14	0,5	50	52,1	0,475
Mahalanobis Range Replacement	14	0,5	100	29,7	0,523
Mahalanobis Range Replacement	14	0,5	150	24,3	0,567
Mahalanobis Range Replacement	14	0,5	200	22,2	0,641
Mahalanobis Range Replacement	15	0,5	50	47,5	0,475
Mahalanobis Range Replacement	15	0,5	100	28,1	0,527
Mahalanobis Range Replacement	15	0,5	150	23,7	0,566
Mahalanobis Range Replacement	15	0,5	200	21,4	0,651
Max Distance Expansion Replacement	6	1	50	212,6	0,303
Max Distance Expansion Replacement	6	1	100	102,5	0,334
Max Distance Expansion Replacement	6	1	150	71,3	0,349
Max Distance Expansion Replacement	6	1	200	56,6	0,365
Max Distance Expansion Replacement	7	1	50	119,3	0,348
Max Distance Expansion Replacement	7	1	100	52,0	0,397
Max Distance Expansion Replacement	7	1	150	35,7	0,427
Max Distance Expansion Replacement	7	1	200	29,8	0,445
Max Distance Expansion Replacement	8	1	50	65,9	0,392
Max Distance Expansion Replacement	8	1	100	33,0	0,434
Max Distance Expansion Replacement	8	1	150	24,3	0,467
Max Distance Expansion Replacement	8	1	200	22,0	0,481
Max Distance Expansion Replacement	9	1	50	46,4	0,415
Max Distance Expansion Replacement	9	1	100	24,4	0,448
Max Distance Expansion Replacement	9	1	150	20,0	0,478

Max Distance Expansion Replacement	9	1	200	17,7	0,492
Max Distance Expansion Replacement	10	0,5	50	192,2	0,308
Max Distance Expansion Replacement	10	0,5	100	73,5	0,369
Max Distance Expansion Replacement	10	0,5	150	51,8	0,393
Max Distance Expansion Replacement	10	0,5	200	43,1	0,404
Max Distance Expansion Replacement	11	0,5	50	128,2	0,342
Max Distance Expansion Replacement	11	0,5	100	51,2	0,400
Max Distance Expansion Replacement	11	0,5	150	37,0	0,423
Max Distance Expansion Replacement	11	0,5	200	30,6	0,438
Max Distance Expansion Replacement	12	0,5	50	90,0	0,370
Max Distance Expansion Replacement	12	0,5	100	40,1	0,423
Max Distance Expansion Replacement	12	0,5	150	29,3	0,444
Max Distance Expansion Replacement	12	0,5	200	26,2	0,448
Max Distance Expansion Replacement	13	0,5	50	66,6	0,386
Max Distance Expansion Replacement	13	0,5	100	33,3	0,432
Max Distance Expansion Replacement	13	0,5	150	25,3	0,448
Max Distance Expansion Replacement	13	0,5	200	23,3	0,460
Max Distance Expansion Replacement	14	0,5	50	52,1	0,403
Max Distance Expansion Replacement	14	0,5	100	29,7	0,429
Max Distance Expansion Replacement	14	0,5	150	24,3	0,445
Max Distance Expansion Replacement	14	0,5	200	22,2	0,454
Max Distance Expansion Replacement	15	0,5	50	47,5	0,407
Max Distance Expansion Replacement	15	0,5	100	28,1	0,430
Max Distance Expansion Replacement	15	0,5	150	23,7	0,439
Max Distance Expansion Replacement	15	0,5	200	21,4	0,450
Max Range Replacement	6	1	50	212,6	0,329
Max Range Replacement	6	1	100	102,5	0,366
Max Range Replacement	6	1	150	71,3	0,382
Max Range Replacement	6	1	200	56,6	0,397
Max Range Replacement	7	1	50	119,3	0,376
Max Range Replacement	7	1	100	52,0	0,428
Max Range Replacement	7	1	150	35,7	0,463
Max Range Replacement	7	1	200	29,8	0,487
Max Range Replacement	8	1	50	65,9	0,416
Max Range Replacement	8	1	100	33,0	0,463
Max Range Replacement	8	1	150	24,3	0,507
Max Range Replacement	8	1	200	22,0	0,532
Max Range Replacement	9	1	50	46,4	0,439
Max Range Replacement	9	1	100	24,4	0,478
Max Range Replacement	9	1	150	20,0	0,523
Max Range Replacement	9	1	200	17,7	0,539
Max Range Replacement	10	0,5	50	192,2	0,335
Max Range Replacement	10	0,5	100	73,5	0,401
Max Range Replacement	10	0,5	150	51,8	0,424
Max Range Replacement	10	0,5	200	43,1	0,439
Max Range Replacement	11	0,5	50	128,2	0,367
Max Range Replacement	11	0,5	100	51,2	0,430

Max Range Replacement	11	0,5	150	37,0	0,455
Max Range Replacement	11	0,5	200	30,6	0,480
Max Range Replacement	12	0,5	50	90,0	0,396
Max Range Replacement	12	0,5	100	40,1	0,453
Max Range Replacement	12	0,5	150	29,3	0,476
Max Range Replacement	12	0,5	200	26,2	0,488
Max Range Replacement	13	0,5	50	66,6	0,410
Max Range Replacement	13	0,5	100	33,3	0,457
Max Range Replacement	13	0,5	150	25,3	0,486
Max Range Replacement	13	0,5	200	23,3	0,507
Max Range Replacement	14	0,5	50	52,1	0,427
Max Range Replacement	14	0,5	100	29,7	0,456
Max Range Replacement	14	0,5	150	24,3	0,485
Max Range Replacement	14	0,5	200	22,2	0,501
Max Range Replacement	15	0,5	50	47,5	0,430
Max Range Replacement	15	0,5	100	28,1	0,459
Max Range Replacement	15	0,5	150	23,7	0,483
Max Range Replacement	15	0,5	200	21,4	0,500

Table 35 - Analytic results for TP index

Since evaluating the results from the above table is difficult due to the large volume of information, each parameter will initially be examined in isolation. It should be highlighted that the decision interval and allowance values are evaluated together, as different allowance values are combined with various decision intervals. The following tables present the mean, standard deviation, maximum, and minimum values for each parameter.

<u>Replacement method</u>	<u>Mean of TP</u>	<u>Standard deviation of TP</u>	<u>Minimum value of TP</u>	<u>Maximum Value of TP</u>	<u>Mean number of epochs</u>
Mahalanobis Range Replacement	0,523	0,099	0,369	0,769	53,1
Max Distance Expansion Replacement	0,413	0,046	0,303	0,492	53,1
Max Range Replacement	0,447	0,052	0,329	0,539	53,1

Table 36 - Index TP results for each replacement method

<u>Combination</u>	<u>Mean of TP</u>	<u>Standard deviation of TP</u>	<u>Minimum value of TP</u>	<u>Maximum Value of TP</u>	<u>Mean number of epochs</u>
k=0,5 / h=10	0,403	0,051	0,308	0,478	90,2
k=0,5 / h=11	0,439	0,057	0,342	0,540	61,8
k=0,5 / h=12	0,463	0,058	0,370	0,570	46,4
k=0,5 / h=13	0,479	0,067	0,386	0,628	37,1
k=0,5 / h=14	0,484	0,067	0,403	0,641	32,1
k=0,5 / h=15	0,485	0,069	0,407	0,651	30,2
k=1,0 / h=6	0,370	0,038	0,303	0,431	110,8
k=1,0 / h=7	0,450	0,070	0,348	0,599	59,2
k=1,0 / h=8	0,504	0,098	0,392	0,732	36,3
k=1,0 / h=9	0,533	0,116	0,415	0,769	27,1

Table 37 - Index TP results for each allowance and decision interval combination

<u>Buffer size</u>	<u>Mean of TP</u>	<u>Standard deviation of TP</u>	<u>Minimum value of TP</u>	<u>Maximum Value of TP</u>	<u>Mean number of epochs</u>
50	0,398	0,048	0,303	0,477	102,1
100	0,448	0,054	0,334	0,586	46,8
150	0,486	0,080	0,349	0,752	34,3
200	0,512	0,095	0,365	0,769	29,3

Table 38 - Index TP results for each buffer size

Regarding the behavior of the Total Performance Score (TP), as observed in the table, for a constant allowance, an increase in the decision interval leads to an improvement in system performance. More specifically, for $k = 1.0$, the mean TP score increases from 0.370 (at $h = 6$) to 0.533 (at $h = 9$), whereas for $k = 0.5$ it ranges between 0.403 and 0.485. This evolution is justified by the fact that the TP score is calculated as the arithmetic mean of four individual metrics: C , R , S' , and PATE-F1. Although the R index degrades as h increases, the concurrent increase in the remaining indexes, and particularly the significant rise in compression due to the reduction of epochs, fully offsets this decline and dictates the positive trend of the mean. In fact, the influence of compression is so strong that it occasionally overshadows individual weaknesses at higher decision intervals, as is the case during the transition from $h = 14$ to $h = 15$. At this point, despite the degradation in detection capability (PATE-F1) caused by the loss of true alarms, the TP score continues to record a marginal increase (from 0.484 to 0.485), as the further elimination of epochs more than compensates for the drop in detection accuracy.

Regarding the standard deviation, it is observed that it starts at low values for small decision intervals (0.038 for $k = 1.0$ and $h = 6$) but follows a significantly upward trend, reaching 0.116 at $h = 9$. This broadening of the dispersion is also evident in the spread between the minimum and maximum values, which expands from $[0.303 - 0.431]$ to $[0.415 - 0.769]$. This behavior is almost exclusively attributed to the instability introduced by the S' index when managing longer-lasting epochs, as the buffer's inability to consistently represent the increased volume of information drives up the overall variability of the system. From the above, it follows that the improvement in mean overall performance, which is primarily driven by the high compression ratio, results in a decrease in system stability and the emergence of larger fluctuations in the final sampling quality. The graph below depicts the relationship between h , k , and TP .

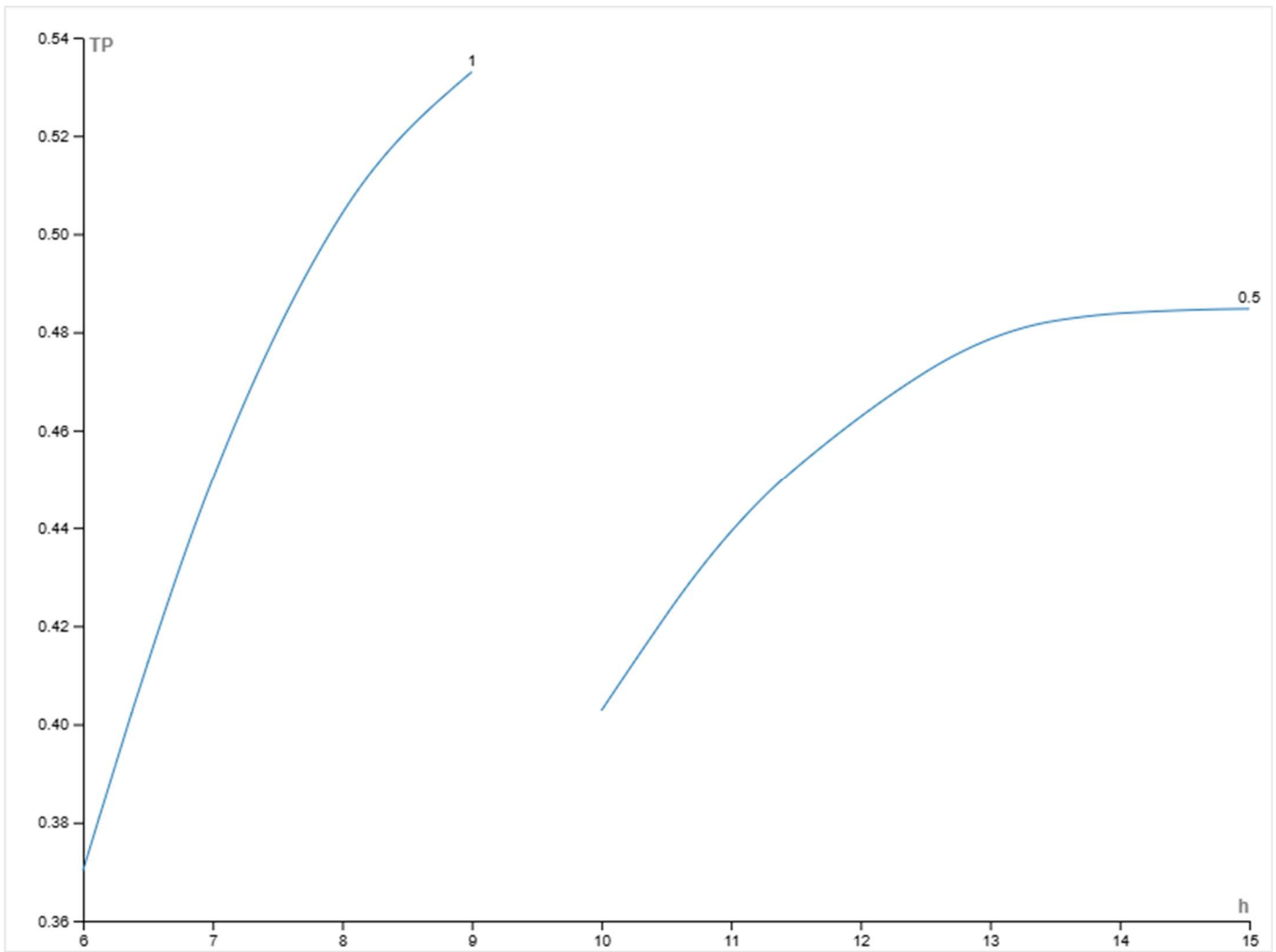


Figure 34 - Variation of index TP as a function of the decision interval h for allowance values k^{32}

From the above graph, a clear non-linear trend is evident. Specifically, for $k = 1.0$, it is observed that the growth rate of the Total Performance Index (TP) is slightly decreasing, giving the curve a strictly increasing, concave shape. On the other hand, for $k = 0.5$, the growth rate decreases more sharply at higher decision intervals and, ultimately, as the decision interval approaches $h = 15$, the curve tends to completely stabilize. This shape visually depicts what was analyzed previously: in the initial steps, the rapid elimination of epochs boosts compression, dynamically pushing the overall score upward. However, at extreme decision intervals, the margin for further compression is exhausted. Consequently, this dynamic is no longer sufficient to overcome with the same momentum the degradation of the detection capability (PATE-F1) and the continuous drop of the Volume Comparison Index (R), a fact characteristically reflected by the halt in the upward trajectory of TP .

Regarding the effect of buffer capacity on the Total Performance Index, the use of larger sizes steadily enhances the final system performance. In particular, the mean score starts from 0.398 for a buffer of 50 measurements and peaks at 0.512 for a size of 200. Although, as noted earlier, the compression capability reaches its maximum point at size 100 and then begins to decline, the overall trend of TP remains upward. This is explained by the simultaneous, continuous improvement of both the volume and generalized variance

³² The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

indices, as well as the PATE-F1 index. The combined increase of these three parameters at larger buffers more than compensates for the decrease in compression, shaping the positive growth rate of the mean.

Examining the standard deviation, it is observed that while at small sizes the variability is kept at low levels (0.048 at size 50), it gradually widens, approaching 0.095 at size 200. This increasing dispersion is also reflected in the corresponding distribution limits, where the range between the minimum and maximum values expands from the initial [0.303 - 0.477] to [0.365 - 0.769]. The cause of this destabilization is primarily located in the difficulty of the S' index to consistently capture the dispersion when managing large epochs. The following boxplot illustrates the distribution of the index values per buffer size.

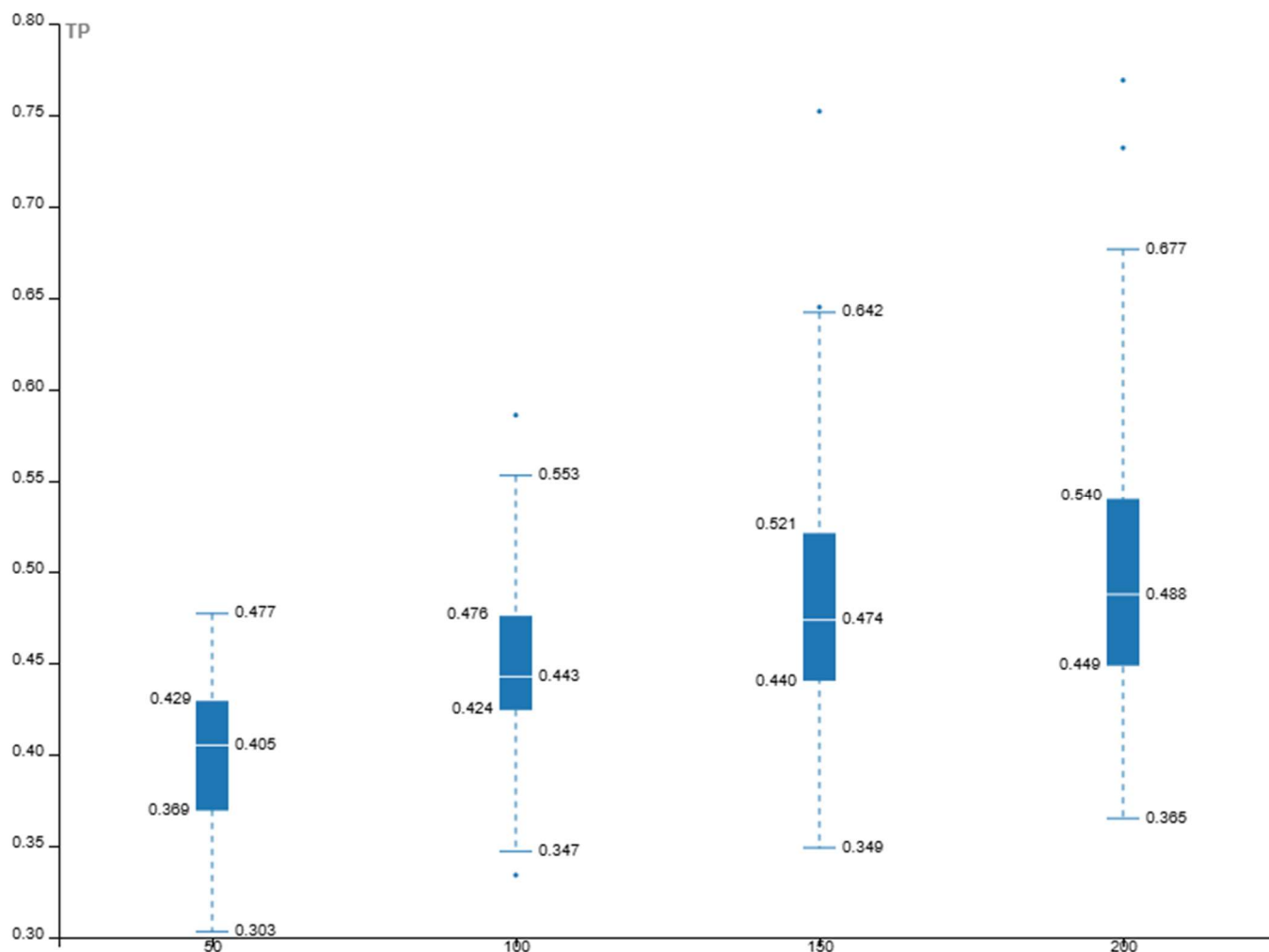


Figure 35 - Boxplot of index TP as a function of the buffer size³³

As observed in the graph, the median value of the index follows an upward trajectory, starting from 0.405 for a capacity of 50 measurements and ending at 0.488 for a size of 200. Concurrently, an expansion of the overall dispersion is observed. Furthermore, while the interquartile range is kept at relatively limited levels at smaller sizes, it increases noticeably at sizes 150 and 200. Additionally, the distribution acquires a positive skewness, which is evident from the elongation of the upper whisker compared to the lower one at sizes 100, 150, and 200. This specific shape demonstrates that larger buffers allow the system to sporadically achieve high performances, while the lower performance bound remains more compact. Finally, regarding

³³ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

outliers, they are completely absent at size 50. The behavior at size 100 is noteworthy, where both a negative and a positive outlier are recorded, whereas at larger sizes (150 and 200) outliers are exclusively located at the upper end, confirming the system's potential for occasional peaks in the *TP*index.

Regarding the effect of replacement methods on the Total Performance Index, it emerges that the Mahalanobis Range Replacement Method outperforms the others in all key statistics. It is followed by the Max Range Replacement Method, while the lowest overall performance is recorded by the strategy. As already mentioned, the compression ratio and detection capability remain perfectly constant for all three replacement approaches. Therefore, the superiority of the Mahalanobis method on the overall index (0.523 compared to 0.447 and 0.413) is attributed exclusively to its ability to maximize the R and S' indexes. This is achieved because the Mahalanobis distance incorporates the correlations and different scales of the variables, more faithfully preserving the multivariate information within the buffer, as opposed to Euclidean methods that treat all dimensions equally.

Regarding the variability of the results, the Mahalanobis method exhibits a noticeably higher standard deviation (0.099), whereas the two Euclidean strategies maintain lower dispersion (0.046 and 0.052). This increased deviation does not constitute an indication of structural instability, but is a direct consequence of the method's operational mode. Strategies based on Euclidean distance fail to capture multivariate extreme states, confining performances to a narrow and conservative range of values. Conversely, the ability of the Mahalanobis method to faithfully represent the actual fluctuations of the system allows the *TP*index to reach maximum values of up to 0.769, a fact that inevitably widens the distribution and increases the standard deviation.

The above conclusions are visualized in the graph below, which illustrates the relationship between the *TP* index and the buffer size for each replacement method. As shown, for all strategies without exception, an increase in buffer capacity leads to an improvement in the index, with the Mahalanobis Range Replacement Method significantly outperforming the rest across every examined size.

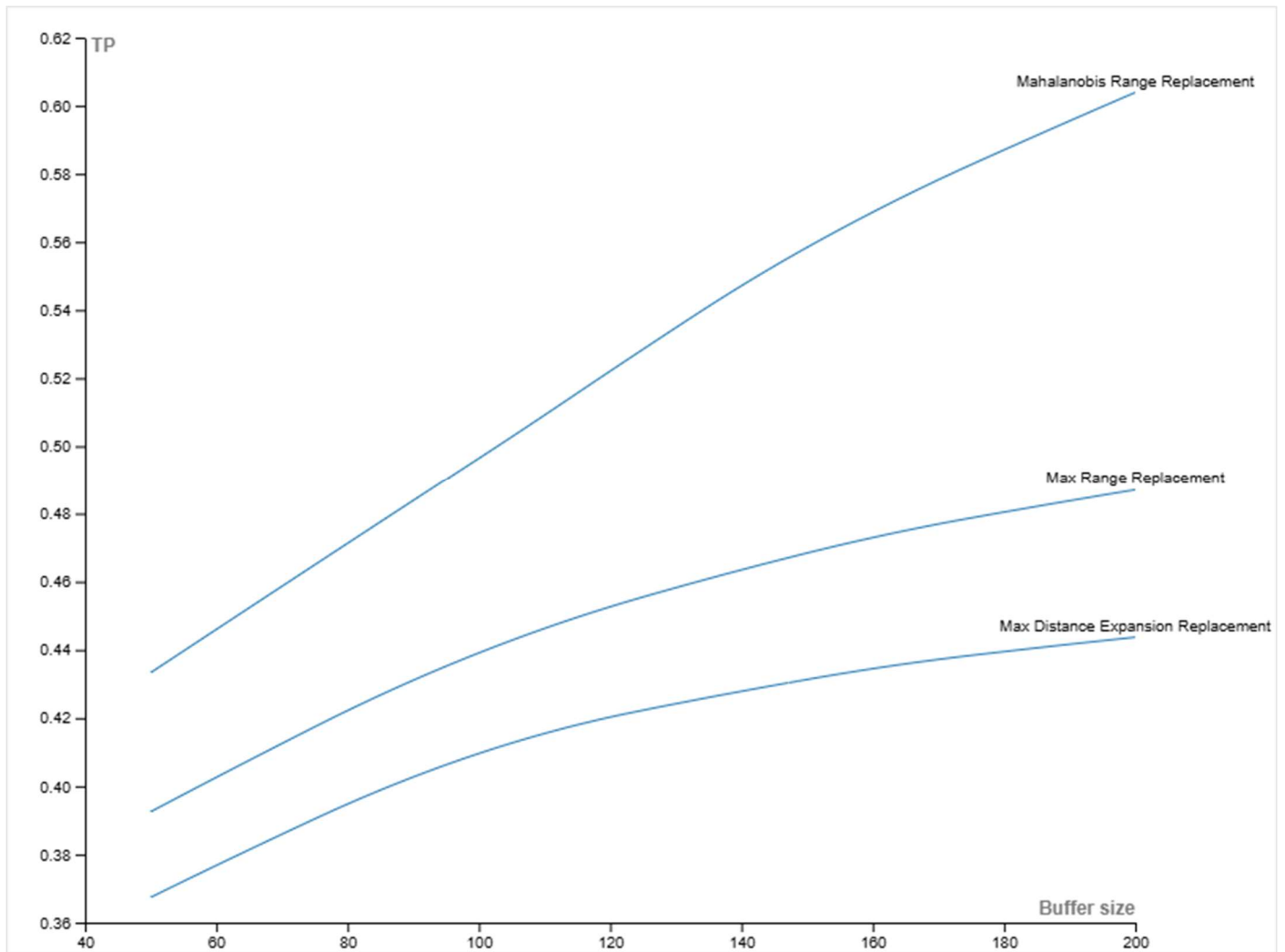


Figure 36 - Variation of index TP as a function of buffer size and replacement method³⁴

³⁴ The graph was created using the tool RAWGraphs (Mauri, M., Elli, T., Caviglia, G., Ubaldi, G., & Azzi, M. 2017. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, 28:1–28:5. New York, NY: ACM. <https://doi.org/10.1145/3125571.3125585>).

10. CONCLUSIONS

The proposed method aims to compress data at the edge node prior to transmitting them to the cloud, while preserving their statistical information. Consequently, the evaluation is not limited to data volume reduction, but also examines the extent to which the retained sample captures the original dataset, particularly regarding its extreme values and variability.

The initial data stream was divided into epochs using the multivariate CUSUM algorithm, which generates alarms when the cumulative sum of measurement deviations from the mean vector exceeds a predefined decision interval. At each epoch transition, the data stored in the buffer are transmitted to the cloud, and the reservoir is cleared to store the measurements of the new epoch.

The selection of data to remain in the buffer is performed via replacement algorithms. Initially, the buffer reaches a full state. Once full, the replacement method is activated, checking whether each new measurement meets specific criteria to replace an existing one. Three such strategies are compared in this study: the Max Range Replacement Method (using Euclidean distance), the corresponding Mahalanobis Range Replacement Method, and the Max Distance Expansion Replacement Method.

The implementation of the method was carried out using a modular design and object-oriented classes. Thus, individual system components, such as the replacement method, can be replaced without requiring changes to the rest of the implementation.

The optimal parameter values derived from the experimental procedure cannot be considered universal, as each application has different requirements. For instance, in a system with limited bandwidth, greater weight may be given to compression, whereas in an application where data are used for further analysis in the cloud, preserving their statistical information may be more important.

For this reason, the evaluation was based on both individual indexes (R , S' , C , PATE-F1) and composite indexes (SQ , SE , TP). The use of multiple indexes allows the examination of data compression, statistical information preservation, and detection capability alike. Furthermore, the composite indexes can be adjusted through the use of different weights, depending on the needs of each application. For example, when compression is deemed more important than statistical accuracy, the C index can be assigned a greater weight in the SE and TP indexes.

10.1.1 CONCLUSIONS ON PARAMETERIZATION

The selection of parameters significantly affects the system's behavior. The three core parameters, namely the allowance (k), the decision interval (h), and the buffer size, affect both the compression ratio and the statistical fidelity of the data.

10.1.1.1 Allowance and decision interval

The increase of parameters k and h affects the individual statistical fidelity indexes differently. Specifically, increasing the decision interval h reduces the sensitivity of CUSUM and leads to fewer and longer epochs. As a result, the Volume Comparison Index (R) exhibits a downward trend, achieving its highest value at $h = 6$ and $k = 1$.

Conversely, the normalized Generalized Variance Comparison Index (S') increases as h increases. Longer epochs allow the retention of more extreme values in the sample, causing S' to reach its maximum value at $h = 9$ and $k = 1$. The opposing trends of R and S' result in relatively stable values of SQ , with the maximum value also occurring at $h = 9$ and $k = 1$.

The compression ratio (C) also improves with increasing h , showing its highest value at $h = 9$ and $k = 1$. This behavior is linked to the reduction in the number of epochs, as fewer epochs entail fewer data transmissions to the cloud.

A similar trend appears in the PATE-F1 index, which achieves its highest value at $h = 9$ and $k = 1$. In the specific experiments, increasing h appears to significantly limit false alarms without a correspondingly large loss of true alarms.

The SE and TP indexes also exhibit an upward trend and reach their maximum value at $h = 9$ and $k = 1$. The improvement in C , S' , and PATE-F1 is sufficient for the overall indexes to increase despite the decline in R . Therefore, for these specific experiments, the combination of $h = 9$ and $k = 1$ represents the best choice based on the overall indexes.

10.1.1.2 Buffer size

Increasing the buffer size improves the statistical fidelity indexes. With a larger capacity, more samples can be retained, increasing the probability of recording extreme values in each dimension. This is reflected in the values of R and S' , which achieve their best performance for a buffer equal to 200. SQ follows the same trend and also presents its maximum value at a size of 200.

The compression ratio (C) exhibits a different behavior. Its mean value is higher for a buffer size of 100 and decreases at sizes 150 and 200. Thus, increasing capacity does not lead to a corresponding improvement in compression, since larger buffers reduce the rate of epoch reduction and ultimately increase the volume of transmitted data.

PATE-F1 steadily improves and achieves its highest value for a buffer size of 200. The larger capacity provides more samples for estimating the covariance matrix, leading to a more stable estimation and, in the specific experiments, a significant reduction in false alarms.

SE and TP increase with buffer size and reach their maximum values at a size of 200. The improvement in R , S' , and PATE-F1 offsets the reduction in compression at larger buffers.

However, sizes 150 and 200 exhibit a higher standard deviation and more outliers in overall performance. Therefore, the 200-sample buffer yields the best average overall performance, but size 100 represents a more balanced choice when greater weight is given to compression. In this case, the compression ratio amounts to $C = 0.813$, while performance variance remains at more restricted levels.

10.1.2 CONCLUSIONS ON REPLACEMENT METHODS

The choice of the replacement method mainly affects the statistical fidelity of the sample, while it does not affect the compression ratio (C) or the detection capability (PATE-F1). This is due to the operation of the two mechanisms: CUSUM determines epoch changes, whereas the replacement method is used to manage the buffer after it becomes full.

Among the three methods, the Mahalanobis Range Replacement Method records the optimal performance in both the Volume Comparison Index (R) and the normalized Generalized Variance Comparison Index (S'). This is because the Mahalanobis distance incorporates the correlations between variables and the different measurement scales, preserving multivariate information more faithfully. In contrast, the Max Range Replacement Method and the Max Distance Expansion Replacement Method, which are based on Euclidean distance, treat all dimensions equally, resulting in a significant lag in capturing the actual multivariate data envelope.

Consequently, the Mahalanobis method completely dominates all three composite indexes (SQ , SE , and TP), systematically presenting the highest mean values across every parameter combination and buffer size. Although this specific strategy exhibits a higher standard deviation in indexes such as S' , this does not indicate structural instability. On the contrary, it proves its ability to faithfully represent the actual fluctuations of the system by pushing maximum values to the limit, unlike Euclidean methods which fail to detect extreme states and confine performances to a narrow range.

10.2 LIMITATIONS AND PROPOSALS FOR FURTHER RESEARCH

The present work focused on demonstrating the functionality and comparative evaluation of the compression architecture and replacement strategies at the edge under controlled conditions. In this context, certain theoretical limitations emerge, which constitute fertile ground for future investigation:

- Validation on real telemetry data: The system evaluation was performed on synthetic data generated via an AR(1) model and a multivariate normal distribution. Although the residuals method removes serial correlation, a natural extension of the research involves testing the architecture on real, raw ship time series. In such environments, deviations from normality, the prominent presence of noise, and momentary sensor failures will allow testing the robustness of Vector CUSUM under real operating conditions.
- Dynamic parameter adaptation: In the present implementation, the CUSUM parameters (h , k) remain constant throughout the stream. An interesting research direction is the development of adaptive mechanisms, where the decision interval h and the allowance k are dynamically readjusted according to the variance and duration of each epoch, achieving an optimized balance between compression and detection accuracy.
- Integration of outlier filtering mechanisms: During the sampling process, the buffer replacement methods seek to retain extreme points for the faithful representation of dispersion. In a real deployment, future research should integrate mechanisms to distinguish between actual extreme operational states and false extreme measurements, so as to prevent the introduction of erroneous dispersion into the summary.

11. REFERENCES

- Crosier, R.B. (1988) 'Multivariate Generalizations of Cumulative Sum Quality-Control Schemes', *Technometrics*, 30(3), pp.291-303. Available at: <https://www.jstor.org/stable/1270083> (Accessed: 29 August 2026).
- Devore, J.L. (2012) *Probability and Statistics for Engineering and the Sciences*. 8th edn. Boston, MA: Cengage Learning.
- Ghorbani, R., Reinders, M.J.T. and Tax, D.M.J. (2024) 'PATE: Proximity-Aware Time Series Anomaly Evaluation', in *KDD '24: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp.872-883. ACM. doi:10.1145/3637528.3671971.
- Hawkins, D.M. and Olwell, D.H. (1998) *Cumulative Sum Charts and Charting for Quality Improvement*. New York: Springer. doi:10.1007/978-1-4612-1686-5.
- Huet, A., Navarro, J.M. and Rossi, D. (2022) 'Local Evaluation of Time Series Anomaly Detection Algorithms', in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. New York: ACM. doi:10.1145/3534678.3539339.
- Johnson, R.A. and Wichern, D.W. (2007) *Applied Multivariate Statistical Analysis*. 6th edn. Upper Saddle River, NJ: Pearson Prentice Hall.
- Karlis, D. (2003) *Polymetavlitiki Statistiki Analysi*. Athens: Ekdoseis Oikonomikou Panepistimiou Athinon.
- Mahmoud, M.A. and Maravelakis, P.E. (2013) 'The performance of multivariate CUSUM control charts with estimated parameters', *Journal of Statistical Computation and Simulation*, 83(4), pp.721-738. doi:10.1080/00949655.2011.633910.
- Mesnil, B. and Petitgas, P. (2009) 'Detection of changes in time-series of indicators using CUSUM control charts', *Aquatic Living Resources*, 22, pp.187-192. doi:10.1051/alr/2008058.
- Montgomery, D.C. (2013) *Introduction to Statistical Quality Control*. 7th edn. Hoboken, NJ: John Wiley & Sons.
- Page, E.S. (1963) 'Controlling the Standard Deviation by Cusums and Warning Lines', *Technometrics*, 5(3), pp.307-315. Available at: <https://www.jstor.org/stable/1266335> (Accessed: 29 August 2026).
- Qiu, P. (2014) *Introduction to Statistical Process Control*. Boca Raton: CRC Press.
- Raykov, T. and Marcoulides, G.A. (2008) *An Introduction to Applied Multivariate Analysis*. New York: Routledge.
- Safaridis, F., Katsarou, E. and Hadjiefthymiades, S. (2025) 'Reducing IoT Data for Highly Efficient Cloud Storage', *Procedia Computer Science*, 257, pp.182-189.
- Siegmund, D. (1985) *Sequential Analysis Tests and Confidence Intervals*. New York: Springer-Verlag.
- Simopoulos, S.E. (1989) *Metriseis Technikon Megethon*. 2nd edn. Athens: Panepistimiakes Ekdoseis E.M.P.

Tillé, Y. (2006) *Sampling Algorithms*. New York: Springer.

Werman, M., Peleg, S. and Rosenfeld, A. (1985) 'A Distance Metric for Multidimensional Histograms', *Computer Vision, Graphics, and Image Processing*, 32, pp.328-336.

Woodall, W.H. and Ncube, M.M. (1985) 'Multivariate CUSUM Quality-Control Procedures', *Technometrics*, 27(3), pp.285-292. doi:10.1080/00401706.1985.10488053.

APPENDIX A: ALGORITHM CORE IMPLEMENTATION

MAIN STREAM MANAGEMENT SCRIPT

```
import os
import pandas as pd
import numpy as np
from numpy.linalg import inv

from buffer import DataBuffer
from cov_mean_calculation import CovMeanCalculation
from vector_cusum import VectorCusum
from max_range_replacement import MaxRangeReplacement
from mahalanobis_range_replacement import MahalanobisRangeReplacement
from max_distance_expansion_replacement import MaxDistanceExpansionReplacement
from synopsis_exporter import SynopsisExporter

# 1. Configuration & Setup
base_dir = os.path.dirname(os.path.abspath(__file__))
parent_dir = os.path.join(base_dir, '..')
output_base_dir = os.path.join(parent_dir, 'mcusum_outputs')

# Replacing methods
replacement_methods = [
    'MaxRangeReplacement',
    'MahalanobisRangeReplacement',
    'MaxDistanceExpansionReplacement'
]

# Folders for each replacement method
for method in replacement_methods:
    method_dir = os.path.join(output_base_dir, method)
    if not os.path.exists(method_dir):
        os.makedirs(method_dir)

# Target parameters
```

```

parameters_grid = [
    {"k": 0.5, "h": 10.0},
    {"k": 0.5, "h": 11.0},
    {"k": 0.5, "h": 12.0},
    {"k": 0.5, "h": 13.0},
    {"k": 0.5, "h": 14.0},
    {"k": 0.5, "h": 15.0},
    {"k": 1.0, "h": 6.0},
    {"k": 1.0, "h": 7.0},
    {"k": 1.0, "h": 8.0},
    {"k": 1.0, "h": 9.0}
]

buffer_sizes = [50, 100, 150, 200]

# Initialize the stateless statistics calculator
calculator = CovMeanCalculation()

print("Starting overall simulation for 10 datasets...\n")

# OUTER LOOP: Iterate over the 10 datasets
for dataset_id in range(1, 11):
    print(f"=== Processing Dataset {dataset_id}/10 ===")

    # Define input path for the current dataset
    data_file_path = os.path.join(parent_dir, 'synthetic_data', f'synthetic_ship_dataset_{dataset_id}.xlsx')

    # Folders for each dataset
    for method in replacement_methods:
        dataset_output_dir = os.path.join(output_base_dir, method, f'Synthetic ship dataset {dataset_id}')
        if not os.path.exists(dataset_output_dir):
            os.makedirs(dataset_output_dir)

    # Load the Excel dataset
    df = pd.read_excel(data_file_path)

```

```

# Exclude Timestamp if it exists to isolate numeric variables for linear algebra operations
if 'Timestamp' in df.columns:
    df = df.drop(columns=['Timestamp'])

feature_names = df.columns.tolist()
data_stream = df.values.astype(np.float32)
total_measurements = len(data_stream)
variables_num = data_stream.shape[1]

print(f"Dataset loaded: {total_measurements} measurements, {variables_num} variables.")

# 2. Main Execution Loop (For the current dataset)
for params in parameters_grid:
    k_val = params["k"]
    h_val = params["h"]

    for b_size in buffer_sizes:

        # For each replacement method
        for method_name in replacement_methods:
            print(f"  -> Running K={k_val}, H={h_val}, Buffer Size={b_size}, Method={method_name}...")

            # Initialize components for the current configuration
            buffer = DataBuffer(b_size, variables_num)
            exporter = SynopsisExporter(h_val, k_val, b_size)

            # Dynamically redirect the exporter's file path to the specific METHOD and DATASET sub-folder
            dataset_output_dir = os.path.join(output_base_dir, method_name, f'Synthetic ship dataset
{dataset_id}')

            exporter.file_name = os.path.join(dataset_output_dir, os.path.basename(exporter.file_name))

            epoch_id = 1
            start_index = 0

```

```

# Pointers for our dynamic objects
cusum = None
replacer = None

previous_measurement = None

for index, new_measurement in enumerate(data_stream):
    # Phase 1: Sequential filling of the Buffer
    if not buffer.isFull():
        buffer.insert(new_measurement, index)

    # Check if the buffer JUST became full (Transition trigger)
    if buffer.isFull():

        # Retrieve buffer views for matrix calculation
        buffer_data, buffer_ids = buffer.getDataAndIds()

        # Calculate Baseline ( $\Sigma$  and  $\mu$ ) using the differentials
        diff_data = calculator.differential_calc(buffer_data)
        mean_vector_diff, covariance_matrix_diff = calculator.load_cov_mean(diff_data)

        # Initialize Crosier's MCUSUM
        cusum = VectorCusum(mean_vector_diff, covariance_matrix_diff, h_val, k_val)

        # Mean and covariance matrix of the raw data, not the differentials
        mean_vector_raw, covariance_matrix_raw = calculator.load_cov_mean(buffer_data)

        # Replacement classes initialization
        if method_name == 'MaxRangeReplacement':
            replacer = MaxRangeReplacement(buffer_data, buffer_ids)
        elif method_name == 'MahalanobisRangeReplacement':
            raw_inv_cov_matrix = inv(covariance_matrix_raw)
            replacer = MahalanobisRangeReplacement(buffer_data, buffer_ids, raw_inv_cov_matrix)
        elif method_name == 'MaxDistanceExpansionReplacement':
            replacer = MaxDistanceExpansionReplacement(buffer_data, buffer_ids)

```

```

# Phase 2: Steady state (Concept drift detection & Sampling)
else:
    diff_measurement = new_measurement - previous_measurement

    # Update CUSUM with the new measurement and check for ALARM simultaneously
    if cusum.update(diff_measurement):

        # ALARM: Concept Drift Detected - End of Epoch
        end_index = index

        # Retrieve final snapshot of the buffer and log it
        buffer_data, _ = buffer.getDataAndIds()
        global_indices = buffer.getGlobalIndices()
        exporter.log_epoch(buffer_data, global_indices, epoch_id, start_index, end_index)

        # Reset the buffer and CUSUM to re-enter Phase 1 for the new Epoch
        buffer.reset()
        cusum.reset()
        epoch_id += 1
        start_index = index + 1

    else:
        # IN-CONTROL: Execute Replacement Strategy
        is_replaced, replace_idx = replacer.check_new_measurement(new_measurement)

        if is_replaced:
            buffer.replacement(new_measurement, replace_idx, index)
            replacer.sync_state() # State synchronization

    previous_measurement = new_measurement

# Post-Processing: Log the final, incomplete epoch at the end of the data stream
if buffer.total_samples_counter > 0:
    end_index = total_measurements - 1

```

```

        buffer_data, _ = buffer.getDataAndIds()
        global_indices = buffer.getGlobalIndices()

        # Extract only the valid rows if the buffer wasn't completely full
        if buffer.total_samples_counter < buffer.size:
            valid_data = buffer_data[:buffer.total_samples_counter]
            valid_indices = global_indices[:buffer.total_samples_counter]
        else:
            valid_data = buffer_data
            valid_indices = global_indices
        exporter.log_epoch(valid_data, valid_indices, epoch_id, start_index, end_index)

    # Write the accumulated synopses to the CSV file
    exporter.export_results(feature_names)

print("\nAll simulations completed successfully for all replacement methods! Check the generated folders.")

```

VECTOR CUSUM

```

import numpy as np
from numpy.linalg import inv
import math

class VectorCusum:
    # The VectorCusum class implements the Vector Multivariate Cumulative Sum algorithm

    def __init__(self, mean_vector, covariance_matrix, decision_interval, reference_value):
        # Init method initiates the class variables and calculates the inverse covariance matrix

        self.mean_vector = np.array(mean_vector)
        self.covariance_matrix = np.array(covariance_matrix)
        self.decision_interval = decision_interval # Decision interval H
        self.reference_value = reference_value # Reference value K

```

```

# Inverse covariance matrix calculation  $\Sigma^{-1}$ 
self.inverse_covariance_matrix = inv(covariance_matrix)

# Number of variables calculation
self.variables_num = mean_vector.size

self.s_vector = np.zeros(self.variables_num)
self.length_c = 0.0
self.test_statistic = 0.0 # Test statistic  $Y_n$ 
self.out_of_control = False # Control variable. Turns true when a change of epoch occurs
self.run_length = 0 # Counter from the last reset

def reset(self):
    # Reset method resets the variables when an epoch change occurs

    self.s_vector = np.zeros(self.variables_num)
    self.run_length = 0
    self.out_of_control = False

def update(self, new_sample):
    # Update method processes a new sample and detects statistical shifts in the stream

    s_temp = (new_sample - self.mean_vector) + self.s_vector # Temporary vector calculation  $s\_temp = (x_n - \mu)$ 
+ s_{n-1}
    self.length_c = math.sqrt((s_temp.transpose())@self.inverse_covariance_matrix@s_temp)
    self.run_length = self.run_length + 1

    # Check if length  $C_n$  is within the reference value  $K$ 
    if self.length_c <= self.reference_value :
        self.s_vector = np.zeros(self.variables_num)
        self.test_statistic = 0.0 # Test statistic  $Y_n$  reset
        self.out_of_control = False

    else:
        self.s_vector = (s_temp) * (1 - self.reference_value / self.length_c)

```

```

        self.test_statistic = self.length_c - self.reference_value # Test statistic  $Y_n$  calculation

        #print(f"Index: {self.run_length}, Test Statistic ( $Y_n$ ): {self.test_statistic}")

        # Check if  $Y_n$  exceeds decision interval  $H$ 
        if self.test_statistic > self.decision_interval:
            self.out_of_control = True # Epoch change signal
        else:
            self.out_of_control = False

        # print("Y_n" + str(self.run_length) + ":" + str(round(self.test_statistic, 2)))
        # print("S_n" + str(self.run_length) + ":" + str(np.round(self.s_vector, 2)))

        return self.out_of_control

```

BUFFER

```

import numpy as np

class DataBuffer:
    # The DataBuffer class implements the Buffer algorithm

    def __init__(self, size, variables_num):
        # Init method initiates the class variables and allocates memory

        self.size = size
        self.data = np.zeros((size, variables_num), dtype=np.float32) # Initialize data array with zeros (size × variables_num)
        self.samples_ids = np.zeros(size, dtype=int) # Initialize samples_ids array with zeros (size × 1)
        self.global_indices = np.zeros(size, dtype=int) # Initialize the indices array
        self.total_samples_counter = 0
        self.is_full = False

    def reset(self):
        # reset method resets the variables when an epoch change occurs

```

```

        self.data.fill(0)
        self.samples_ids.fill(0)
        self.global_indices.fill(0)
        self.total_samples_counter = 0
        self.is_full = False

def getDataAndIds(self):
    # getDataAndIds method returns read-only views of the data and sample_ids_read_only arrays.

    data_read_only = self.data.view()
    samples_ids_read_only = self.samples_ids.view()

    data_read_only.setflags(write=False)
    samples_ids_read_only.setflags(write=False)

    return data_read_only, samples_ids_read_only

def getGlobalIndices(self):
    # Gets the indices array

    global_indices_read_only = self.global_indices.view()
    global_indices_read_only.setflags(write=False)
    return global_indices_read_only

def replacement (self,new_data, old_data_index, global_index):
    # Replacement method implements Phase 2: In-place replacement

    if old_data_index < 0 or old_data_index >= self.size:
        return False # Index out of bounds
    else:
        self.data[old_data_index] = new_data
        # Assign new unique ID to maintain chronological info
        self.samples_ids[old_data_index] = self.total_samples_counter
        self.global_indices[old_data_index] = global_index

```

```

        self.total_samples_counter = self.total_samples_counter + 1
        return True

def insert(self, new_data, global_index):
    # insert method implements Phase 1: Sequential filling

    self.data[self.total_samples_counter] = new_data
    self.samples_ids[self.total_samples_counter] = self.total_samples_counter
    self.global_indices[self.total_samples_counter] = global_index
    self.total_samples_counter = self.total_samples_counter + 1

    # Check if the buffer is full to signal Phase 2 transition
    if self.total_samples_counter >= self.size:
        self.is_full = True

def isFull(self):
    # isFull method returns the status of the buffer

    return self.is_full

```

COVARIANCE MATRIX AND MEAN CALCULATION

```

import numpy as np

class CovMeanCalculation:
    # The CovMeanCalculation class implements the calculation of covariance and mean matrices directly from an in-memory buffer

    def __init__(self):
        # Init method initializes the class as a stateless entity to minimize resource allocation in Edge environments
        pass

    def differential_calc(self, buffer_data):
        return np.diff(buffer_data, axis=0) # Calculation of consecutive differences

```

```

def load_cov_mean(self, data):

    # Vectorized calculation of the mean vector and the covariance matrix
    mean_vector = np.mean(data, axis=0)
    covariance_matrix = np.cov(data, rowvar=False)

    # Addition of a small epsilon to the diagonal to ensure mathematical stability during matrix inversion
    epsilon = 1e-4
    np.fill_diagonal(covariance_matrix, covariance_matrix.diagonal() + epsilon)

    #print(np.round(covariance_matrix, 4))

    return mean_vector, covariance_matrix

```

MAHALANOBIS RANGE REPLACEMENT METHOD

```

import numpy as np
import math

class MahalanobisRangeReplacement:
    # The MahalanobisRangeReplacement class implements the Mahalanobis Range Replacement Strategy algorithm.

    def __init__(self, buffer, samples_ids, inv_cov_matrix):
        # Init method initiates the class variables

        self.buffer = buffer
        self.samples_ids = samples_ids
        self.inv_cov_matrix = inv_cov_matrix

        # Calculate the mean vector across variables (axis=0) to represent M1
        self.mean_vector = np.mean(self.buffer, axis=0)

        # Calculate initial distances for all points in the buffer
        self.distances = self.distance_calc(self.buffer, self.mean_vector)

```

```

        # Retrieve IDs and max distance
        self.min_distance_id, self.max_distance_id, self.max_distance = self.min_max_calc(self.distances,
self.samples_ids)

def distance_calc(self, buffer, mean_vector):
    # Calculate Mahalanobis distances for all points using a loop for maximum clarity.

    n_samples = buffer.shape[0]
    distances = np.zeros(n_samples)

    # Loop through each sample in the buffer and calculate its Mahalanobis distance
    for i in range(n_samples):
        diff = buffer[i] - mean_vector
        distances[i] = math.sqrt(diff.transpose() @ self.inv_cov_matrix @ diff)

    return distances

def min_max_calc(self, distances, samples_ids):
    # Calculation of the max distance and the min and max distance ids

    # Identify the indices for the minimum (dmin) and maximum (dmax) distances
    self.min_idx = np.argmin(distances)
    max_idx = np.argmax(distances)

    # Extract the corresponding IDs and maximum distance
    min_distance_id = samples_ids[self.min_idx]
    max_distance_id = samples_ids[max_idx]
    max_distance = distances[max_idx]

    return min_distance_id, max_distance_id, max_distance

def check_new_measurement(self, new_measurement):
    # The method checks if the new measurement expands the cluster statistically

    # Get the number of samples (N) currently in the buffer

```

```

n_samples = self.buffer.shape[0]

# Retrieve the actual vector to be replaced (d_min) using its index
min_vector = self.buffer[self.min_idx]

# Calculate the new mean vector to represent M2
temp_mean_vector = self.mean_vector + (new_measurement - min_vector) / n_samples

# Calculate the distance of all existing points to the temp mean (M2)
temp_distances = self.distance_calc(self.buffer, temp_mean_vector)

# Correct the distance for the newly inserted point which replaces d_min
diff_new = new_measurement - temp_mean_vector
new_point_distance = math.sqrt(diff_new.transpose() @ self.inv_cov_matrix @ diff_new)
temp_distances[self.min_idx] = new_point_distance

# Find the new maximum distance (d_max')
new_max_distance = np.max(temp_distances)

# Evaluate the replacement criterion (d_max' > d_max)
if new_max_distance > self.max_distance:
    # Update method's variables
    self.mean_vector = temp_mean_vector
    self.distances = temp_distances
    return True, self.min_idx # Returns true and the index of the measurement that will be replaced
else:
    return False, None

def sync_state(self):
    # Syncs the state

    self.min_distance_id, self.max_distance_id, self.max_distance = self.min_max_calc(self.distances,
self.samples_ids)

```

MAX DISTANCE EXPANSION REPLACEMENT METHOD

```
import numpy as np
from numpy.linalg import norm

class MaxDistanceExpansionReplacement:
    # The MaxDistanceExpansionReplacement class implements a distance-based replacement strategy that expands the
    overall spatial cluster

    def __init__(self, buffer, samples_ids):
        # Init method initiates the class variables

        self.buffer = buffer
        self.samples_ids = samples_ids

        # Perform the initial calculation of distances and states
        self.sync_state()

    def distance_calc(self, buffer, reference_vector):
        # Calculate Euclidean distances for all points to a reference vector

        distances = np.linalg.norm(buffer - reference_vector, axis=1)

        return distances

    def check_new_measurement(self, new_measurement):
        # The method checks if the new measurement expands the cluster

        # Calculate the distance of the new measurement to all existing points
        temp_distances = self.distance_calc(self.buffer, new_measurement)

        # Find the new maximum distance (d_max') and the index it connects to
        new_max_distance = np.max(temp_distances)
        new_max_idx = np.argmax(temp_distances)
```

```

# Evaluate the replacement criterion (d_max' > d_max)
if new_max_distance > self.max_distance:
    # If the new point creates the max distance WITH one of the min points, keep that min point!
    if new_max_idx == self.min_idx_1:
        replace_idx = self.min_idx_2
    else:
        replace_idx = self.min_idx_1

    return True, replace_idx # Returns true and the index of the measurement to be replaced
else:
    return False, None

def sync_state(self):
    # Syncs the state by completely recalculating all distances from scratch

    # Get the number of samples (N)
    n = self.buffer.shape[0]

    # Initialize the NxN distance matrix with zeros
    pairwise_distances = np.zeros((n, n))

    # Iterate to calculate distance of each point (i) from every other point (j)
    for i in range(n):
        for j in range(n):
            # Compute the Euclidean distance (norm)
            diff = self.buffer[i] - self.buffer[j]
            pairwise_distances[i, j] = np.linalg.norm(diff)

    # Find Maximum Distance
    np.fill_diagonal(pairwise_distances, 0.0) # Ignore self-distances (0.0) on the diagonal for max calculation
    self.max_distance = np.max(pairwise_distances)

    # Find Minimum Distance Pair
    current_min_distance = float('inf') # Initialization
    min_index_1 = -1 # Initialization

```

```

min_index_2 = -1 # Initialization

# Iterate through the matrix to find the absolute minimum distance pair
for i in range(n):
    for j in range(i + 1, n): # We start j from (i + 1) to only check the upper triangle of the matrix.
        if pairwise_distances[i, j] < current_min_distance:
            current_min_distance = pairwise_distances[i, j]
            min_index_1 = i
            min_index_2 = j

# Save the indices of the closest pair found
self.min_idx_1 = min_index_1
self.min_idx_2 = min_index_2

```

MAX RANGE REPLACEMENT METHOD

```

import numpy as np
from numpy.linalg import norm

class MaxRangeReplacement:
    # The MaxRangeReplacement class implements the Max Range Replacement Strategy algorithm

    def __init__(self, buffer, samples_ids):
        # Init method initiates the class variables

        self.buffer = buffer
        self.samples_ids = samples_ids

        # Calculate the mean vector across variables (axis=0) to represent M1
        self.mean_vector = np.mean(self.buffer, axis=0)

        # Calculate distances
        self.distances = self.distance_calc(self.buffer, self.mean_vector)

        # Retrieve IDs and max distance

```

```

        self.min_distance_id,    self.max_distance_id,    self.max_distance    =    self.min_max_calc(self.distances,
self.samples_ids)

def distance_calc(self, buffer, mean_vector):
    # Calculate Euclidean distances (di) for all points

    distances = np.linalg.norm(buffer - mean_vector, axis=1)

    return distances

def min_max_calc(self, distances, samples_ids):
    # Calculation of the max distance and the min and max distance ids

    # Identify the indices for the minimum (dmin) and maximum (dmax) distances
    self.min_idx = np.argmin(distances)
    max_idx = np.argmax(distances)

    # Extract the corresponding IDs and maximum distance
    min_distance_id = samples_ids[self.min_idx]
    max_distance_id = samples_ids[max_idx]
    max_distance = distances[max_idx]

    return min_distance_id, max_distance_id, max_distance

def check_new_measurement(self, new_measurement):
    #The method checks if the new measurement expands the cluster

    # Get the number of samples (N) currently in the buffer
    n_samples = self.buffer.shape[0]

    # Retrieve the actual vector to be replaced (d_min) using its index
    min_vector = self.buffer[self.min_idx]

    # Calculate the new mean vector to represent M2
    temp_mean_vector = self.mean_vector + (new_measurement - min_vector) / n_samples

```

```

# Calculate the distance of all existing points to the temp mean (M2)
temp_distances = self.distance_calc(self.buffer, temp_mean_vector)

# Correct the distance for the newly inserted point which replaces d_min
new_point_distance = np.linalg.norm(new_measurement - temp_mean_vector)
temp_distances[self.min_idx] = new_point_distance

# Find the new maximum distance (d_max')
new_max_distance = np.max(temp_distances)

# Evaluate the replacement criterion (d_max' > d_max)
if new_max_distance > self.max_distance:
    # Update method's variables
    self.mean_vector = temp_mean_vector
    self.distances = temp_distances
    # print("New distances array: " + str(self.distances)) #For test
    return True, self.min_idx #Returns true and the index of the measurement that will be replaced
else:
    return False, None

def sync_state(self):
    # Syncs the state

    self.min_distance_id, self.max_distance_id, self.max_distance = self.min_max_calc(self.distances,
self.samples_ids)

```

SYNOPSIS EXPORTER

```

import csv

class SynopsisExporter:
    # The SynopsisExporter class makes a csv file with the exported synopses from the buffer at the end of each
    Epoch.

```

```

def __init__(self, h_value, k_value, buffer_size):
    # Creates the file name of the csv file and initiates the synopsis list.

    self.file_name = f"synopsis__h_{h_value}__k_{k_value}__size_{buffer_size}.csv"
    self.synopsis = []

def log_epoch(self, buffer_data, global_indices, epoch_id, start_index, end_index):
    # Appends the buffer data to the synopsis list, tagged with epoch metadata.

    for i, row in enumerate(buffer_data):
        # Rounding to 2 decimal places
        rounded_row = [round(float(val), 2) for val in row]

        # Combine metadata (epoch_id, start, end) and the multi-dimensional measurement into a single list.
        global_index = int(global_indices[i])
        row_data = [epoch_id, start_index, end_index, global_index] + rounded_row
        self.synopsis.append(row_data)

def export_results(self, feature_names):
    # Writes the accumulated synopsis data to a CSV file.

    headers = ["Epoch_ID", "Start_Index", "End_Index", "Measurement_Index"] + feature_names

    with open(self.file_name, mode='w', newline='') as file:
        writer = csv.writer(file)
        writer.writerow(headers)
        writer.writerows(self.synopsis)

    print(f"Successfully exported {len(self.synopsis)} reduced data points to {self.file_name}")

```

APPENDIX B: DATASET GENERATION

```
import os
import numpy as np
import pandas as pd

# Initialization of system parameters for the Data Stream simulation
N = 25000
fi = 0.95 # Autoregression coefficient (phi) representing system inertia
stability_factor = 0.01

# Selected variables for the multivariate Data Stream
columns = [
    'SPEEDKNOTS', 'MERPM', 'MEPOWER',
    'RANGEX', 'RANGEY', 'INCLINOMETERXZC', 'INCLINOMETERYZC'
]

# Initial mean vector representing the normal operational state
means = np.array([
    5.74,      # SPEEDKNOTS
    17.29,     # MERPM
    3268.67,   # MEPOWER
    12.44,     # RANGEX
    12.14,     # RANGEY
    10.79,     # INCLINOMETERXZC
    6.56       # INCLINOMETERYZC
])

# Standard deviations corresponding to the selected variables
std_devs = np.array([
    5.20,      # SPEEDKNOTS
    22.22,     # MERPM
    5553.74,   # MEPOWER
    15.05,     # RANGEX
    8.95,      # RANGEY
])
```

```

    14.21,      # INCLINOMETERXZC
    9.27       # INCLINOMETERYZC
])

# Correlation matrix defining the interdependence of the variables
R = np.array([
    # SPD    RPM    POW    RX    RY    XZC    YZC
    [ 1.00, 0.85, 0.80, -0.25, -0.30, -0.15, -0.25], # SPEED
    [ 0.85, 1.00, 0.95, 0.15, 0.20, 0.10, 0.15], # RPM
    [ 0.80, 0.95, 1.00, 0.20, 0.25, 0.15, 0.20], # POWER
    [-0.25, 0.15, 0.20, 1.00, 0.60, 0.70, 0.40], # RANGEX
    [-0.30, 0.20, 0.25, 0.60, 1.00, 0.40, 0.70], # RANGEY
    [-0.15, 0.10, 0.15, 0.70, 0.40, 1.00, 0.50], # XZC
    [-0.25, 0.15, 0.20, 0.40, 0.70, 0.50, 1.00] # YZC
])

# Calculation of the initial covariance matrix based on standard deviations and correlation
D = np.diag(std_devs)
cov_matrix = D @ R @ D

# Adjustment of the covariance matrix using the stability factor to restrict white noise variance
adjusted_cov = cov_matrix * stability_factor * (1 - fi**2)

# The mean vector for the multivariate white noise is zero
mean_noise = np.zeros(7)

dynamic_means = np.zeros((N, 7))

# --- Definition of dynamic means for Epoch simulation via concept drifts ---

# 1. Rows 0 - 499: Initial Epoch (Normal operational state)
dynamic_means[0:500] = means

# 2. Rows 500 - 999: Port conditions Epoch
dynamic_means[500:1000] = [0.0, 0.0, 0.0, 2.0, 2.0, 1.0, 1.0]

```

```

# 3. Rows 1000 - 1999: Voyage initiation Epoch
dynamic_means[1000:2000] = [14.0, 70.0, 20000.0, 10.0, 10.0, 8.0, 5.0]

# 4. Rows 2000 - 3999: Storm conditions Epoch (Proportional recovery based on RANGEX)
dynamic_means[2000:3000] = [14.0, 75.0, 25000.0, 35.0, 30.0, 25.0, 18.0]
dynamic_means[3000:3400] = [14.0, 74.5, 24600.0, 33.0, 28.3, 23.6, 17.0]
dynamic_means[3400:3700] = [14.0, 73.4, 23600.0, 28.0, 24.2, 20.1, 14.5]
dynamic_means[3700:4000] = [14.0, 71.5, 22000.0, 20.0, 17.5, 14.5, 10.5]

# 5. Rows 4000 - 7999: Calm sea Epoch (Proportional acceleration based on SPEEDKNOTS)
dynamic_means[4000:6000] = [14.0, 68.0, 19000.0, 5.0, 5.0, 4.0, 3.0]
dynamic_means[6000:6800] = [14.2, 68.8, 19800.0, 5.1, 5.1, 4.1, 3.1]
dynamic_means[6800:7500] = [15.0, 72.2, 23000.0, 5.2, 5.2, 4.2, 3.2]
dynamic_means[7500:8000] = [16.0, 76.5, 27000.0, 5.5, 5.5, 4.5, 3.5]

# 6. Rows 8000 - 15999: Increased speed Epoch
dynamic_means[8000:11000] = [18.0, 85.0, 35000.0, 6.0, 6.0, 5.0, 4.0]
dynamic_means[11000:12000] = [17.9, 84.7, 34800.0, 6.3, 6.3, 5.2, 4.2]
dynamic_means[12000:13000] = [17.7, 83.9, 34100.0, 7.3, 7.1, 6.0, 4.8]
dynamic_means[13000:14000] = [17.2, 82.1, 32800.0, 9.2, 8.8, 7.4, 6.0]
dynamic_means[14000:14800] = [16.7, 80.0, 31200.0, 11.6, 10.8, 9.2, 7.5]
dynamic_means[14800:15400] = [15.7, 76.4, 28400.0, 15.7, 14.3, 12.1, 10.0]
dynamic_means[15400:15800] = [14.6, 72.2, 25200.0, 20.6, 18.4, 15.7, 13.0]
dynamic_means[15800:16000] = [13.0, 66.4, 20800.0, 27.0, 24.0, 20.5, 17.0]

# 7. Rows 16000 - 25000: Severe storm Epoch
dynamic_means[16000:18000] = [10.0, 55.0, 12000.0, 40.0, 35.0, 30.0, 25.0]
dynamic_means[18000:19500] = [10.0, 55.1, 12100.0, 39.7, 34.7, 29.8, 24.8]
dynamic_means[19500:21000] = [10.1, 55.4, 12200.0, 39.0, 34.2, 29.3, 24.4]
dynamic_means[21000:22500] = [10.3, 55.8, 12400.0, 37.8, 33.1, 28.4, 23.6]
dynamic_means[22500:24000] = [10.5, 56.8, 13000.0, 35.2, 30.9, 26.5, 22.0]
dynamic_means[24000:25000] = [11.1, 58.7, 14000.0, 30.2, 26.6, 22.7, 18.8]

```

```

# --- Directory Setup for Datasets ---
output_dir = os.path.join('..', 'synthetic_data')
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# --- Data Stream Generation ---
base_seed = 42 # Base seed for reproducibility

for run_id in range(1, 11): # Loop from 1 to 10

    # Set the unique seed for this specific run
    np.random.seed(base_seed + run_id)

    # Initialization of the dataset array for the simulated Data Stream
    data = np.zeros((N, 7))

    # Establishment of the initial state (n=0)
    initial_state = means

    data[0] = initial_state + np.random.multivariate_normal(mean_noise, adjusted_cov)

    # Generation of the synthetic Data Stream using the Autoregressive Model of First Order AR(1)
    for i in range(1, N):
        e_n = np.random.multivariate_normal(mean_noise, adjusted_cov)
        data[i] = (fi * data[i-1]) + ((1 - fi) * dynamic_means[i]) + e_n

    # --- Application of physical and mechanical limits (clipping) ---
    # 1. SPEEDKNOTS: Restricted between 0 and 26 knots
    data[:, 0] = np.clip(data[:, 0], a_min=0.0, a_max=26.0)

    # 2. MERPM: Restricted between -31.20 (Astern) and 104.0 (Full Ahead)
    data[:, 1] = np.clip(data[:, 1], a_min=-31.20, a_max=104.0)

    # 3. MEPOWER: Restricted between 0 and 57100 kW
    data[:, 2] = np.clip(data[:, 2], a_min=0.0, a_max=57100)

```

```

# 4. RANGEX & RANGEY: Restricted between 0 and 90 degrees
data[:, 3] = np.clip(data[:, 3], a_min=0.0, a_max=90)
data[:, 4] = np.clip(data[:, 4], a_min=0.0, a_max=90)

# 5. INCLINOMETER ZC: Restricted to a maximum of 60 crossings
data[:, 5] = np.clip(data[:, 5], a_min=0.0, a_max=60.0)
data[:, 6] = np.clip(data[:, 6], a_min=0.0, a_max=60.0)
# -----

# Rounding values to 2 decimal places to simulate real telemetry sensors
data = np.round(data, 2)

# Export the generated Data Stream to an Excel file in the designated folder
df = pd.DataFrame(data, columns=columns)

filename = os.path.join(output_dir, f'synthetic_ship_dataset_{run_id}.xlsx')
df.to_excel(filename, index=False)

print(f"Dataset {run_id}/10 successfully generated and saved to {filename}")

print("All datasets have been created successfully")

```

APPENDIX C: SIMULATION

```
import numpy as np
from vector_cusum import VectorCusum

# The script performs a Monte Carlo simulation on ideal multivariate normal data
# to calculate the theoretical ARL0 for various combinations of h and k.

print("1. Constructing the theoretical baseline ( $\mu$  and  $\Sigma$ )...")

# Mean vector from the provided table
mean_vector = np.array([5.74, 17.29, 3268.67, 12.44, 12.14, 10.79, 6.56])

# Standard deviations from the provided table
std_devs = np.array([5.20, 22.22, 5553.74, 15.05, 8.95, 14.21, 9.27])

# Correlation matrix from the provided table
corr_matrix = np.array([
    [ 1.00,  0.85,  0.80, -0.25, -0.30, -0.15, -0.25],
    [ 0.85,  1.00,  0.95,  0.15,  0.20,  0.10,  0.15],
    [ 0.80,  0.95,  1.00,  0.20,  0.25,  0.15,  0.20],
    [-0.25,  0.15,  0.20,  1.00,  0.60,  0.70,  0.40],
    [-0.30,  0.20,  0.25,  0.60,  1.00,  0.40,  0.70],
    [-0.15,  0.10,  0.15,  0.70,  0.40,  1.00,  0.50],
    [-0.25,  0.15,  0.20,  0.40,  0.70,  0.50,  1.00]
])

# Covariance Matrix calculation:  $\Sigma_{ij} = R_{ij} * \sigma_i * \sigma_j$ 
covariance_matrix = np.outer(std_devs, std_devs) * corr_matrix

print("2. Generating Ideal Data Stream...")
# Generate 500,000 samples
TOTAL MOCK SAMPLES = 500000
ideal_data_stream = np.random.multivariate_normal(mean_vector, covariance_matrix, TOTAL MOCK SAMPLES)
```

```

# k values: from 0.5 to 5.0 with a step of 0.5
k_values = np.arange(0.5, 5.5, 0.5)

# h values: from 5.0 to 20.0 with a step of 1.0 for precise decision interval detection
h_values = np.arange(5.0, 21.0, 1.0)

required_epochs = 50 # Number of test Epochs per combination to calculate average

print("\n3. Starting Monte Carlo Simulation for ARL0...\n")
print(f'{'k (Ref Value)':<15} | {'h (Dec Interval)':<15} | {'Theoretical ARL0'}")
print("-" * 55)

# Iteration through parameter grid
for k in k_values:
    for h in h_values:

        # Init method initiates the Crosier's MCUSUM for the specific parameters
        cusum = VectorCusum(mean_vector, covariance_matrix, h, k)

        run_lengths = []
        data_index = 0

        # Collect required_epochs to calculate a statistically significant average
        while len(run_lengths) < required_epochs:

            sample = ideal_data_stream[data_index % TOTAL MOCK SAMPLES]

            # Update method checks for theoretical shifts
            is_out_of_control = cusum.update(sample)

            if is_out_of_control:
                # Store the length of the current theoretical Epoch
                run_lengths.append(cusum.run_length)
                cusum.reset()

```

```

        data_index += 1

# Calculate the theoretical ARL0 (Average Run Length in-control)
theoretical_arl = np.mean(run_lengths)

print(f"{k:<15.1f} | {h:<15.1f} | {theoretical_arl:.2f}")

# Break mechanism: Stop evaluating higher h values if ARL0 exceeds 500
if theoretical_arl > 500:
    print(f"--- Decision interval exceeded (>500). Stopping h-search for k={k:.1f} ---")
    break # Breaks out of the inner (h) loop and moves to the next k

print("\nSimulation Complete. Select (k, h) combinations that yield an ARL0 between 150 and 500.")

```

APPENDIX D: EVALUATION

GLUE CODE

```
import os
import glob
import pandas as pd

from compression_ratio_evaluator import CompressionRatioEvaluator
from volume_comparison_evaluator import VolumeComparisonEvaluator
from generalized_variance_evaluator import GeneralizedVarianceEvaluator
from pateF1_evaluator import PateF1Evaluator

# Determine file paths
script_dir = os.path.dirname(os.path.abspath(__file__))
parent_dir = os.path.abspath(os.path.join(script_dir, '..'))
mcusum_outputs_dir = os.path.join(parent_dir, 'mcusum_outputs')
synthetic_data_dir = os.path.join(parent_dir, 'synthetic_data')

# List of replacement strategies
replacement_methods = [
    'MahalanobisRangeReplacement',
    'MaxDistanceExpansionReplacement',
    'MaxRangeReplacement'
]

# List to store all evaluation results
all_evaluation_results = []

# List to store records of problematic files
problematic_records = []

# Feature columns used for multivariate analysis
feature_columns = [
    'SPEEDKNOTS', 'MERPM', 'MEPOWER',
    'RANGEX', 'RANGEX', 'RANGEX', 'INCLINOMETERXZC', 'INCLINOMETERYZC'
```

```

]

# Drift points and buffer zone for PATE-F1 evaluation
drifts = [
    500, 1000, 2000, 3000, 3400, 3700, 4000, 6000, 6800, 7500,
    8000, 11000, 12000, 13000, 14000, 14800, 15400, 15800,
    16000, 18000, 19500, 21000, 22500, 24000
]

buffer_zone = 12

print("Beginning of calculations for the 10 datasets...")

# LOOP 1: Iterate over each replacement method
for method in replacement_methods:
    method_dir = os.path.join(mcusum_outputs_dir, method)
    print(f"Method: {method}")

    # LOOP 2: Iterate over each dataset
    for dataset_id in range(1, 11):
        dataset_outputs_folder = os.path.join(method_dir, f"Synthetic ship dataset {dataset_id}")

        # Load the corresponding original dataset
        original_data_path = os.path.join(synthetic_data_dir, f"synthetic_ship_dataset_{dataset_id}.xlsx")
        original_df = pd.read_excel(original_data_path)

        # Find all synopsis CSV files for various h, k, size parameters
        synopsis_files = glob.glob(os.path.join(dataset_outputs_folder, "synopsis__*.csv"))

        for syn_file in synopsis_files:
            filename = os.path.basename(syn_file)

            # Extract parameters from filename
            parts = filename.replace('.csv', '').split('__')
            h_val = float(parts[1].split('_')[1])

```

```

k_val = float(parts[2].split('_')[1])
size_val = int(parts[3].split('_')[1])

synopsis_df = pd.read_csv(syn_file)

# 1. Calculate Compression Ratio (C)
c_score = CompressionRatioEvaluator(original_df, synopsis_df).run()

# 2. Calculate Volume Ratio (R)
vol_evaluator = VolumeComparisonEvaluator(original_df, synopsis_df)
r_score, r_zero_epochs, r_zero_epoch_ids, r_total_epochs = vol_evaluator.run()

# 3. Calculate Generalized Variance Ratio (S) - Raw value
gv_evaluator = GeneralizedVarianceEvaluator(original_df, synopsis_df)
s_score, s_zero_epochs, s_zero_epoch_ids, s_total_epochs = gv_evaluator.run()

# Check for problematic epochs (zero volume or zero generalized variance)
if r_zero_epochs > 0 or s_zero_epochs > 0:
    problematic_records.append({
        'Method': method,
        'Dataset_ID': dataset_id,
        'h': h_val,
        'k': k_val,
        'Size': size_val,
        'Volume_Zero_Epochs': r_zero_epochs,
        'GV_Zero_Epochs': s_zero_epochs,
        'Total_Epochs': r_total_epochs,
        'Volume_Zero_Epoch_IDs': ','.join(map(str, r_zero_epoch_ids)),
        'GV_Zero_Epoch_IDs': ','.join(map(str, s_zero_epoch_ids))
    })

# 4. Calculate PATE-F1, True & False Alarms
pate_results = PateF1Evaluator(synopsis_df).run()

f1_score = pate_results.get("F1_Score", 0.0)

```

```

count_TP = pate_results.get("True_Alarms", 0)
count_FP = pate_results.get("False_Alarms", 0)

# Store raw results for current run
all_evaluation_results.append({
    'Method': method,
    'Dataset_ID': dataset_id,
    'h': h_val,
    'k': k_val,
    'Size': size_val,
    'Compression_Ratio_C': c_score,
    'Volume_Ratio_R': r_score,
    'Gen_Variance_Ratio_S': s_score,
    'True_Alarms': count_TP,
    'False_Alarms': count_FP,
    'F1_Score': f1_score
})

# 5. Export averages over the 10 datasets and save
results_df = pd.DataFrame(all_evaluation_results)

# Group by parameters and calculate the mean for all metrics (the final 120 combinations)
summary_df = results_df.groupby(['Method', 'h', 'k', 'Size']).mean(numeric_only=True).reset_index()

# Apply Min-Max Normalization to create S' from the averaged S values
s_min = summary_df['Gen_Variance_Ratio_S'].min()
s_max = summary_df['Gen_Variance_Ratio_S'].max()

summary_df["Gen_Variance_Ratio_S'"] = (summary_df['Gen_Variance_Ratio_S'] - s_min) / (s_max - s_min)

# Compute combined indices using the normalized S' values
summary_df['Sampling_Quality_SQ'] = (summary_df['Volume_Ratio_R'] + summary_df["Gen_Variance_Ratio_S'"]) / 2.0
summary_df['Statistical_Efficiency_SE'] = (summary_df['Volume_Ratio_R'] + summary_df["Gen_Variance_Ratio_S'"] +
summary_df['Compression_Ratio_C']) / 3.0

```

```

summary_df['Total_Performance_TP'] = (summary_df['Compression_Ratio_C'] + summary_df['Volume_Ratio_R'] +
summary_df["Gen_Variance_Ratio_S"] + summary_df['F1_Score']) / 4.0

# Select and order columns for the final report
output_columns = [
    'Method', 'h', 'k', 'Size',
    'Compression_Ratio_C', 'Volume_Ratio_R', 'Gen_Variance_Ratio_S', "Gen_Variance_Ratio_S'",
    'Sampling_Quality_SQ', 'Statistical_Efficiency_SE', 'Total_Performance_TP',
    'True_Alarms', 'False_Alarms', 'F1_Score'
]
summary_df = summary_df[output_columns]

# Save the summary to Excel file
final_output_path = os.path.join(script_dir, 'combined_evaluation_summary.xlsx')
summary_df.to_excel(final_output_path, index=False, engine='openpyxl')

# Save problematic files to a separate Excel file
problematic_df = pd.DataFrame(problematic_records)
problematic_path = os.path.join(script_dir, 'problematic_epochs.xlsx')
problematic_df.to_excel(problematic_path, index=False, engine='openpyxl')

print(f"\nThe process completed successfully!")
print(f"The results (average of 10 datasets) were saved to: {final_output_path}")
print(f"Problematic files saved to: {problematic_path}")

```

GENERALIZED VARIANCE INDEX

```

import numpy as np
import pandas as pd

class GeneralizedVarianceEvaluator:
    # Initialization of the Evaluator with the original and synopsis datasets
    def __init__(self, original_dataset_df, synopsis_df):
        self.original_dataset_df = original_dataset_df
        self.synopsis_df = synopsis_df

```

```

# Selected variables for the multivariate Data Stream
self.feature_columns = [
    'SPEEDKNOTS', 'MERPM', 'MEPOWER',
    'RANGEX', 'RANGEY', 'INCLINOMETERXZC', 'INCLINOMETERYZC'
]

# Calculation of the generalized variance ( $|\Sigma|$ ) in the n-dimensional space
def calculate_generalized_variance(self, data_matrix):
    cov_matrix = np.cov(data_matrix, rowvar=False) # Calculate the covariance matrix ( $\Sigma$ ).
    generalized_variance = np.linalg.det(cov_matrix) # Calculation of the generalized variance as the
determinant of the covariance matrix
    return generalized_variance

# Execution of the generalized variance comparison method across all Epochs
def evaluate_epochs(self):
    epochs = self.synopsis_df['Epoch_ID'].unique() # Extraction of the unique Epoch IDs from the Buffer synopsis
    variance_ratios = [] # List of the epochs with zero generalized variance

    zero_gv_epochs = 0 # Keep track of how many epochs have zero volume (problematic ones)
    total_epochs = len(epochs) # Total number of epochs in the synopsis file
    zero_gv_epoch_ids = []

    # Loop through each Epoch to calculate the specific generalized variance ratio
    for epoch_id in epochs:
        # Isolation of the synopsis records for the current Epoch
        epoch_synopsis = self.synopsis_df[self.synopsis_df['Epoch_ID'] == epoch_id]

        # Retrieval of the global start and end indices
        start_idx = int(epoch_synopsis['Start_Index'].iloc[0])
        end_idx = int(epoch_synopsis['End_Index'].iloc[0])

        # Isolation of the corresponding measurements from the original dataset
        # (+1 is added to ensure the end_idx is included in the Pandas slice)

```

```

epoch_dataset = self.original_dataset_df.iloc[start_idx : end_idx + 1]

# Extraction of the n-dimensional feature columns as NumPy arrays
synopsis_matrix = epoch_synopsis[self.feature_columns].values
dataset_matrix = epoch_dataset[self.feature_columns].values

# Calculation of the generalized variances ( $|\Sigma|$ ) for the synopsis and the original dataset
gv_synopsis = self.calculate_generalized_variance(synopsis_matrix)
gv_dataset = self.calculate_generalized_variance(dataset_matrix)

# Calculation of the generalized variance ratio for the current Epoch
if gv_dataset == 0: # Check if original volume is zero
    zero_gv_epochs += 1
    zero_gv_epoch_ids.append(epoch_id)
    if gv_synopsis == 0:
        ratio = 1.0 # Both are zero thus, synopsis keeps the property
    else:
        ratio = np.nan # The Original is zero but synopsis is not, set nan
else:
    ratio = gv_synopsis / gv_dataset # The normal case

variance_ratios.append(ratio)

# Save stats so glue code can check for problematic epochs
self.zero_gv_epochs = zero_gv_epochs
self.total_epochs = total_epochs
self.zero_gv_epoch_ids = zero_gv_epoch_ids

# Calculation of the overall Sampling Quality Score based on the generalized variance ratio
final_ratio = np.mean(variance_ratios)

# Return the final score rounded to two decimal places
return round(final_ratio, 2)

# Main orchestration method to execute the evaluation process

```

```

def run(self):
    # Execution of the internal evaluation method
    final_R_score = self.evaluate_epochs()

    return (final_R_score, self.zero_gv_epochs, self.zero_gv_epoch_ids, self.total_epochs)

```

VOLUME COMPARISON INDEX

```

import numpy as np
import pandas as pd

class VolumeComparisonEvaluator:

    # Initialization of the Evaluator with the original and synopsis datasets
    def __init__(self, original_dataset_df, synopsis_df):
        self.original_dataset_df = original_dataset_df
        self.synopsis_df = synopsis_df

        # Selected variables for the multivariate Data Stream
        self.feature_columns = [
            'SPEEDKNOTS', 'MERPM', 'MEPOWER',
            'RANGEX', 'RANGEY', 'INCLINOMETERXZC', 'INCLINOMETERYZC'
        ]

        # Calculation of the bounding box volume in the n-dimensional space
        def calculate_bounding_box_volume(self, data_matrix):
            # Determination of the minimum and maximum values for each dimension
            mins = np.min(data_matrix, axis=0)
            maxs = np.max(data_matrix, axis=0)

            ranges = maxs - mins # Calculation of the ranges for each variable
            volume = np.prod(ranges) # Calculation of the total volume as the product of all ranges

            return volume

```

```

# Execution of the volume comparison method across all Epochs
def evaluate_epochs(self):

    epochs = self.synopsis_df['Epoch_ID'].unique() # Extraction of the unique Epoch IDs from the Buffer synopsis
    volume_ratios = []

    zero_volume_epochs = 0 # Keep track of how many epochs have zero volume (problematic ones)
    total_epochs = len(epochs) # Total number of epochs in the synopsis file
    zero_volume_epoch_ids = [] # List of the epochs with zero volume

    # Loop through each Epoch to calculate the specific volume ratio
    for epoch_id in epochs:
        # Isolation of the synopsis records for the current Epoch
        epoch_synopsis = self.synopsis_df[self.synopsis_df['Epoch_ID'] == epoch_id]

        # Retrieval of the global start and end indices
        start_idx = int(epoch_synopsis['Start_Index'].iloc[0])
        end_idx = int(epoch_synopsis['End_Index'].iloc[0])

        # Isolation of the corresponding measurements from the original dataset
        # (+1 is added to ensure the end_idx is included in the Pandas slice)
        epoch_dataset = self.original_dataset_df.iloc[start_idx : end_idx + 1]

        # Extraction of the n-dimensional feature columns as NumPy arrays
        synopsis_matrix = epoch_synopsis[self.feature_columns].values
        dataset_matrix = epoch_dataset[self.feature_columns].values

        # Calculation of the volumes for the synopsis and the original dataset
        v_synopsis = self.calculate_bounding_box_volume(synopsis_matrix)
        v_dataset = self.calculate_bounding_box_volume(dataset_matrix)

        # Calculation of the volume ratio for the current Epoch
        if v_dataset == 0: # Check if original volume is zero
            zero_volume_epochs += 1

```

```

        zero_volume_epoch_ids.append(epoch_id)
    if v_synopsis == 0:
        ratio = 1.0 # Both are zero thus, synopsis keeps the property
    else:
        ratio = np.nan # The Original is zero but synopsis is not, set nan
    else:
        ratio = v_synopsis / v_dataset # The normal case

    volume_ratios.append(ratio)

# Save stats so glue code can check for problematic epochs
self.zero_volume_epochs = zero_volume_epochs
self.total_epochs = total_epochs
self.zero_volume_epoch_ids = zero_volume_epoch_ids

# Calculation of the overall Sampling Quality Score based on the volume ratio
final_ratio = np.mean(volume_ratios)

# Return the final score rounded to two decimal places
return round(final_ratio, 2)

# Main orchestration method to execute the evaluation process
def run(self):
    # Execution of the internal evaluation method
    final_R_score = self.evaluate_epochs()

    return (final_R_score, self.zero_volume_epochs, self.zero_volume_epoch_ids, self.total_epochs)

```

INDEX PATE-F1

```

import numpy as np
import pandas as pd

class PateF1Evaluator:
    # Initialization of the Evaluator directly with the synopsis DataFrame

```

```

def __init__(self, synopsis_df, buffer_zone=12, total_measurements=25000):
    self.synopsis_df = synopsis_df

    # Extraction of unique alarm timestamps (End_Index)
    raw_alarms = self.synopsis_df['End_Index'].unique().tolist()

    # The final forced flush at the end of the data stream is not an alarm.
    last_index = total_measurements - 1
    if last_index in raw_alarms:
        raw_alarms.remove(last_index)

    # The dynamic list of alarm time indices
    self.alarms = sorted(raw_alarms)

    # The static list of time indices representing the actual epoch changes
    self.drifts = [
        500, 1000, 2000, 3000, 3400, 3700, 4000, 6000, 6800, 7500,
        8000, 11000, 12000, 13000, 14000, 14800, 15400, 15800,
        16000, 18000, 19500, 21000, 22500, 24000
    ]

    # The buffer zone
    self.buffer_zone = buffer_zone

    # Execution of the PATE-F1 weight calculation and absolute alarm counting
    def _calculate_weights(self):
        w_TP = 0.0
        w_FP = 0.0
        w_FN = 0.0

        # Counters for the absolute operational metrics
        count_TP = 0

        # Loop through each actual concept drift to evaluate detection performance
        for t_k in self.drifts:

```

```

# Definition of the upper bound of the buffer zone for the current Epoch transition
upper_bound = t_k + self.buffer_zone

# Isolation of the alarms that fall within the current buffer zone
valid_alarms = [t_l for t_l in self.alarms if t_k <= t_l <= upper_bound]

if valid_alarms:
    # Selection of the first chronologically recorded alarm within the zone
    t_l = valid_alarms[0]

    # Calculation of the true positive weight (wTP) based on detection speed
    current_w_tp = (self.buffer_zone - (t_l - t_k)) / self.buffer_zone
    w_TP += current_w_tp

    # Calculation of the complementary false positive weight (wFP)
    w_FP += (1.0 - current_w_tp)

    # Increment the absolute True Alarms counter
    count_TP += 1

    # Removal of the selected alarm from the list to prevent re-evaluation.
    self.alarms.remove(t_l)
else:
    # In case of a missed anomaly, the false negative weight is penalized
    w_FN += 1.0

# The length of the remaining list exactly represents the absolute False Alarms
count_FP = len(self.alarms)

# Each remaining alarm contributes a maximum weight of 1.0 to wFP.
w_FP += float(count_FP)

return w_TP, w_FP, w_FN, count_TP, count_FP

# Calculation of the final performance metrics (Precision, Recall, F1-Score)

```

```

def _calculate_metrics(self, w_TP, w_FP, w_FN):
    # Precision calculation
    if (w_TP + w_FP) > 0:
        precision = w_TP / (w_TP + w_FP)
    else:
        precision = 0.0

    # Recall calculation
    if (w_TP + w_FN) > 0:
        recall = w_TP / (w_TP + w_FN)
    else:
        recall = 0.0

    # F1-Score calculation (Harmonic mean of Precision and Recall)
    if (precision + recall) > 0:
        f1_score = 2 * (precision * recall) / (precision + recall)
    else:
        f1_score = 0.0

    return precision, recall, f1_score

# Main orchestration method to execute the evaluation process
def run(self):
    # Execution of the internal weight calculation and counting method
    w_TP, w_FP, w_FN, count_TP, count_FP = self._calculate_weights()

    # Execution of the metrics calculation method
    precision, recall, f1_score = self._calculate_metrics(w_TP, w_FP, w_FN)

    # Return a dictionary containing the absolute counts and the final evaluation metrics
    return {
        'True_Alarms': count_TP,
        'False_Alarms': count_FP,
        'w_TP': round(w_TP, 2),
        'w_FP': round(w_FP, 2),

```

```

        'w_FN': round(w_FN, 2),
        'Precision': round(precision, 4),
        'Recall': round(recall, 4),
        'F1_Score': round(f1_score, 4)
    }

```

COMPRESSION RATIO

```

import numpy as np
import pandas as pd

class CompressionRatioEvaluator:
    # Initialization of the Evaluator with the original and synopsis datasets
    def __init__(self, original_dataset_df, synopsis_df):
        self.original_dataset_df = original_dataset_df
        self.synopsis_df = synopsis_df

    # Calculation of the global compression ratio across the entire data stream
    def _evaluate_compression(self):
        # Determination of the total number of measurements in the original data stream (N)
        N = len(self.original_dataset_df)

        # Determination of the total number of synopsis elements flushed from the Buffer (M)
        M = len(self.synopsis_df)

        # Calculation of the Compression Ratio (CR)
        compression_ratio = 1.0 - (M / N)

        return compression_ratio

    # Main orchestration method to execute the evaluation process
    def run(self):
        # Execution of the internal evaluation method
        final_CR_score = self._evaluate_compression()

```

```
# Return the final score rounded to two decimal places  
return round(final_CR_score, 2)
```